

Elettronica

Esercitazioni su circuiti sequenziali

Uso dei display

Schede Nexys4-DDR e Zedboard

Obiettivo

- Misurare il tempo durante il quale un bottone viene premuto
- Sulla scheda abbiamo a disposizione un clock a 100 MHz
- Pertanto con un contatore possiamo misurare il tempo con una precisione di 10 ns
- Vogliamo mostrare il risultato su di un display a 7 segmenti a più cifre

Diodo LED

- Ogni LED è un diodo che si accende se l'**anodo** è ad una tensione positiva rispetto al **catodo**

Display a 7 segmenti

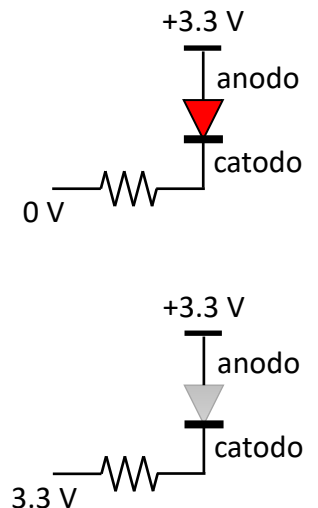
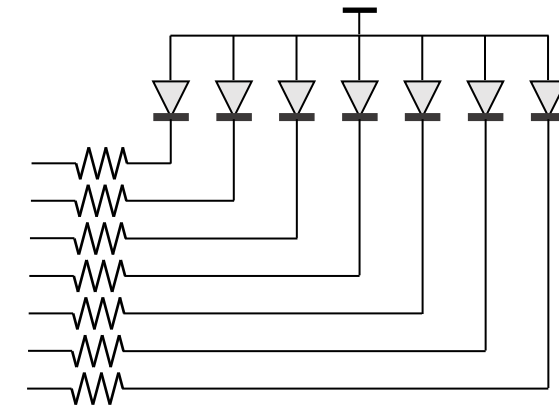
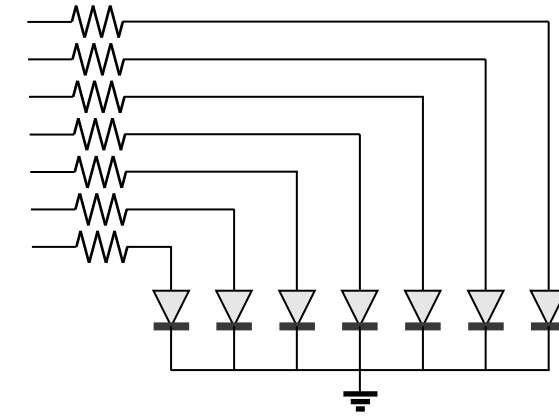
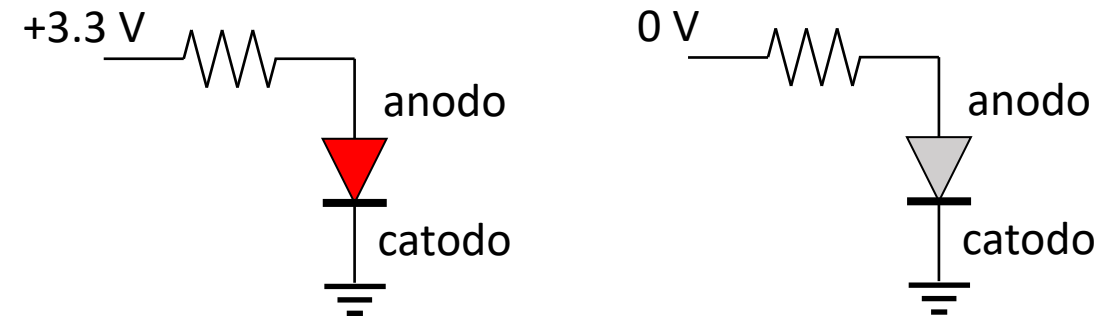
- Ogni display è costituito da 7 LED più il punto decimale

Connessione a catodo comune

- Solitamente **abbiamo supposto che i catodi fossero tutti connessi a massa, e che la logica pilotasse gli anodi**
- Un 1 logico accende (tensione positiva), lo 0 logico spegne

Connessione ad anodo comune

- Nella scheda Nexys4-DDR, gli **anodi** dei LED di ciascuna cifra sono tutti collegati assieme
- La scheda pilota invece i catodi
- Uno 0 logico accende, un 1 logico spegne

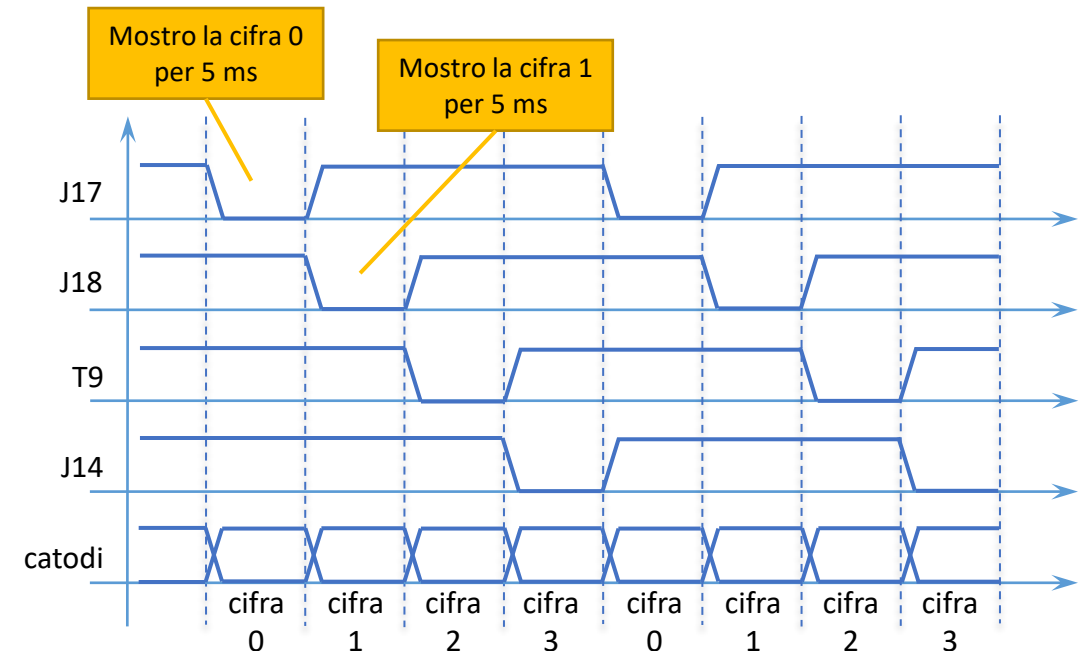
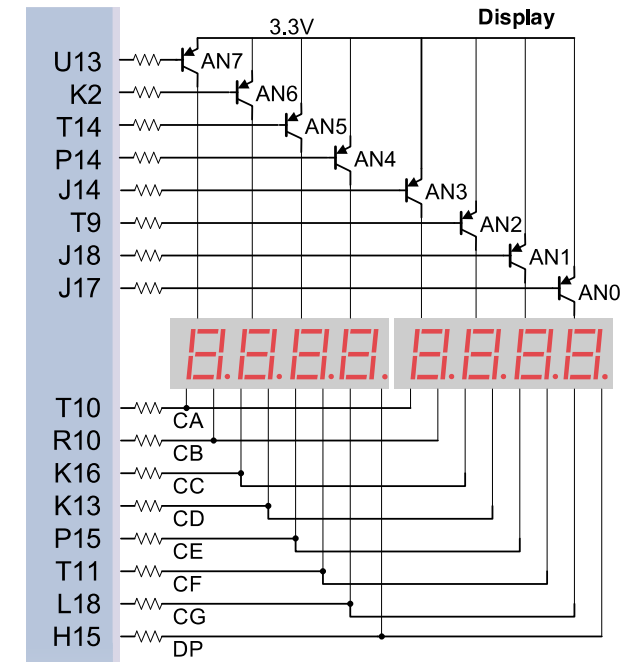


Connessione

- Nella scheda sono presenti 8 display a 7 segmenti
- Per controllarli tutti contemporaneamente ci sarebbe bisogno di $8 * 8 = 64$ connessioni
- Per semplificare si collegano assieme i catodi
- I catodi dei segmenti a di tutti i display sono collegati assieme, e pilotati da un singolo segnale
- Lo stesso per gli altri segmenti
- Uso solo 8 connessioni per i catodi
- Però, se accendo un segmento, si accende su tutti display!

Multiplexing

- Anodi connessi a V_{DD} tramite un transistor controllato
- La scheda può scegliere quali display accendere
- Poiché c'è un transistor pilota, il display si accende se il segnale di comando è allo 0 logico
- Accendo solo il primo display, e imposto il valore della cifra sui catodi
- Poi accendo il secondo display, e imposto i catodi in modo da vedere la seconda cifra
- Così via fino all'ottavo, e poi riprendo dal primo
- Se lo faccio abbastanza velocemente (e.g., 100 volte al secondo), tutti i display sembreranno accesi, ed avranno cifre diverse
- In realtà un solo display è acceso alla volta!



Contatore divisore

- Piloto il contatore a 100 MHz, ma **guardo i tre bit più significativi** per determinare quale cifra accendere
- Guardando tre bit conto da 0 a 7, per selezionare ognuna delle 8 cifre
- Per esempio, i bit 17, 18 e 19 contano a $100 \text{ MHz} / 2^{17} = 763 \text{ Hz}$, periodo 1.3 ms, faccio tutto il display in 10 ms, i.e., 95 volte al secondo
- I bit 19, 20 e 21 contano a $100 \text{ MHz} / 2^{19} = 191 \text{ Hz}$, periodo 5.2 ms, faccio tutto il display in 42 ms, i.e., 24 volte al secondo

Multiplexer 8 a 1

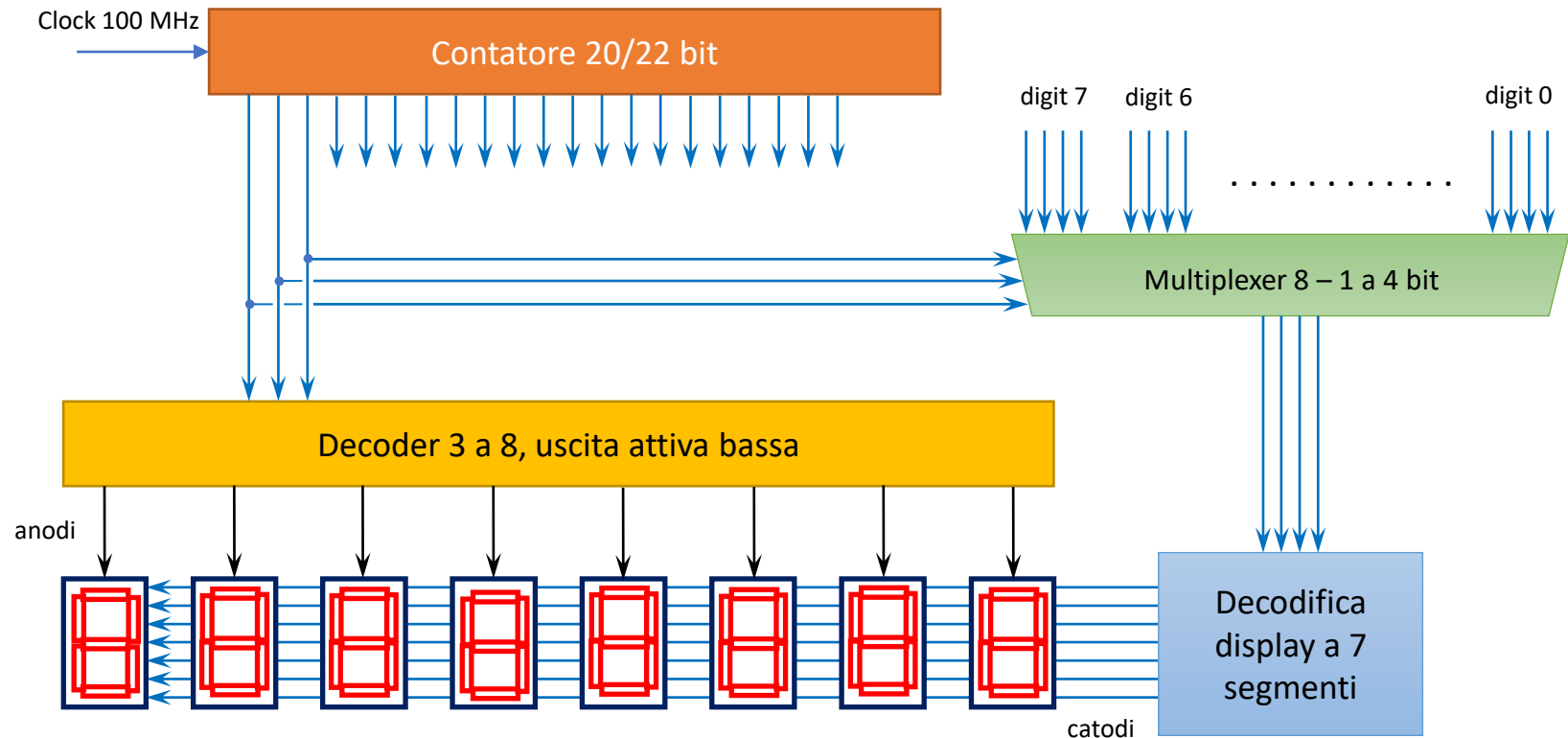
- Prende in ingresso **tutte** le cifre da mostrare
- **Seleziona solo la cifra da mostrare** in ogni istante, sulla base del valore del contatore
- Sincronizzato con il decoder 3 a 8

Decoder 3 a 8

- Accende una sola cifra alla volta, sulla base del valore del contatore
- Sincronizzata con il multiplexer
- L'uscita è attiva bassa per soddisfare il tipo di collegamento sulla scheda

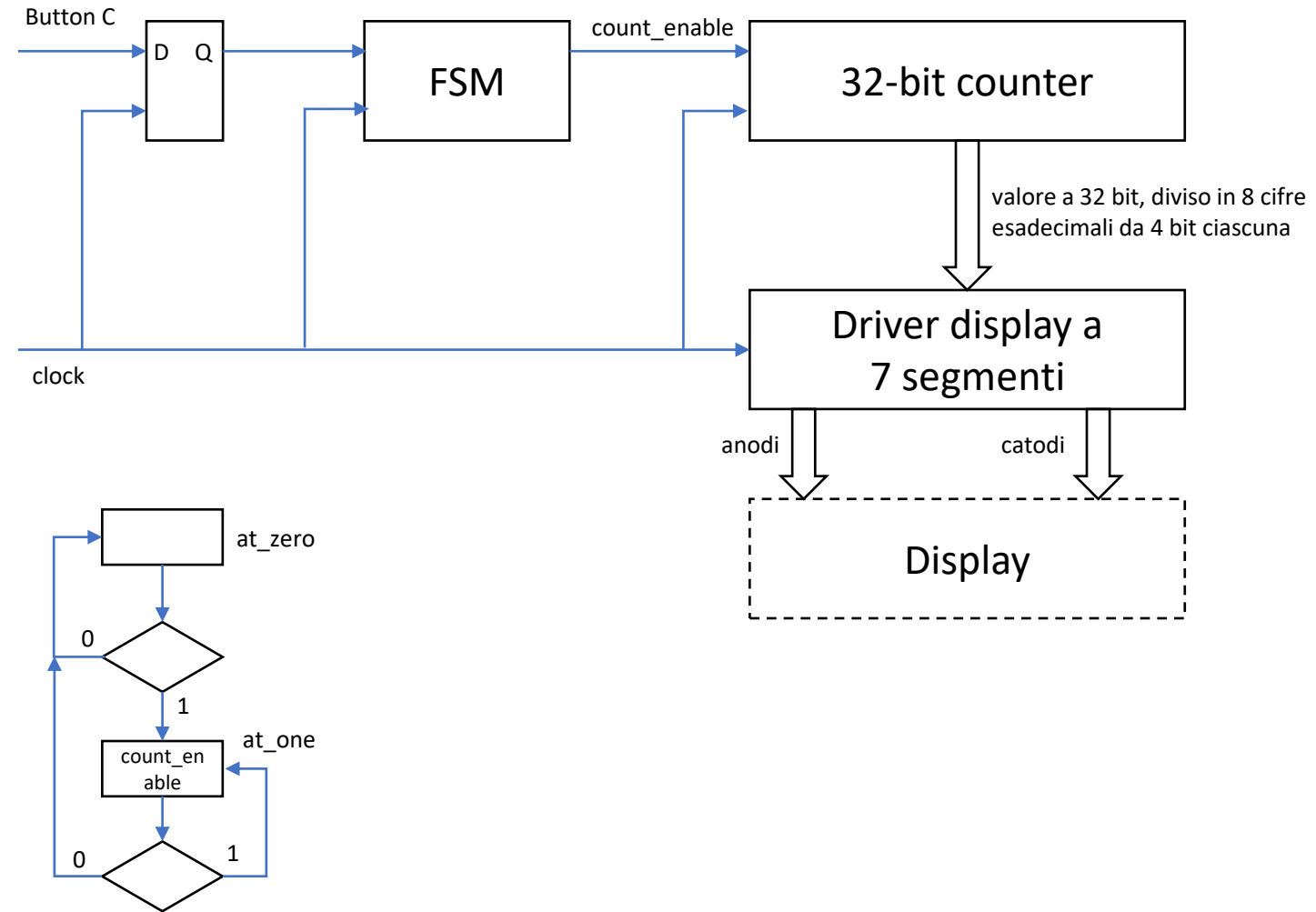
Decodifica 7 segmenti

- Ne basta una perché mostriamo solo una cifra alla volta
- Uscite attive basse, perché connessa ai catodi



Struttura sistema

- Un registro sincronizza il segnale dal bottone con il nostro sistema, in modo da evitare eventuali corse critiche sugli ingressi
- Una macchina a stati finiti comanda il segnale di count enable del contatore che misura la lunghezza dell'impulso. Questa MSF è abbastanza inutile ma è stata aggiunta per complicare un po' il progetto.
- Il contatore a 32 bit misura la lunghezza dell'impulso. La sua uscita a 32 bit viene divisa in 8 cifre ognuna da 4 bit, che corrispondono ad una cifra esadecimale.
- Mostriamo il risultato del conteggio in esadecimale, per evitare di fare conversioni da binario a decimale
- Il driver corrisponde al circuito visto nella slide precedente



architecture Behavioral **of** seven_segment_driver **is**

-- We will use a counter to derive the frequency for the displays
-- Clock is 100 MHz, we use 3 bits to address the display, so we count every
-- size - 3 bits

signal flick_counter : **unsigned**(size - 1 **downto** 0);

-- The digit is temporarily stored here

signal digit : **std_logic_vector**(3 **downto** 0);

-- Collect the values of the cathodes here

signal cathodes : **std_logic_vector**(7 **downto** 0);

begin

-- Divide the clock

process (clock, reset) **begin**

if reset = '0' **then**

 flick_counter <= (**others** => '0');

elsif rising_edge(clock) **then**

 flick_counter <= flick_counter + 1;

end if;

end process;

-- Select the anode

with flick_counter(size - 1 **downto** size - 3) **select**

AN <=

"11111110" **when** "000",

"11111101" **when** "001",

"11111011" **when** "010",

"11110111" **when** "011",

"11101111" **when** "100",

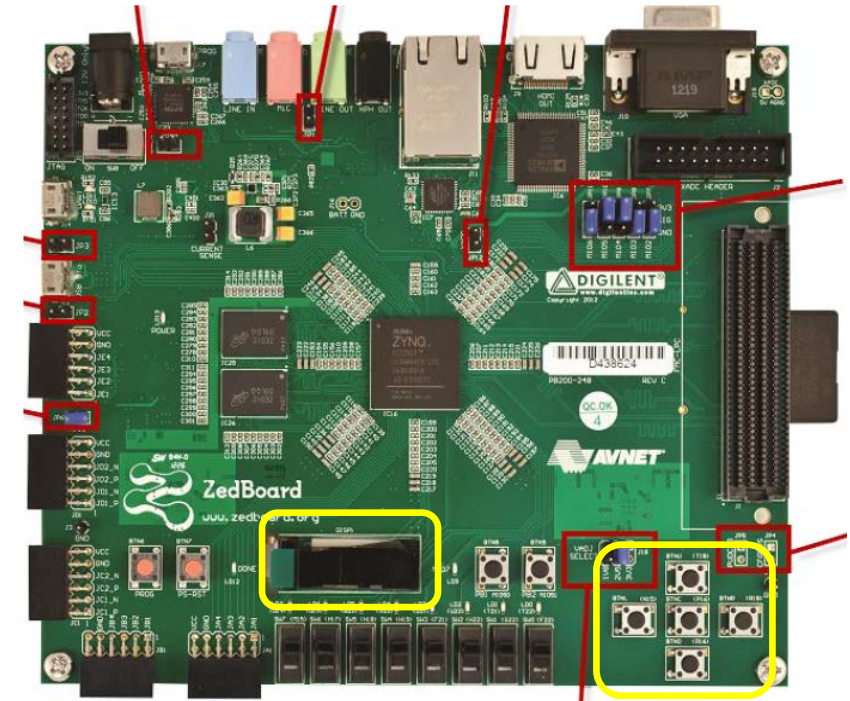
"11011111" **when** "101",

"10111111" **when** "110",

"01111111" **when others**;

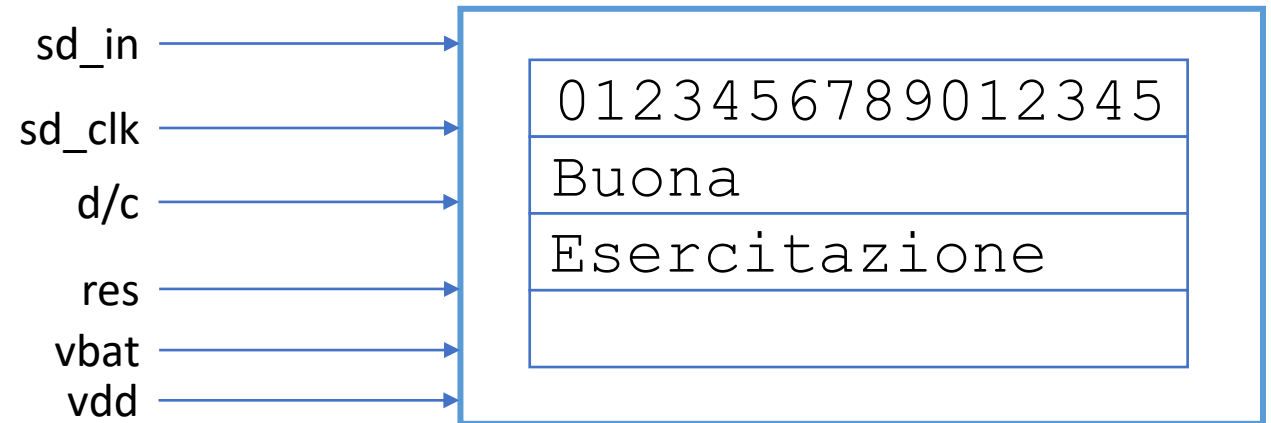
Scheda ZedBoard

- Il dispositivo Zynq contiene al suo interno una FPGA e due processori ARM Cortex-A9 (ci gira Linux)
- Diverse periferiche, ma non i display a 7 segmenti
- Usiamo invece il display OLED



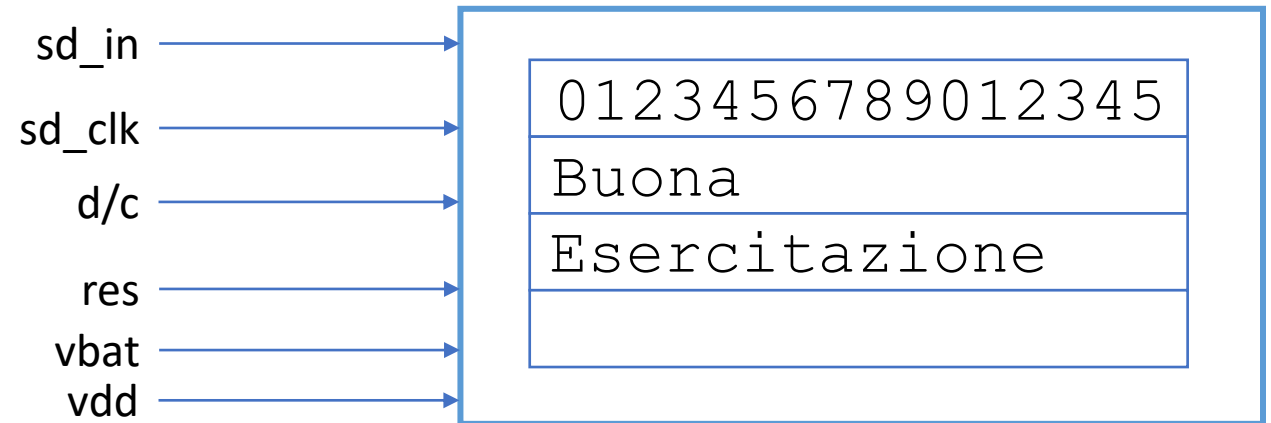
Driver display OLED

- Più complicato dei display a 7 segmenti
 - Il modulo ha una matrice di 64x128 pixel, è possibile accenderli e spegnerli singolarmente
 - Lo stato dei pixel è memorizzato internamente al display, e può essere modificato tramite appositi comandi
 - Si possono fare scritte o disegni



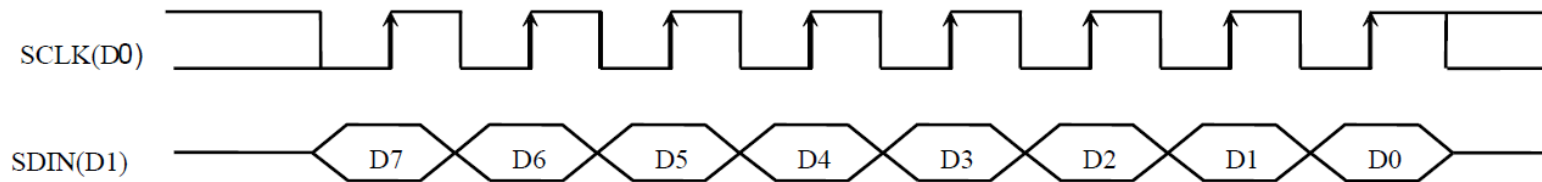
Driver display OLED

- Il modulo riceve comandi e dati tramite una interfaccia seriale
 - Occorre fornire sia il clock (sd_clk) sia i dati (sd_in)
 - Il segnale d/c determina se inviare dati o comandi



Generazione dati seriali

- Periodo del clock > 100 ns (frequenza < 10 MHz)
 - Noi la facciamo a 3.125 MHz
- Dati gli 8 bit da mandare, si genera il clock e i dati tramite una macchina a stati (andiamo a 100 MHz)
- (vedi file spi_controller.vhd)
- Protocollo usato per interfacciarsi a diverse periferiche nell'ambito embedded

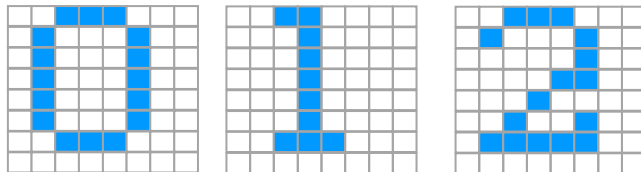


Sequenza di inizializzazione

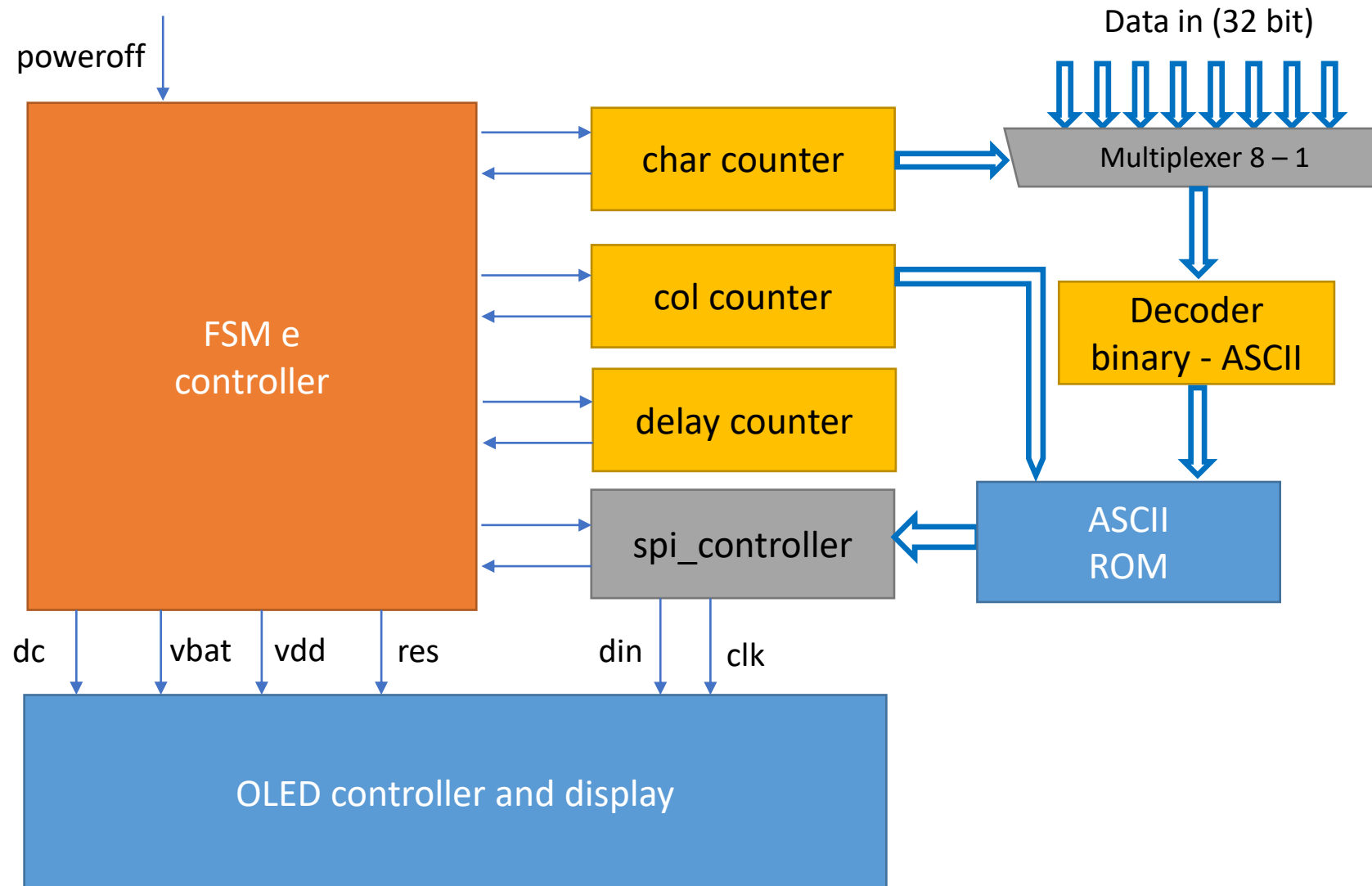
- All'inizio occorre accendere il display con una precisa sequenza
 - V_{DD} on
 - Send display off command
 - Clear screen
 - V_{CC} on
 - Delay 100 ms
- Sequenza realizzata con macchina a stati in modulo driver
 - Usa lo spi_controller per mandare i comandi
 - Usa un contatore per determinare il delay di 100 ms (delay counter)

Scrittura dati

- Si seleziona una riga tramite un apposito comando
- Si mandano valori a 8 bit che indicano per ogni colonna quali pixel devono essere accesi e quali spenti
- Per ogni cifra occorre mandare 8 valori da 8 bit, per fare una matrice 8x8
- I valori per tutti i codici ASCII sono contenuti in una memoria ROM
- Il driver determina l'indirizzo della memoria, legge i valori, e li manda al display tramite lo spi_controller



Schema del driver



Sequenza di power off

- Anche per spegnere occorre seguire una sequenza predefinita
 - Guardare i data sheet sul sito
- La sequenza viene eseguita quando il segnale poweroff viene attivato
- La scrittura dei dati richiede tempo
 - A 3.125 MHz, ogni bit impiega 0.32 us
 - Ogni colonna sono 8 bit: $8 * 3.2 \text{ us} = 2.56 \text{ us}$
 - Ogni cifra sono 8 colonne: $8 * 2.56 \text{ us} = 20.48 \text{ us}$
 - Sono 16 cifre per riga: $16 * 20.48 \text{ us} = 327.68 \text{ us}$
 - Durante questo tempo la FSM NON guarda il poweroff
 - Il poweroff deve quindi rimanere attivo per un po' (ma se lo attacchiamo a un bottone siamo a posto)

Progettare un misuratore di
lunghezza di impulsi

Misuratore di impulsi

Specifica

Realizzare un dispositivo che indichi la durata degli impulsi a 1 di un ingresso L

Come rappresentiamo il dato?

- Numero binario
- Numero in BCD
- ...

In ogni caso ci vorranno diversi bit in uscita



Come misuriamo la durata?

- Numero di cicli di clock
- Tempo in secondi

Raffinamento specifica

Come sappiamo quando il dato è corretto?

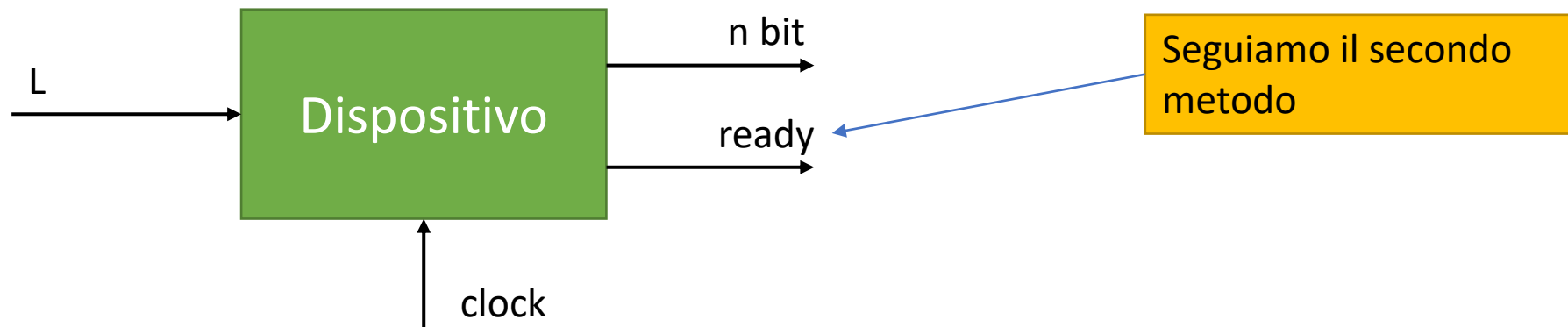
- Presumibilmente l'uscita si incrementa man mano...

Soluzioni

1. Tenere l'uscita al valore 0, tranne quando vogliamo mostrare un risultato
2. Far aggiornare il risultato, ma prevedere una seconda uscita che si attivi solo quando il risultato è valido

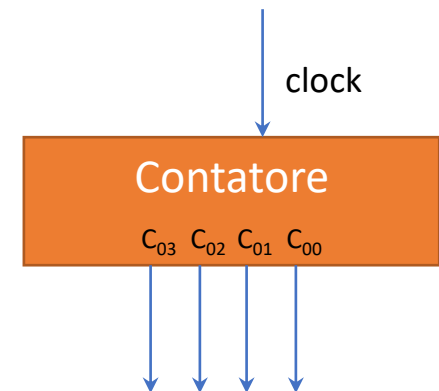
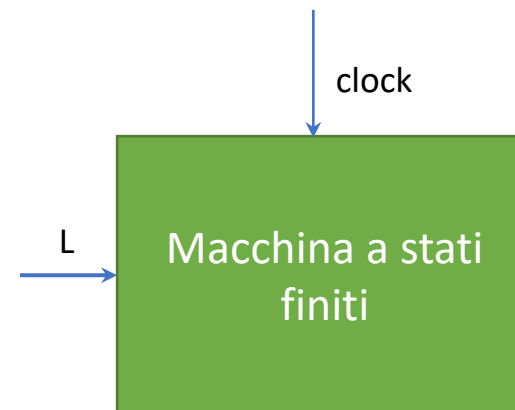
Quando è valido il risultato?

- Per esempio quando l'ingresso L torna da 1 a 0



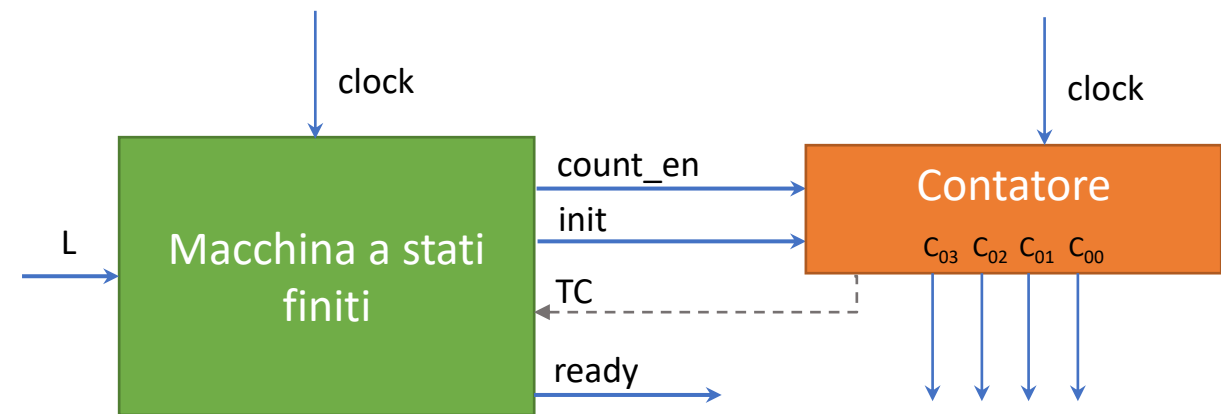
Struttura

- **Singola macchina a stati**
 - In pratica si costruisce un contatore
 - Non conviene
 - Per esempio non sappiamo neanche come mettere il fondo scala
- **Contatore + macchina a stati**
 - Contatore nel data path
 - Macchina a stati di controllo
 - Il fondo scala si può decidere in un secondo momento, per il momento mettiamo 4 bit

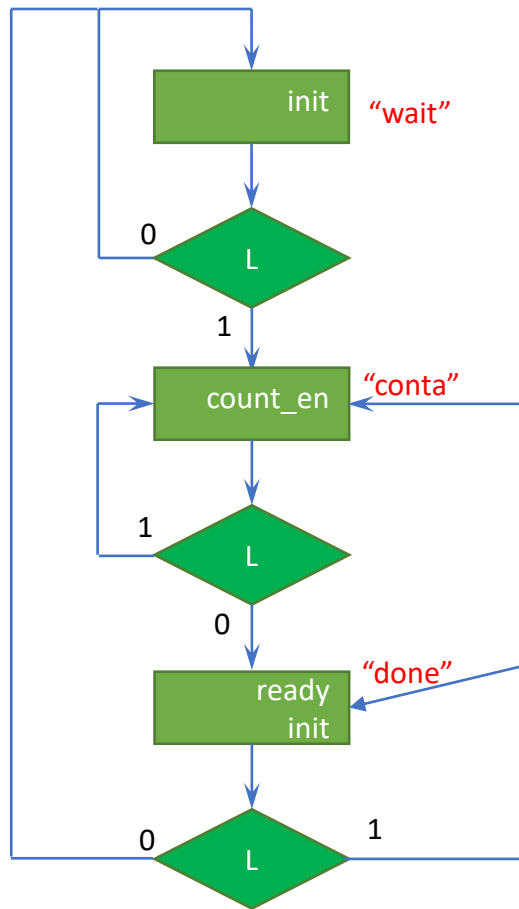


Segnali di controllo e stato

- **Vogliamo poter abilitare il contatore al conteggio**
 - Usiamo un segnale di count enable
- **Finito un conteggio ne vogliamo iniziare un altro (se L torna a 1)**
 - Ci vuole un segnale di inizializzazione (init)
- **Occorre indicare la condizione di ready**
 - Compito della macchina a stati
- **Verifichiamo l'overflow**
 - Segnale di terminal count
 - Per questo esercizio lo ignoriamo



Sviluppo macchina a stati



Stato wait

- Supponendo che l'ingresso L sia inizialmente a 0, attende che vada a 1
- Contemporaneamente inizializza il contatore

Stato conta (si entra con L = 1)

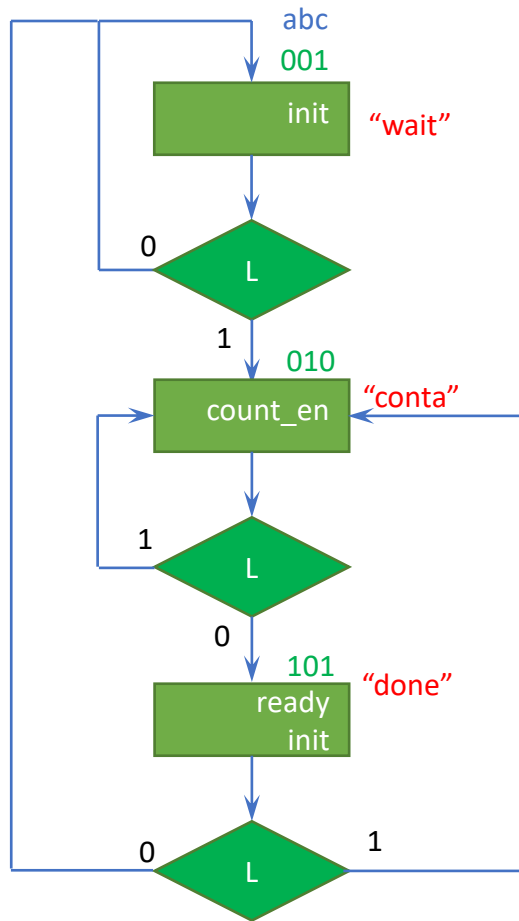
- Abilita il contatore al conteggio
- Attende che L torni a 0

- init è sincrono, quindi il contatore va a 0 solo nel ciclo di clock dopo
- In particolare, il contatore ha il valore corretto mentre si è in done e ready è attivo

Stato done (si entra con L = 0)

- Fornisce il segnale di ready
- Ready attivo per un ciclo di clock (scelta progettuale, non c'era nelle specifiche)
- L potrebbe andare subito a 1, quindi
 - Inizializza subito il contatore
 - Verifico il valore di L
 - Se L = 0, torno allo stato wait
 - Se L = 1, vado direttamente allo stato conta

Codifica macchina a stati



Ci sono tre stati

- Possiamo usare semplicemente due variabili di stato
- Oppure seguire una codifica alternativa

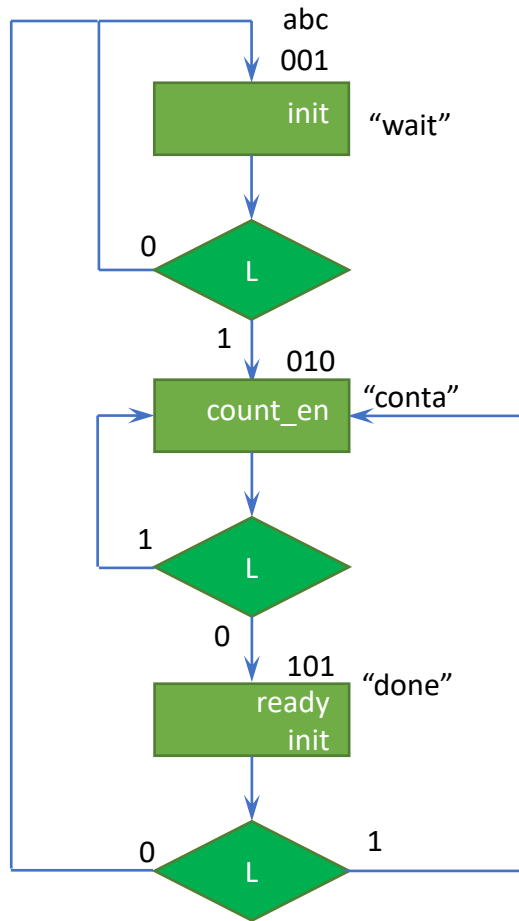
Codifica one-hot

- Possibile, occorrono tre variabili di stato

Variante sul one-hot

- Abbiamo anche tre uscite
 - Codifichiamo lo stato in modo che ogni variabile di stato corrisponda ad una uscita
 - Possibile perché non ci sono stati con la medesima combinazione delle uscite
-
- $ready = a$
 - $count_en = b$
 - $init = c$

Mappe di Karnaugh



$a \setminus bc$	00	01	11	10
0	-	wait	-	conta
1	-	done	-	-

Mappa degli stati

$a \setminus bc$	00	01	11	10
0	-	0	-	L'
1	-	0	-	-

$$a_{+1} = L'b$$

$a \setminus bc$	00	01	11	10
0	-	L	-	L
1	-	L	-	-

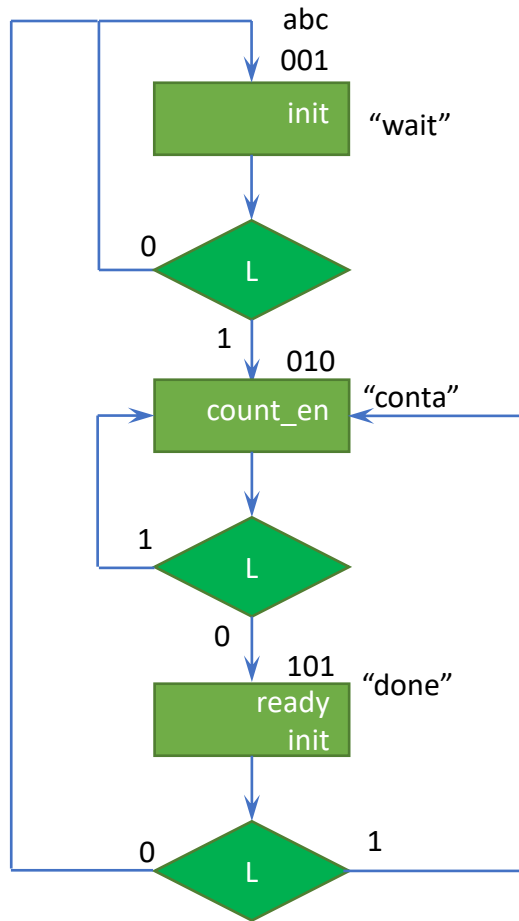
$$b_{+1} = L$$

$a \setminus bc$	00	01	11	10
0	-	L'	-	L'
1	-	L'	-	-

$$c_{+1} = L'$$

$init = c$
 $count_en = b$
 $ready = a$

Circuito



$$\begin{aligned} a_{+1} &= L'b \\ b_{+1} &= L \\ c_{+1} &= L' \end{aligned}$$

$$\begin{aligned} init &= c \\ count_en &= b \\ ready &= a \end{aligned}$$

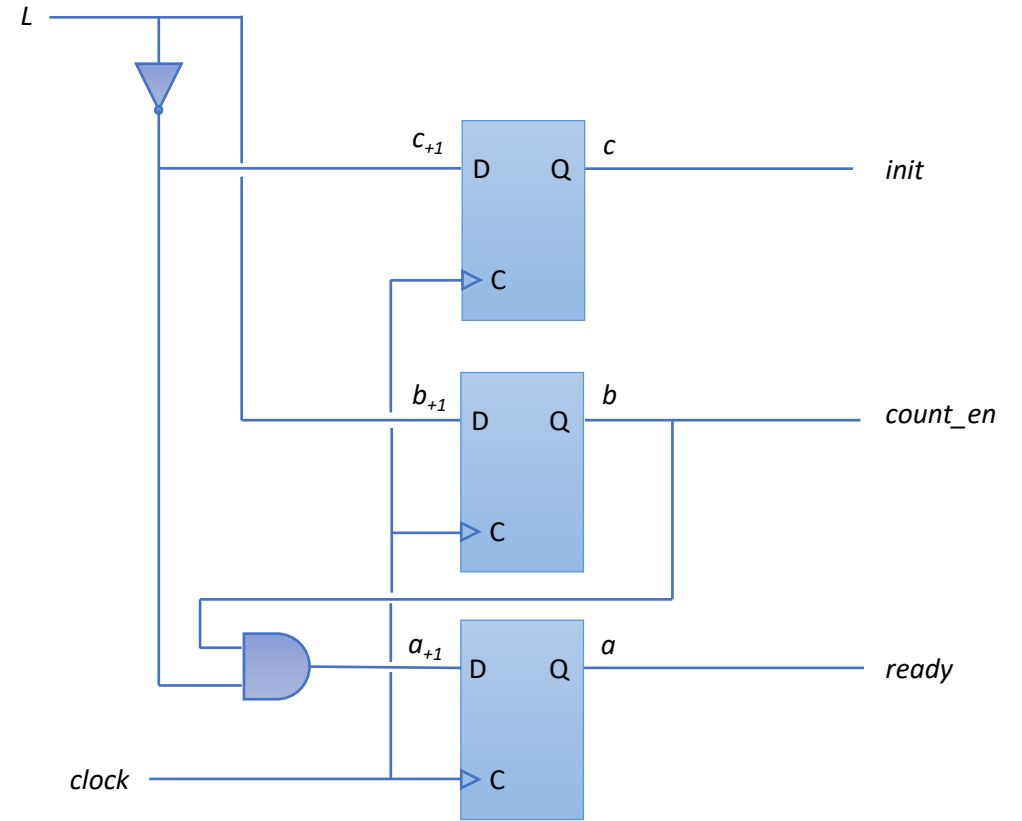
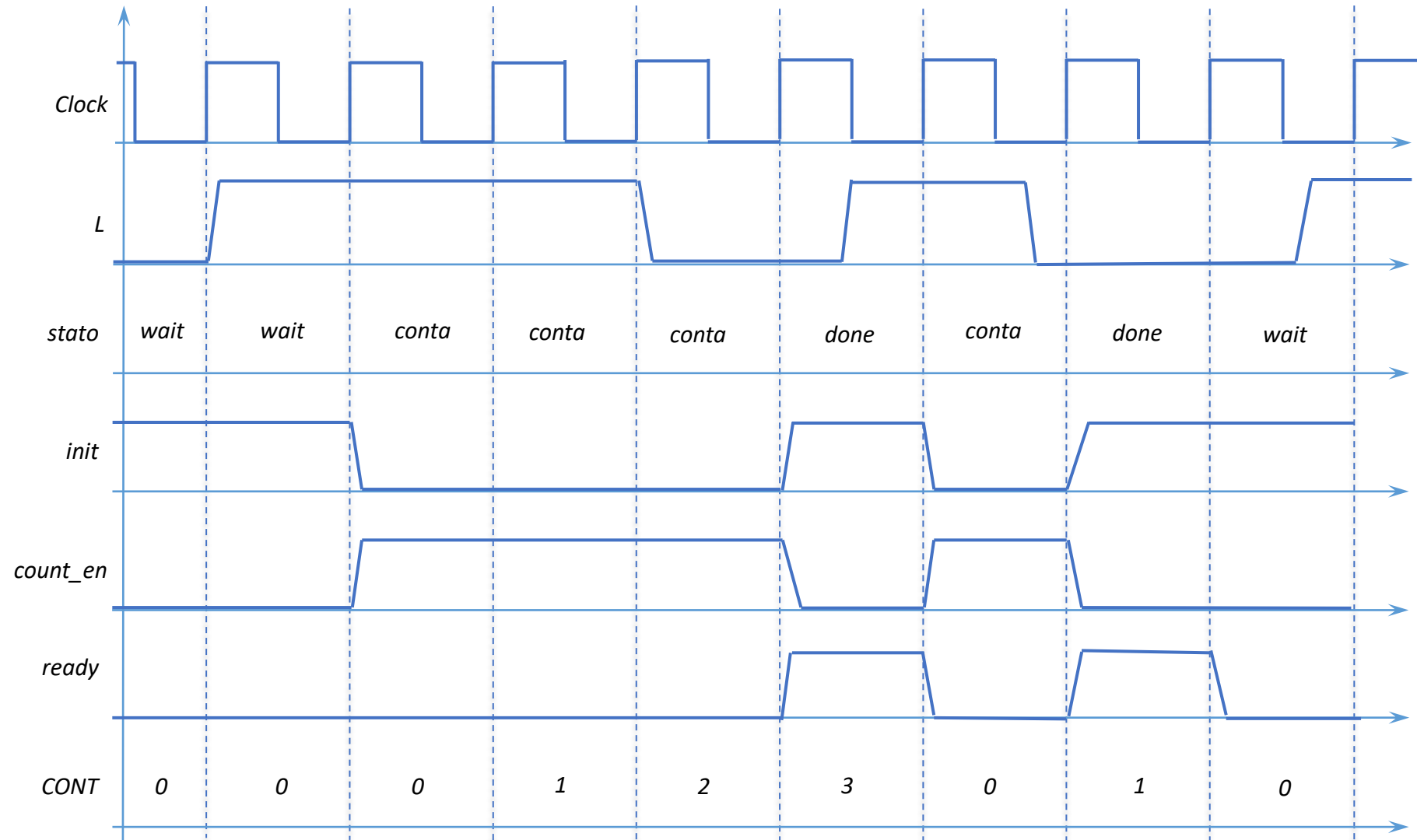
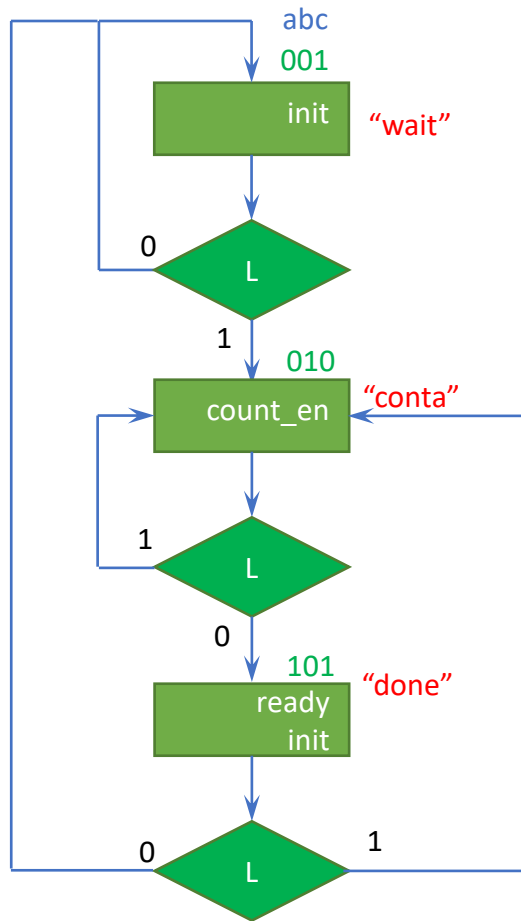


Diagramma temporale



Verifica sequenze

Sembra facile...

Ready, set, go!



Riattivazione

- Terminata una sequenza, dopo lo *stop*, il dispositivo deve essere pronto a gestire una nuova sequenza

Tre ingressi collegati a pulsanti

- Vogliamo verificare la corretta sequenza di attivazione dei pulsanti
- Informazione fornita sulle due uscite *active* ed *error*

Active

- Attivare *active* se si riceve una sequenza in cui prima *ready* viene premuto e quindi rilasciato, ed infine *go* viene premuto
- L'uscita *active* deve attivarsi **subito**, non appena *go* viene premuto, **senza aspettare il fronte di clock successivo**
- L'uscita *active* deve essere disattivata quando viene premuto *stop*, indipendentemente dallo stato di *go*

Error

- L'uscita *error* viene attivata qualora *go* venga premuto **prima** o **mentre** è premuto *ready*
- Anche *error* deve essere disattivato quando viene premuto *stop*

Diagramma a stati

Stato a

- Attende che si prema *ready*
- Ma se premo *go* è chiaramente un errore, vado in **e**
- Se premo *ready* e non *go* si va in **b**

Stato b

- Attende che *ready* sia rilasciato
- Ma se premo *go* prima di rilasciare *ready* è un errore, vado in **e**
- Se rilascio *ready* e non premo *go* si va in **c**

Stato c

- Attende che *go* sia premuto
- Una uscita condizionata attiva *active* immediatamente, senza aspettare il fronte del clock
- Al fronte si va in **d**
- Bisogna guardare *stop*?

Stato d

- Attende che sia premuto *stop*
- Mantiene *active* a 1, poi torna in **a**

Stato e

- Attiva *error*, quando vede *stop* torna in **a**

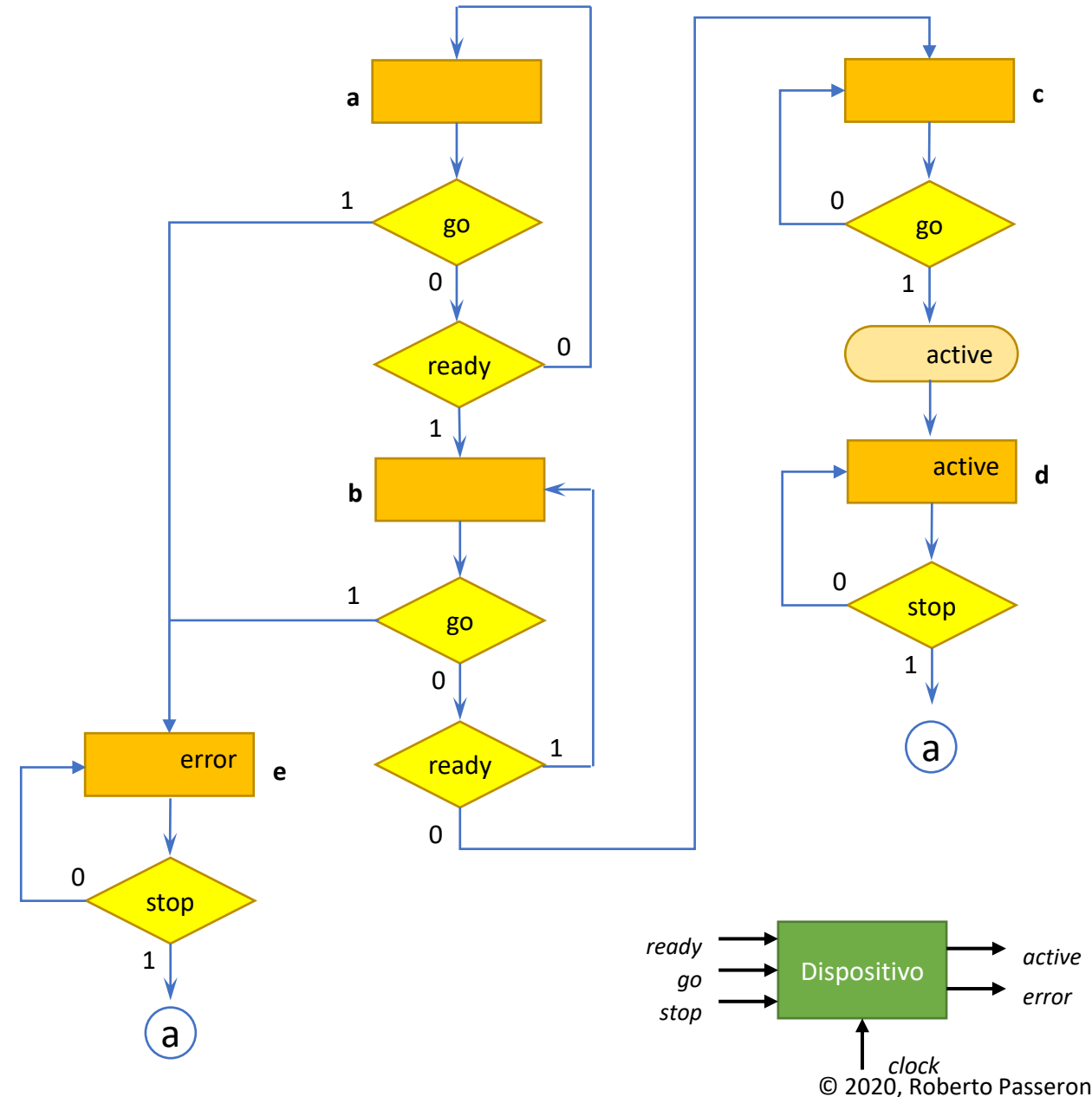


Diagramma temporale

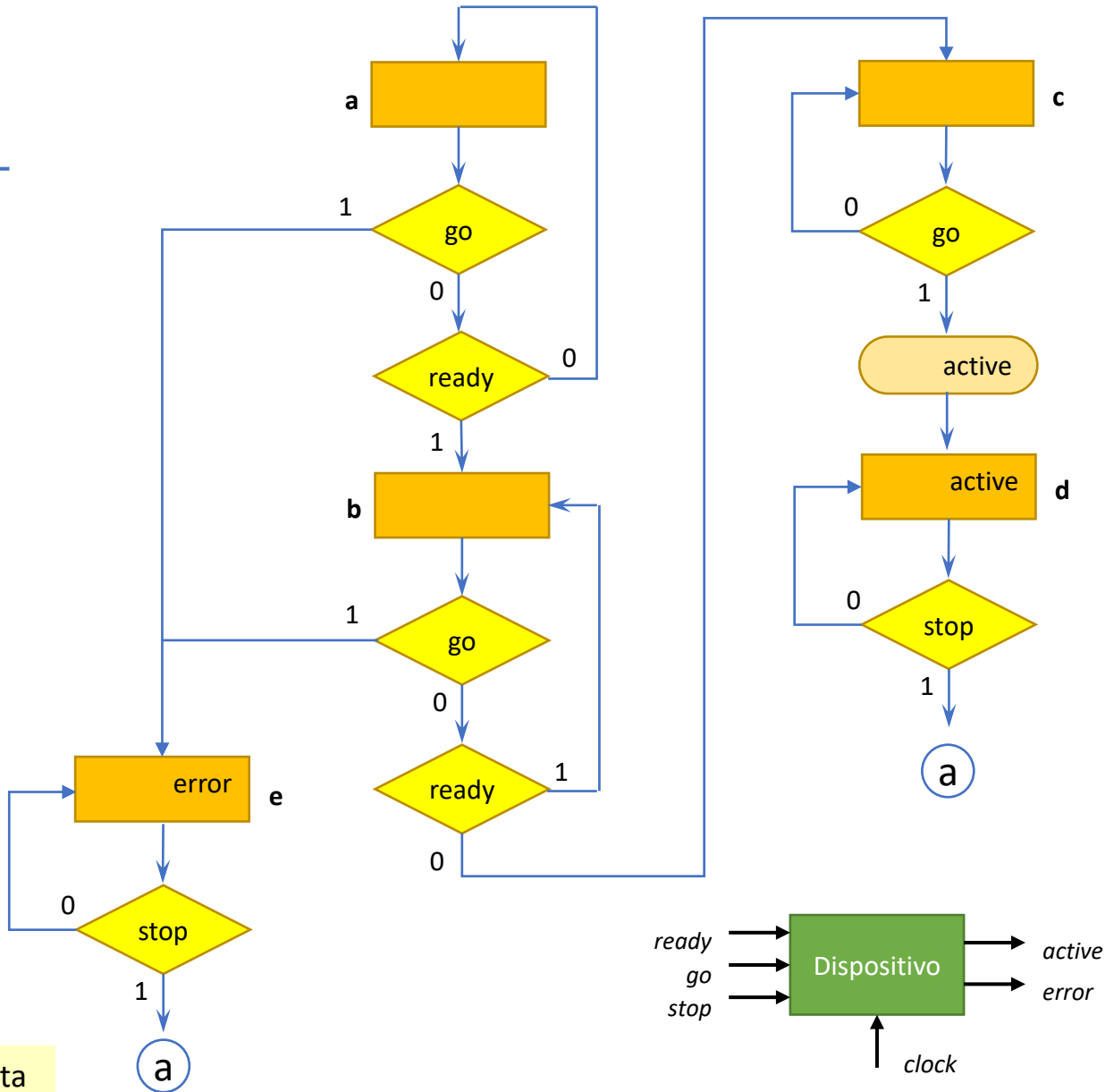
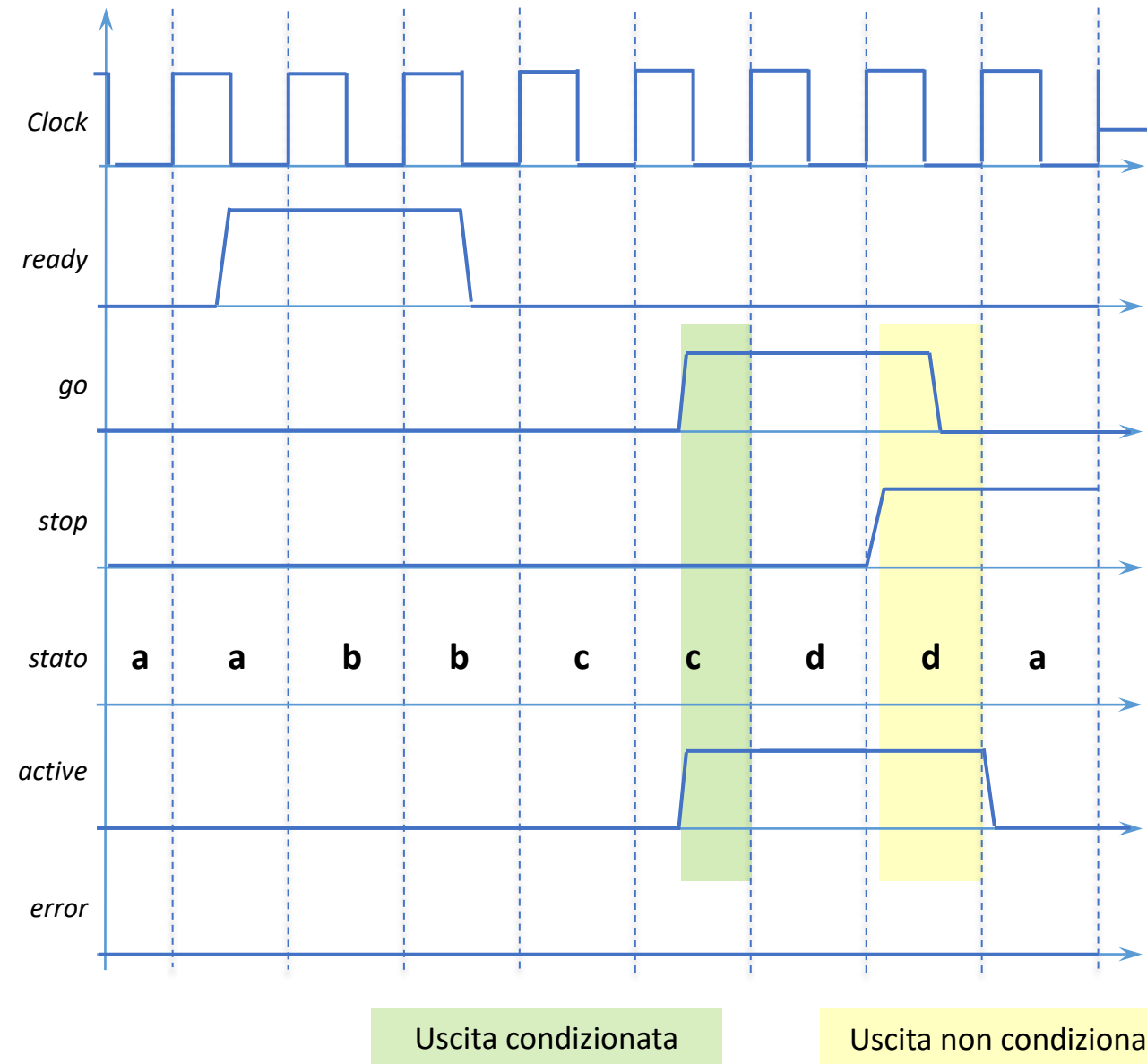
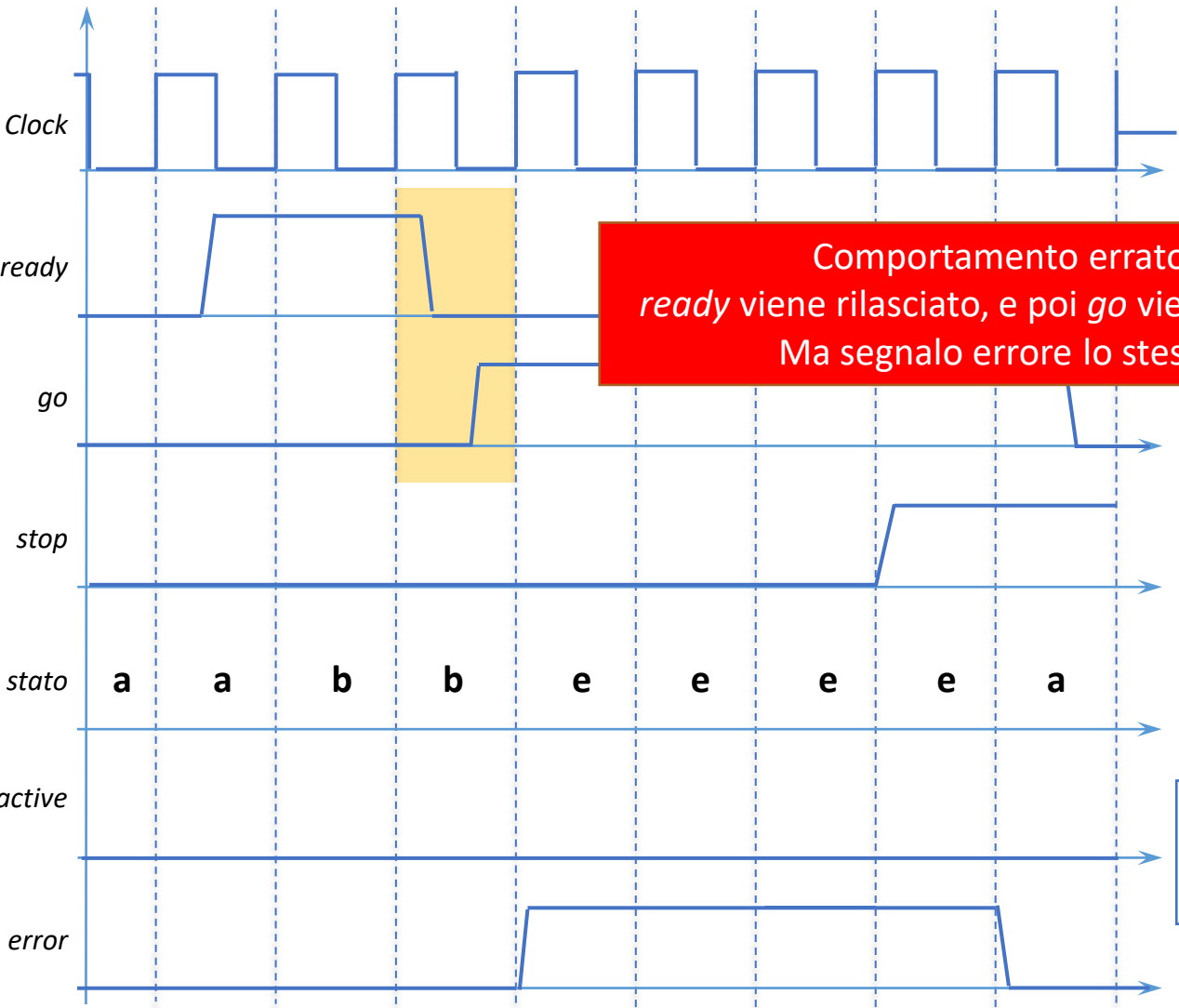


Diagramma temporale



Comportamento errato!
ready viene rilasciato, e poi *go* viene premuto
Ma segnale errore lo stesso!

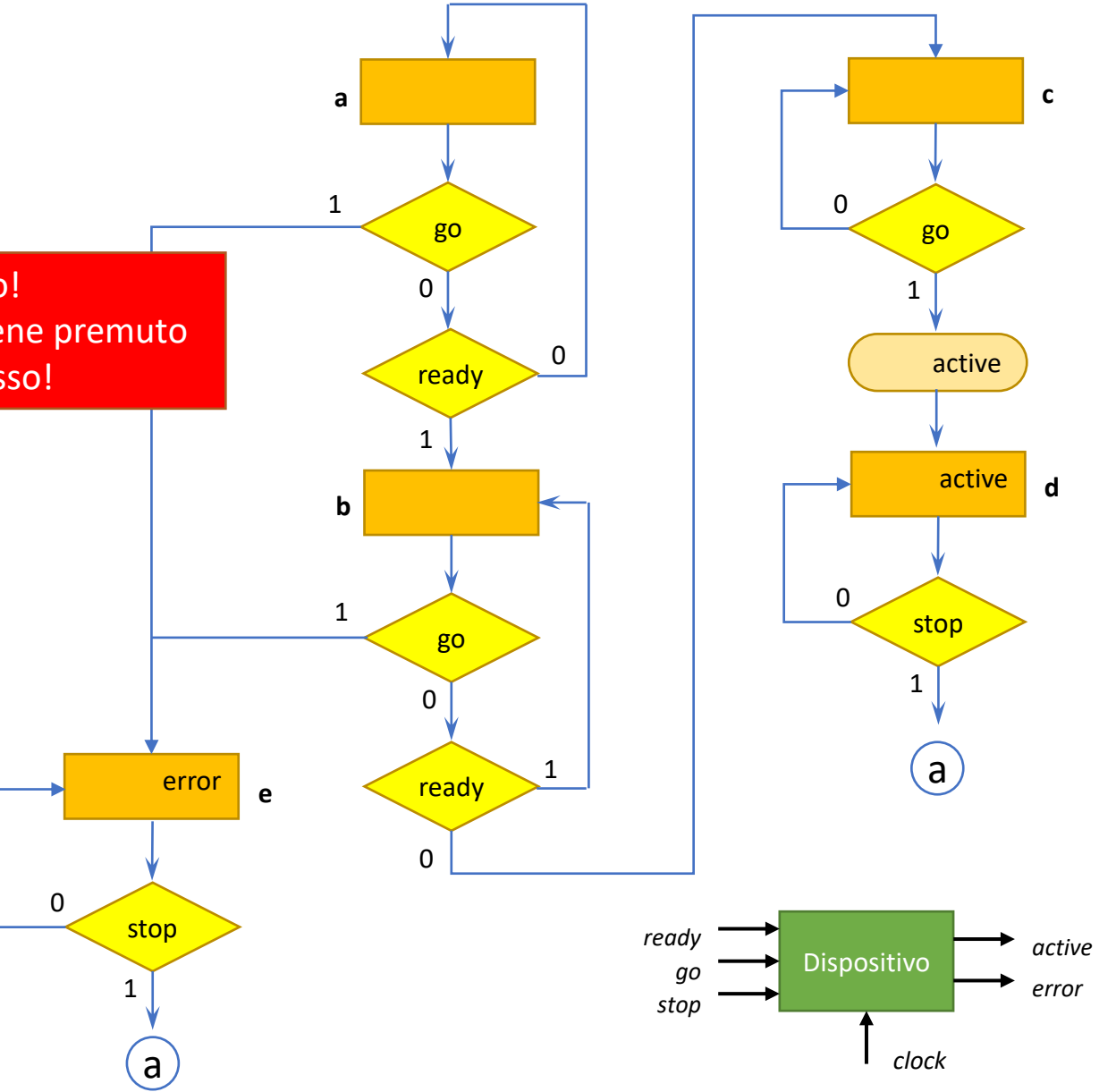


Diagramma temporale

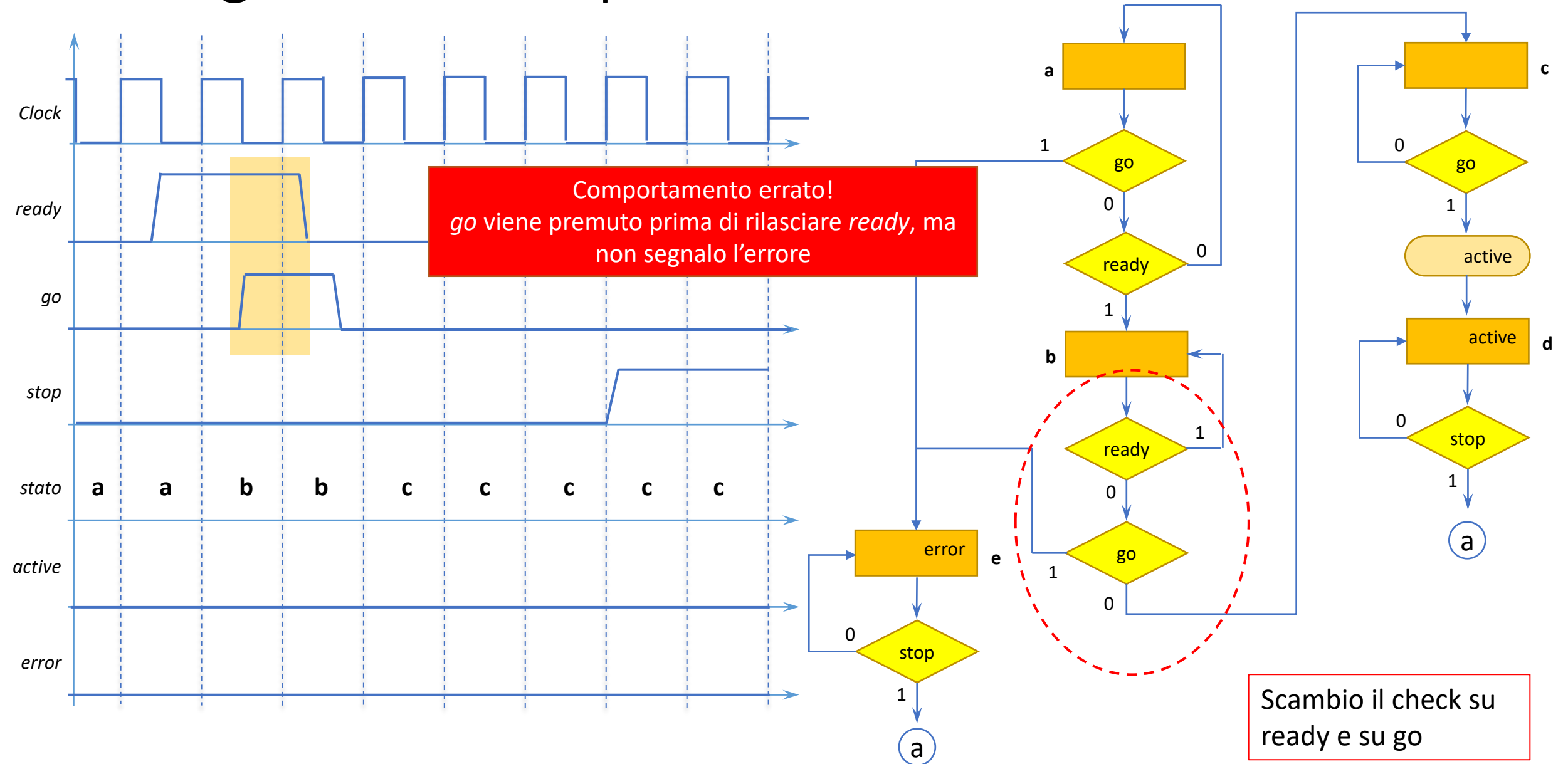


Diagramma temporale

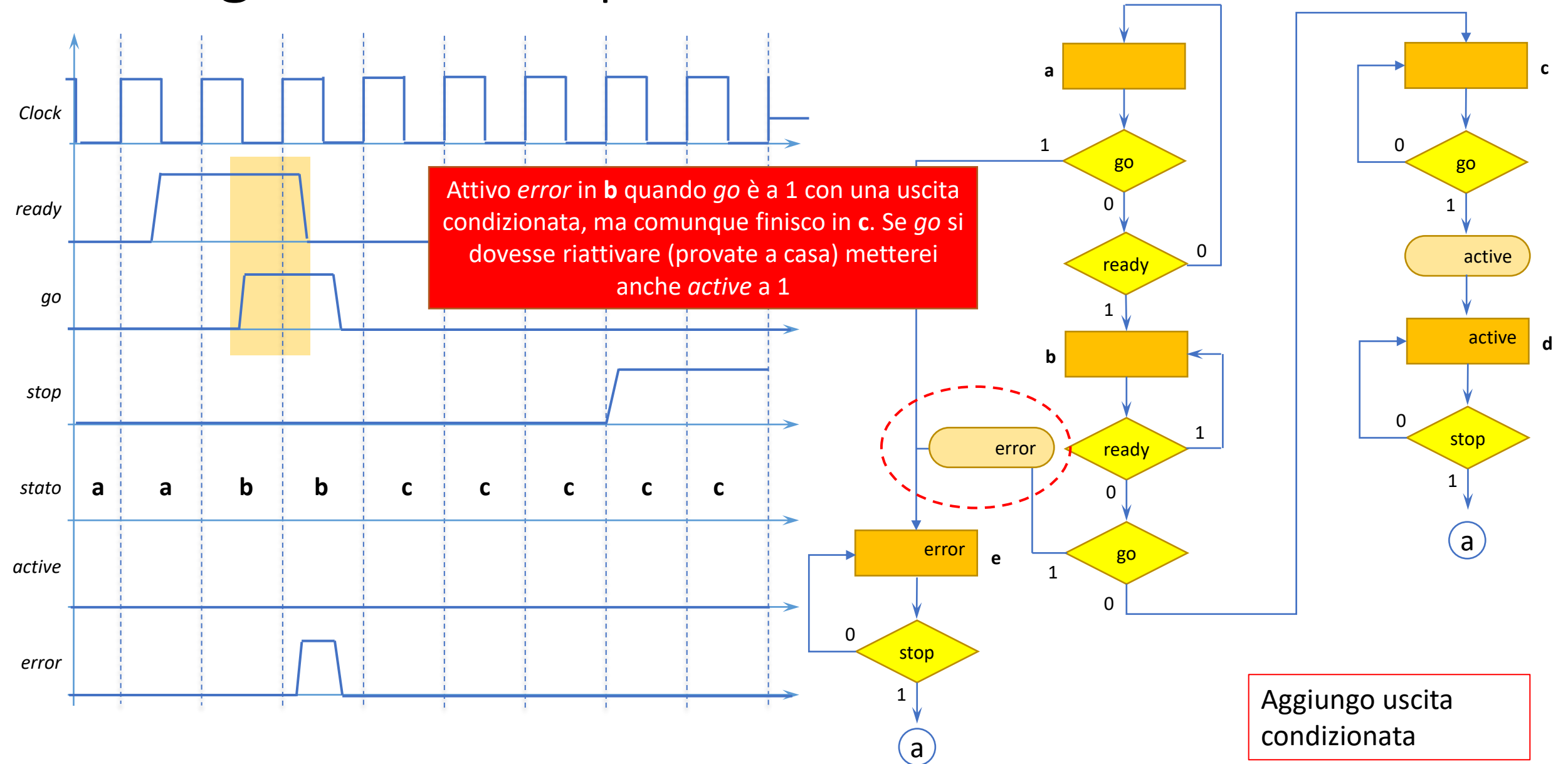
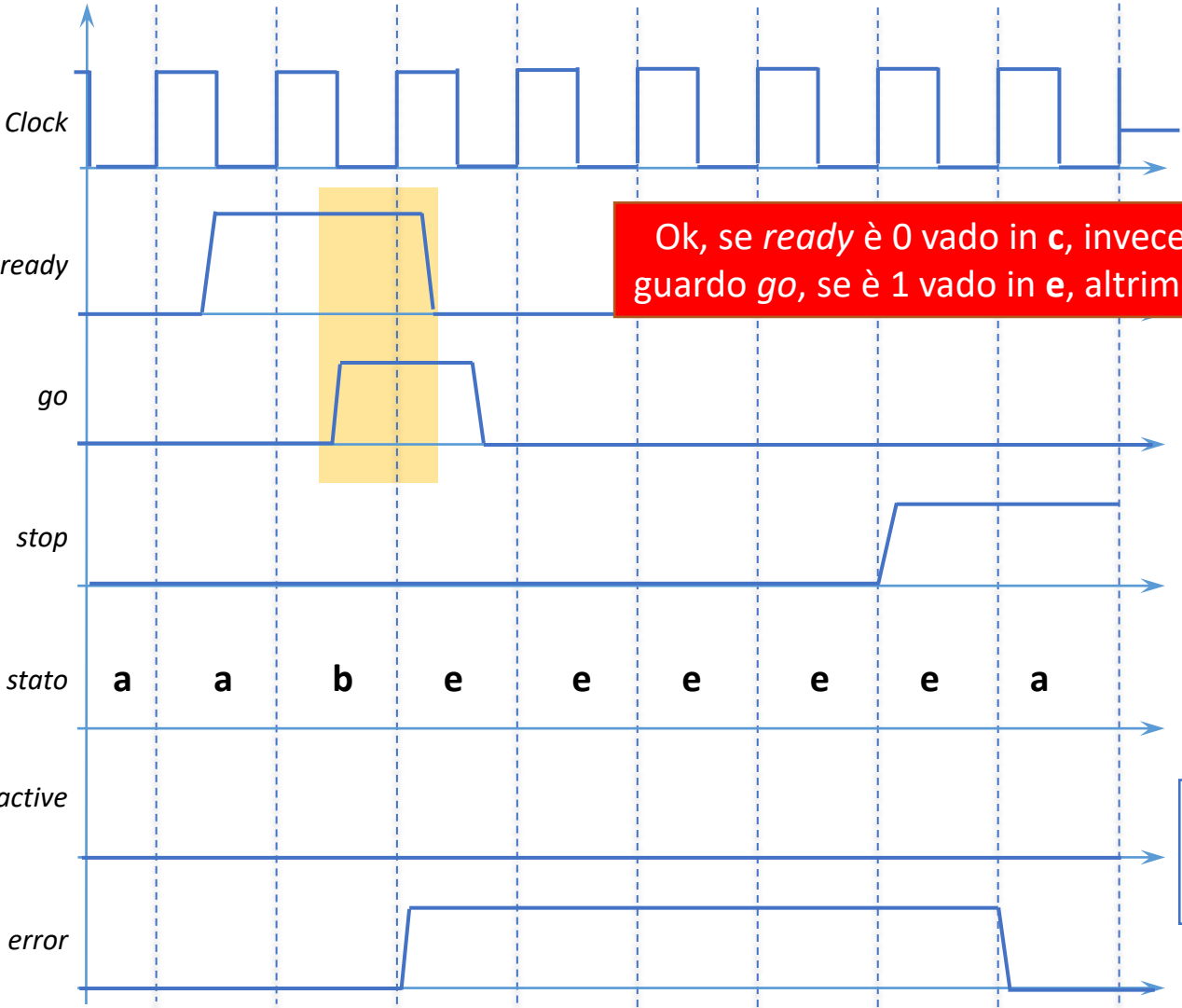
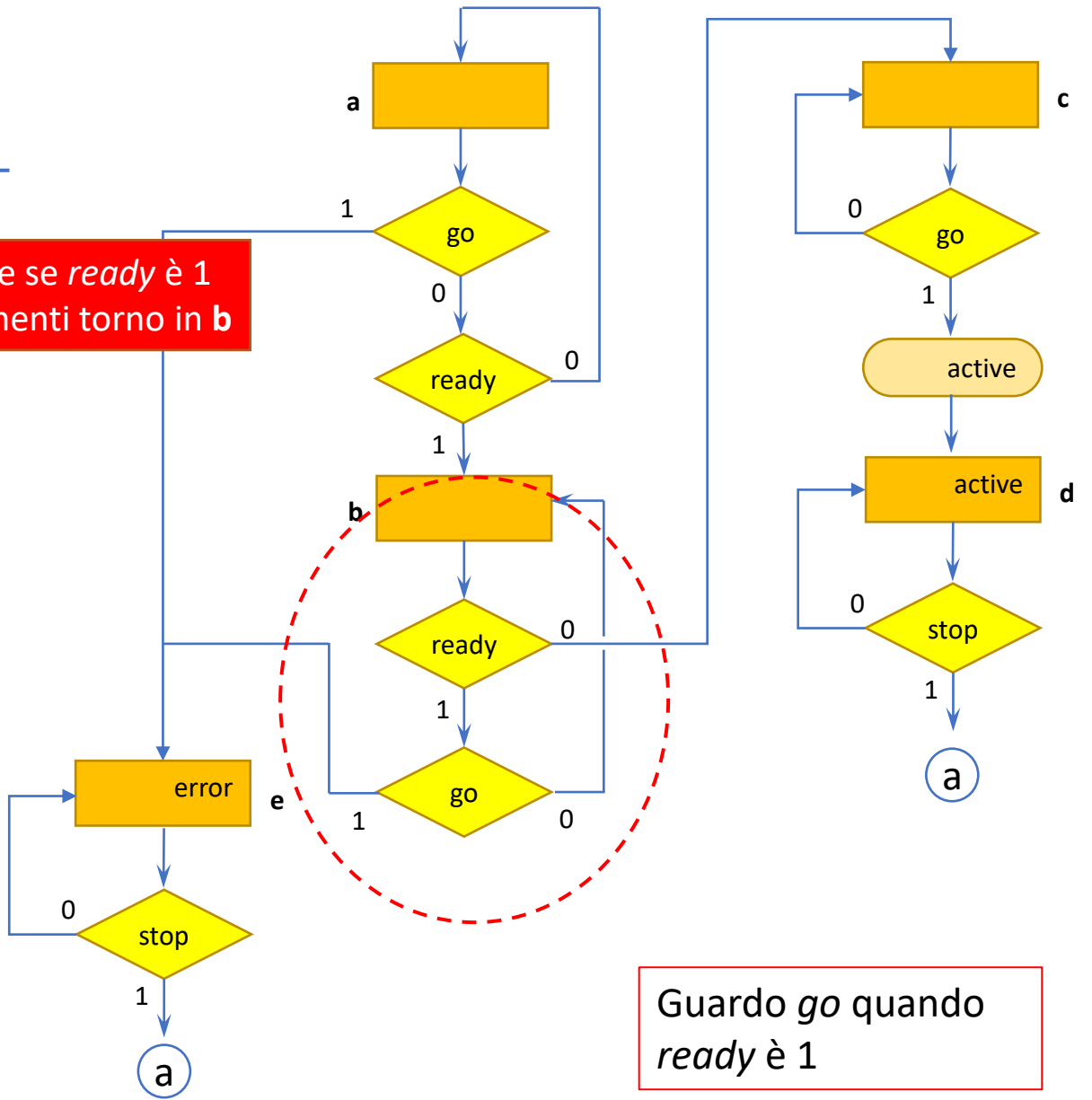


Diagramma temporale



Ok, se *ready* è 0 vado in *c*, invece se *ready* è 1 guardo *go*, se è 1 vado in *e*, altrimenti torno in *b*



Guardo *go* quando *ready* è 1

Diagramma temporale

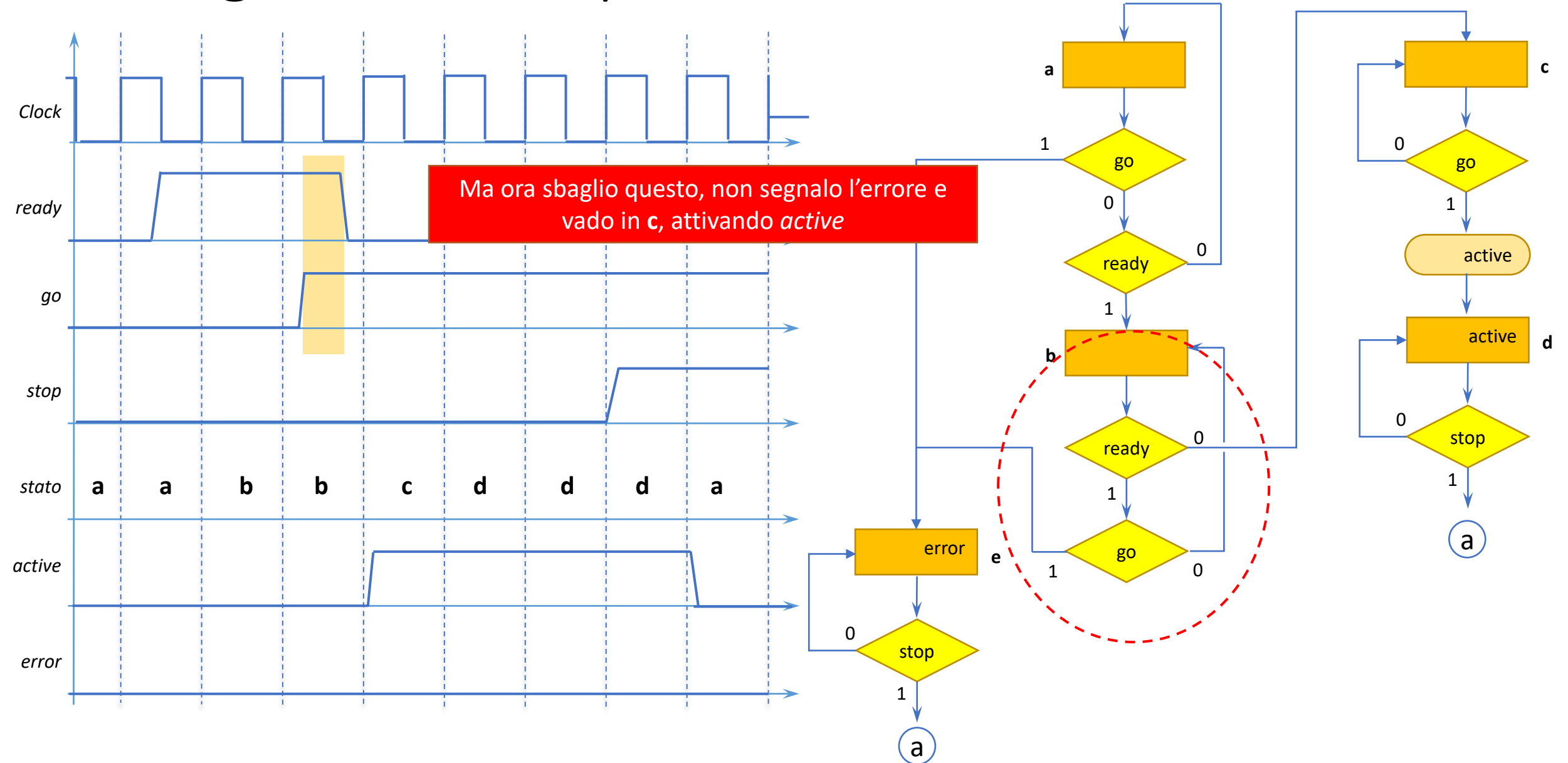


Diagramma temporale

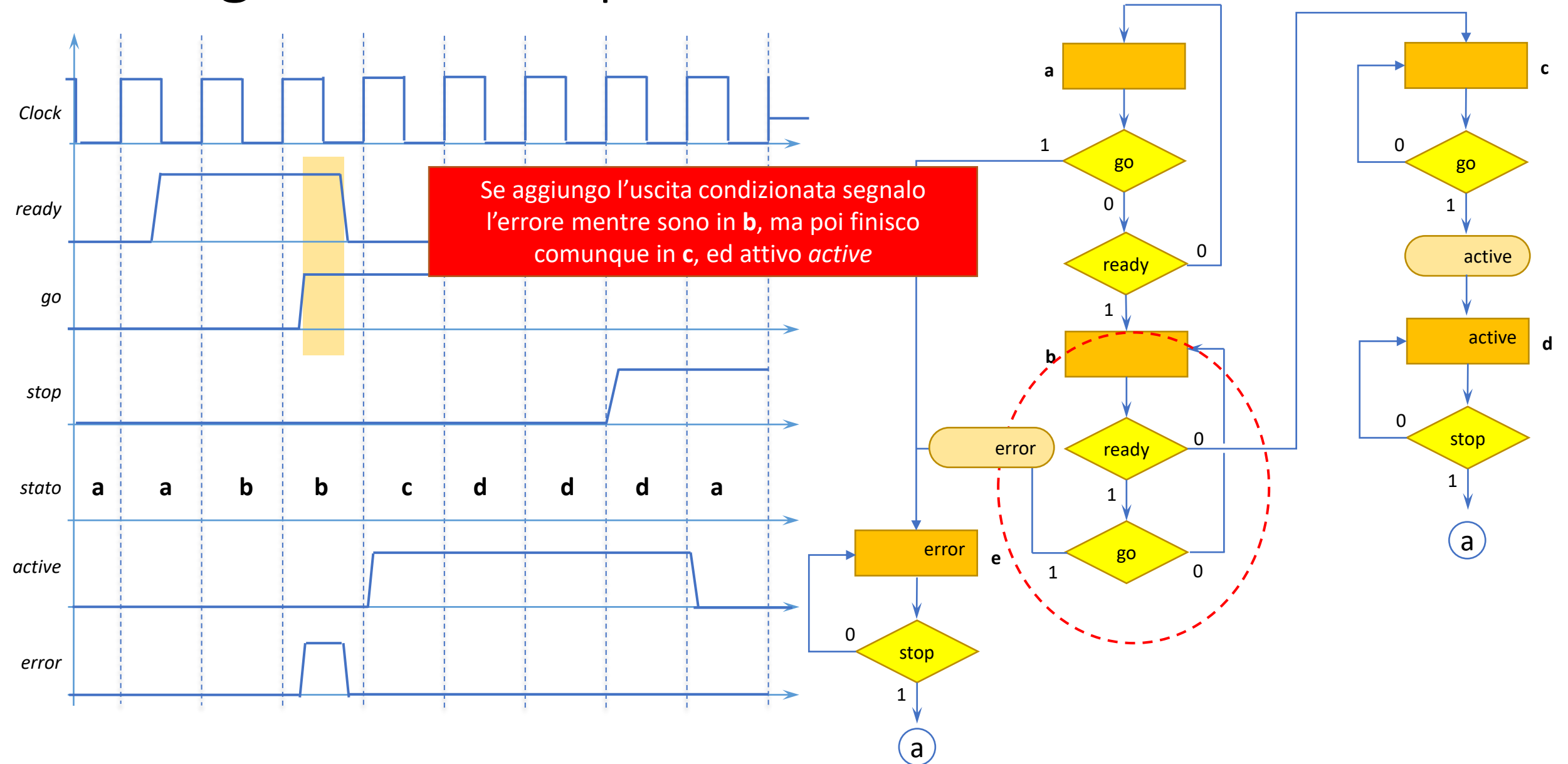
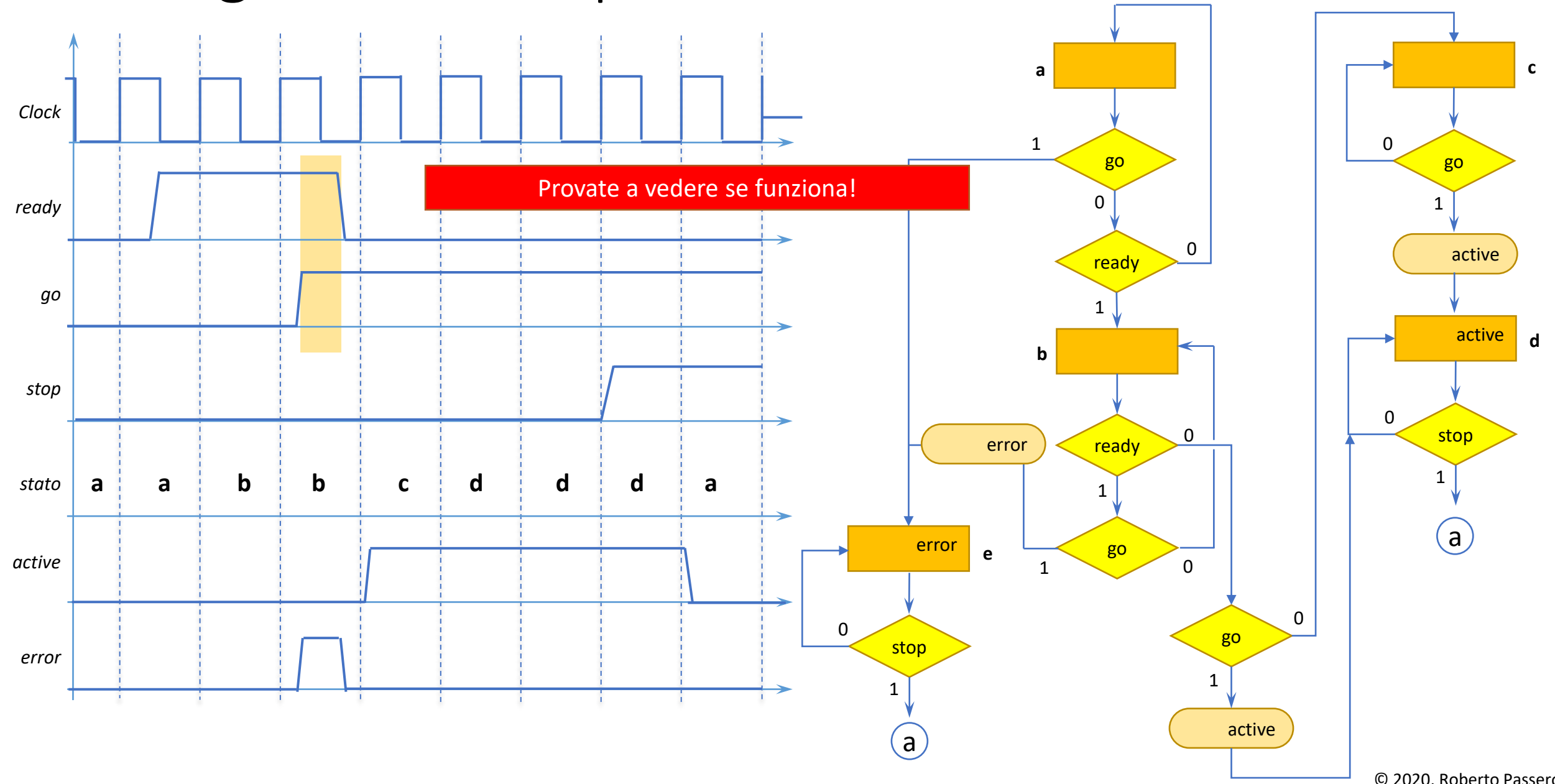


Diagramma temporale



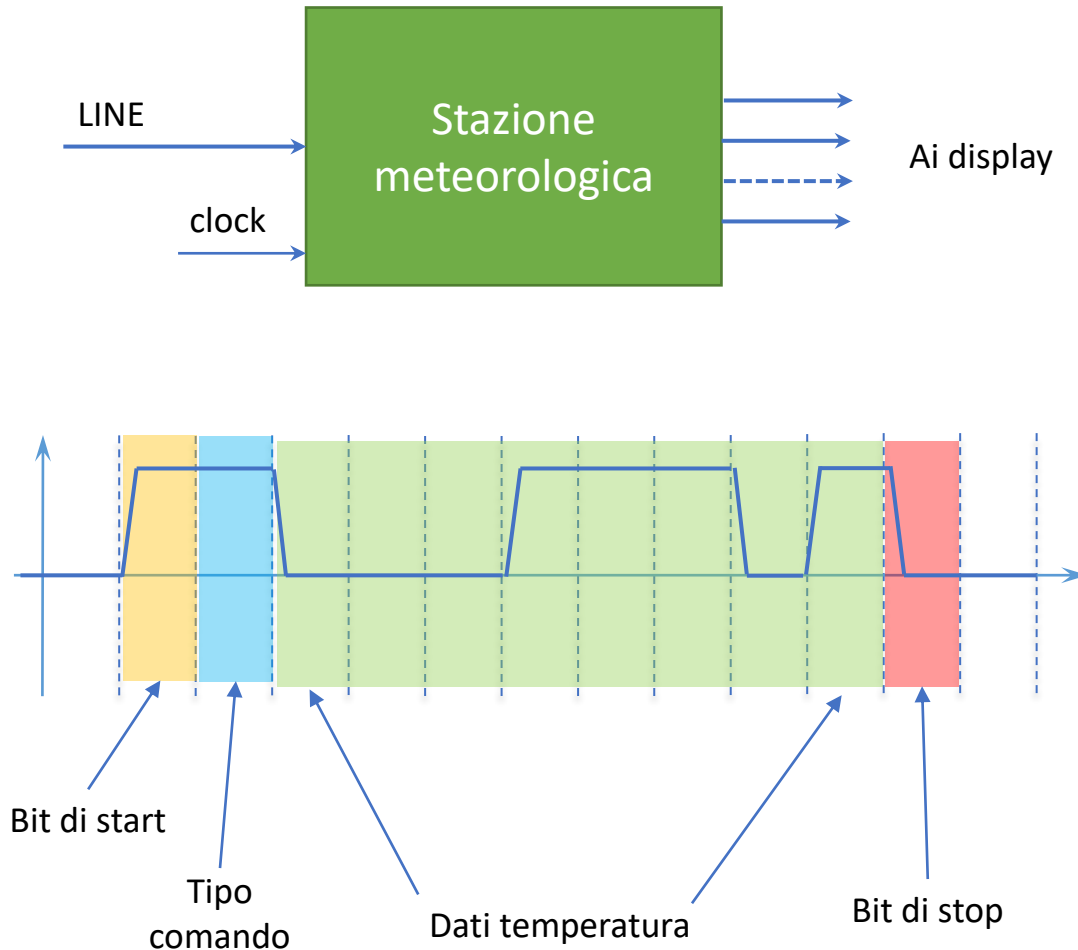
Morale

- Non c'è modo di risolvere il problema con un circuito sincrono
- Il clock deve essere molto più veloce dei segnali di ingresso
- Questi non devono attivarsi o disattivarsi nello stesso ciclo di clock
 - Se ho un clock molto veloce, riesco ad approssimare bene questa situazione
- In alternativa, si può fare un circuito **asincrono**
 - I flip flop SR commutano di stato appena commuta l'ingresso
 - Risoluzione temporale idealmente infinita, di fatto limitata dai ritardi delle porte logiche

Stazione meteorologica

Tema d'esame del gennaio 2010

Specifica



La stazione riceve dati di temperatura su una linea seriale LINE

- Temperatura codificata su 8 bit
- La stazione deve mostrare il dato su un display a 7 segmenti
- La stazione deve tenere traccia della temperatura minima

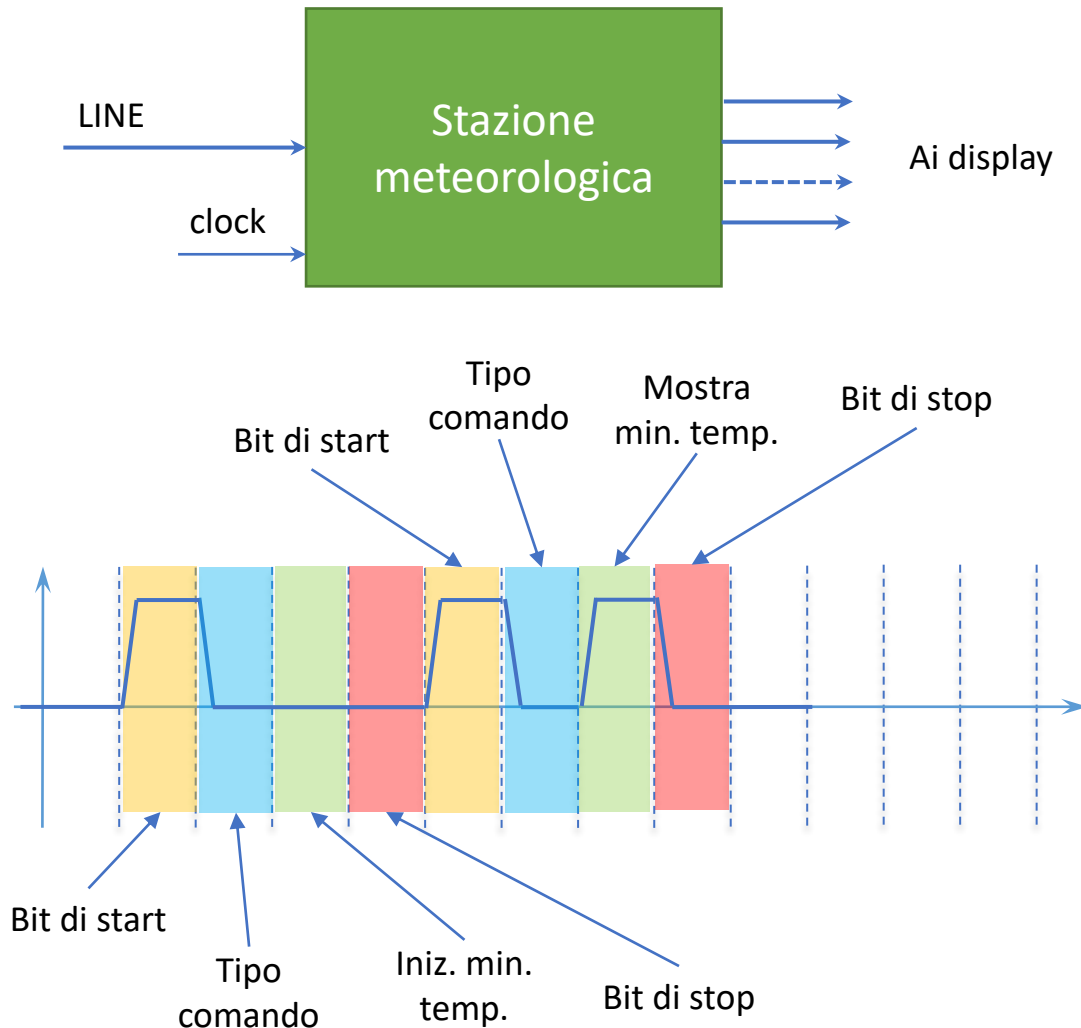
Stazione controllata dalla linea seriale

- Sulla linea vengono inviati dei comandi, tra cui la temperatura
- LINE è normalmente a 0.
- Ogni bit del comando dura un ciclo di clock
- I comandi sono individuati da un **bit di start a 1**, seguito dai dati, e da un **bit di stop a 0**

Tre comandi

- Nuovo dato: **1** – 1 – T_0 – T_1 – T_2 – T_3 – T_4 – T_5 – T_6 – T_7 – **0**
- Inizializza temp. minima: **1** – 0 – 0 – 0
- Mostra temperatura minima: **1** – 0 – 1 – 0
- **La temperature minima deve essere mostrata sul display fino all'arrivo del prossimo dato**

Specifica



La stazione riceve dati di temperatura su una linea seriale LINE

- Temperatura codificata su 8 bit
- La stazione deve mostrare il dato su un display a 7 segmenti
- La stazione deve tenere traccia della temperatura minima

Stazione controllata dalla linea seriale

- Sulla linea vengono inviati dei comandi, tra cui la temperatura
- LINE è normalmente a 0.
- Ogni bit del comando dura un ciclo di clock
- I comandi sono individuati da un **bit di start a 1**, seguito dai dati, e da un **bit di stop a 0**

Tre comandi

- Nuovo dato: **1** – 1 – $T_0 - T_1 - T_2 - T_3 - T_4 - T_5 - T_6 - T_7$ – **0**
- Inizializza temp. minima: **1** – 0 – 0 – **0**
- Mostra temperatura minima: **1** – 0 – 1 – **0**
- **La temperature minima deve essere mostrata sul display fino all'arrivo del prossimo dato**

Progetto

- Progettare un **data path** ed una **macchina a stati** che lo controlli
 - Prevedere appositi registri per memorizzare i dati e per interfacciarsi alla linea seriale (e.g., un registro a scorrimento)
 - Provate a evitare di usare un contatore per contare i bit in ingresso del dato
 - Durante la ricezione di un dato il display deve continuare a mostrare il dato precedente (o la temperatura minima)
 - Fare la macchina a stati che interpreti i comandi e piloti il data path in modo appropriato
- Sviluppare un diagramma temporale per verificare il funzionamento
 - Verificare in particolare che il sistema non perda comandi se arrivano uno dietro l'altro
- Supponendo di voler misurare temperature da -28 a 99.5 gradi celsius con precisione di 0.5 gradi
 - Inventarsi una codifica per i dati

Registro a scorrimento Q

- Ingresso seriale: *serial_in*
- Abilitazione allo shift: *sh_enable*
- Init sincrono: *azzera*
- Uscita Q_0 visibile alla MSF

Registro T

- Memorizza ultimo dato visto
- Load enable: *ld_enable*

Registro M

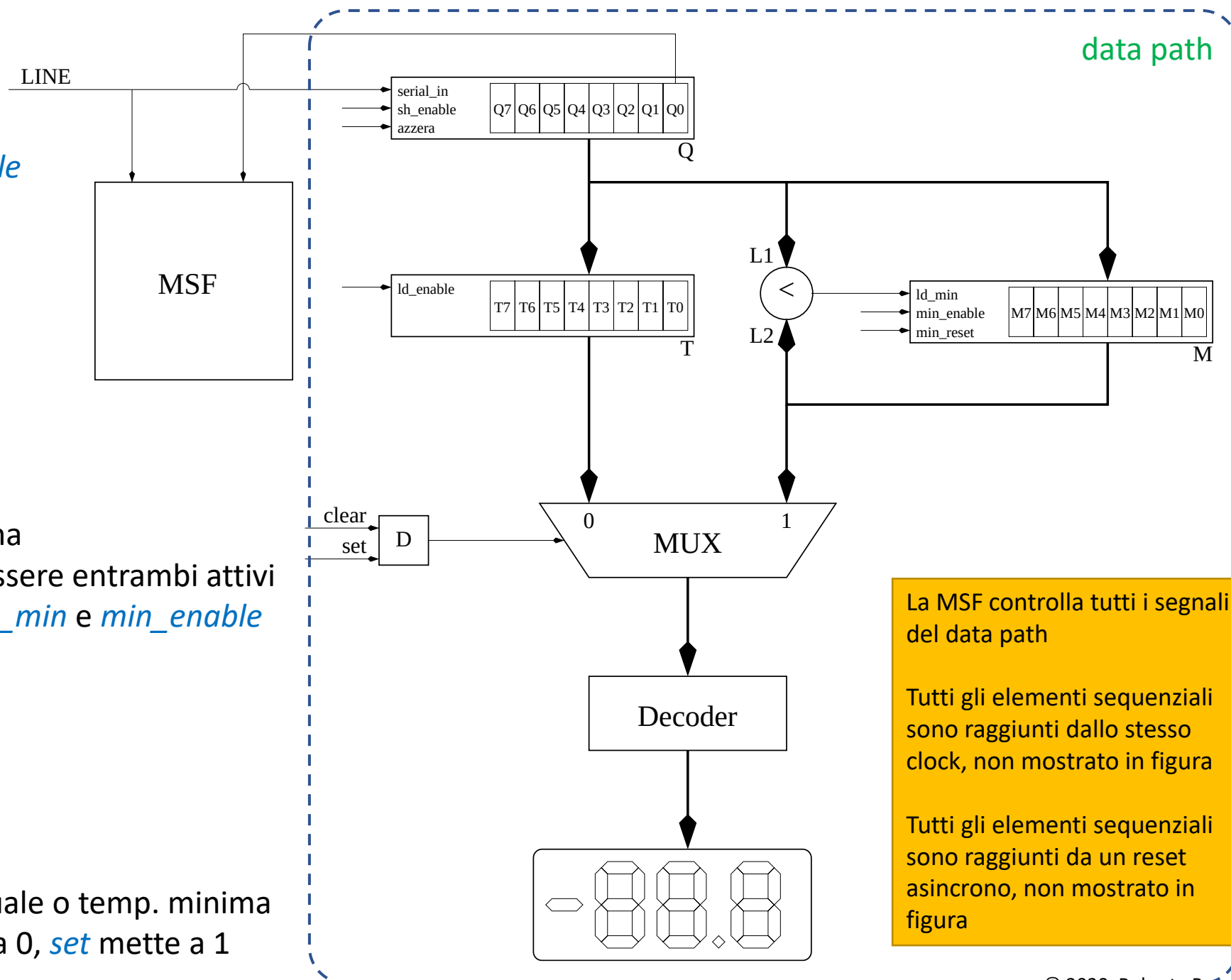
- Memorizza temperatura minima
- Doppio load enable (devono essere entrambi attivi per abilitare il caricamento): *ld_min* e *min_enable*
- Init sincrono: *min_reset*

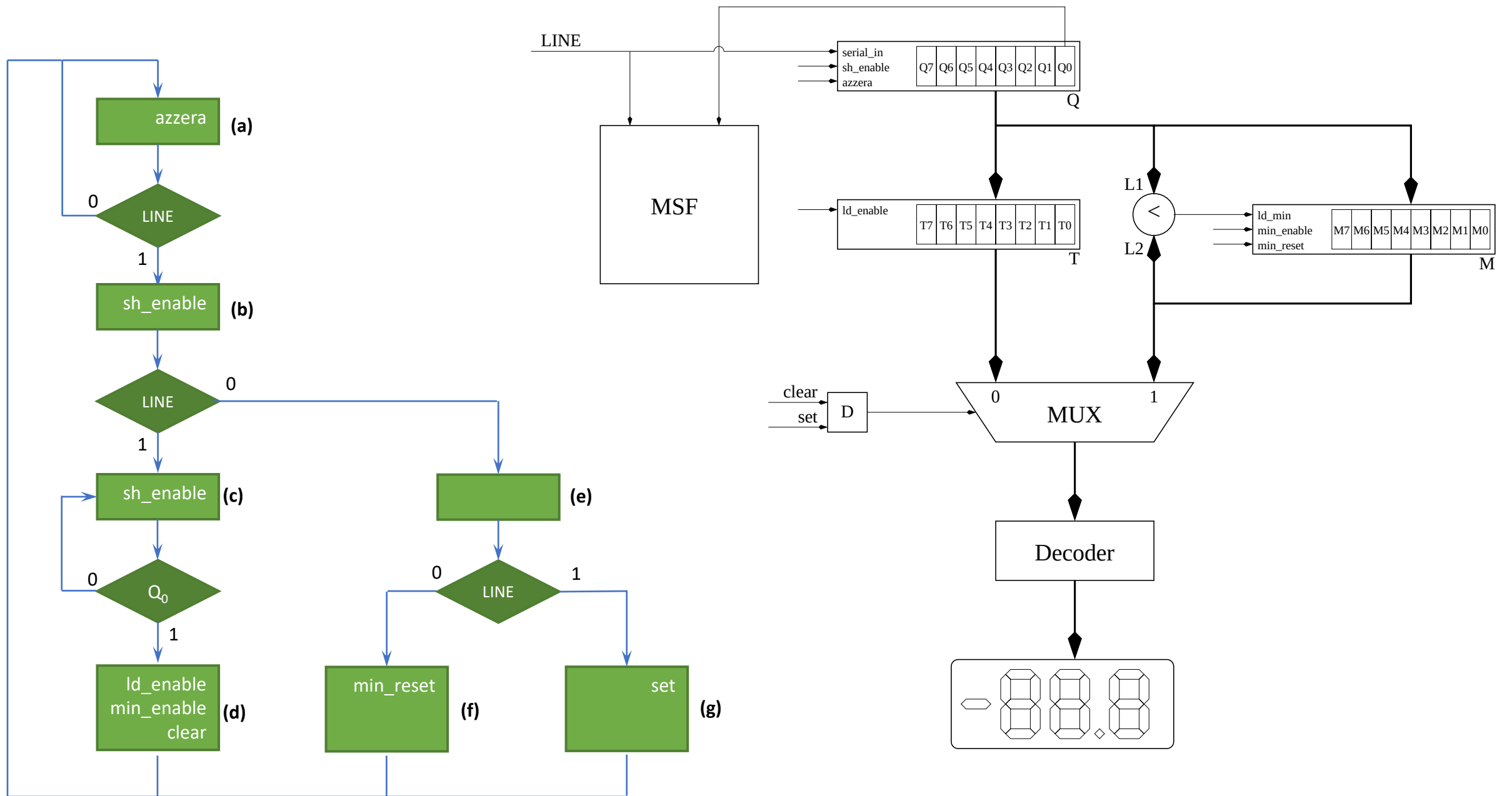
Comparatore

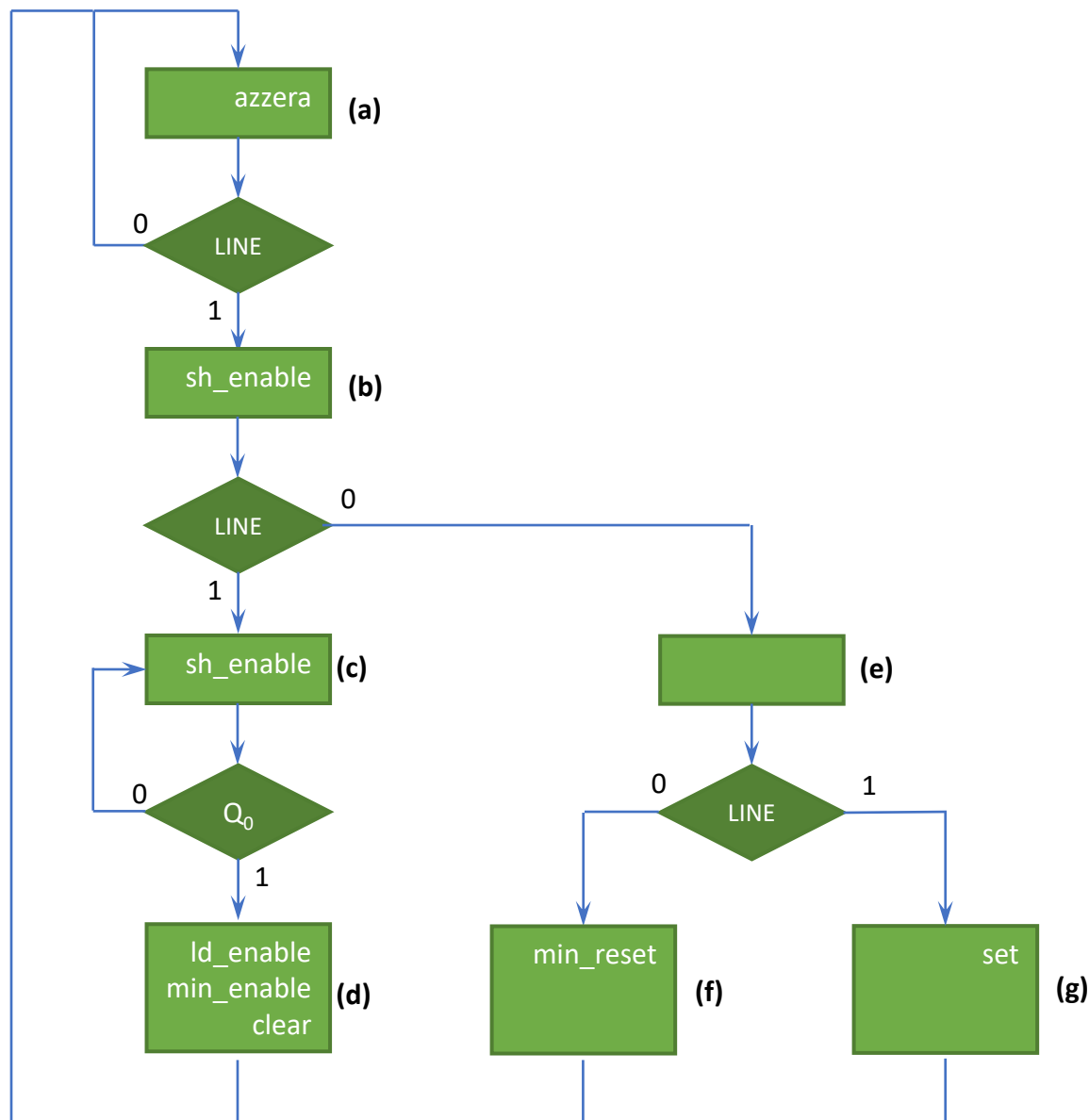
- Uscita attiva se $L1 < L2$

Flip flop D

- Ricorda se mostrare temp. attuale o temp. minima
- Controlli sincroni: *clear* mette a 0, *set* mette a 1







Stato a

- Aspetta un comando (cioè che LINE vada a 1)
- Inizializza Q (*azzera*), così che contenga tutti 0, in particolare azzera Q_0

Stato b

- Abilita lo scorrimento (nota: il bit di start non entra in Q, ma entra il bit dopo)
- Se LINE = 1 è un nuovo dato, va in c
 - Entra 1 in Q, quando arriva a Q_0 segnala la fine
- Se LINE = 0 è un comando diverso, va in e

Stato c

- Continua a shiftare (*sh_enable*) fino a che Q_0 diventa 1

Stato d

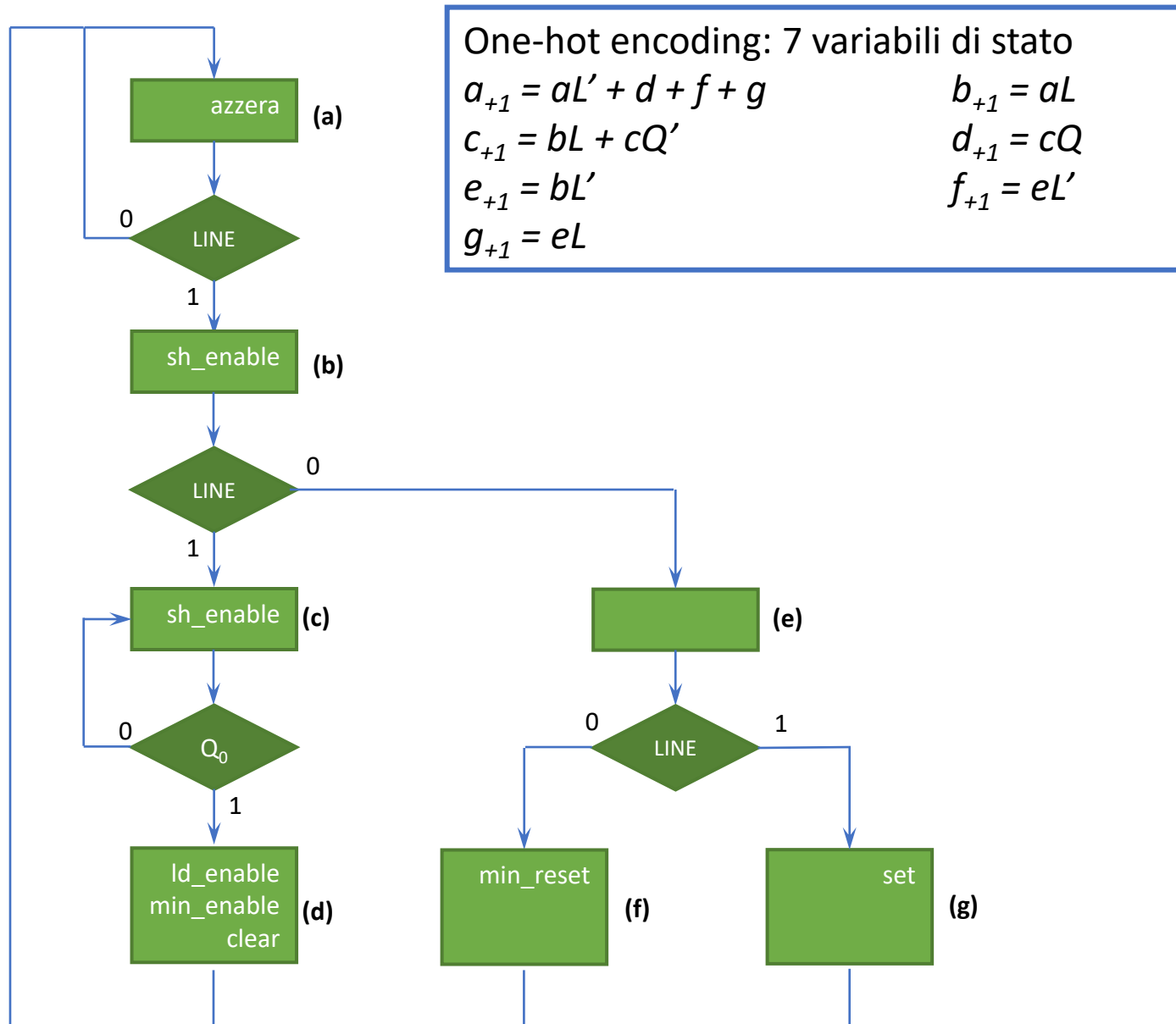
- Carica il nuovo dato (*ld_enable*)
- Aggiorna la temperatura minima (*min_enable*), ma solo se è minore di quella già memorizzata
- Mette D a 0 (*clear*) in modo da mostrare nuovo dato

Stati e, f, g

- Verificano il comando su LINE, attivano *min_reset* o *set* a seconda di cosa occorre fare

Riavvio

- Gli stati **d**, **f** e **g** tornano in **a** senza guardare LINE, perché siamo al bit di stop
- Volendo si può controllare che il bit di stop sia effettivamente a 0



Codifica minima a 3 bit, possibile codifica

Mappa stati

$x \setminus yz$	00	01	11	10
0	-	e	b	c
1	f	g	a	d

$x \setminus yz$	00	01	11	10
0	-	1	0	Q
1	1	1	L'	1

$$x_{+1} = y' + xz' + L'x + Qz'$$

$x \setminus yz$	00	01	11	10
0	-	0	L	1
1	1	1	1	1

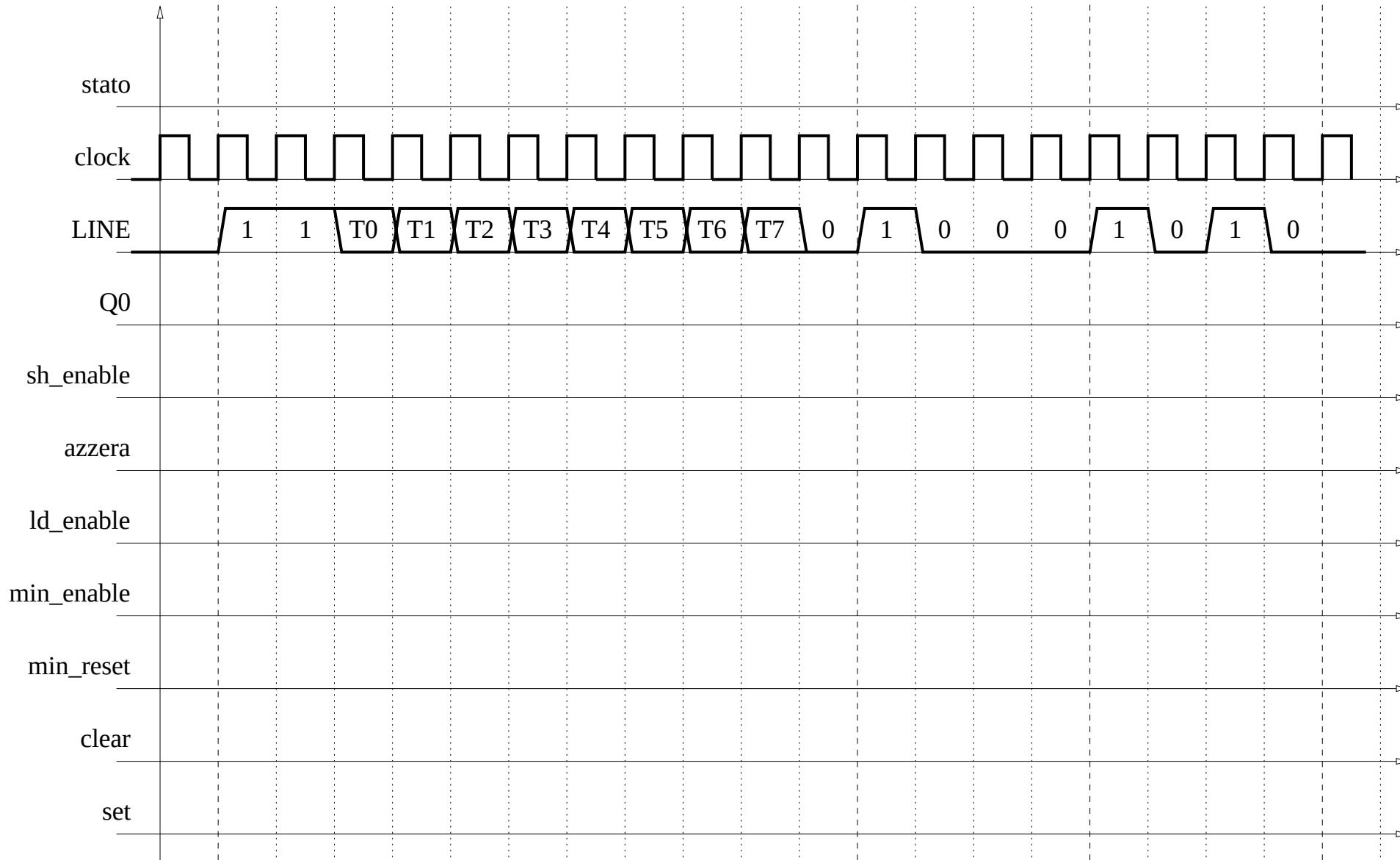
$$y_{+1} = x + z' + Ly$$

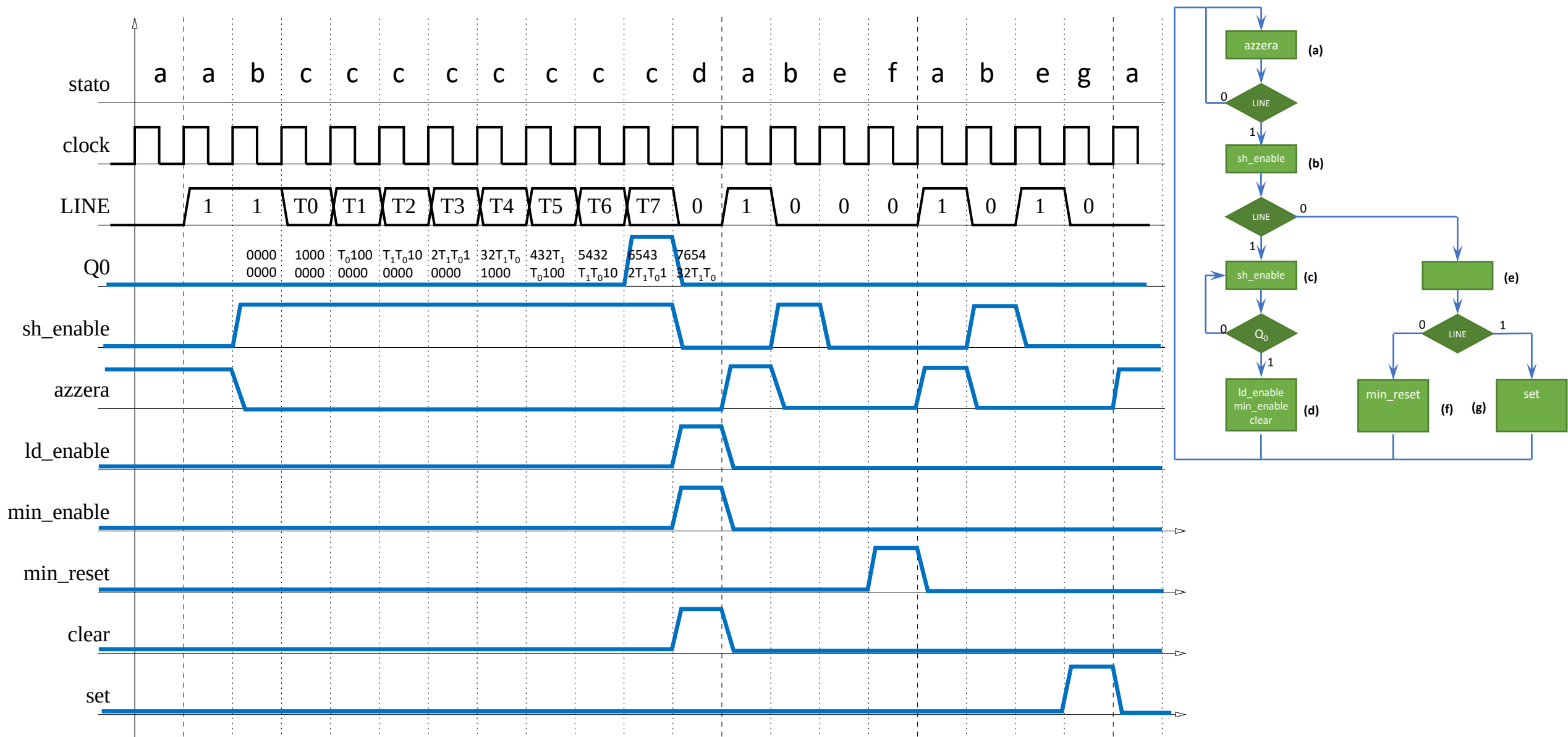
$x \setminus yz$	00	01	11	10
0	-	L	L'	0
1	1	1	1	1

$$z_{+1} = x + Ly' + L'yz$$

Per casa, fare le uscite

Diagramma temporale:
Un nuovo dato viene seguito
da due altri comandi





Codifica

Con 8 bit, non possiamo codificare in binario o in complemento a 2

- 1 bit per la parte decimale (precisione di 0.5)
- 7 bit per la parte intera $\rightarrow [-64, 63]$

Ci sono in ogni caso 256 diverse combinazioni

- -28, -27.5, -27, -26.5, ..., ..., -1, -0.5, 0, 0.5, 1, 1.5, 2, ..., ..., 98, 98.5, 99, 99.5
- Quindi 8 bit sono sufficienti
- Vogliamo codificare in modo che funzioni un comparatore binario
- Spostiamo semplicemente il valore dello 0

Assegniamo 0 a -28, e 255 a 99.5

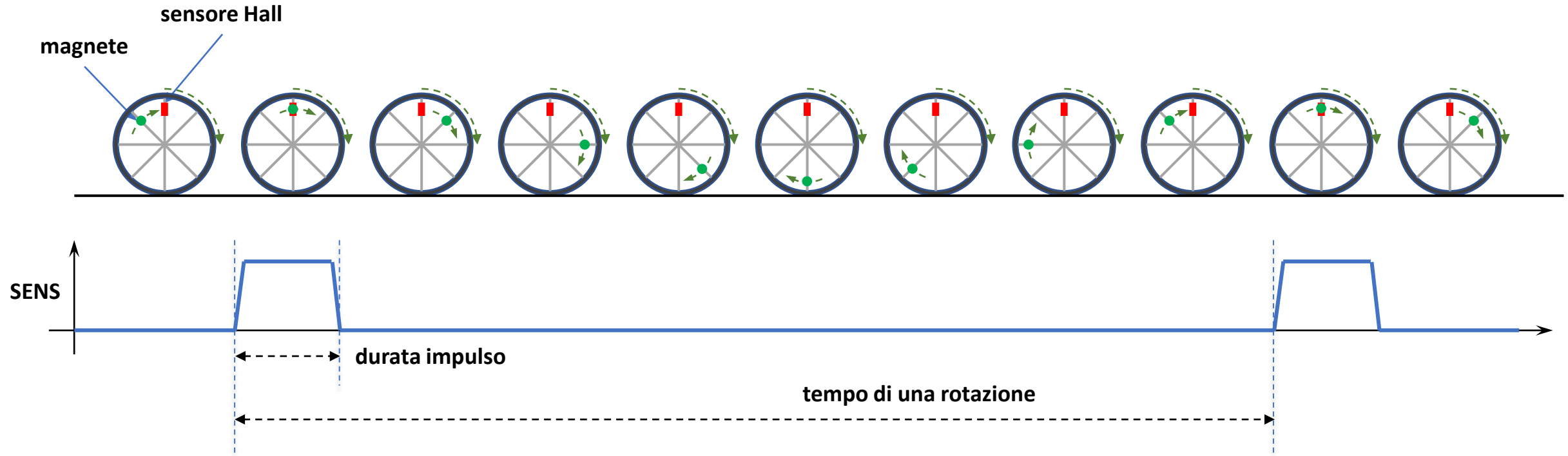
- Calcoliamo i valori intermedi
- $\text{code} = 2 \cdot \text{temp} + 56$
- La decodifica si occupa di fare la trasformazione contraria

Temp.	Code
-28	00000000
-27.5	00000001
-27	00000010
-26.5	00000011
...	...
-1	00110110
-0.5	00110111
0	00111000
0.5	00111001
1	00111010
1.5	00111011
2	00111100
...	...
98.5	11111101
99	11111110
99.5	11111111

Computer da bicicletta

Tema d'esame del gennaio 2011

Funzionamento

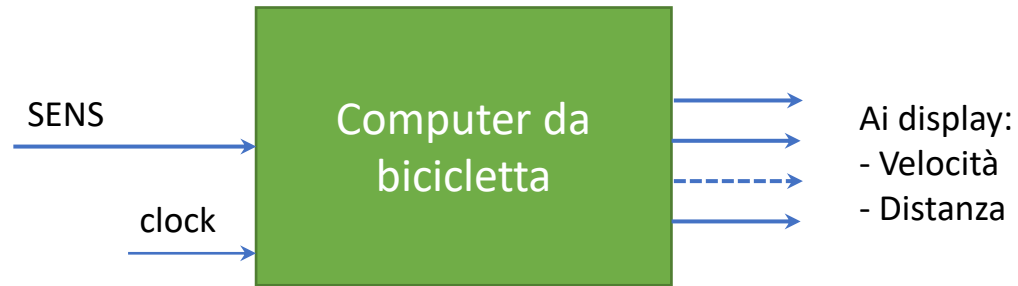


Magnete su raggio gira con la ruota

Sensore ad effetto Hall fisso rileva il passaggio del magnete

- Quando il magnete passa davanti al sensore, viene inviato un impulso su un segnale SENS
- Il tempo tra un impulso e quello successivo è inversamente proporzionale alla velocità: $t = \frac{\pi d}{v}$
- Durata dell'impulso anche dipendente dalla velocità

Specifica



Progettare il datapath

- Suggerimento: è più facile misurare il tempo tra un impulso e il successivo, la velocità può essere poi calcolata successivamente
- Inizialmente non preoccuparsi della dimensione dei registri e contatori (fondo scala), da aggiustare dopo
- Il computer conosce la dimensione della circonferenza della ruota

Il computer riceve in ingresso gli impulsi sul segnale SENS

- Si tratta di un singolo bit

Il computer deve misurare

- La velocità della bicicletta, da aggiornare ogni due giri della ruota per fare una media
- La distanza totale percorsa, da aggiornare ogni giro della ruota

Progettare il controllo

- Per aiutare: la durata dell'impulso (tra fronte di salita e discesa) dipende dalla velocità, ma si assuma che non sia mai inferiore a 3 cicli di clock (tuttavia può essere di più)
- Ignorare possibili corse critiche sugli ingressi
- Frequenza del clock da determinare successivamente

Datapath

Tempo di rotazione

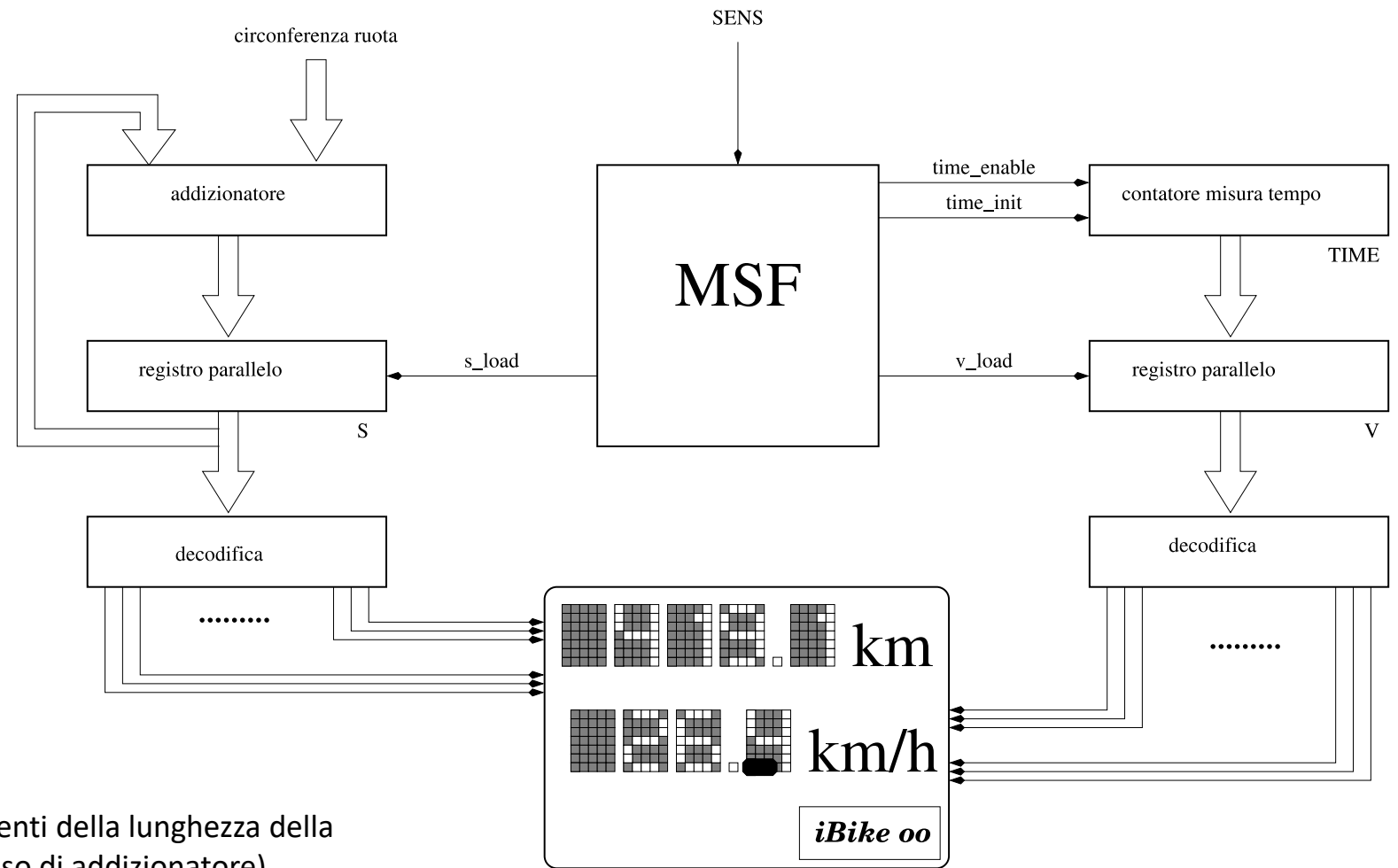
- Usiamo il contatore TIME con **init** per misurare il tempo tra un impulso e l'altro
- Ogni due giri salviamo il valore in un secondo registro V (load enable: **v_load**). In questo modo il display mostra un valore che non riparte ogni volta da zero
- La decodifica esegue anche il calcolo della velocità a partire dal tempo (da progettare in un secondo momento)

Distanza

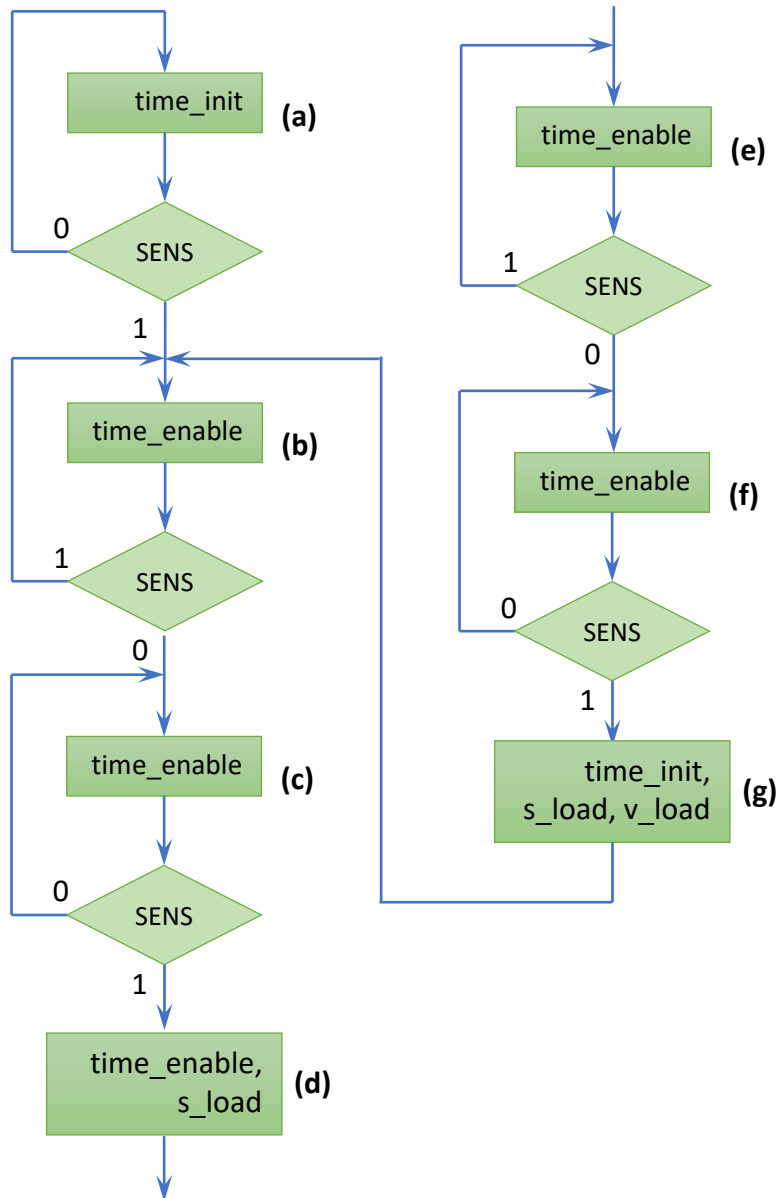
- Potremmo usare un contatore per contare il numero di giri, e poi moltiplicare per la circonferenza
- Più conveniente usare un contatore che si incrementi della lunghezza della circonferenza ad ogni giro (moltiplicatore più grosso di addizionatore)
- La figura mostra questa soluzione in modo esplicito. Il segnale **s_load** determina quando far contare, l'addizionatore di quanto far contare
- Anche in questo caso la decodifica da progettare in un secondo momento

Display

- Per cambiare usiamo un display a matrice di punti 7x5
- Occorre fare una decodifica diversa con 35 uscite per ogni cifra



MSF



Metodo

- Aspettiamo i vari fronti del segnale **SENS**
- Nel frattempo abilitiamo i contatori e i registri nei momenti opportuni

Time e V

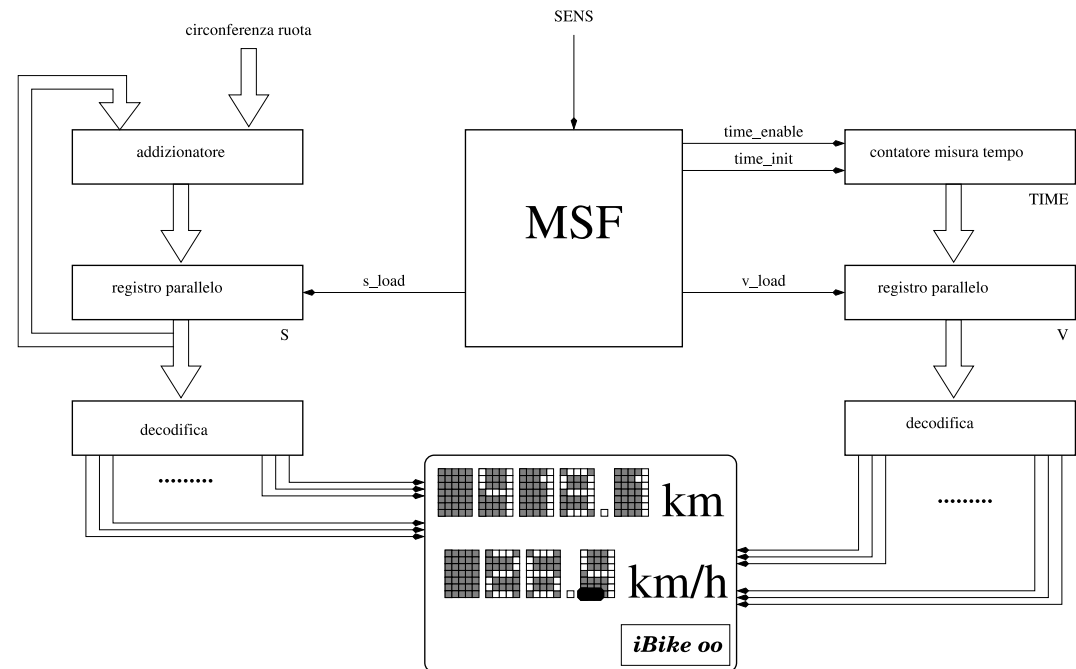
- TIME fondamentale sempre attivo. Lo iniziamo ogni due giri (**time_init**, stato **g**) per ricominciare il conteggio
- V carica il valore di TIME ogni due giri (**v_load**, sempre stato **g**). Segnali sincroni, quindi V carica il valore di TIME e contemporaneamente TIME si inizializza

Registro S

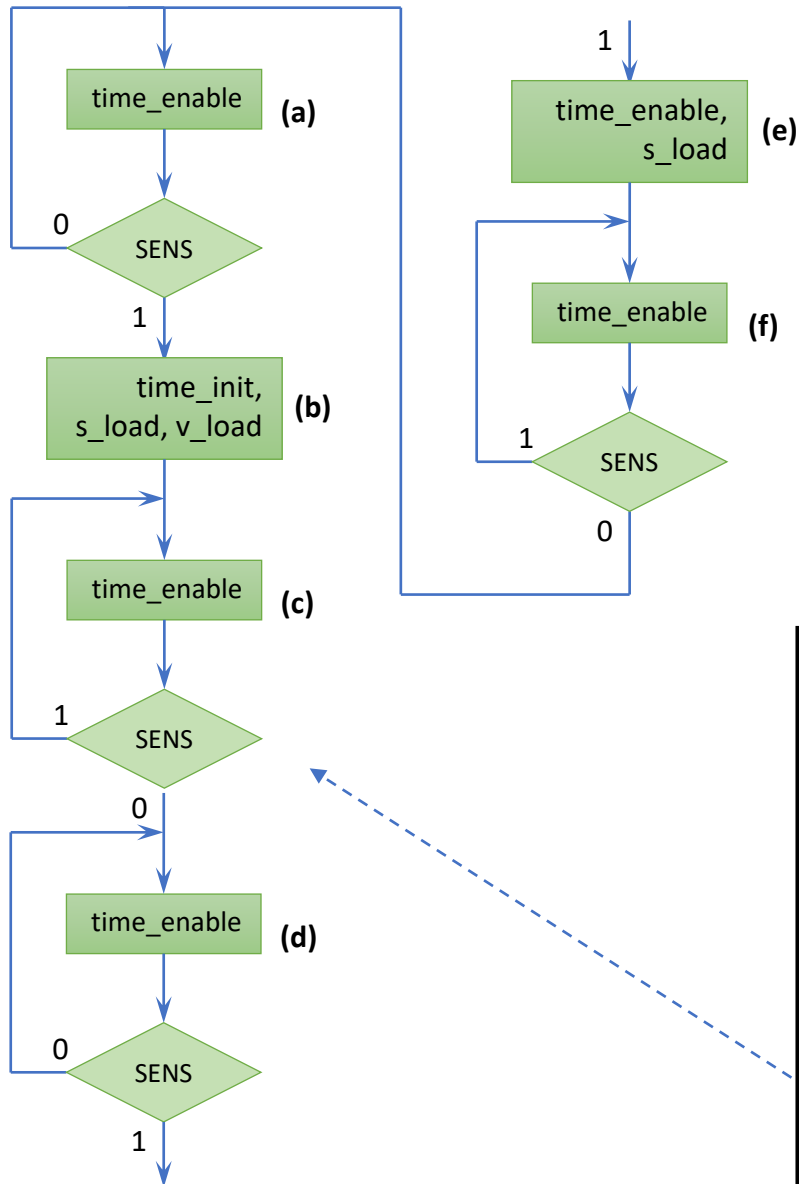
- Carica nuovo valore ad ogni giro (**s_load**, stati **d** e **g**)

Stati d e g

- Non osservano il valore di SENS
- Non è un problema perché sappiamo che SENS resta attivo per almeno 3 cicli di clock



MSF



Ottimizzazione

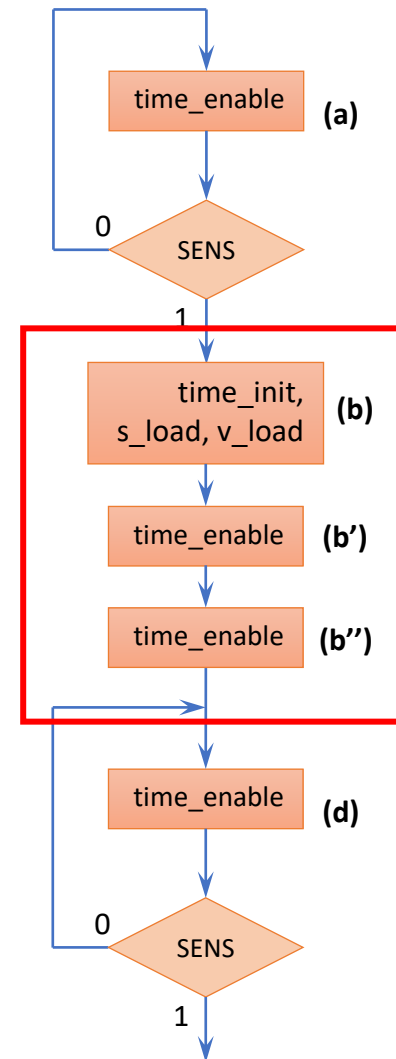
- Lo stato **a** precedente serve solo all'inizio, poi non ci si entra più
- Lo sostituiamo con lo stato **f** precedente, che ora diventa lo stato iniziale **a**
- Facciamo magari un piccolo errore sulla velocità all'inizio, ma lo annulliamo dopo due giri della ruota
- Contiamo forse un giro in più per la distanza, ma l'errore all'inizio è comunque inevitabile
- Riduciamo di uno stato, teniamo in pratica solo il loop interno

Nota bene

- Attiviamo **time_init** nel ciclo dopo quello in cui S è andato a 1
- Perdiamo un ciclo di clock nella misura?
- No perché **time_enable** anche rimane attivo nel ciclo in cui S va 1, due giri dopo, recuperando il ciclo perso

Da non fare!

- Usare 3 stati per contare tre cicli di clock in cui SENS sta a 1
- SENS sta a 1 per **almeno** tre cicli di clock, **ma potrebbe rimanervi di più**
- Si rischia di finire in **d** con SENS ancora a 1, e quindi andare erroneamente in **e**
- Occorre invece esplicitamente attendere la transizione su SENS



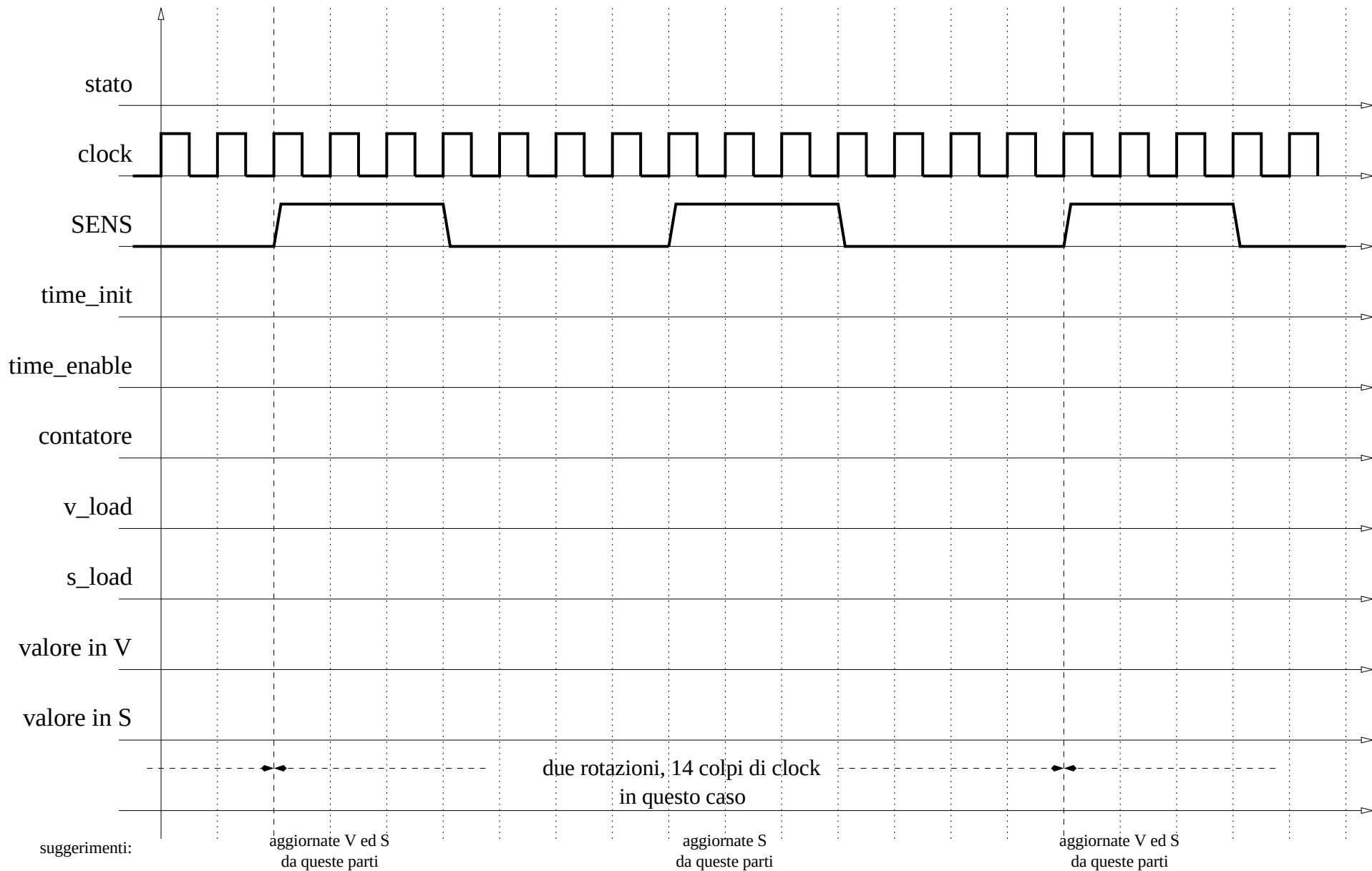
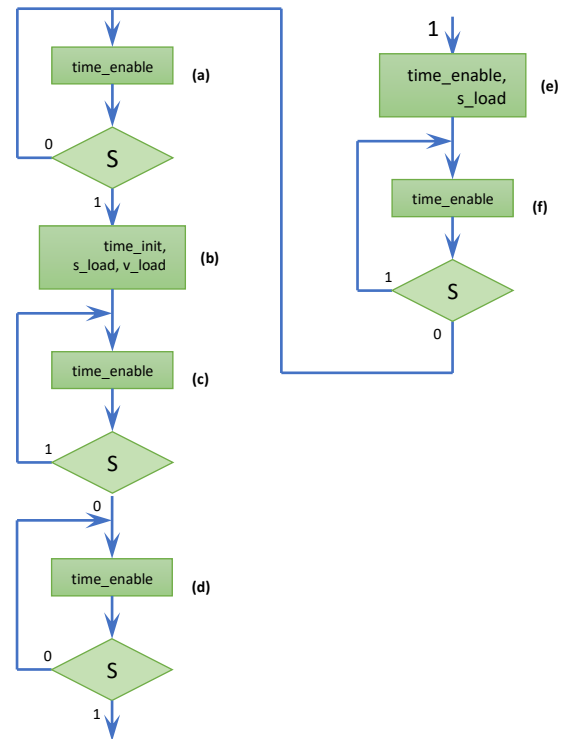
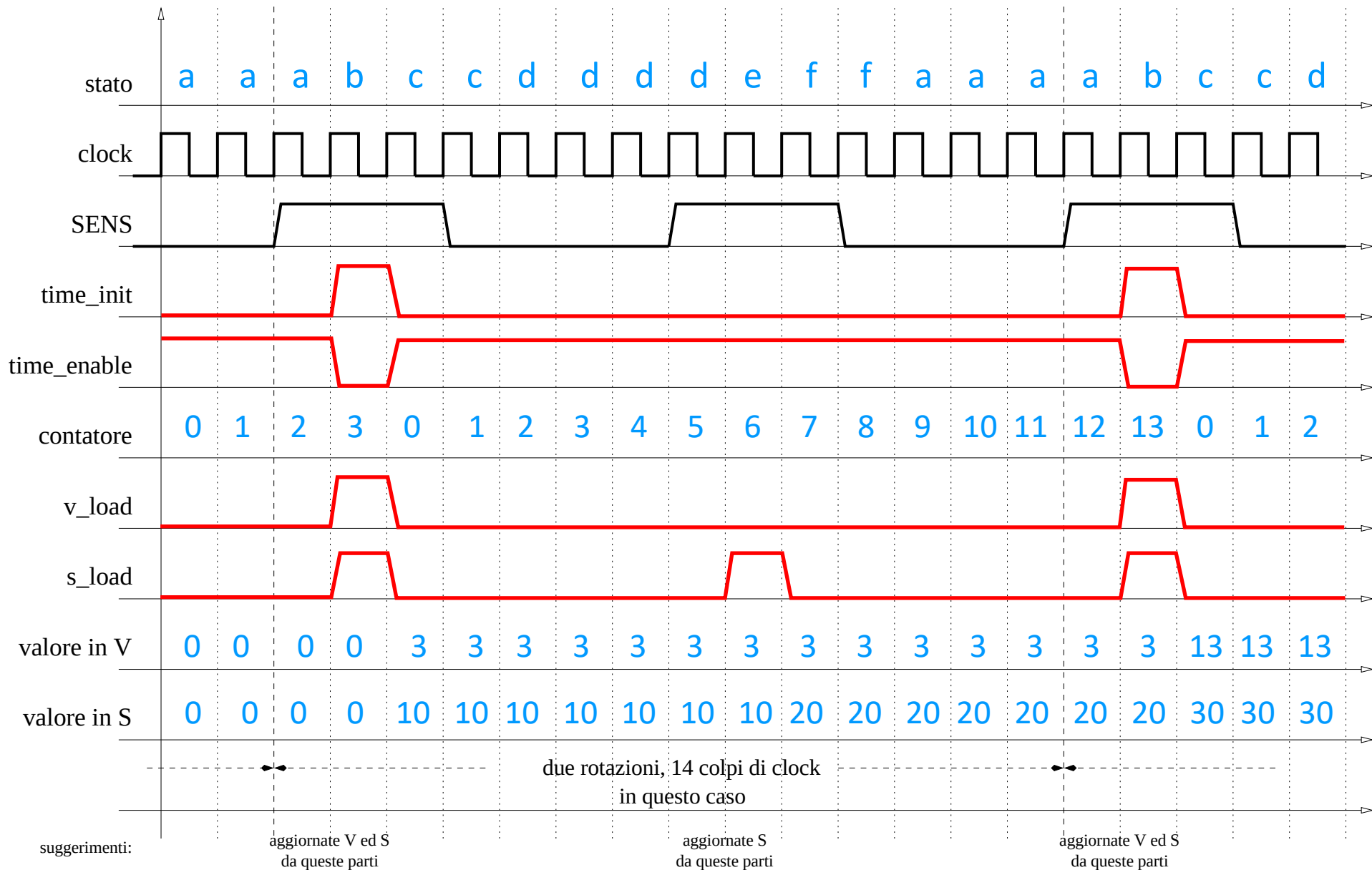


Diagramma temporale:
Due rotazioni impiegano 14
cicli di clock

Indicare il valore contenuto
nel contatore, e nei registri V
ed S

Supponiamo siano tutti
inizialmente a 0

Supponiamo la circonferenza
abbia lunghezza 10



Dimensione del contatore e clock

Diamoci delle specifiche

- Velocità massima 60 km/h, Velocità minima 1 km/h, Precisione di 0.1 km/h
- Ruota da 26 pollici di diametro

Clock: sufficientemente veloce alla velocità massima con la precisione richiesta

- $\frac{\text{circonferenza}}{\text{velocità}} = \frac{26 \cdot 0.0254 \cdot \pi}{60 \cdot 1000 / 3600} = \frac{7.468948038}{60} = 0.124482467 \text{ s} = T_c \cdot n$
- $\frac{\text{circonferenza}}{\text{velocità}} = \frac{26 \cdot 0.0254 \cdot \pi}{60.1 \cdot 1000 / 3600} = \frac{7.468948038}{60.1} = 0.124275341 \text{ s} = T_c \cdot (n - 1)$
- $0.124482467 - 0.124275341 = 207.126 \cdot 10^{-6} = T_c \cdot n - T_c \cdot (n - 1) = T_c$
- $f_c = 4827.979105 \text{ Hz} \cong 5 \text{ kHz}$
- La metà se consideriamo due rotazioni

Dimensione contatore: quanti cicli di clock per 2 rotazioni a 1 km/h?

- Durata rotazione = $\frac{7.469}{1} = 7.469 \text{ s}$
- Cicli di clock per 2 rotazioni = $7.469 \cdot 2 \cdot 5000 = 74690$

Così precisione di 0.05 km/h

Quindi il contatore deve avere almeno 17 bit

- Fondo scala di 131072 clock
- Velocità minima effettiva di 0.57 km/h
- Con 16 bit velocità minima di 1.14 km/h

Calcolo da fare per determinare la velocità in km/h in base al valore n del contatore

$$v = \frac{7.469 \cdot 5000 \cdot 2}{n} = \frac{74690}{n}$$

Misuratore qualità del segnale

Tema d'esame del febbraio 2010

Qualità del segnale: controllo di parità

Supponiamo di trasmettere 8 bit

- Chi trasmette aggiunge un nono bit in modo che il numero complessivo di bit a 1 sia pari (bit di parità)
- Chi riceve conta il numero di bit a 1 complessivi ricevuti
 - Se sono **pari**, il dato è **probabilmente corretto**
 - Se sono **dispari**, il dato è **certamente errato**

I bit possono essere ricevuti incorrettamente a causa del rumore durante la trasmissione

	dato → <u>10001010</u> parità → <u>1</u>	ci sono 3 bit a uno (<i>dispari</i>) nel dato. Metto un altro uno, così diventano 4 (<i>pari</i>)
	se ricevo...	
	<u>10001010</u> <u>1</u>	Vedo 4 bit a 1. Ritengo il dato corretto
Uno 0 diventa 1	<u>10011010</u> <u>1</u>	Vedo 5 bit a 1. Ritengo il dato errato
Un 1 diventa 0	<u>10000010</u> <u>1</u>	Vedo 3 bit a 1. Ritengo il dato errato
Corrotto il bit di parità	<u>10001010</u> <u>0</u>	Vedo 3 bit a 1. Ritengo il dato errato
Doppio errore, non lo rilevo	<u>10101011</u> <u>1</u>	Vedo 6 bit a 1. Ritengo il dato corretto

Qualità del segnale: controllo di parità

Supponiamo di trasmettere 8 bit

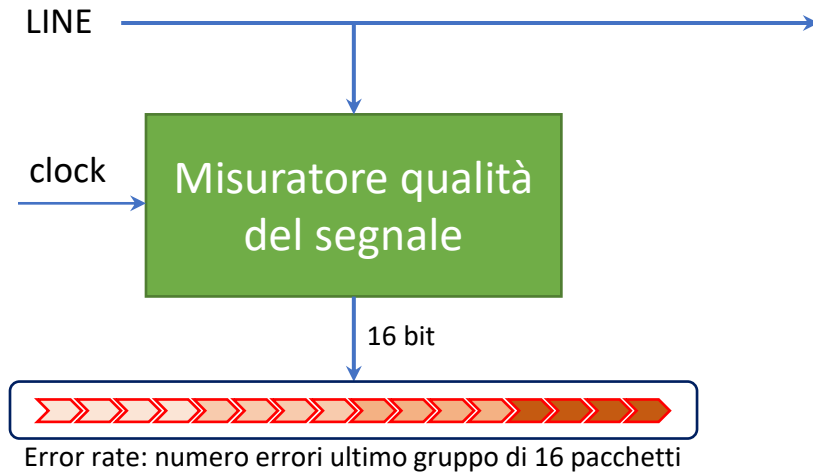
- Chi trasmette aggiunge un nono bit in modo che il numero complessivo di bit a 1 sia pari (bit di parità)
- Chi riceve conta il numero di bit a 1 complessivi ricevuti
 - Se sono **pari**, il dato è **probabilmente corretto**
 - Se sono **dispari**, il dato è **certamente errato**

I bit possono essere ricevuti incorrettamente a causa del rumore durante la trasmissione

	dato → <u>11001010</u> parità → 0	ci sono 4 bit a uno (<i>pari</i>) nel dato. Metto uno zero, così rimangono 4 (<i>pari</i>)
	se ricevo...	
	<u>11001010</u> 0	Vedo 4 bit a 1. Ritengo il dato corretto
Uno 0 diventa 1	<u>11011010</u> 0	Vedo 5 bit a 1. Ritengo il dato errato
Un 1 diventa 0	<u>11000010</u> 0	Vedo 3 bit a 1. Ritengo il dato errato
Corrotto il bit di parità	<u>11001010</u> 1	Vedo 5 bit a 1. Ritengo il dato errato
Doppio errore, non lo rilevo	<u>11101011</u> 0	Vedo 6 bit a 1. Ritengo il dato corretto

Nota: si può decidere che il numero di bit a 1 debba essere **dispari** invece di pari. Non cambia nulla, l'importante è mettersi d'accordo tra chi trasmette e chi riceve!

Specifica



Progettare il datapath

- Si può calcolare la parità acquisendo l'intero dato e facendo il calcolo nel datapath. Provate a non farlo: **il calcolo della parità lo deve fare il controllo**
- Non c'è quindi bisogno di memorizzare il dato
- Il datapath deve tenere traccia del numero di bit visti e del numero di pacchetti visti
- Inoltre tiene traccia del numero di errori visti negli ultimi 16 pacchetti

Mostrare su un display in scala termometrica il numero di errori rilevati in gruppi di 16 pacchetti

- 16 LED da accendere in base al numero di errori
- Ogni 16 pacchetti in modo da fare una sorta di media

Dati provenienti in forma seriale

- Ogni dato è preceduto da un **bit di start a 1**, seguito dal **dato a 8 bit**, dal un **bit di parità**, e da **3 bit di stop a 0**
- Il bit di parità è trasmesso in modo tale che il numero di bit a 1 totali, escluso il bit di start e i bit di stop, sia **pari**
 - **1-10001010-1-000**, **1-10001011-0-000**

Progettare il controllo

- Per aiutare: 3 bit di stop danno tempo dopo il calcolo della parità di eseguire le operazioni accessorie senza usare uscite condizionate
- **Calcolare la parità in modo incrementale**
- Verificare con i dati presenti nel datapath quando finisce un dato e quando occorre ricominciare con il conto (ogni 16 pacchetti)
- Non è necessario verificare che i bit di stop siano a 0

Datapath

Contatore di bit

- Usiamo il contatore BIT COUNTER per contare i bit ricevuti sulla linea LINE
- Segnali: **init_bit** inizializza a 0, **count_bit** abilita al conteggio, **TC_bit** è il terminal count
- Fondo scala da decidere in base alla macchina a stati

Contatore di pacchetti

- Usiamo il contatore DATA COUNTER per contare il numero di pacchetti ricevuti e ricominciare il conto ogni 16
- Segnali: **init_data** inizializza a 0, **count_data** abilita al conteggio, **TC_data** è il terminal count
- Contatore modulo 16 (da 0 a 15)

Contatore di errori

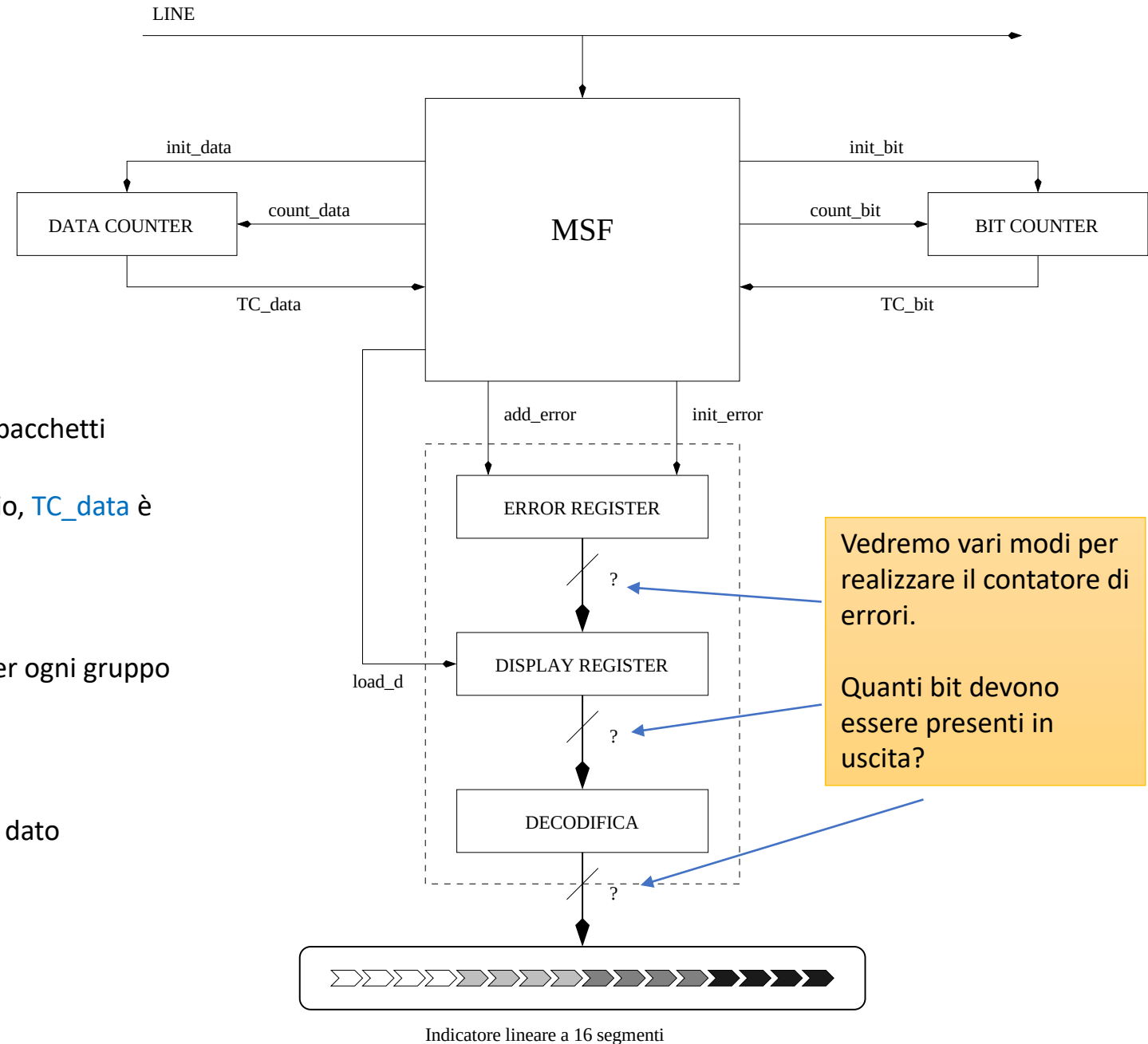
- Usiamo il contatore ERROR_REGISTER per contare gli errori per ogni gruppo di 16 pacchetti
- Segnali: **add_error** incrementa di 1, **init_error** inizializza a 0

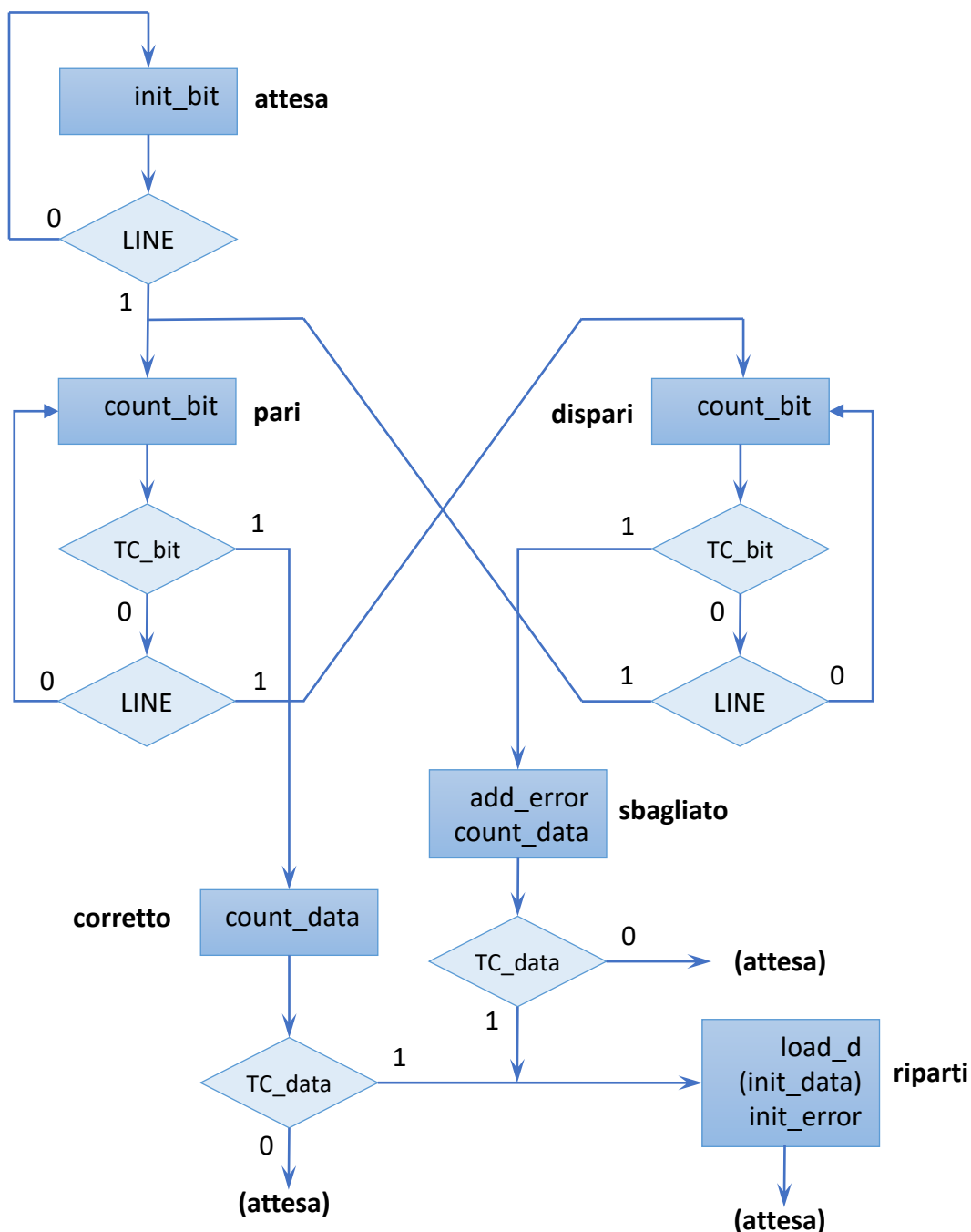
Display register

- Per non mostrare il conto che si incrementa, memorizziamo il dato precedente nel registro DISPLAY REGISTER
- Segnali: **load_d** comanda il caricamento

Decodifica

- Pilota i 16 LED in base al valore del display register





Stato attesa

- Aspetta il bit di start, inizializza il contatore di bit (`init_bit`)

Stati pari e dispari

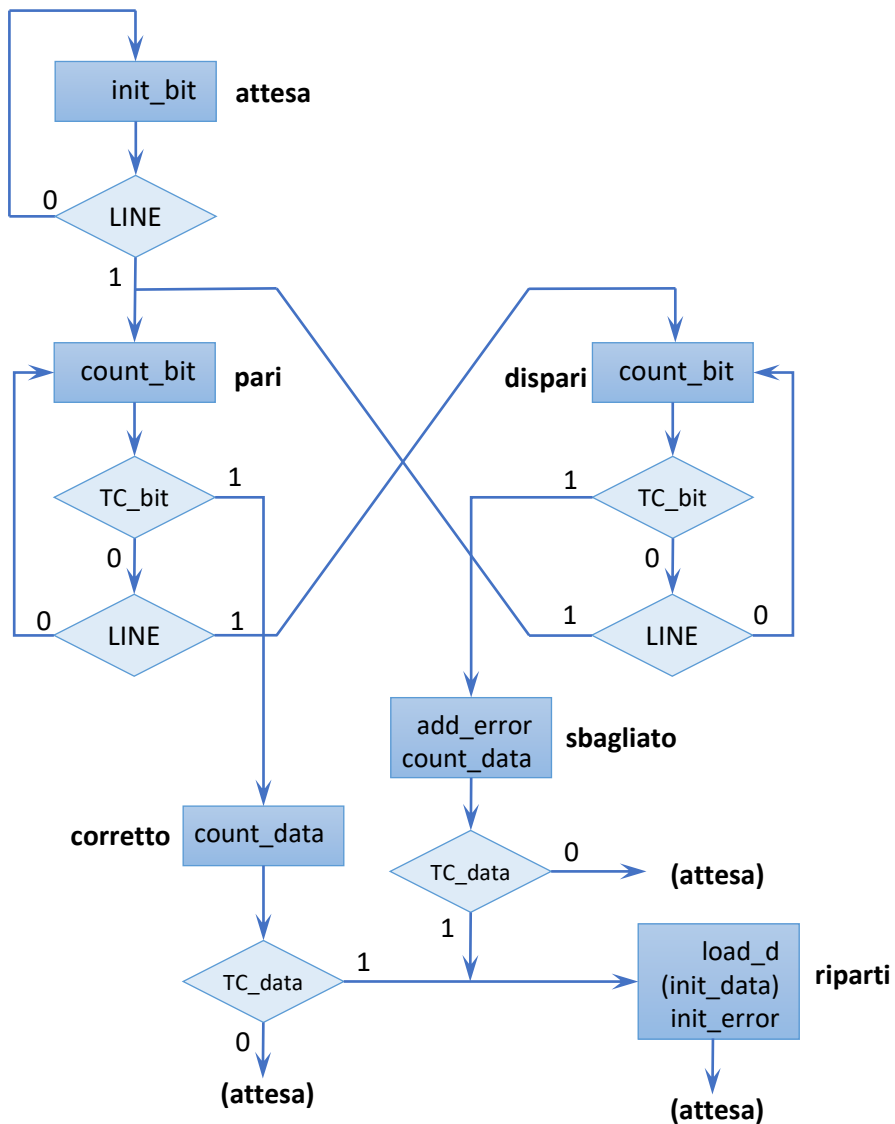
- Mentre riceviamo i bit, teniamo traccia se il numero di 1 visti è pari oppure dispari. Non è necessario contarli!
- Inizialmente il numero di 1 è zero (non abbiamo visto ancora nulla), quindi è pari
- Ogni volta che si vede un bit `LINE` a 1, se il numero di 1 era pari diventa dispari, se era dispari diventa pari. Se vedo 0 invece rimane com'era
- Contemporaneamente guardo anche se sono arrivato alla fine dei dati (`TC_bit`), incluso il bit di parità. Per come è fatta la MSF, devo contare da 0 a 9 (risulterà chiaro dal diagramma temporale)
- Sono possibili altre MSF (e.g., guardo `LINE` anche nel ramo dopo che vedo `TC_bit` a 1) in cui il contatore conta da 0 a 8
- Se esco dallo stato **pari** il dato è corretto, se esco da **dispari** è errato

Stati corretto e sbagliato

- Ho visto un pacchetto, quindi incremento il contatore con `count_data`
- Se il dato è errato aggiungo l'errore al contatore (`add_error`)
- Guardo se ho fatto 16 pacchetti (`TC_data`): se no ritorno in **attesa** ad aspettare un nuovo pacchetto, altrimenti vado in **riparti**

Stato riparti

- Carica il numero degli errori nel display register (`load_d`)
- Inizializzo il conto degli errori (`init_error`) e dei pacchetti (`init_data`). Questo è superfluo se il data counter fa overflow e ricomincia da 0



Contatore modulo 2

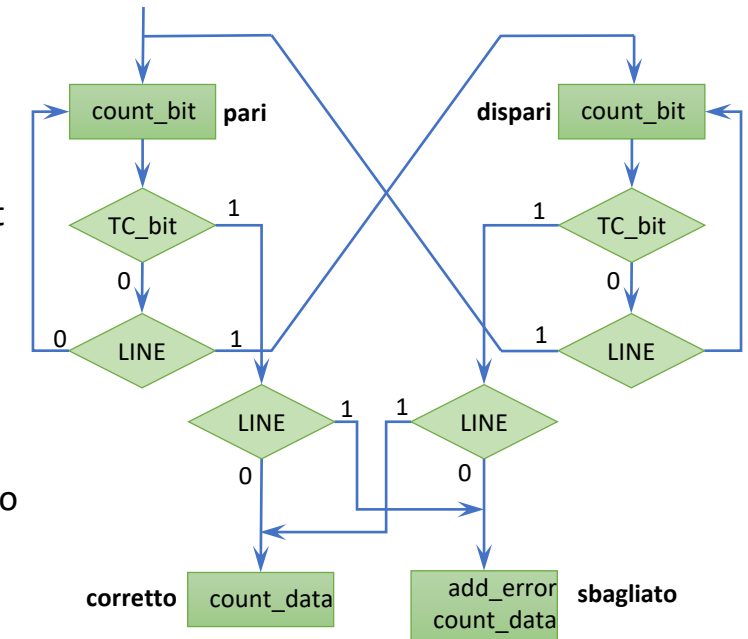
- Gli stati **pari** e **dispari** realizzano un contatore modulo 2 (conta da 0 a 1) usando **LINE** come count enable e ricominciando da 0 su overflow

Bit di stop

- Non li guardo (dove andrebbero verificati?)
- In questa realizzazione mentre sono sul bit di parità vado ancora in **pari** oppure in **dispari** per decidere sull'ultimo bit. Il **TC_bit** arriva quindi mentre siamo già sul primo bit di stop
- Durante il secondo bit di stop sono in **corretto** o in **sbagliato**
- Se ho finito i pacchetti, lo stato **riparti** è in corrispondenza del terzo bit di stop
- Solo a quel punto vado in **attesa** e sono pronto per un nuovo pacchetto
- Bene quindi avere 3 bit di stop!!

Come velocizzare

- Posso dare il **TC_bit** durante il bit di parità. Occorre allora guardare **LINE** sul ramo in cui **TC_bit** è 1 per decidere se il dato è corretto oppure no. Si può ridurre di un bit di stop. Il bit counter conta ora fino a 8
- Usare uscite condizionate per **count_data** e **add_error**
- Attenzione: **add_error** deve venire un ciclo prima di **load_d**, altrimenti si caricherebbe nel display register il dato non ancora incrementato (quindi no **load_d** uscita condizionata fuori da **sbagliato**)



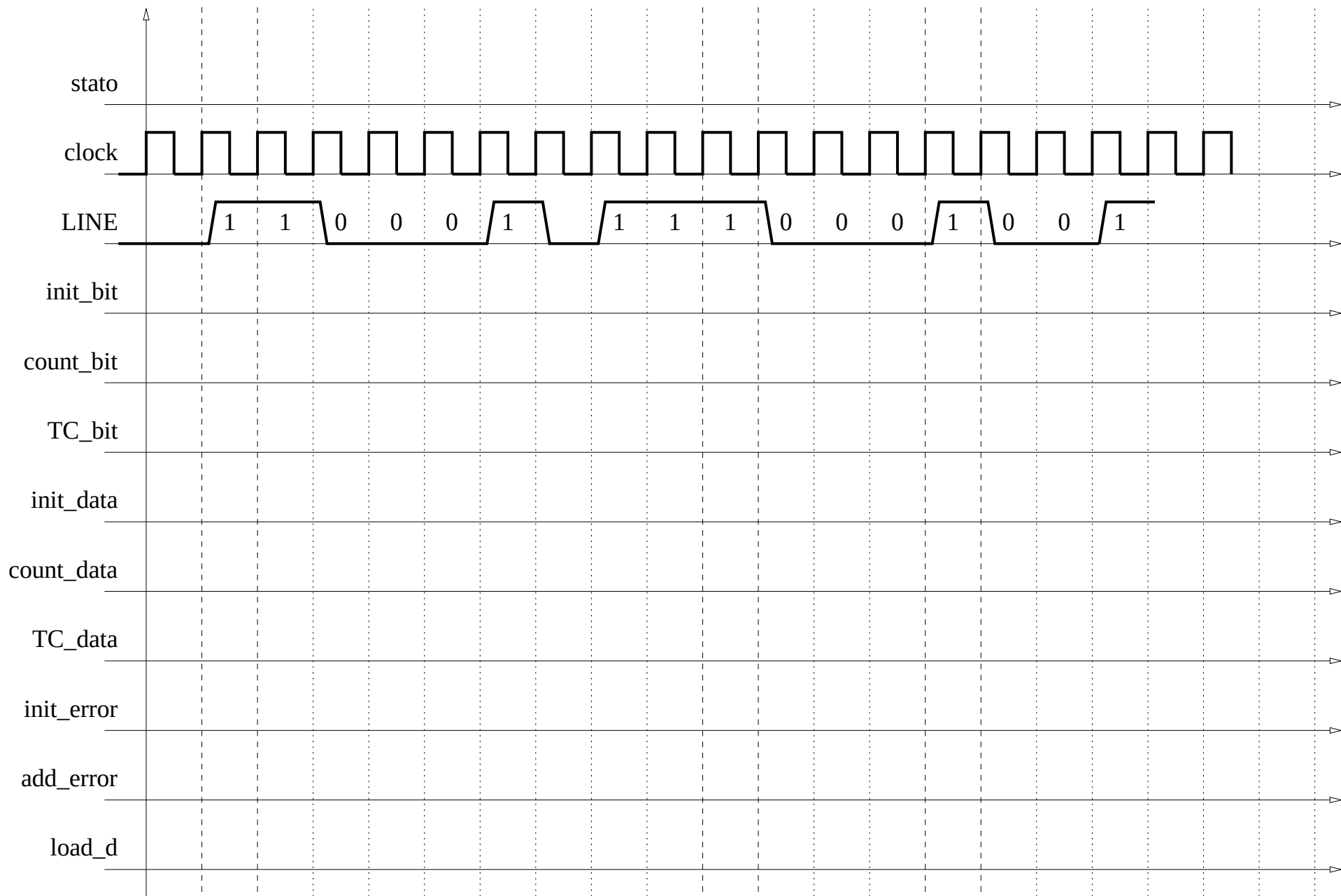


Diagramma temporale:

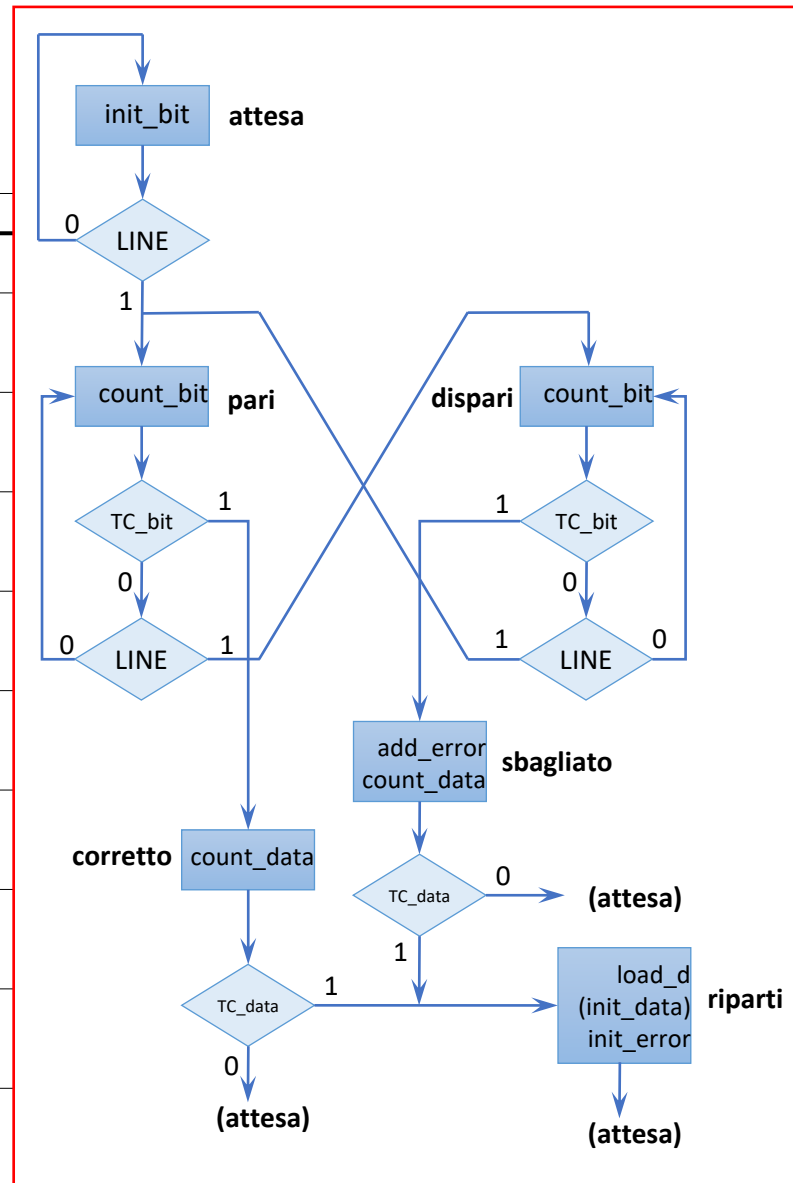
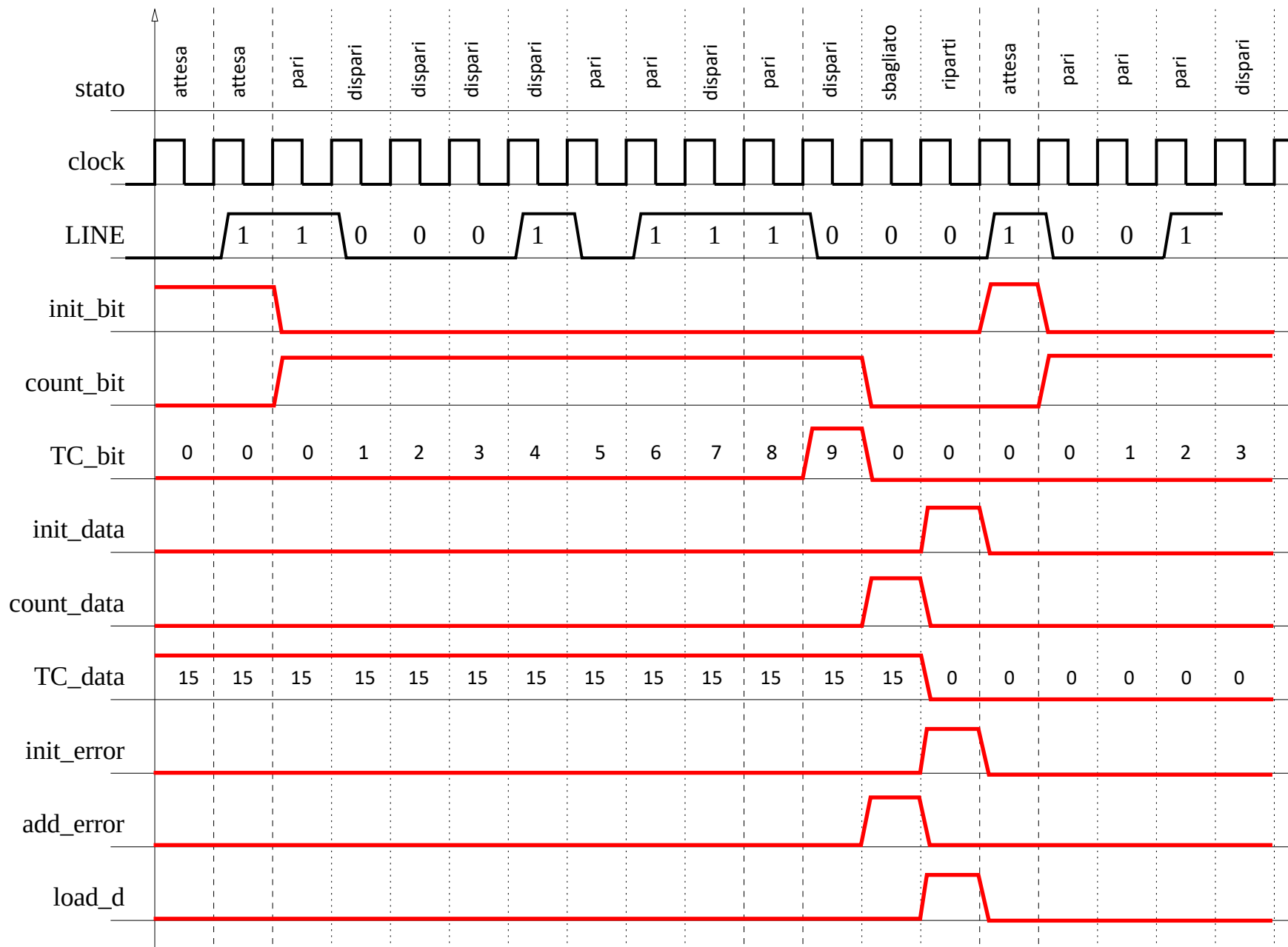
- Bit di start
- Dato: 10001011
- Parità: 1

Il dato è errato (ci sono 5 bit a 1)

- Tre bit di stop
- Nuovo bit di start
- Altri dati non finiti

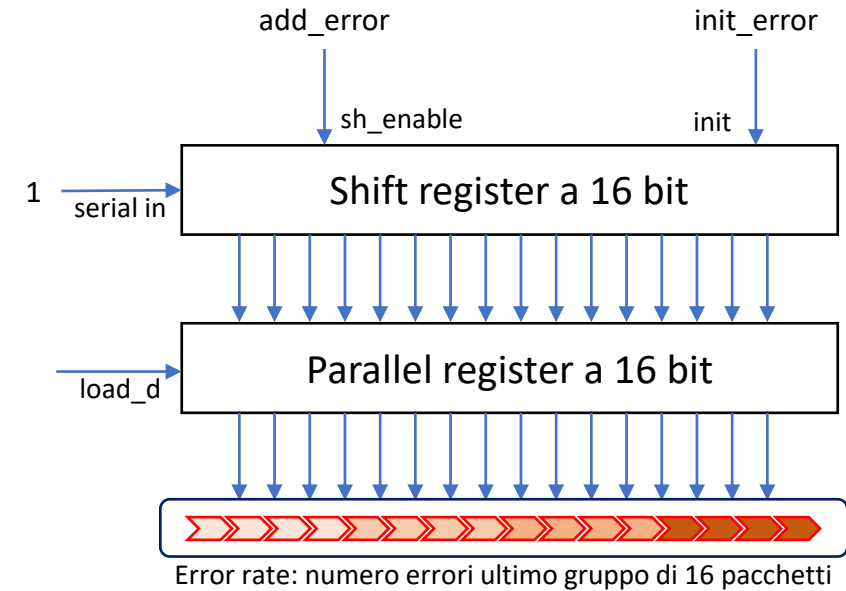
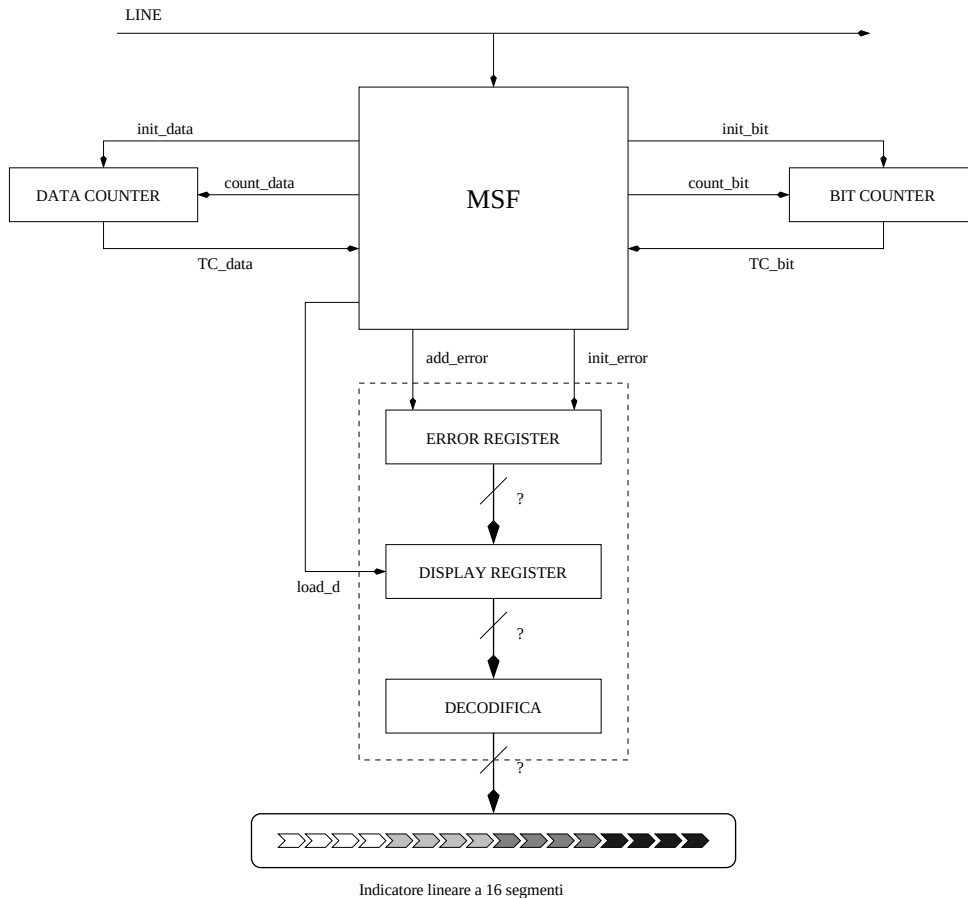
Indicare il valore contenuto nel bit counter nella riga di TC_bit. Assumere che abbia valore iniziale 0

Indicare il valore contenuto nel data counter nella riga di TC_data. Assumere che abbia valore iniziale 15 (siamo cioè all'ultimo pacchetto)



Quanti bit sono necessari per contare gli errori?

- Devo contare da 0 a 16 errori, estremi inclusi (nessun pacchetto errato, tutti i pacchetti errati)
- Sono allora 17 combinazioni!**
- Ho bisogno di **5 bit**
- Display register anche a 5 bit, decodifica da numero binario a codifica termometrica (colonna di LED)



Alternativa

- Contiamo direttamente in termometrico (simile contatore Johnson)**
- Per esempio usiamo un registro a scorrimento a 16 bit con un 1 sull'ingresso seriale
- Il segnale **add_error** abilita allo scorrimento. Ogni volta fa entrare un nuovo 1 mentre gli altri scorrono
- Inizializziamo mettendo il registro a 0
- Registro display register parallelo anche a 16 bit
- Nota: **non c'è bisogno della decodifica**, i dati sono già nel formato richiesto
- Qual è meglio?

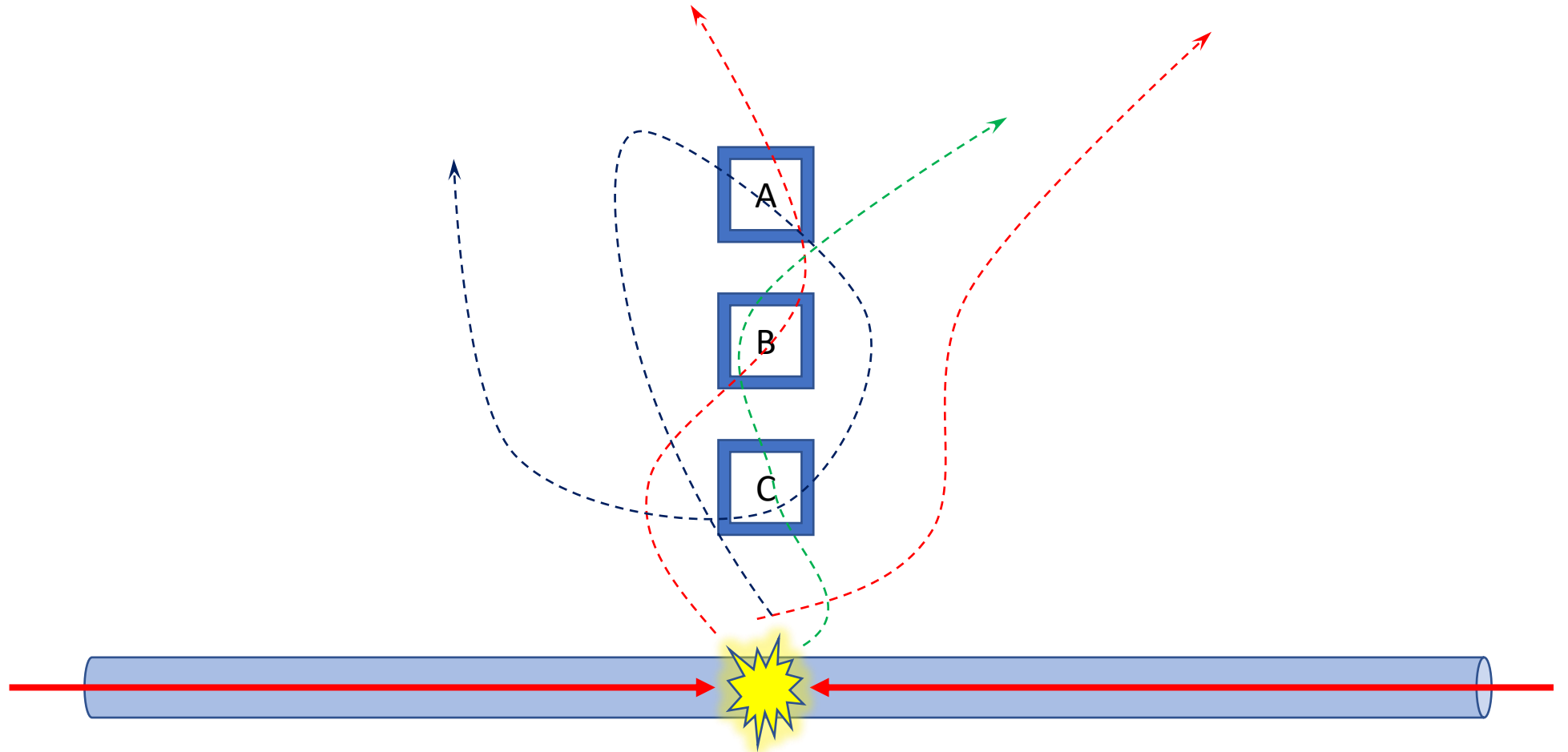
Misuratore di correlazione spaziale

Tema d'esame del giugno 2015

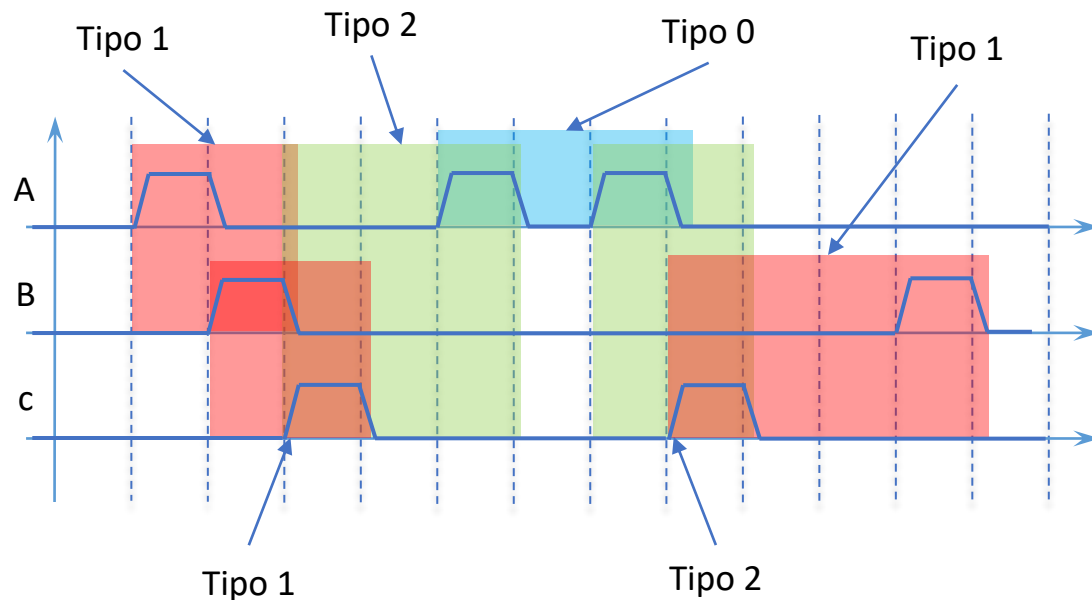
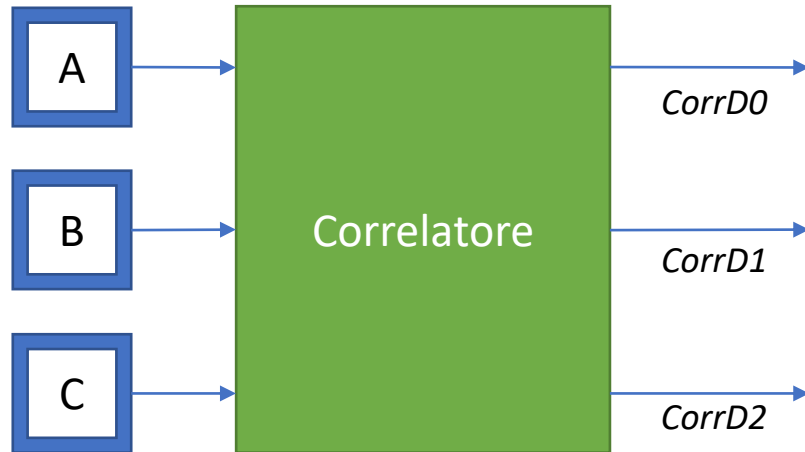
Un acceleratore produce flussi di particelle tramite delle collisioni

Tre sensori, A-B-C, rilevano il passaggio delle particelle

Vogliamo misurare eventuali correlazioni tra i dati rilevati dai vari sensori



Specifica



I rivelatori mandano un impulso per ogni evento rivelato

- L'impulso dura esattamente un ciclo di clock
- Per semplicità, supponiamo che in ogni ciclo di clock possa essere attivo il segnale di un solo rivelatore

Vogliamo misurare la correlazione tra gli eventi

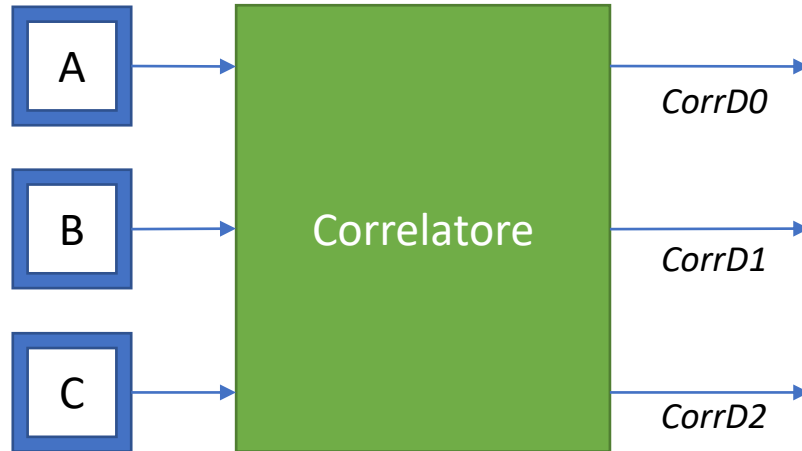
- Diciamo che due eventi sono tra loro *successivi* se hanno luogo in due cicli di clock tra loro successivi, o anche da più cicli se non intervengono altri eventi

Consideriamo tre tipi di correlazione tra eventi

- **Tipo 0:** lo stesso sensore rivela due eventi in successione
- **Tipo 1:** due sensori adiacenti (A e B, oppure B e C) rivelano un evento ciascuno in successione
- **Tipo 2:** due sensori lontani (A e C) rivelano un evento ciascuno in successione
- **Nota:** il secondo evento di una successione diventa il primo evento di quella successiva

Dati gli eventi, vogliamo innanzi tutto contare quante correlazioni di tipo 0, 1 e 2 hanno luogo

Specifica



Progettare il datapath

- Relativamente semplice
- Occorre contare il numero delle correlazioni sulla base delle indicazioni dell'unità di controllo
- Il calcolo delle uscite può essere eseguito sulla base dei conteggi tramite un componente dedicato

Comportamento delle uscite

- Le uscite del correlatore si attivano per indicare se un tipo di correlazione è più preponderante rispetto alle altre

Più in specifico:

- Le uscite *CorrD0*, *CorrD1* e *CorrD2* si attivano se il numero di correlazioni osservate del tipo corrispondente (0, 1 e 2, rispettivamente) è almeno il doppio degli altri due

Progettare il controllo

- Bisogna ricordarsi ogni volta quale sensore ha prodotto l'ultimo evento
- Dato un secondo evento, occorre incrementare il contatore giusto, e **nel ciclo di clock successivo essere subito pronti a gestire un nuovo evento**

Datapath

Contatori di correlazioni

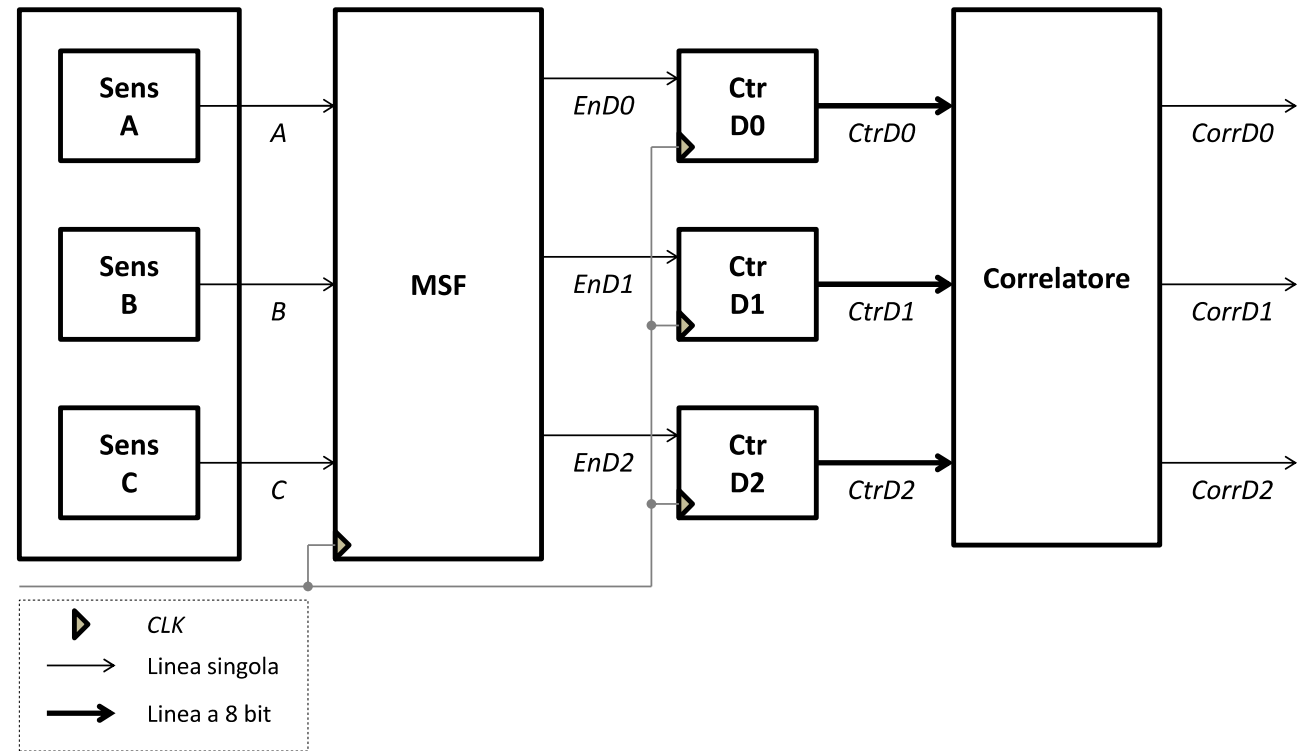
- Tre contatori (CtrD0, CtrD1, CtrD2) consentono di contare il numero di correlazioni dei vari tipi
- Ognuno si suppone inizializzato all'accensione
- Ognuno abilitato al conteggio tramite un apposito segnale dalla macchina a stati
- Fondo scala indeterminato: supponiamo siano dimensionati in modo sufficiente per contare il numero di correlazioni di interesse

Correlatore

- Prende in ingresso i valori dei tre contatori
- Determina il valore delle uscite
- Non viene richiesto di progettarlo, ma potete provare a pensarci lo stesso (e.g., come si calcola il doppio del valore di un contatore per fare i confronti?)
- Componente del tutto combinatorio

Macchina a stati

- Traduce i segnali dei sensori in comandi per i contatori



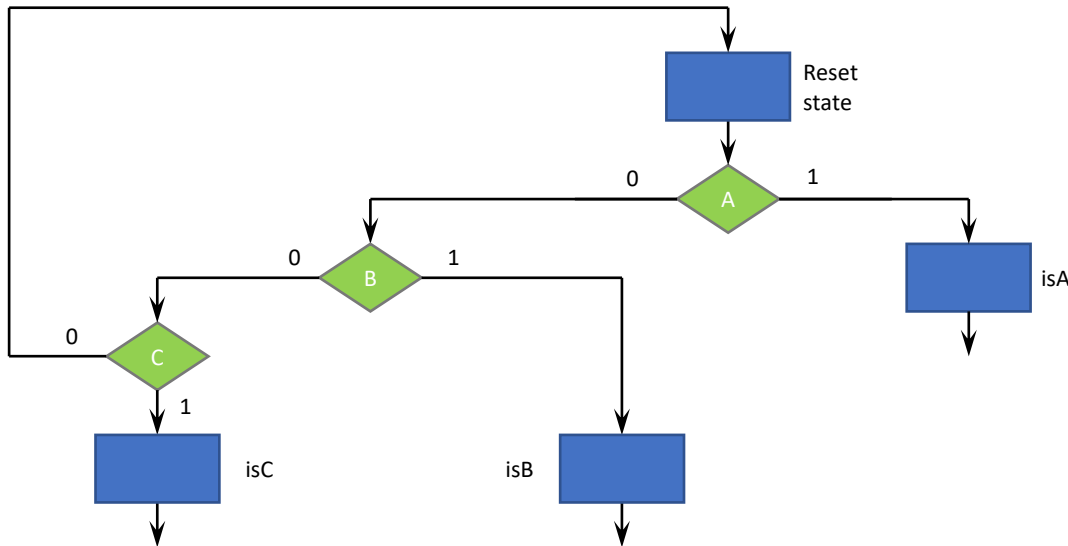
MSF: Funzionamento in fase iniziale

Stato di reset

- Stato iniziale della macchina a stati
- Non è ancora successo nulla, quindi questo stato semplicemente aspetta che vi sia un evento da uno qualunque dei rivelatori
- Possiamo guardare A, B e C praticamente singolarmente, perché sappiamo che non hanno luogo contemporaneamente
- Arrivato l'evento, non si attiva nessun contatore al conteggio perché si tratta del primo

Stati isA, isB e isC

- Ciascuno di questi stati tiene traccia del fatto che l'ultimo evento visto abbia avuto luogo sul sensore A, B o C, rispettivamente
- A partire da questi stati, un qualunque altro evento dovrà essere conteggiato come correlazione di tipo 0, 1 o 2
- Il nuovo evento determina anche in quale stato si andrà a finire (vedi prossima slide)



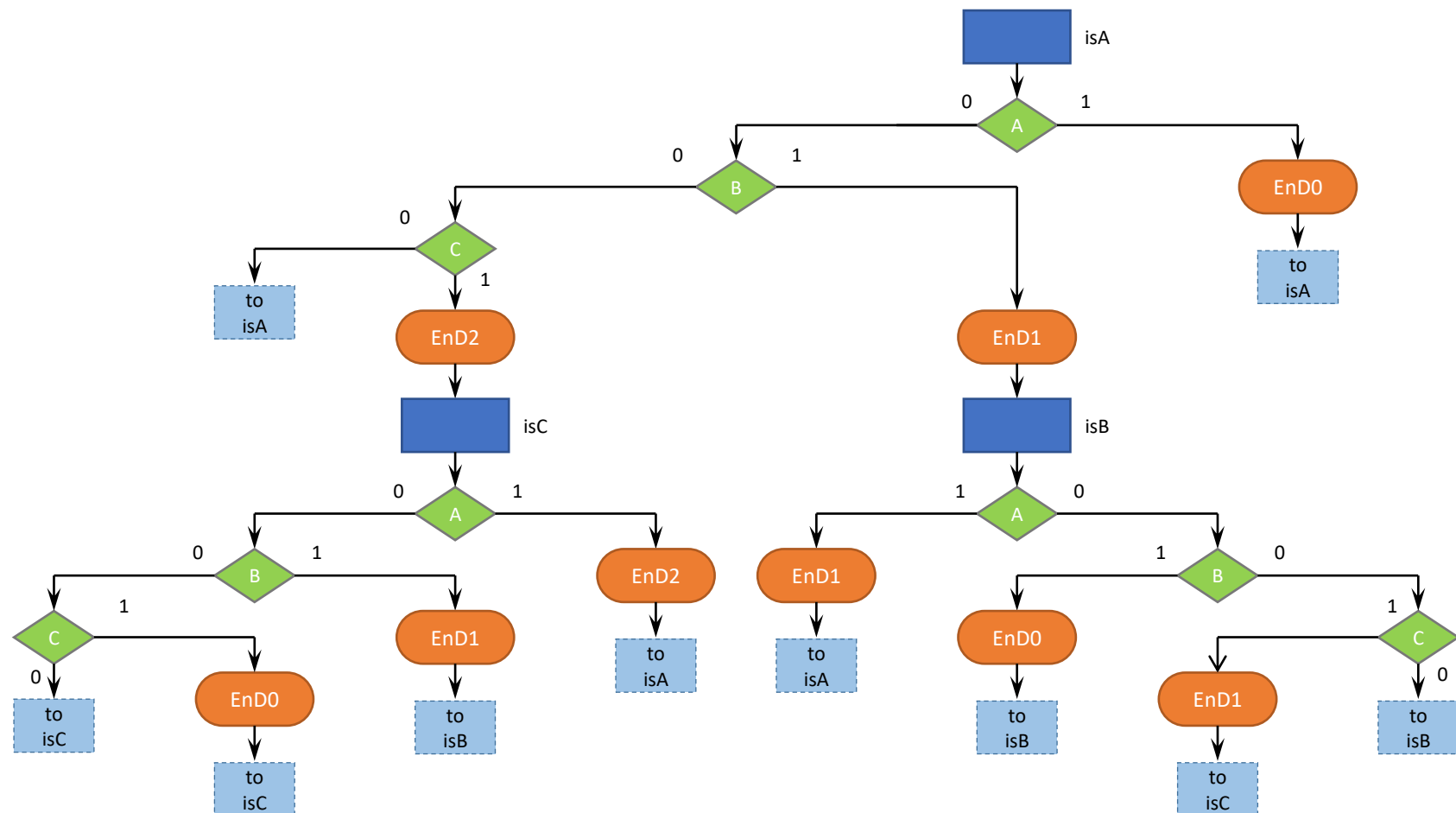
Funzionamento a regime

Da ogni stato si osservano A, B e C

- Sulla base dell'evento, si abilita il contatore corrispondente al conteggio
- Per esempio, da **isA**, se si vede di nuovo A, si abilita EnD0 (Tipo 0)
- Se invece si vede B, si abilita EnD1 (Tipo 1), e se si vede C si abilita EnD2 (Tipo 2)
- Da **isC**, se si vede C si abilita EnD0, per B EnD1 e per A EnD2

Uscite condizionate e transizioni

- Le uscite condizionate eseguono subito l'operazione e non necessitano di uno stato a parte
- La transizione poi termina nello stato **isA**, **isB** o **isC** a seconda dell'evento rivelato
- Per esempio, da **isA**, se si vede A si ritorna in **isA**, se si vede B si finisce in **isB**, e con C si finisce in **isC**
- E così per tutti gli altri stati

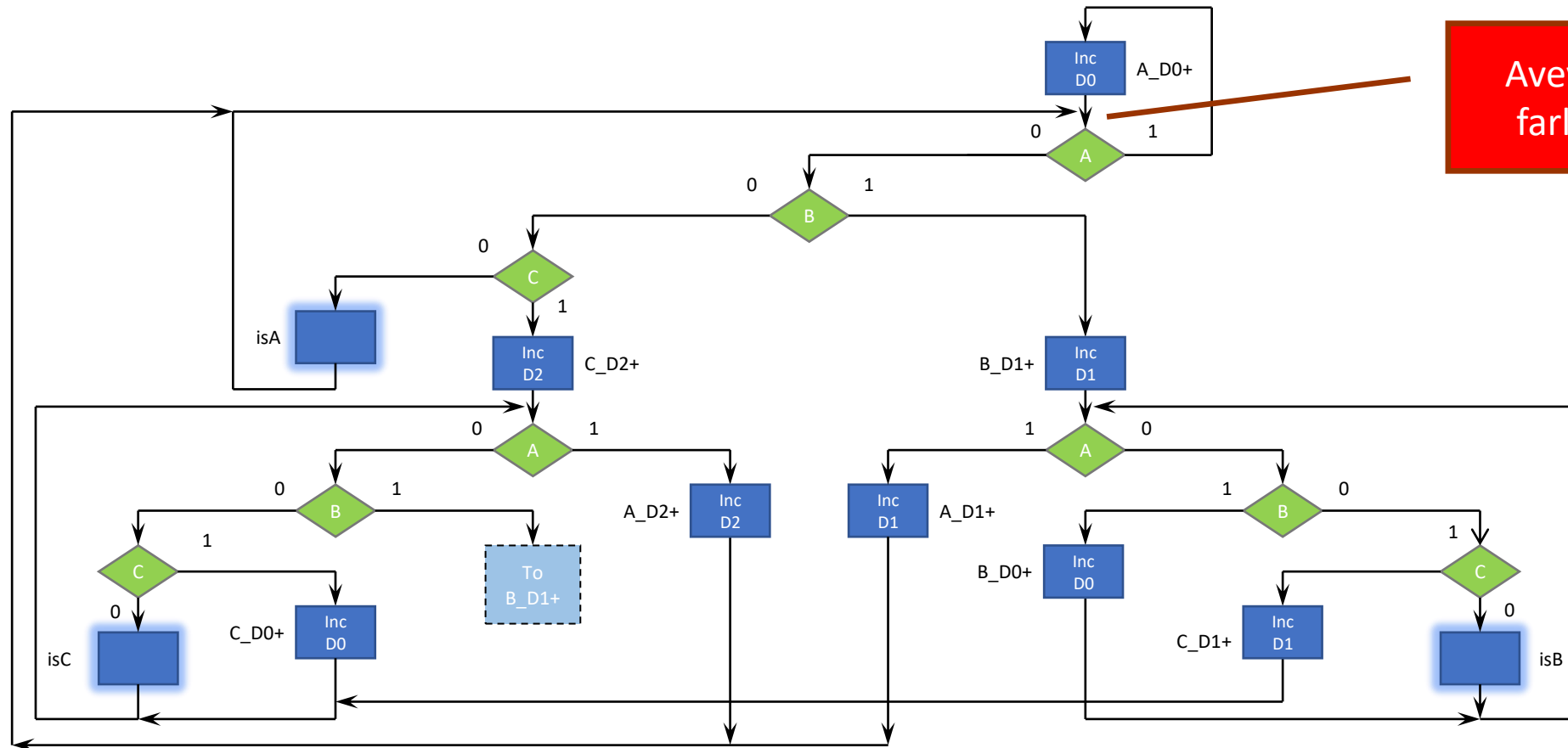


Macchina di Mealy

- Conveniente non tanto perché la risposta è più veloce, **ma perché permette di risparmiare stati**
- Dovendo essere pronti a rispondere già al ciclo successivo, una macchina di Moore dovrebbe duplicare gli stati per ricordarsi l'evento precedente e pilotare il corretto segnale di incremento sui contatori

Macchina di Moore

- Molti più stati (11 invece di 3)
- **isA** + 3 stati per dire di aver visto A e contemporaneamente incrementare D0, D1 o D2
- **isB** + 2 stati per dire di aver visto B e contemporaneamente incrementare D0 o D1
- **isC** + 3 stati per dire di aver visto C e contemporaneamente incrementare D0, D1 o D2
- Struttura delle transizioni decisamente più complessa: mi sono concesso la scorciatoia che dico sempre di non seguire!!



Avevo detto di non farlo! E ribadisco!

NOME e MATRICOLA: Aretha Franklin

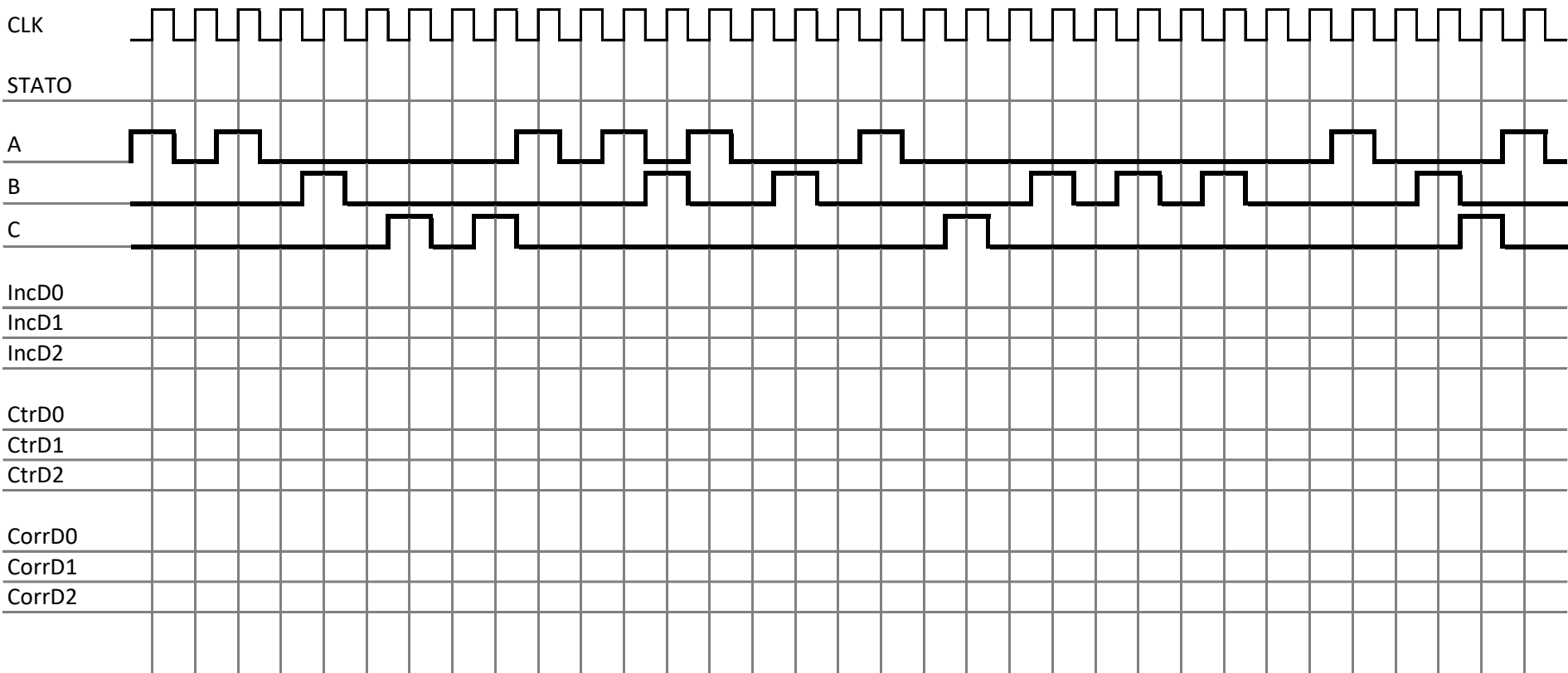


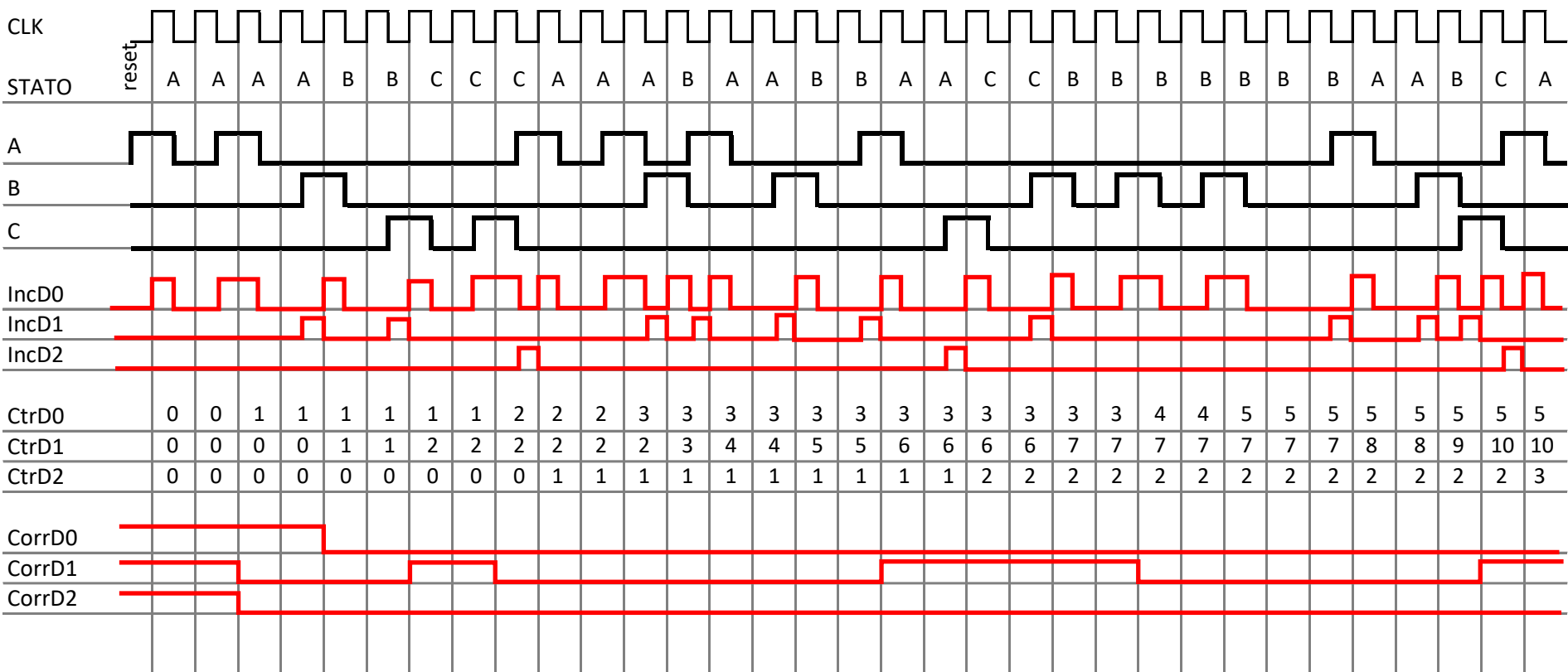
Diagramma temporale:

Vari eventi da tenere in considerazione

Fare molta attenzione a quando si attivano le uscite condizionate

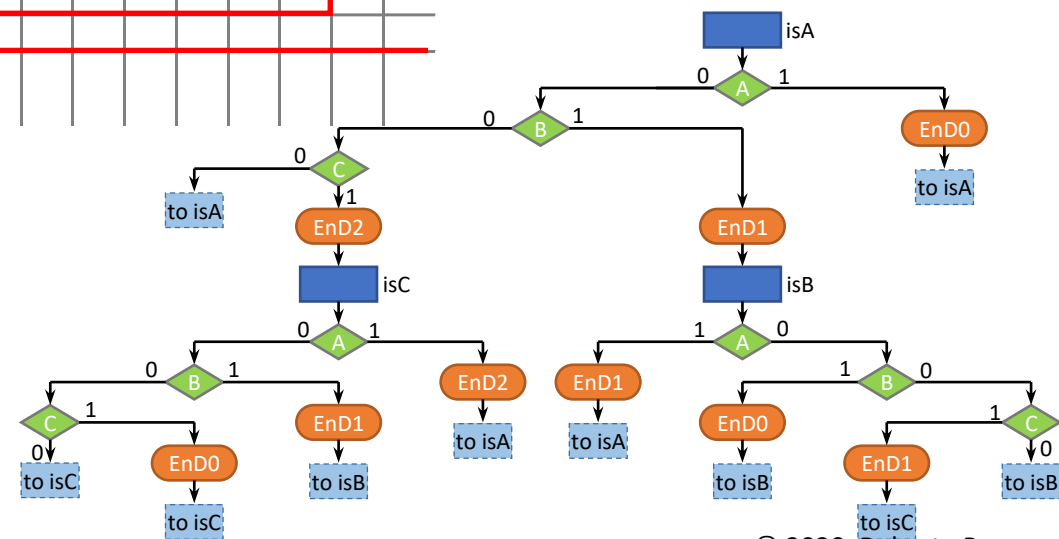
Per cambiare, i segnali dai sensori variano in mezzo al ciclo di clock

Indicare il valore contenuto nei contatori nella riga corrispondente

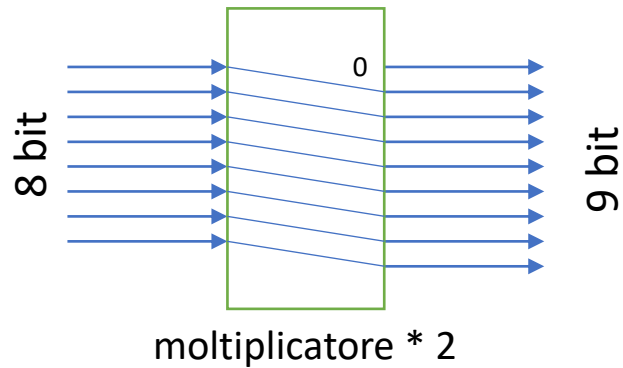
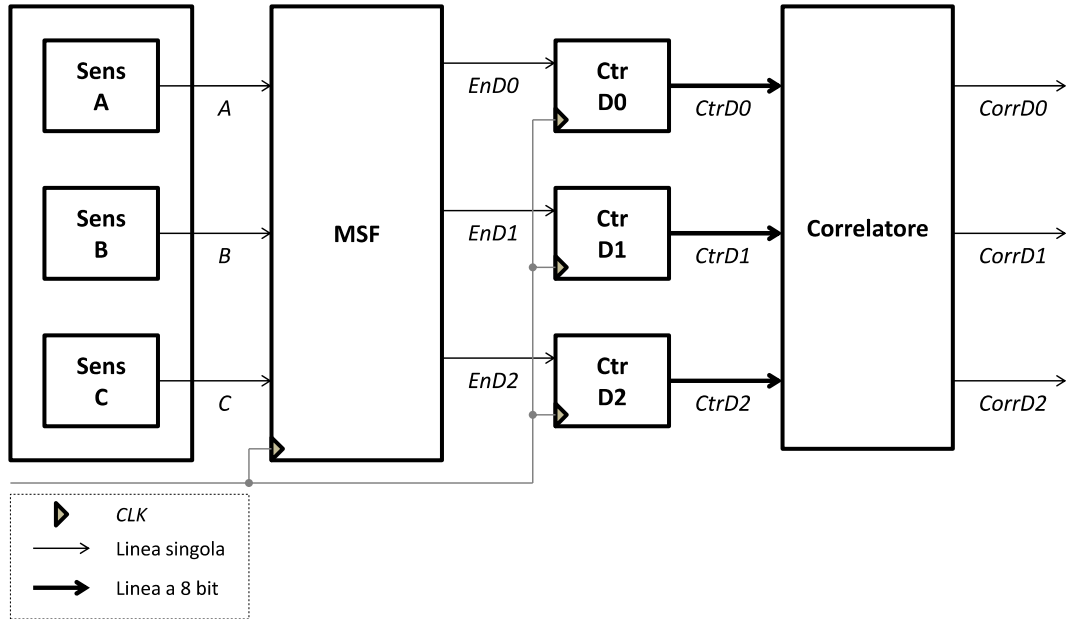


Uscite condizionate

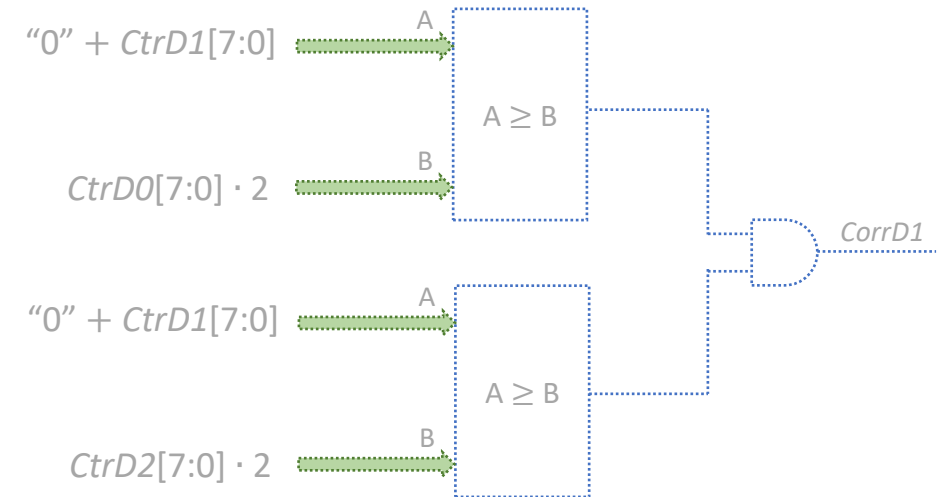
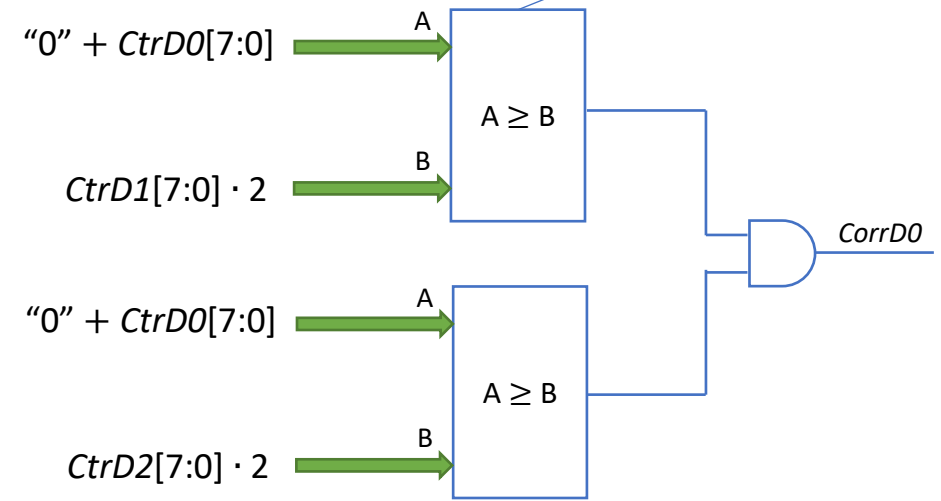
- Si attivano appena il segnale del rivelatore si attiva
- **Attenzione che cambiano anche al fronte del clock, perché la macchina a stati cambia di stato!!**
- L'uscita ha effetto sui contatori solo se è attiva alla fine del ciclo di clock



Correlatore

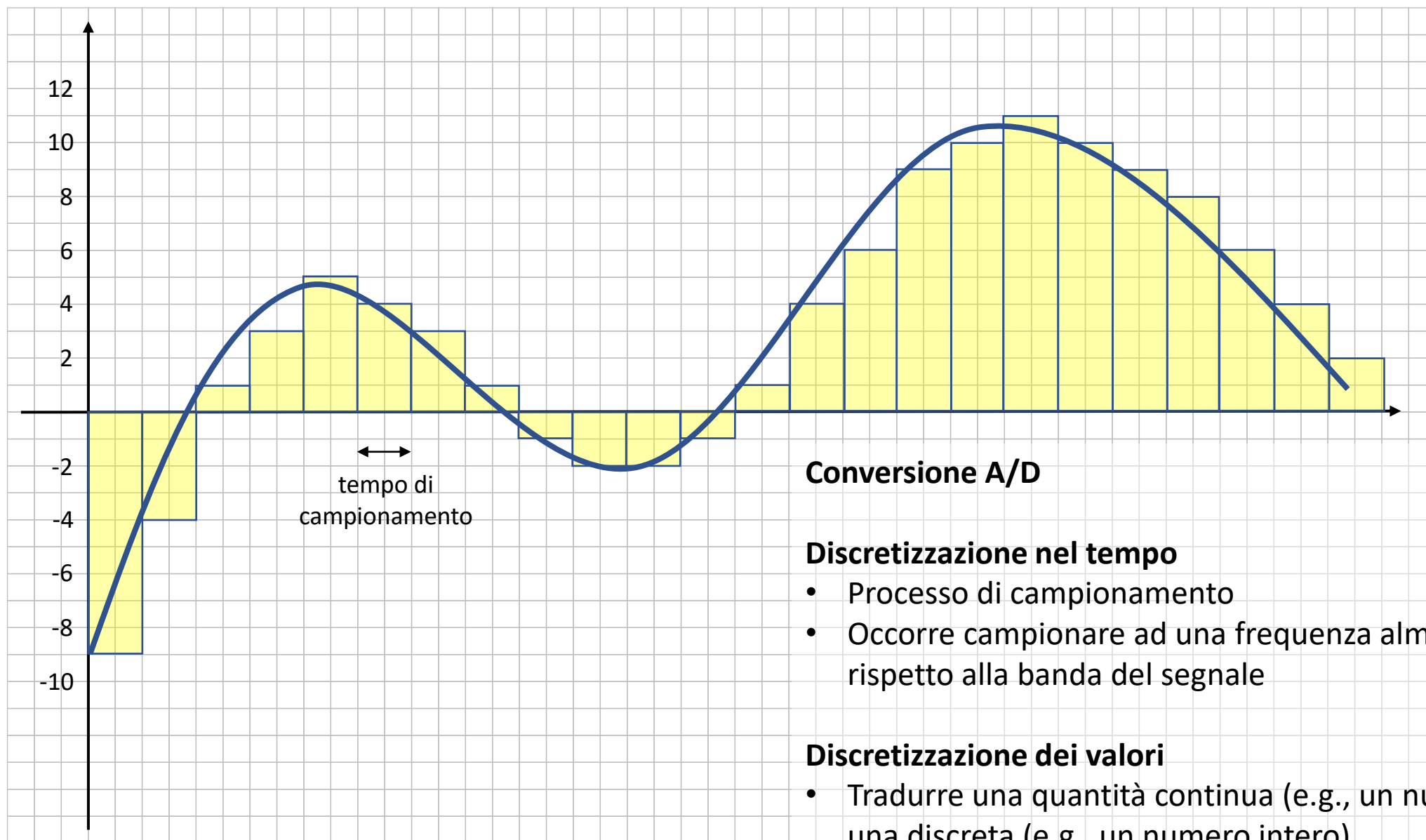


comparatore 9 bit



Convertitore Analogico/Digitale

Tema d'esame del giugno 2011



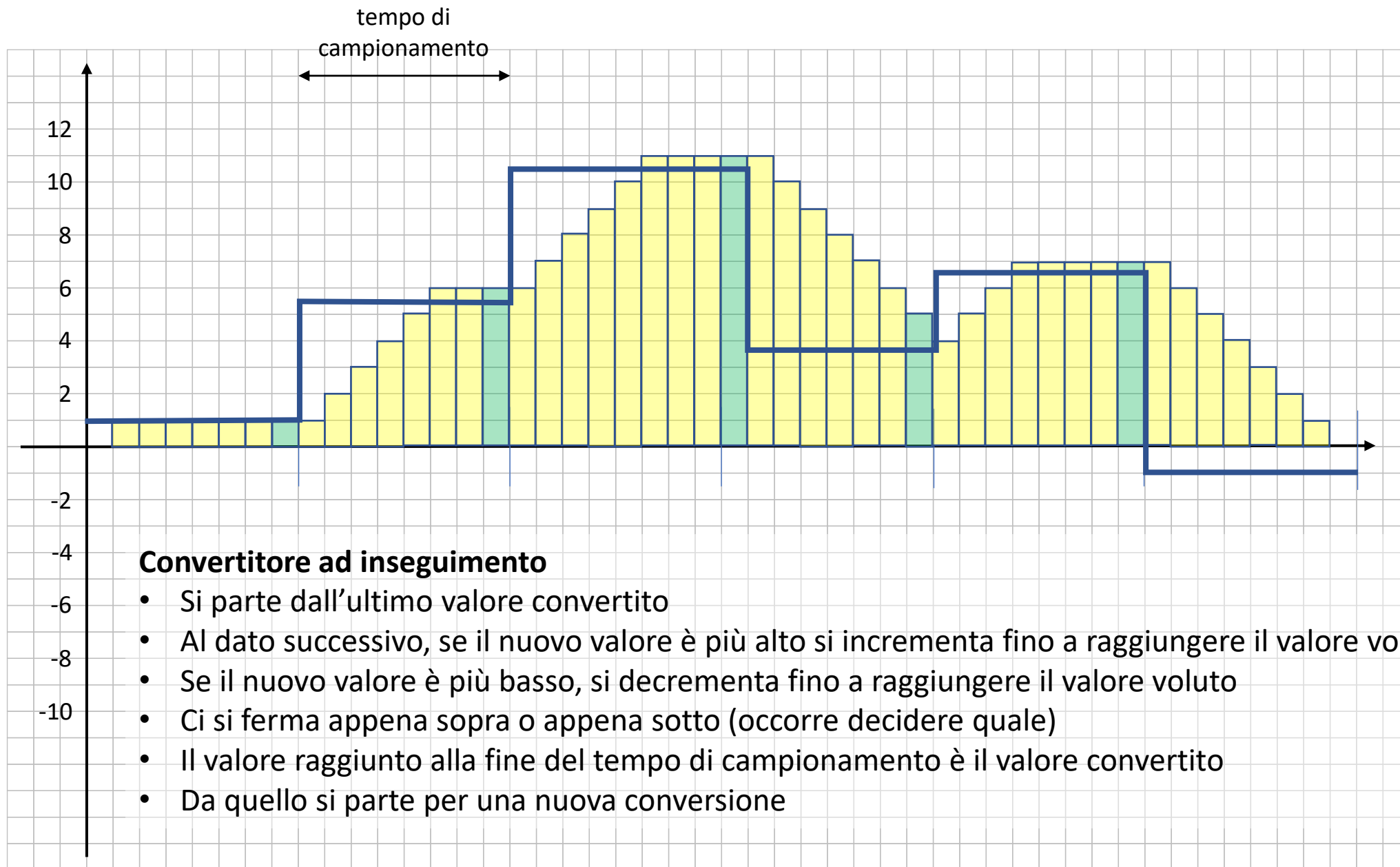
Conversione A/D

Discretizzazione nel tempo

- Processo di campionamento
- Occorre campionare ad una frequenza almeno doppia rispetto alla banda del segnale

Discretizzazione dei valori

- Tradurre una quantità continua (e.g., un numero reale) in una discreta (e.g., un numero intero)
- Ci occuperemo di questo secondo processo



Convertitore Digitale/Analogico

- Il convertitore digitale analogico trasforma il valore digitale contenuto nel contatore in un livello di tensione continua
- Per esempio il valore 128 potrebbe corrispondere a 0.5 V, 255 a 1 V
- Generalmente più semplice da progettare di quello analogico/digitale

Contatore dato corrente (up/down)

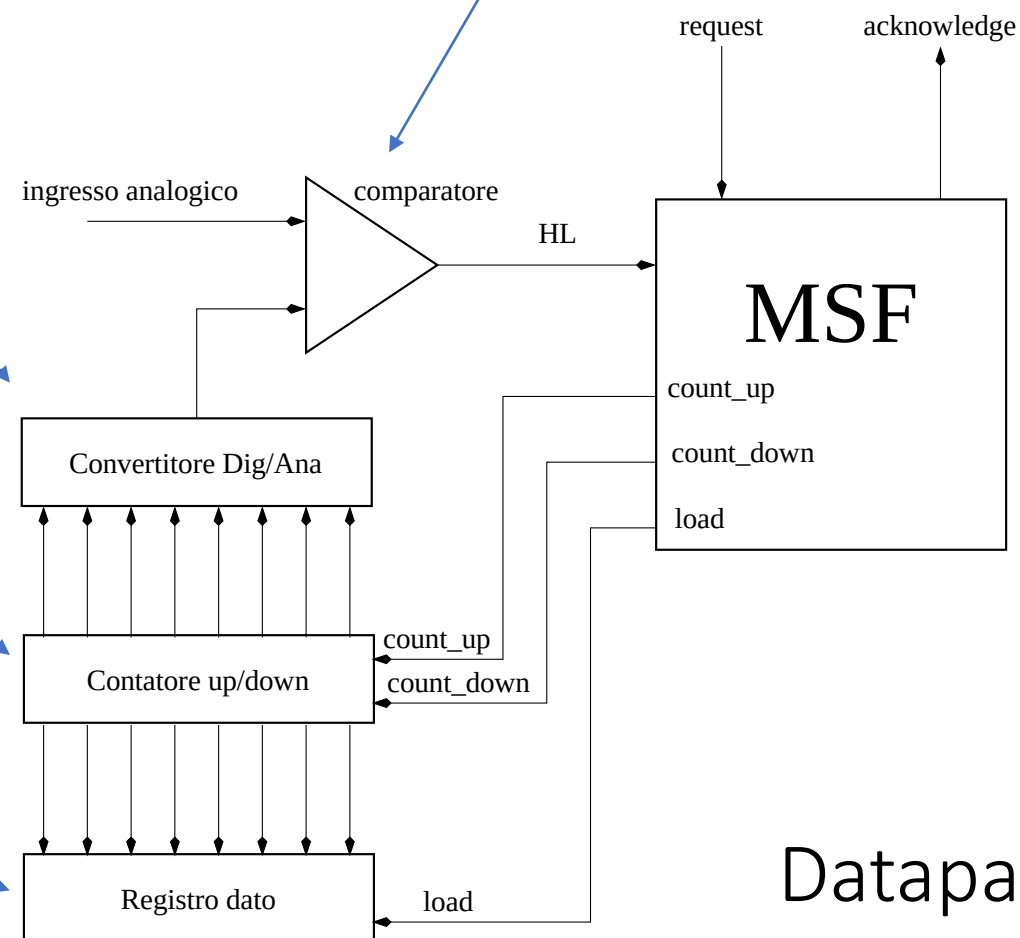
- Il contatore registra il valore corrente della conversione
- E' un contatore *avanti-indietro*, cioè in grado di incrementare e decrementare
- I due count enable **count_up** e **count_down** comandano quando incrementare o decrementare

Registro di dato

- Non si vuole rendere visibile il valore del contatore mentre conta durante la conversione
- Il registro di dato viene usato per memorizzare il valore valido dell'ultima conversione eseguita

Comparatore analogico

- Un comparatore analogico confronta il valore (analogico) del contatore con il valore del segnale, già campionato
- L'uscita digitale **HL** sarà 1 se l'ingresso analogico è maggiore o uguale a quello digitale, 0 altrimenti

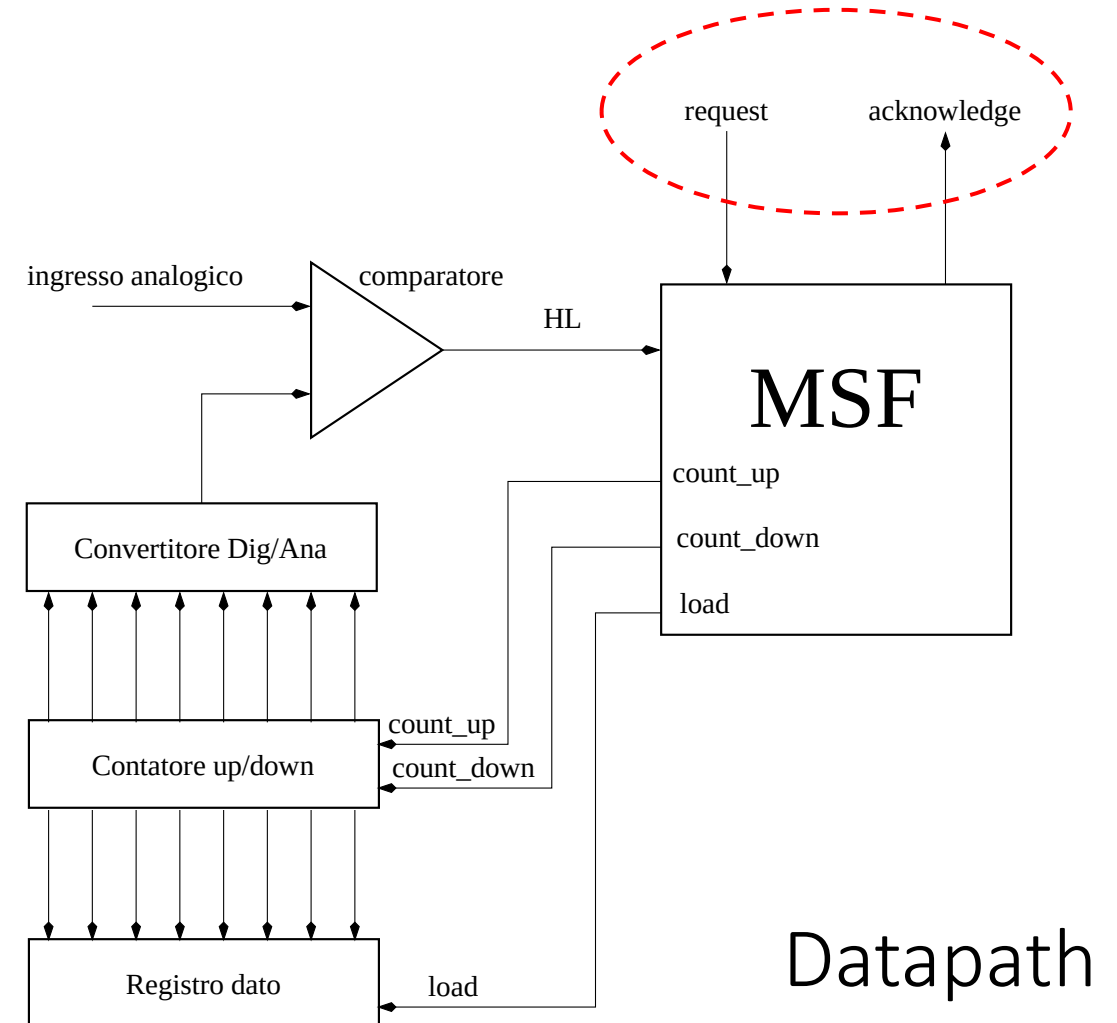
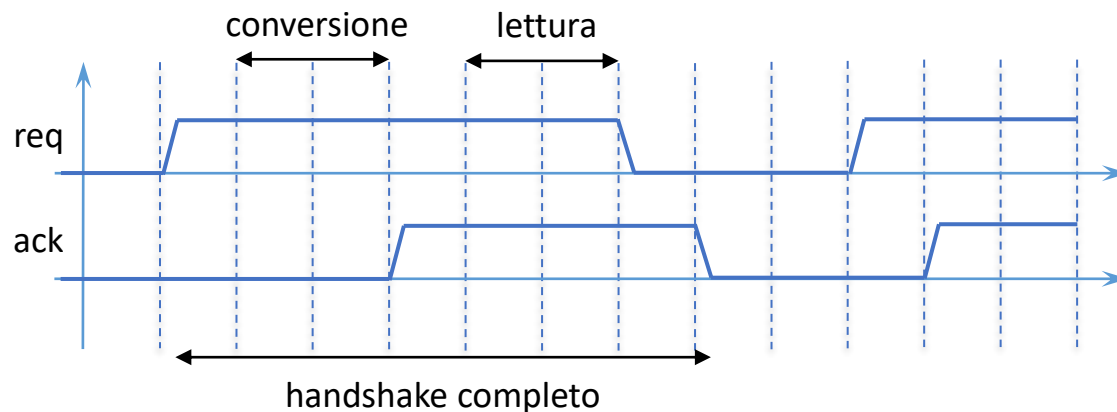


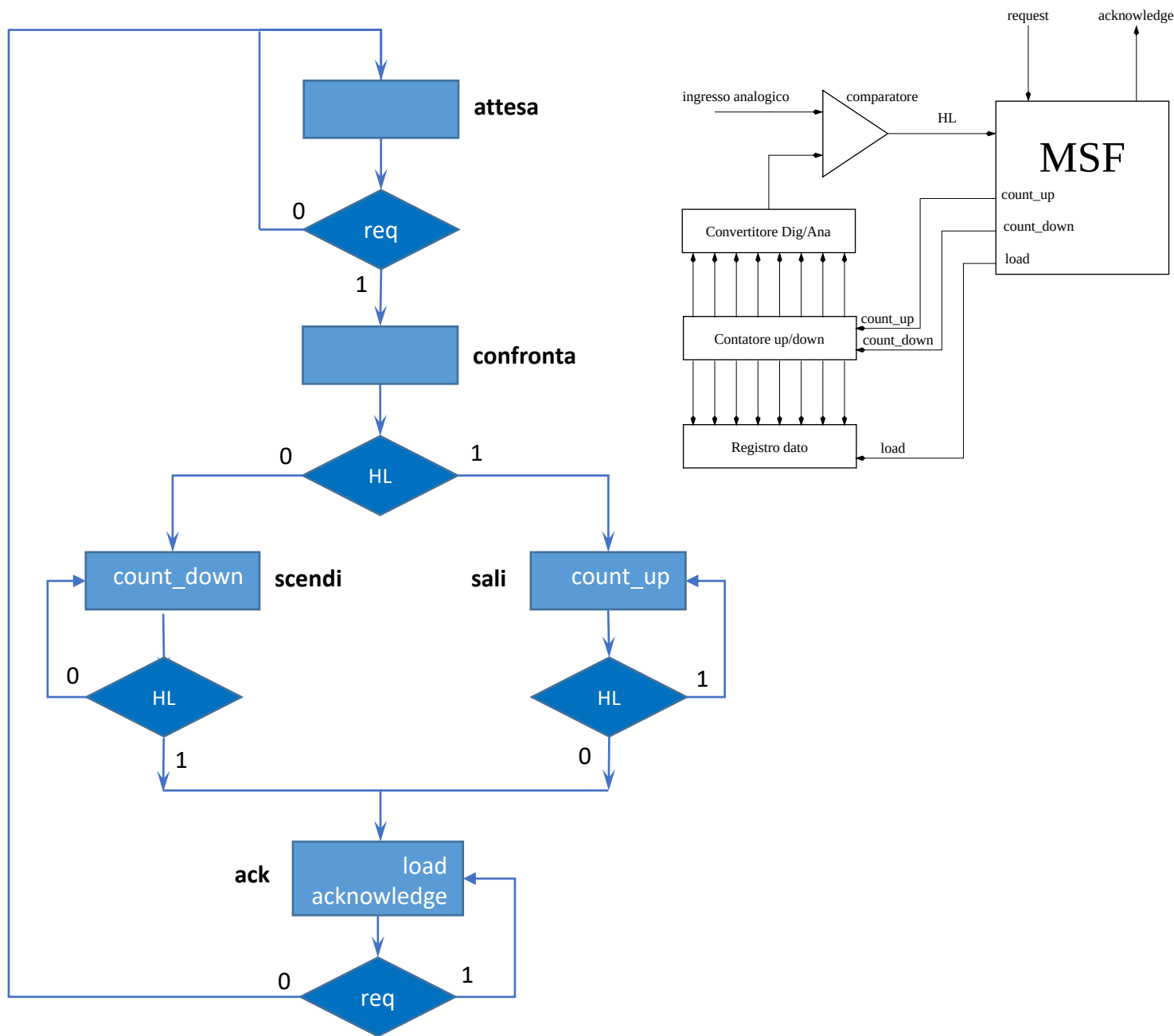
Datapath

MSF: ingressi e uscite di controllo

Handshake completo

- I segnali di **request** e **acknowledge** vengono usati per costruire un handshake completo
- Quando **request** va a 1, viene richiesta una nuova conversione
- La MSF esegue la conversione pilotando il contatore sulla base dell'uscita del comparatore
- Terminata la conversione (quando termina?), la MSF pone **acknowledge** a 1
- Il risultato viene letto e **request** viene messo a 0
- Dopo che **request** va a 0, la MSF può mettere **acknowledge** a 0
- Non è permesso all'utente esterno rimettere **request** a 1 se **acknowledge** non è ancora andato a 0





Stato attesa

- Aspetta che **request** vada a 1

Stato confronta

- Guarda il valore di **HL** per decidere se incrementare o decrementare il contatore
- Se **HL** = 0 occorre decrementare, altrimenti incrementare

Stato scendi

- Decrementa il contatore (**count_down**)
- Continua a decrementare fin tanto che il contatore è maggiore del segnale analogico, i.e., fin tanto che **HL** = 0

Stato sali

- Incrementa il contatore (**count_up**)
- Continua a decrementare fin tanto che il contatore è minore o uguale del segnale analogico, i.e., fin tanto che **HL** = 1

Stato ack

- Comanda il caricamento del dato nel registro
- Attiva **acknowledge**
- Attende che **request** torni a 0
- Il segnale **acknowledge** andrà a 0 quando si torna in attesa

Nome: Appello di Giugno 2011

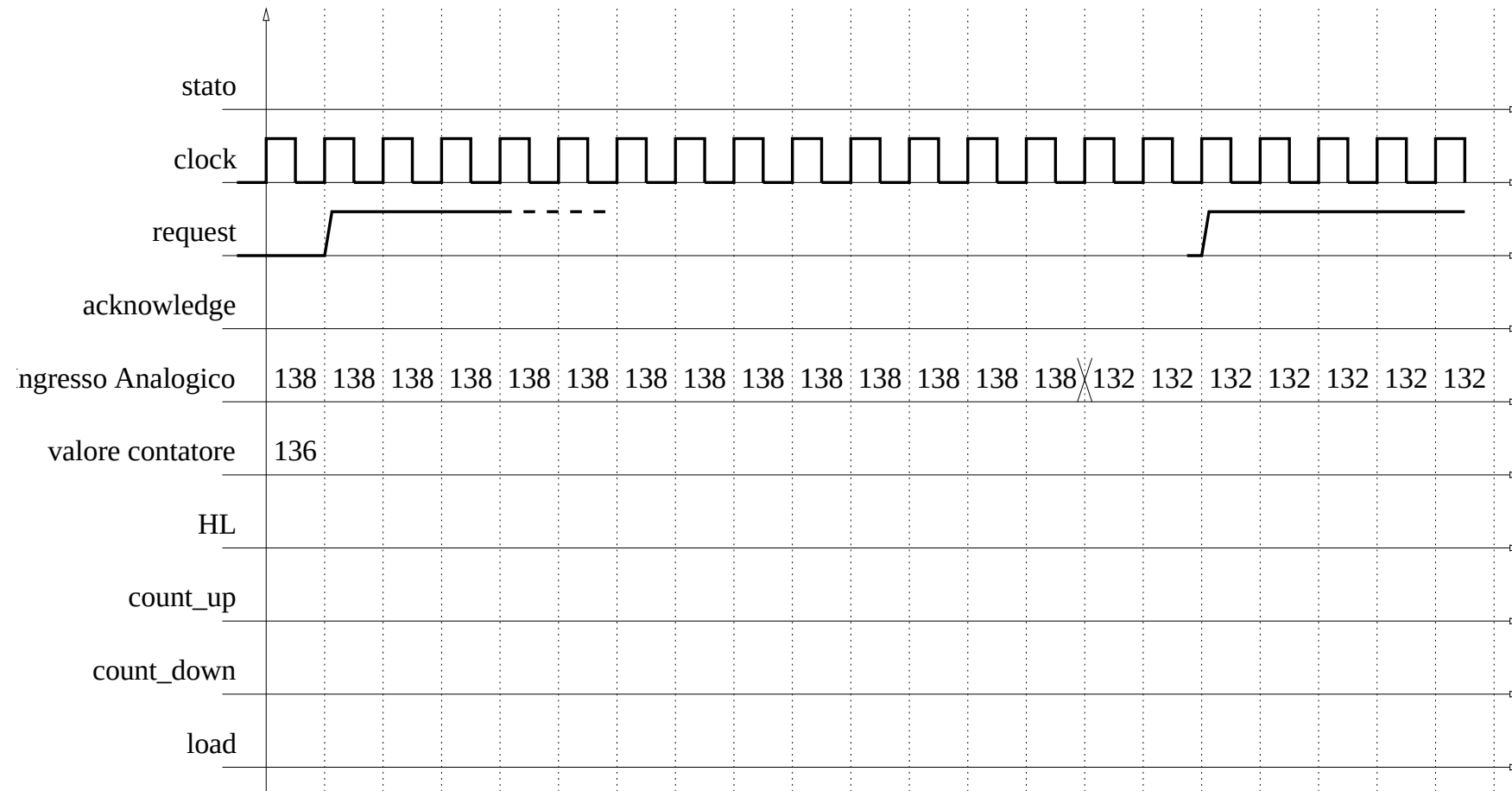


Diagramma temporale:

Il contatore comincia al valore 136

Ingresso analogico al valore 138

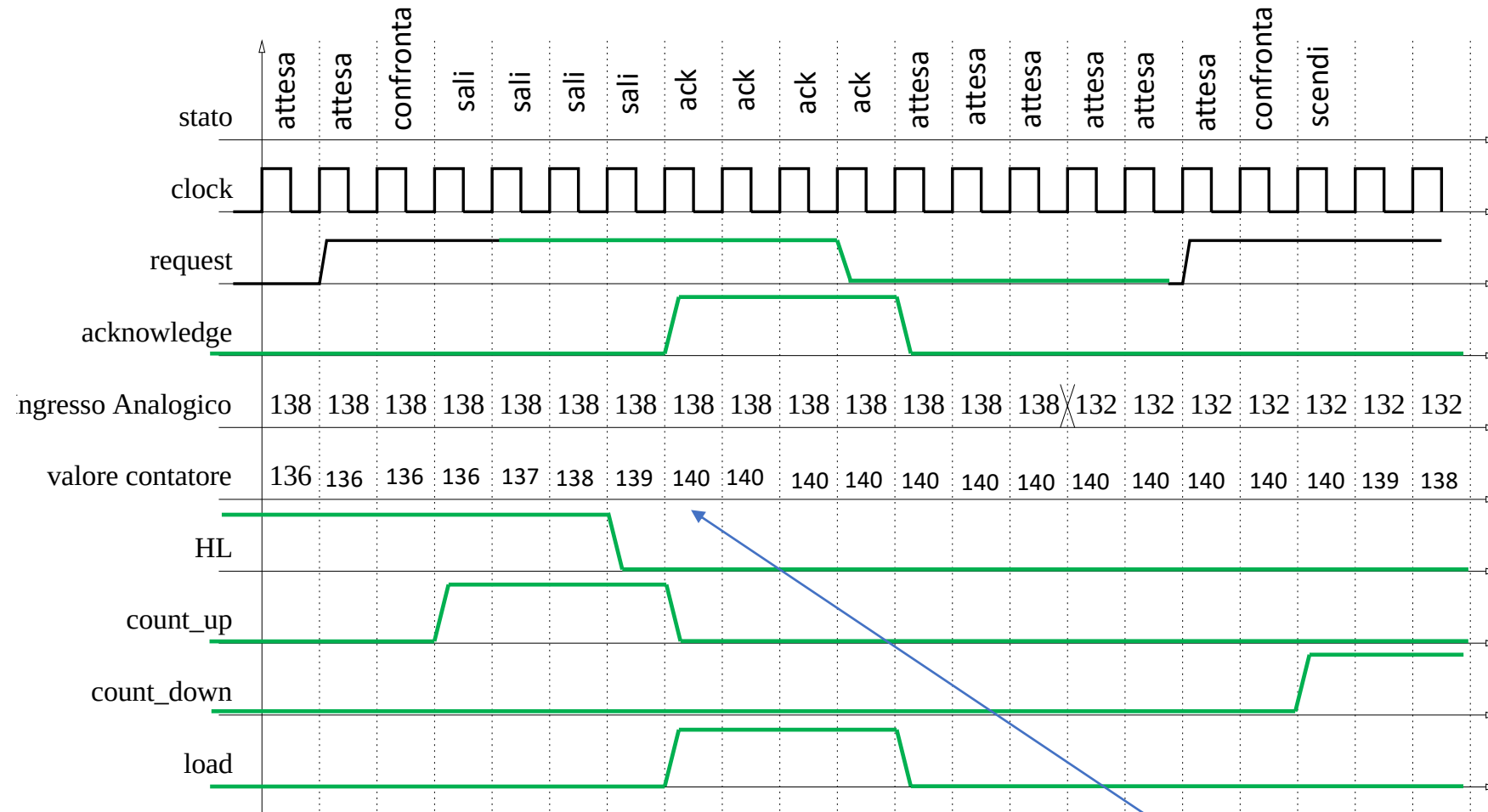
Request viene attivato al secondo ciclo

Far eseguire la conversione, e fare scendere il request al momento opportuno

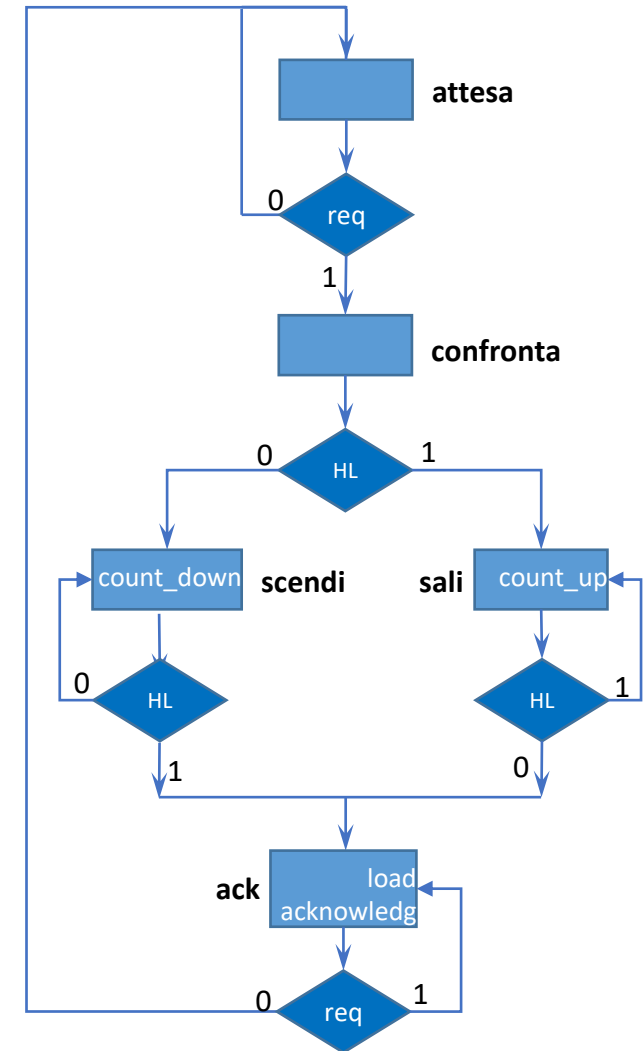
Ingresso analogico scende a 132 e viene richiesta una nuova conversione

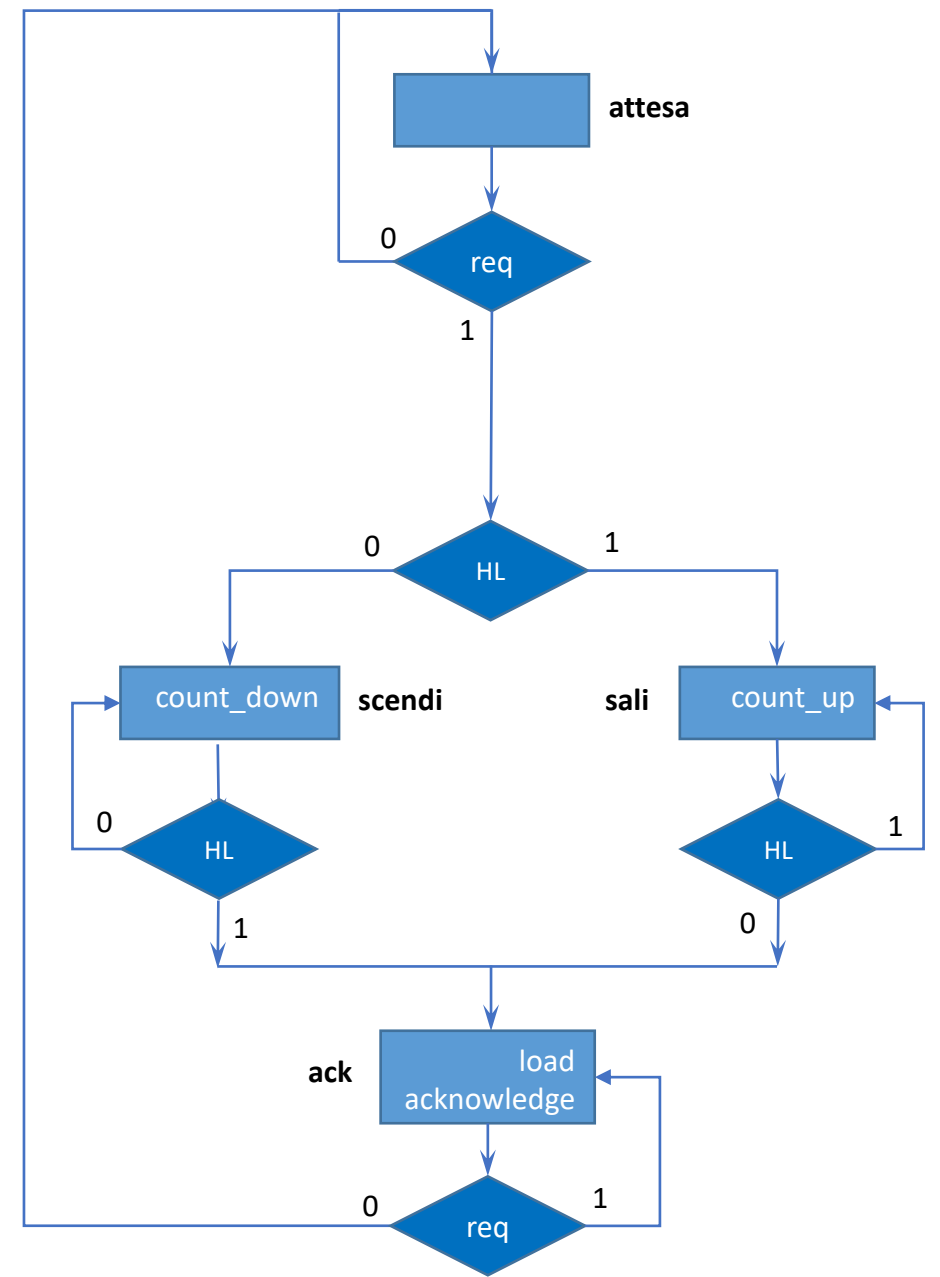
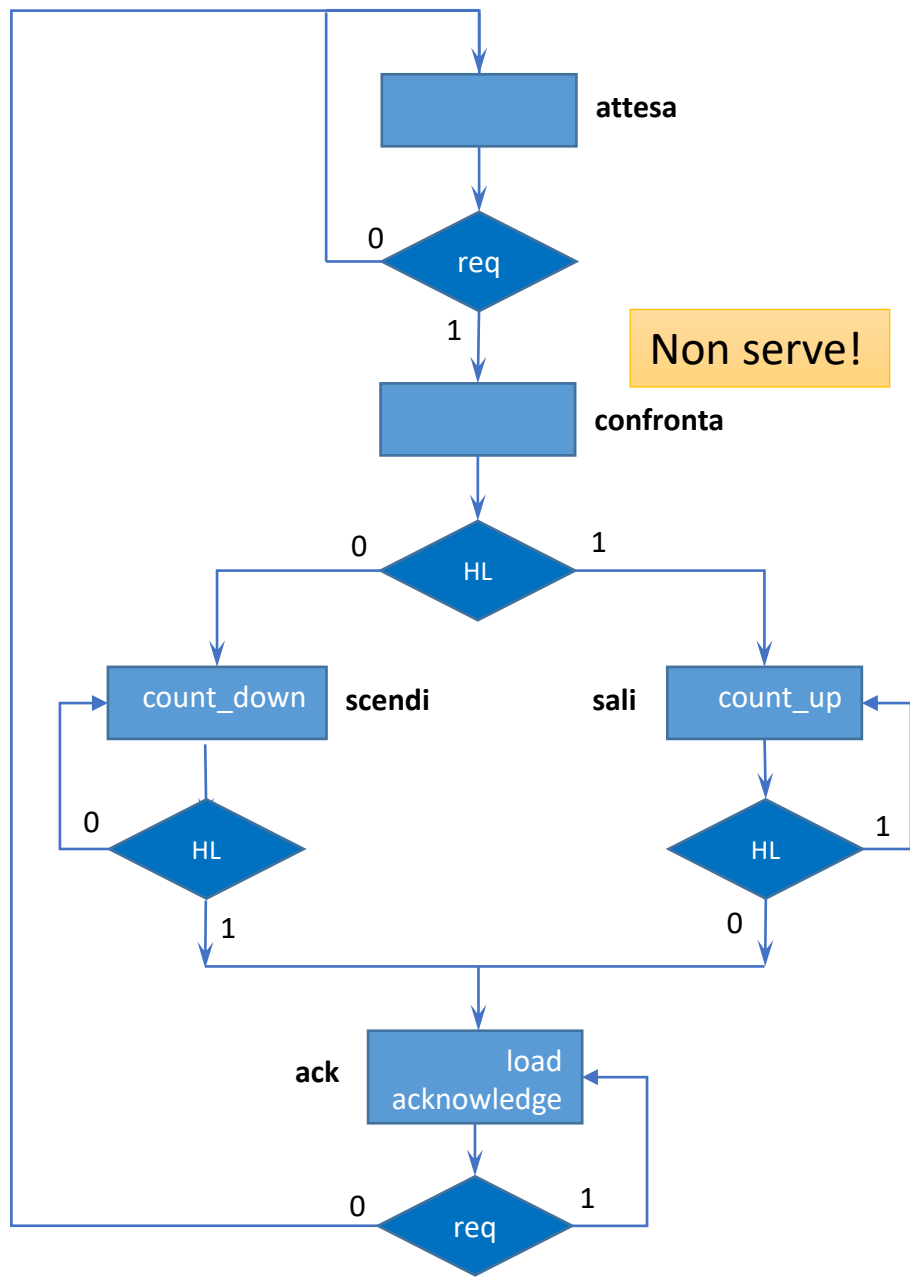
Nome: Albert Einstein

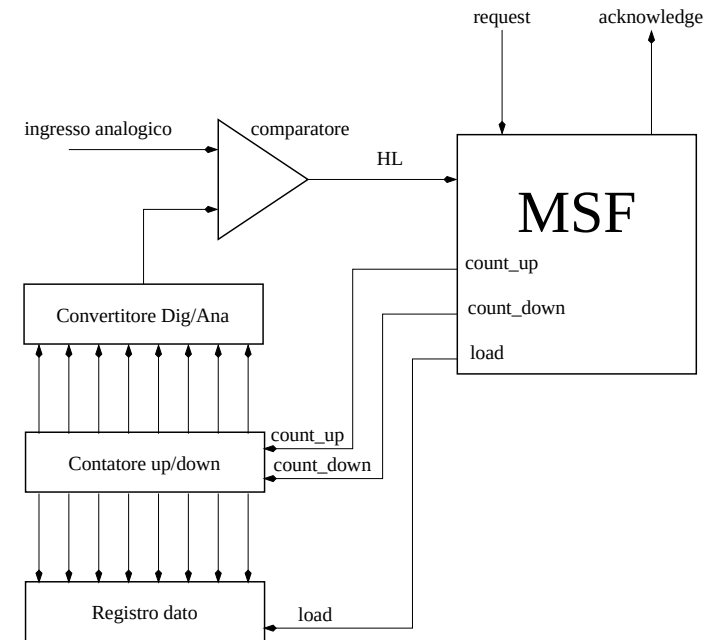
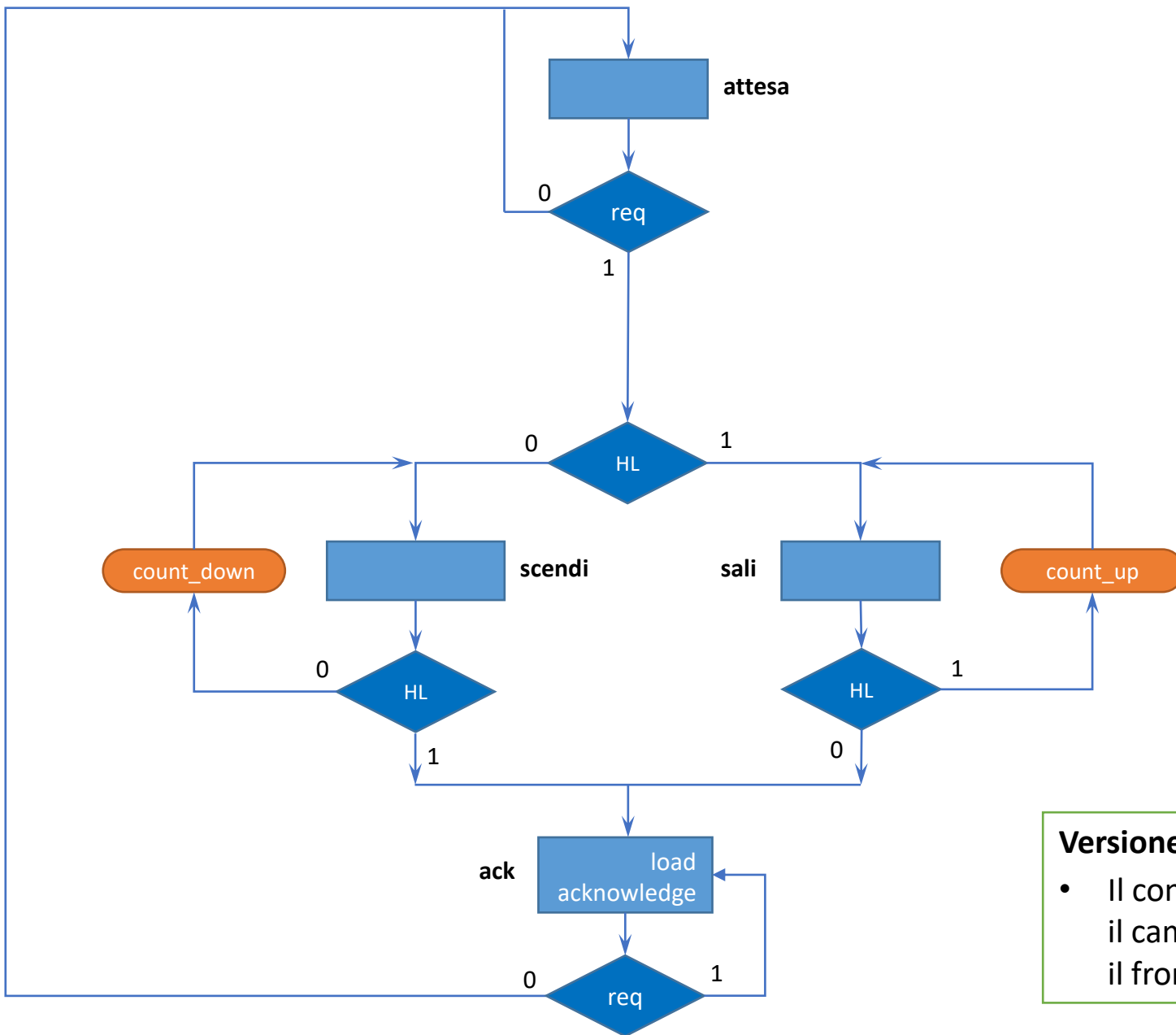
Appello di Giugno 2011



La combinazione di come funziona il comparatore e della velocità della macchina a stati fanno andare il valore del contatore a 140





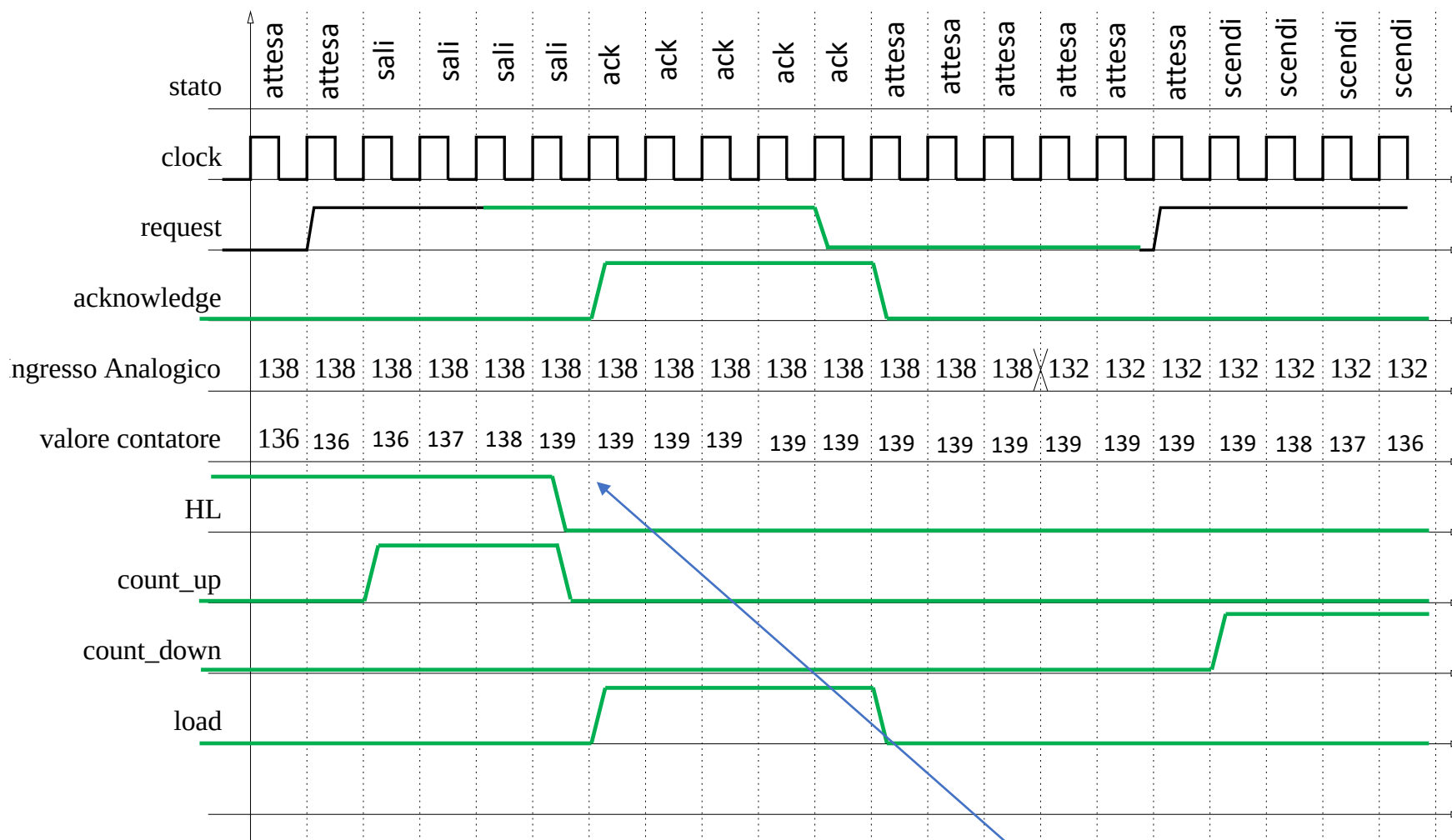


Versione di Mealy

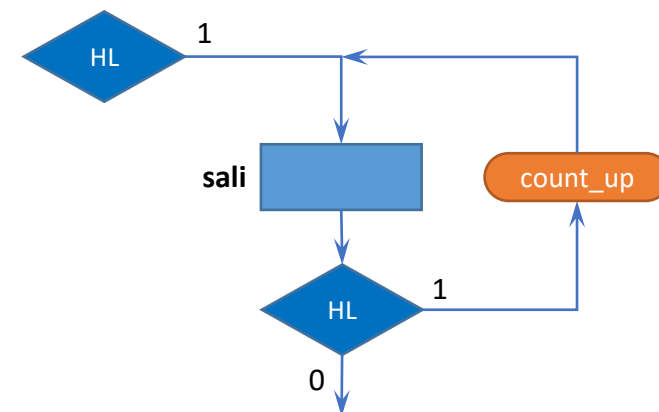
- Il conteggio viene terminato non appena si vede il cambiamento del segnale HL, senza aspettare il fronte del clock

Nome: Albert Einstein

Appello di Giugno 2011



Si reagisce più in fretta e ci si ferma a 139 invece di 140



Prestazioni

- A che velocità deve andare il clock per una comunicazione telefonica?
 - Campionamento a 8 kHz
 - Discretizzazione a 8 bit (256 livelli)
- E per un CD?
 - Campionamento a 44.1 kHz
 - Discretizzazione a 16 bit (65536 livelli)
- E per audio professionale?
 - Campionamento a 192 kHz
 - Discretizzazione a 24 bit (16,777,216 livelli)
- Come modificare il circuito per gestire questi casi?

Frequenza di clock: telefono

- A 8 kHz abbiamo 125 μ s di tempo per la conversione
- Con 256 livelli potenzialmente abbiamo bisogno di 256 cicli per la conversione
- Ogni ciclo deve essere $125 / 256 = 0.488 \mu$ s
- Frequenza clock di 2.048 MHz ($= 8 \cdot 256$)

Frequenza di clock: CD

- A 44.1 kHz abbiamo 22.675 μ s di tempo per la conversione
- Abbiamo bisogno di 65536 cicli per la conversione (caso peggiore)
- Frequenza clock di 2.89 GHz ($= 44.1 \cdot 65536$)

Esistono molte architetture di conversione diverse. Il convertitore ad approssimazioni successive funziona in modo simile a quello visto, ma richiede per ogni conversione un numero di cicli pari al numero di bit.

Per il CD il clock andrebbe a 705.6 kHz



Interfaccia RS232

Tema d'esame del gennaio 2013

Interfaccia RS232

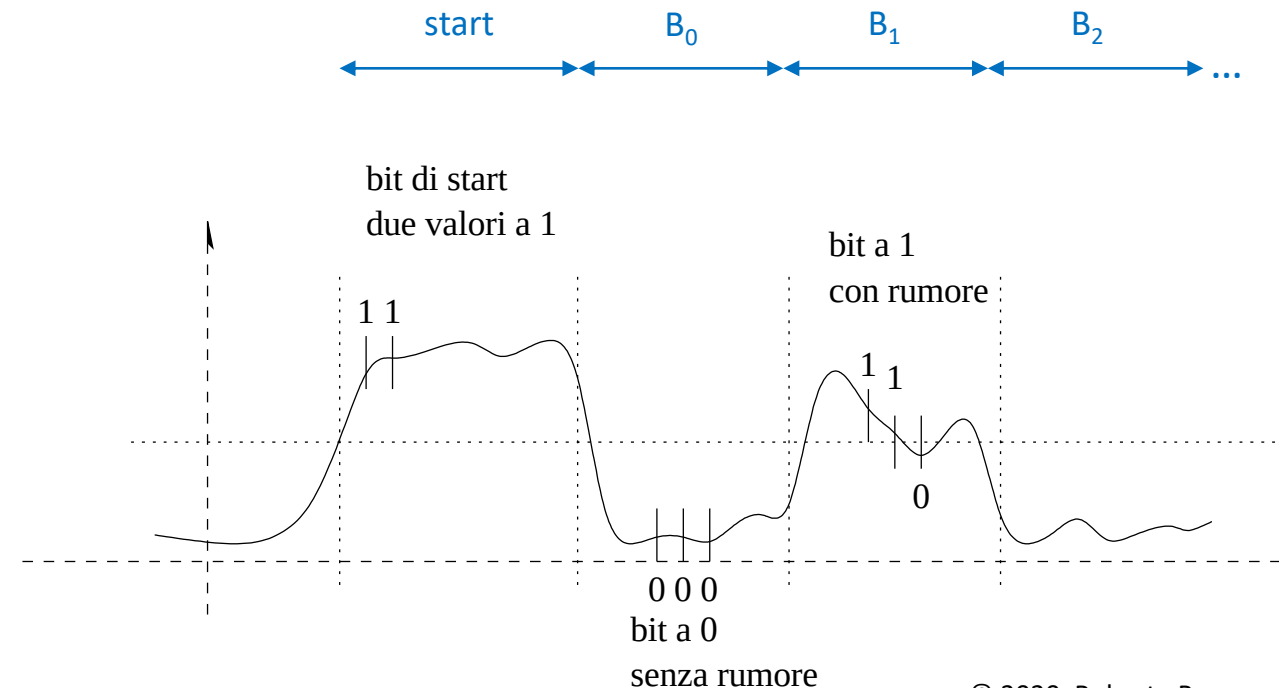
- Primo esempio di sistema di comunicazione seriale
- Precursore dell'USB
- Caratterizzato da **segnale sbilanciato** tra -12 e +12 V, **asincrono**, semplice da realizzare ma molto sensibile al rumore (anche con un cavetto schermato)

Misure per ridurre l'effetto del rumore

- Il circuito di interfaccia campiona il valore del segnale della RS232 più volte per ogni bit
- Per semplificare, **il bit di start deve essere visto a 1 per almeno due campionamenti consecutivi**
- Il bit di start consente anche di sincronizzare la trasmissione con la ricezione (chi riceve sa quando partire)
- **I bit successivi vengono invece campionati 3 volte nell'intorno del centro presunto del bit**
- Se tutti i valori sono a 0 o tutti a 1, il bit è considerato 0 o 1 correttamente
- Se invece non sono concordi, **si considera la maggioranza**, ma si indica anche su un segnale apposito la presenza di rumore

Per saperlo:

Tipicamente, nella RS232 si usa un bit di start, 8 bit di dato, un bit di parità, ed un bit di stop



Realizzazione

- L'interfaccia riceve e campiona il segnale dalla RS232 in arrivo sull'ingresso L
- Si assuma che L venga campionato con un clock a frequenza pari a 8 volte quella del segnale RS232: ogni bit su L corrisponde a 8 cicli di clock della MSF
- Significa che ogni bit viene campionato 8 volte

Cosa fare:

- Attendere il bit di start (devono essere due 1 consecutivi)
- Per ogni bit successivo, guardare i 3 campioni in mezzo al bit e decidere il valore finale **calcolando la maggioranza**
- Inserire (**shift_en**) ogni bit così determinato nel *data shift register* tramite l'uscita **Q** della MSF
- Per ogni bit, indicare nel *noise shift register* se si è rilevato del rumore (dati non concordi)
- Al termine della trasmissione, il dato potrà essere letto in forma parallela dal *data shift register*

Separazione dei compiti

- La macchina a stati si occupa di rilevare il bit di start, di campionare il segnale e di determinare la maggioranza
- Il componente *synch_gen* dice alla MSF quando campionare (segnale **SS**: start sampling) e quando siamo arrivati alla fine del dato (**TC**)
- Occorre campionare L nei tre fronti di clock **successivi** a quelli in cui si vede **SS = 1**
- *synch_gen* viene innescato dalla macchina a stati quando vede il bit di start attivando il segnale di **init**.

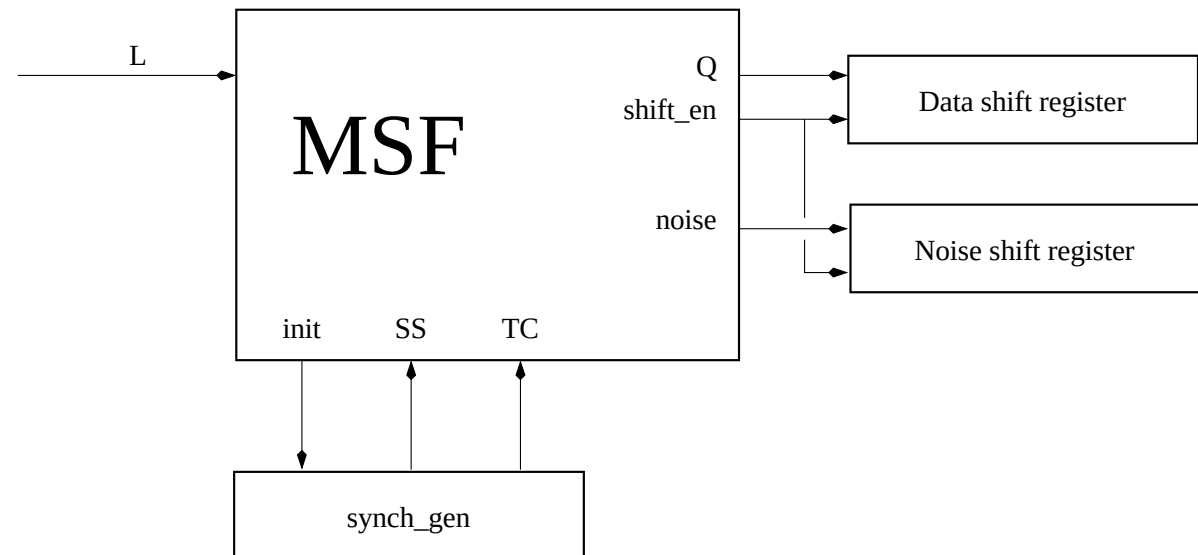
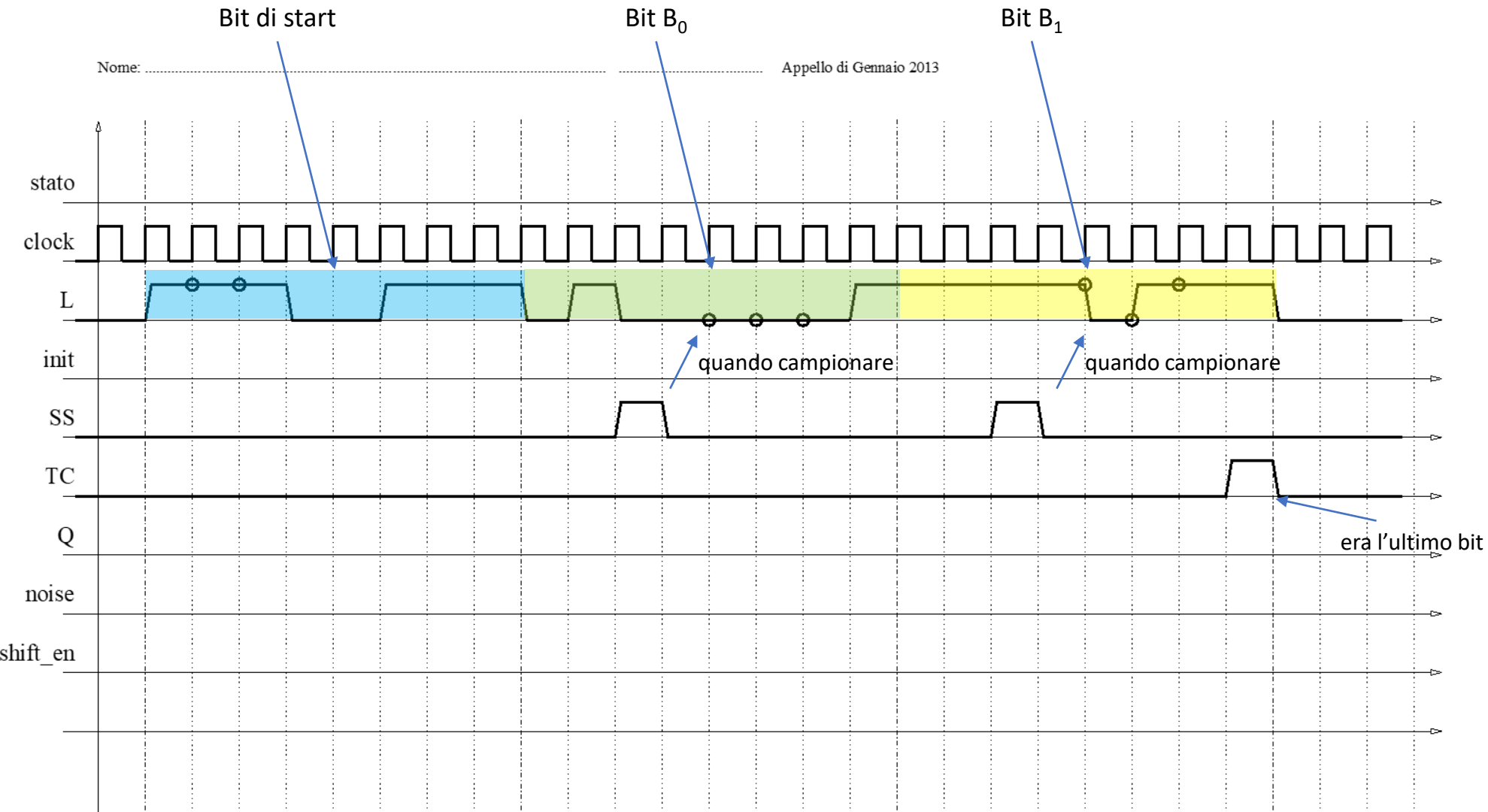


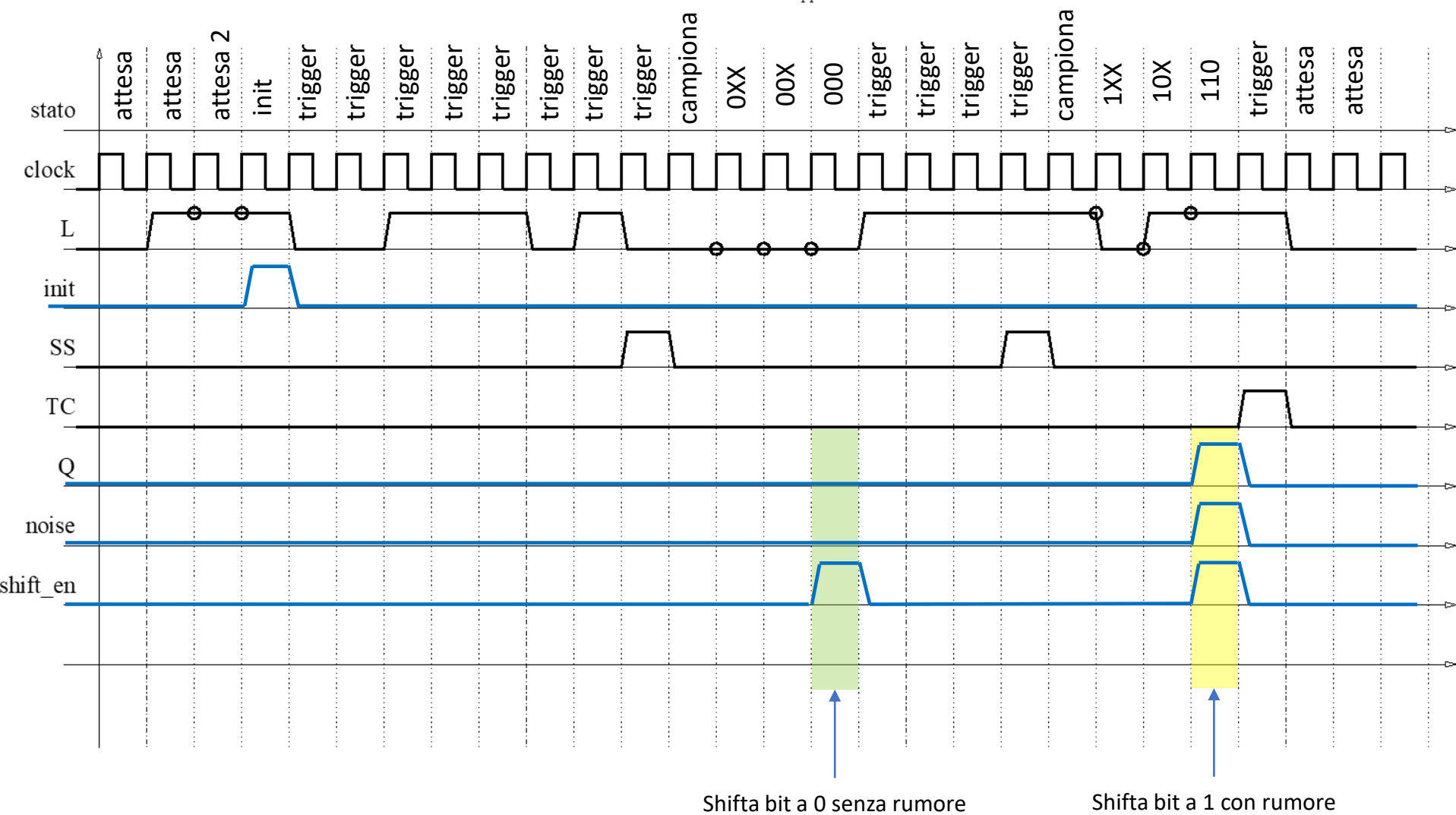
Diagramma temporale:

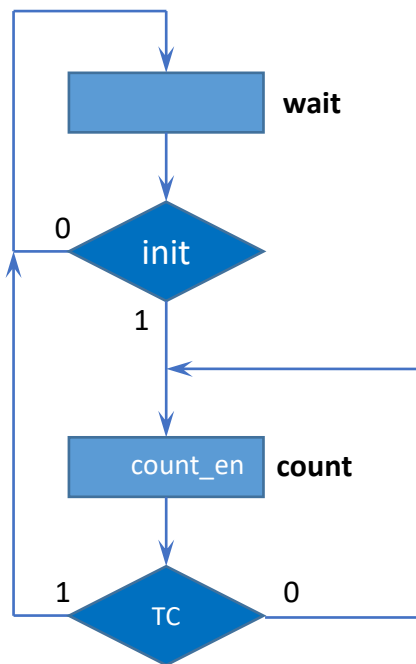
Viene trasmesso un bit di start seguito da due bit di dato

Rumore sul secondo bit
Rumore anche sul bit di start, ma non lo guardiamo

synch_gen fornisce le temporizzazioni giuste per quando guardare il valore di L





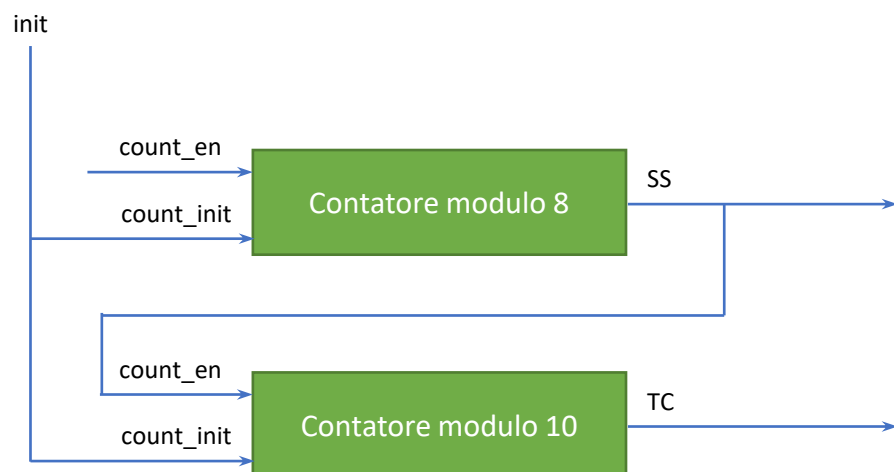


Generatore di sincronismi

- La macchina a stati si interfaccia con la MSF del progetto precedente tramite i segnali **init** e **TC**
- Abilita i due contatori al conteggio tramite **count_en**

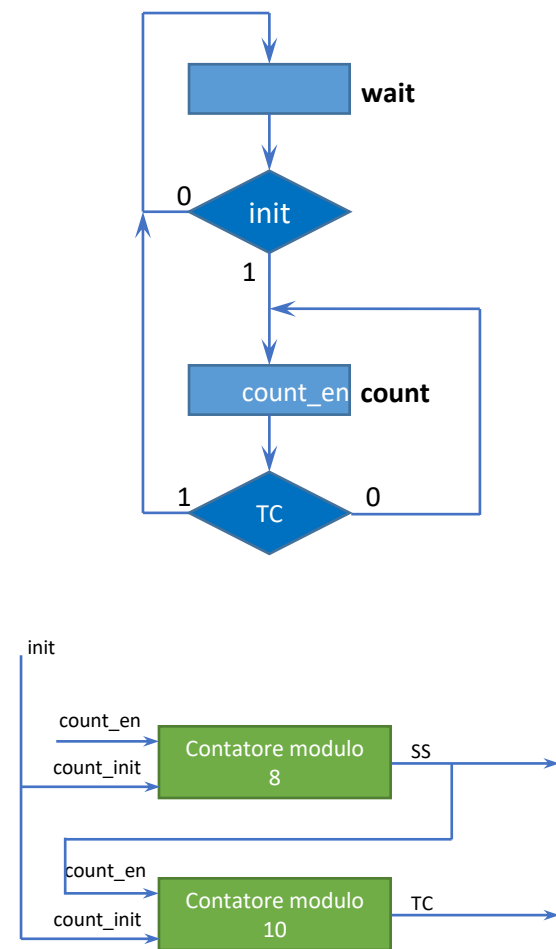
Contatori

- Il primo contatore conta i campionamenti di ciascun bit della trasmissione seriale (8 campionamenti per bit)
- Il secondo contatore conta i bit (8 bit di dato, un bit di parità e un bit di stop = 10)
- Secondo contatore attivo nel ciclo di clock in cui SS è attivo
- **TC** attivo per più tempo rispetto a quanto mostrato nel diagramma temporale



Si ipotizza il TC al valore 2 del secondo contatore

Appello di Gennaio 2013

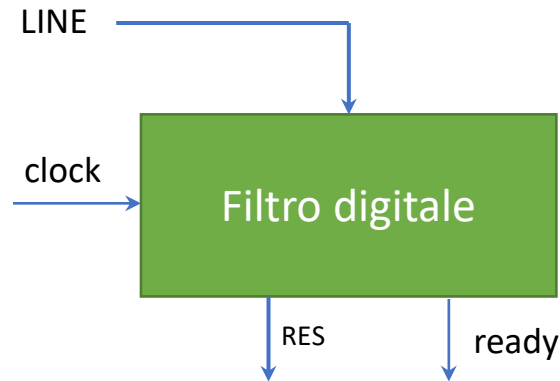




Filtro digitale

Tema di esame molto antico

Specifica



Progettare il datapath

- Questa volta usate un contatore per contare i bit in ingresso
- Potete usare sommatore e un moltiplicatore
- Il sommatore ha un ritardo trascurabile
- Il moltiplicatore impiega quasi due cicli di clock ad eseguire l'operazione
- I parametri dei filtri possono essere memorizzati in registri con uscita tri-state ed abilitazione

Dati in arrivo su linea seriale

- Bit di start a 1, seguito da 8 bit di dato, e un bit di stop a 0

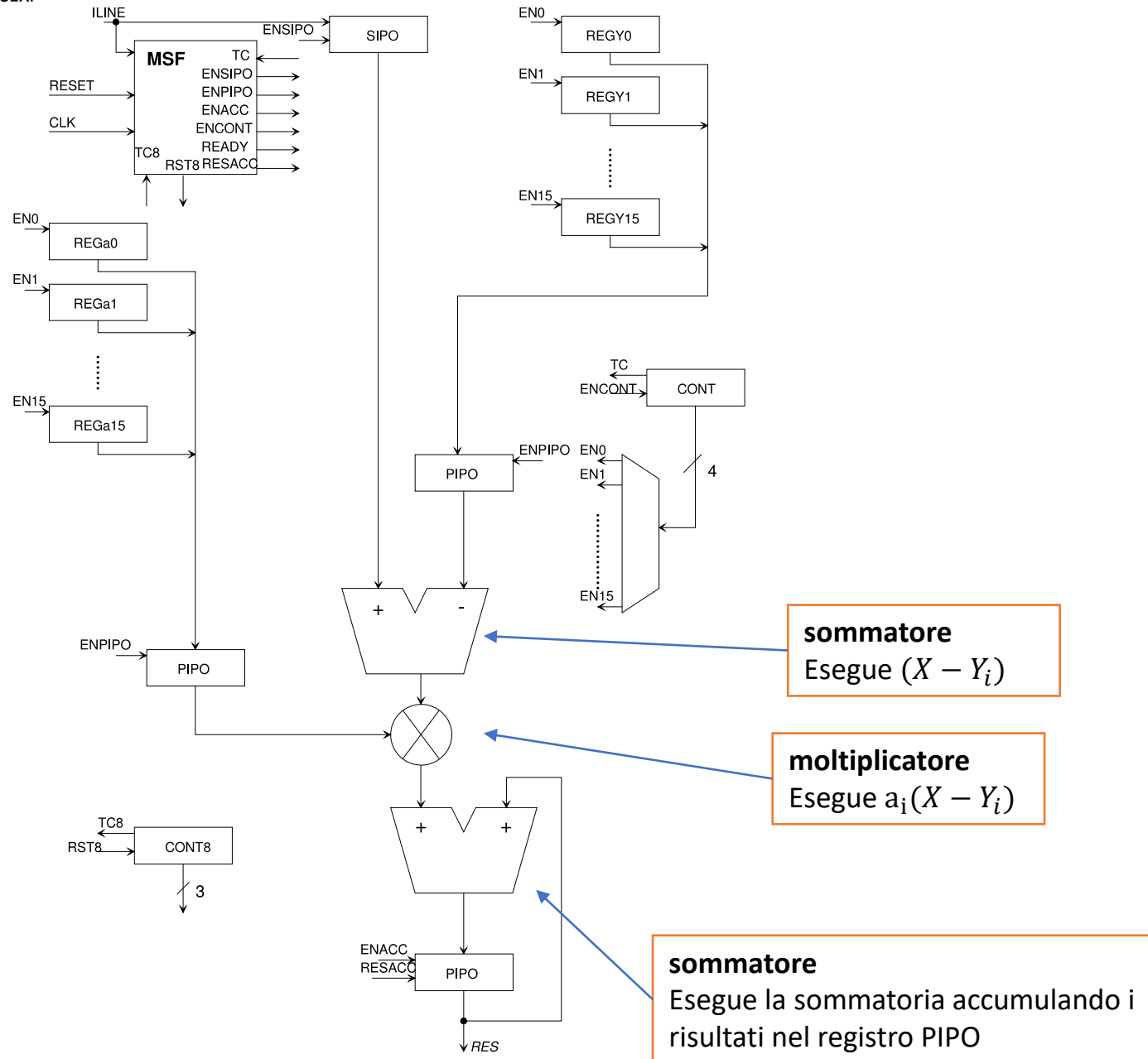
Operazione di filtro

- Sono dati 16 pesi a_i e 16 campioni Y_i
- Calcolare il valore dato dalla seguente formula, dove X è il dato acquisito

$$RES = \sum_{i=0}^{15} a_i(X - Y_i)$$

Progettare il controllo

- Una volta acquisito il dato c'è tempo per eseguire l'operazione
- Eseguire la sommatoria in 16 fasi distinte
- Attenzione al ritardo del moltiplicatore, occorre attendere che termini l'operazione
- Segnale ready attivo quando il filtro è pronto per un nuovo calcolo



SIPO

- Acquisisce il dato seriale
- **ENSIPO**: shift enable

CONT8

- Contatore modulo 8 usato per contare i bit
- Conta sempre (non ha il count enable), ma può essere inizializzato con il segnale sincrono **RST8**
- **TC8**: terminal count (**TC8** = 1 quando il conteggio arriva a 7)

CONT

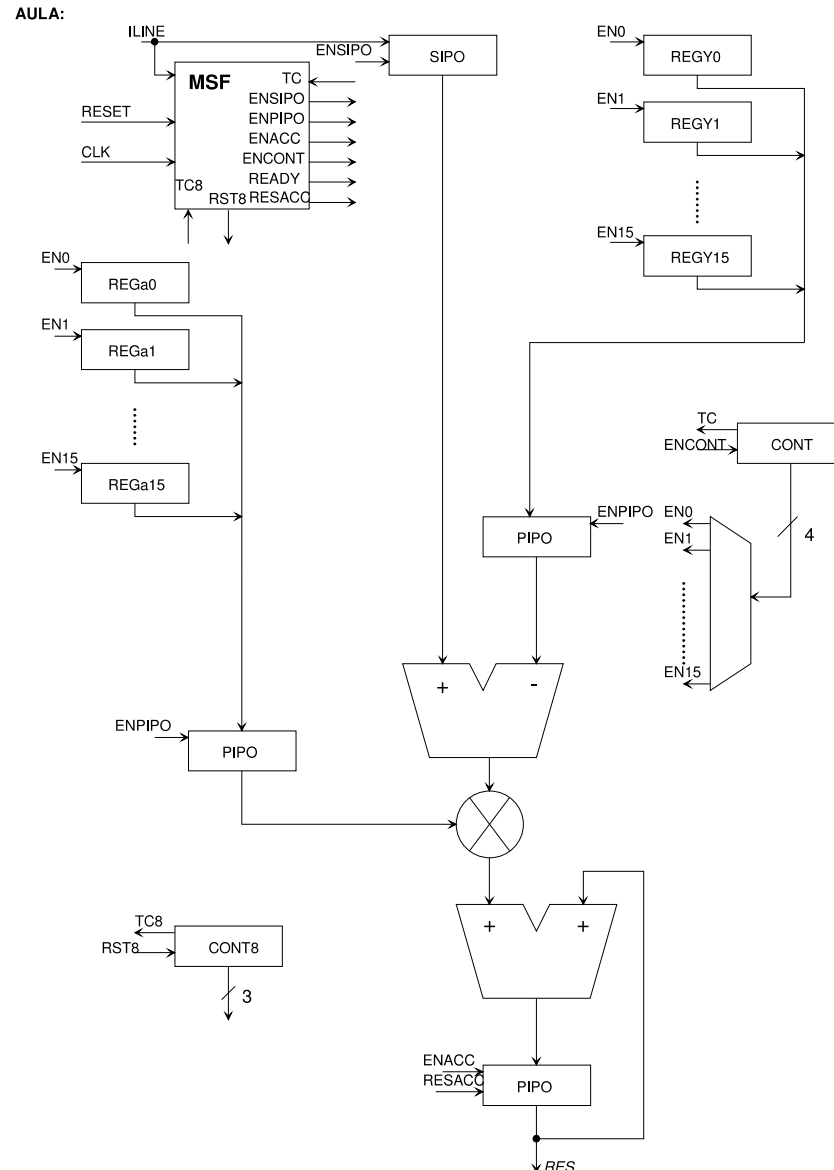
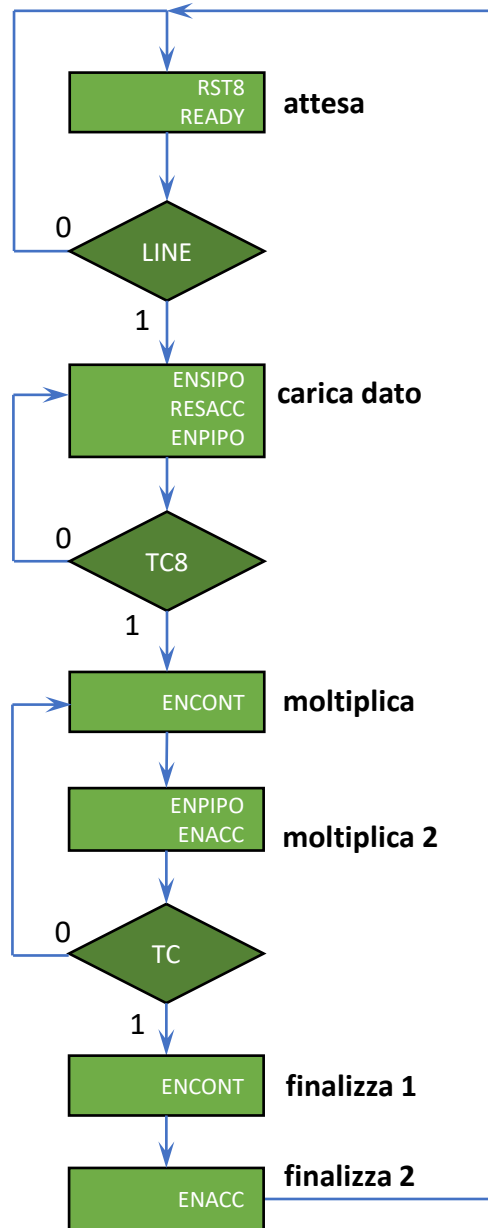
- Contatore modulo 16 usato per indirizzare i registri dei coefficienti (**REGa** e **REGY**)
- **ENCONT**: count enable
- **TC**: terminal count
- La decodifica a valle abilita l'uscita tri-state di una coppia di registri alla volta

PIPO

- Registro usato per memorizzare i coefficienti in ogni fase
- **ENPIPO**: load enable

PIPO (RES)

- Registro accumulatore usato per memorizzare il risultato durante le varie fasi
- **ENACC**: load enable
- **RESACC**: inizializza il registro a 0



Stato attesa

- Aspetta che **LINE** vada a 1 per il bit di start
- Inizializza **CONT8**, e attiva **READY** per dire di essere pronto

Stato carica dato

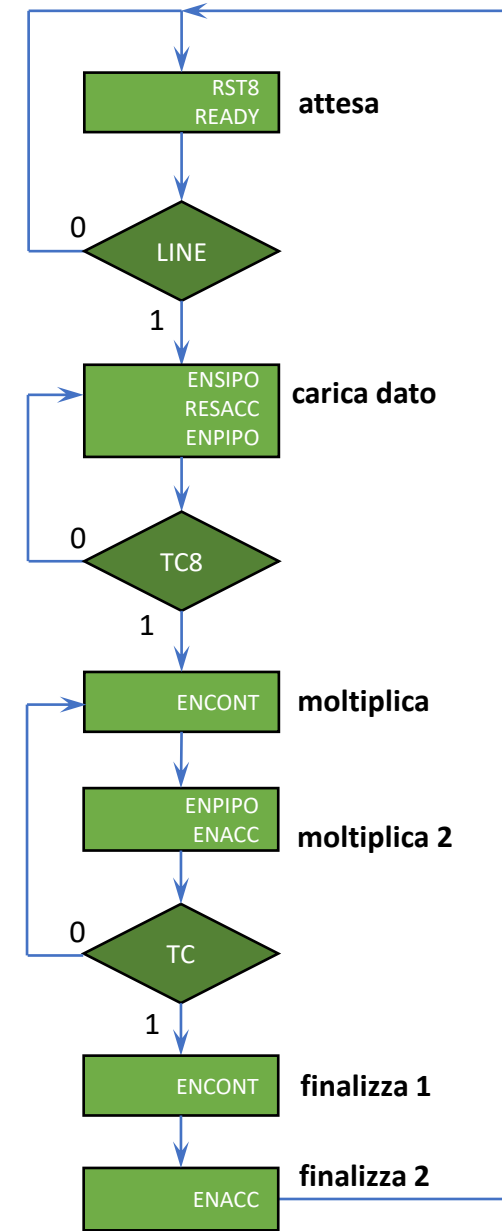
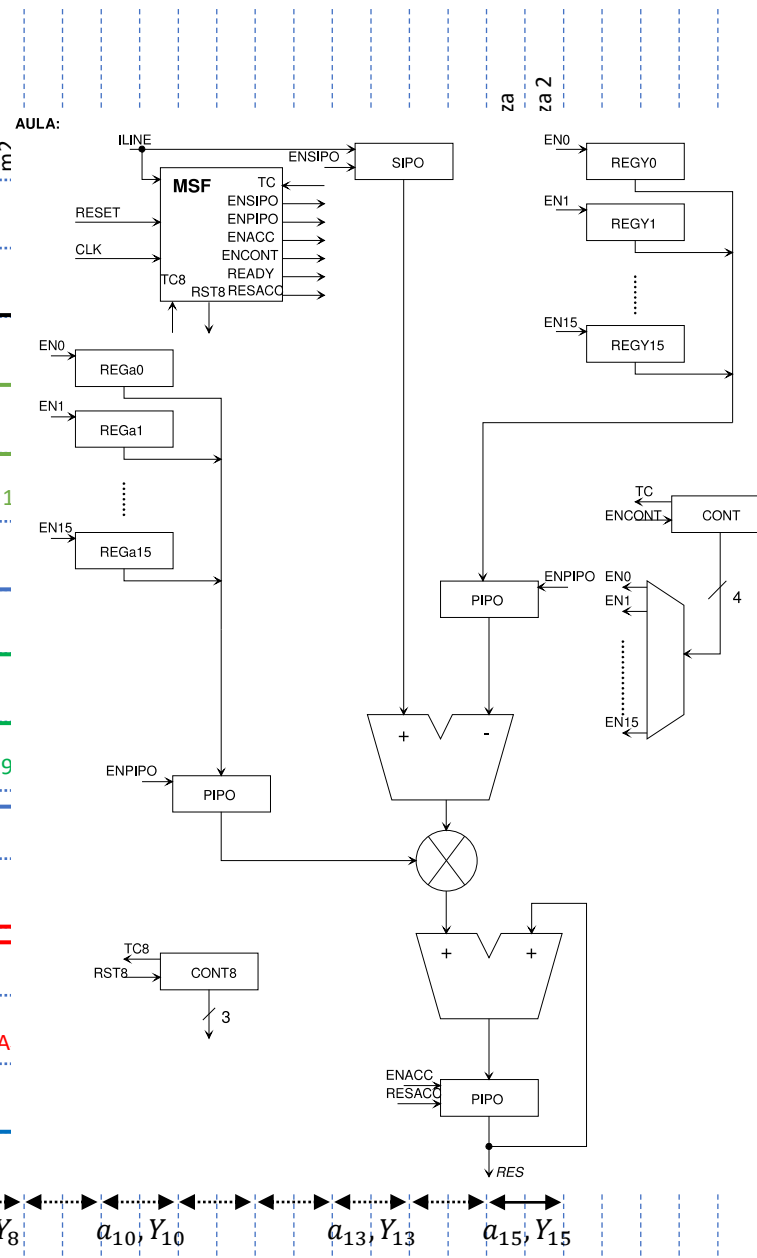
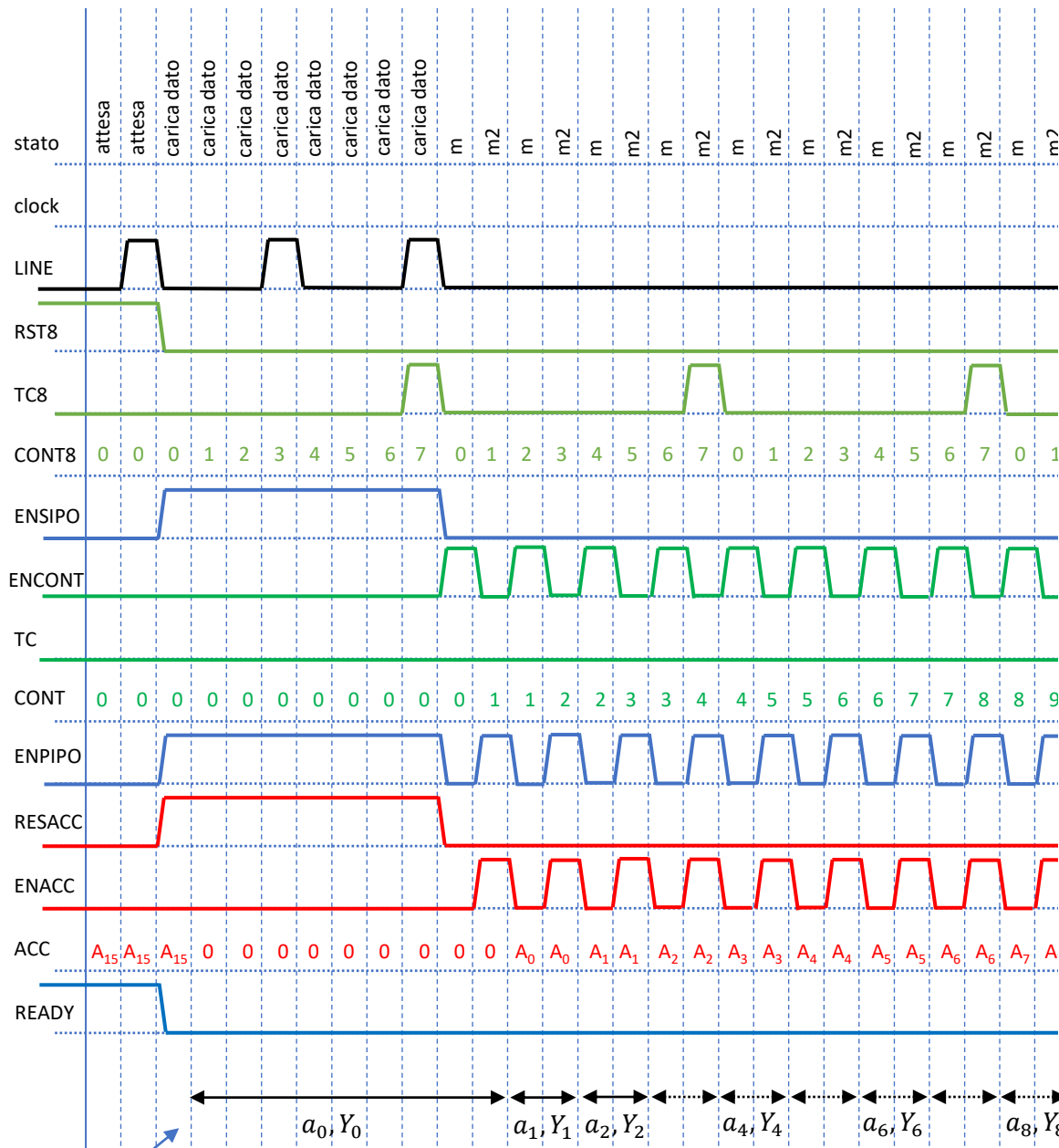
- Abilita il registro a scorrimento (**ENSIPO**) e inizializza il registro accumulatore (**RESACC**)
- Carica i coefficienti a_0 e Y_0 nei registri PIPO (**ENPIPO**)
- Attende che il dato sia acquisito (**TC8**). Non c'è bisogno del count enable per **CONT8**

Stato multiplica e multiplica2

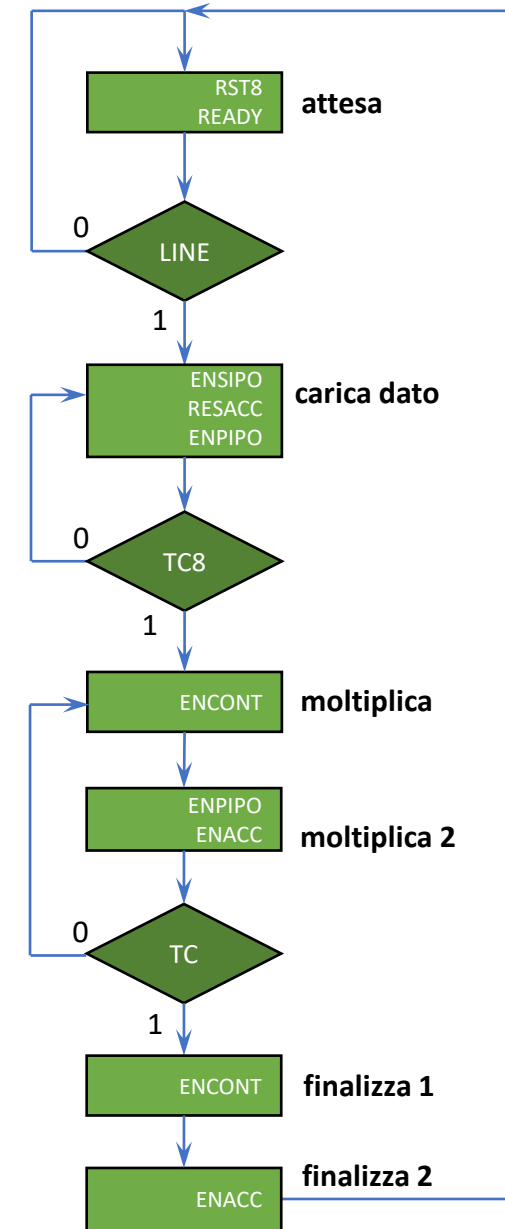
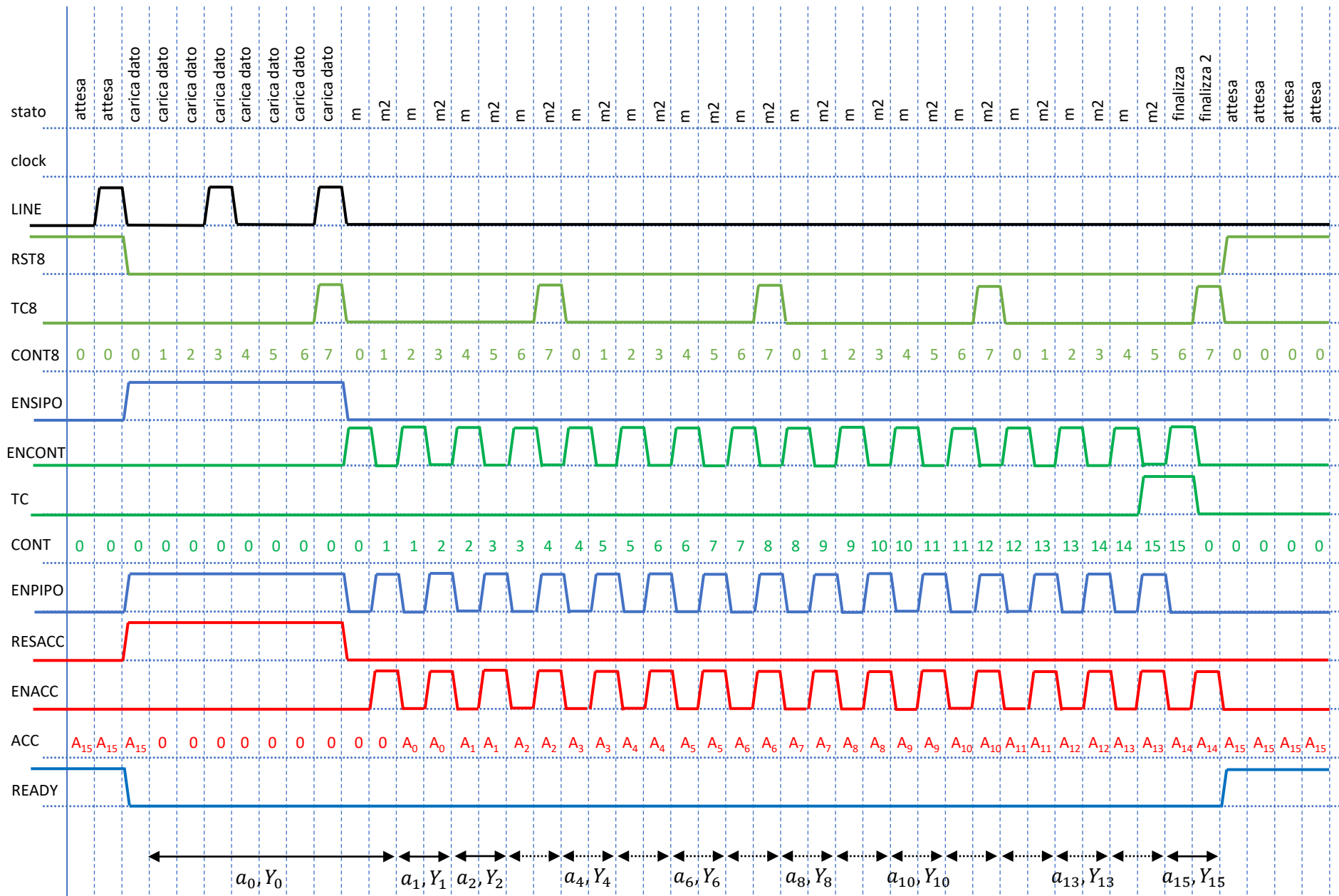
- Attendono che il moltiplicatore termini la sua operazione
- **ENCONT** incrementa il contatore dei coefficienti
- **ENPIPO** carica il nuovo coefficiente selezionato nei PIPO
- **ENACC** abilita il registro accumulatore a caricare il risultato del moltiplicatore, sommandolo al valore precedente
- Il segnale **TC** indica quando si è arrivati all'ultimo coefficiente

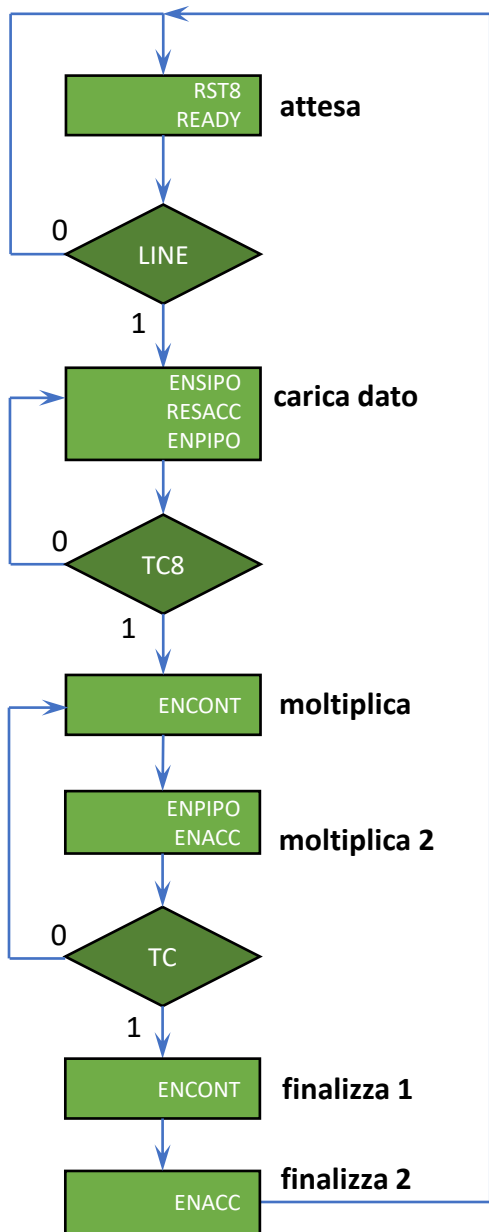
Stati finalizza1 e finalizza2

- Poiché **ENCONT** viene abilitato in **multiplica**, quando **TC** va a 1 abbiamo in realtà caricato l'ultimo coefficiente in PIPO, ma occorre ancora fare l'operazione
- Questi due stati terminano il lavoro
- **ENCONT** manda **CONT** in overflow, che ricomincia quindi da 0



In uscita dai PIPO





$a \setminus bc$	00	01	11	10
0	att	caric	m	m2
1	fin2	-	-	fin

Mappa degli stati

a	b	c	L	$TC8$	TC	a_{+1}	b_{+1}	c_{+1}	U
0	0	0	0	-	-	0	0	0	...
0	0	0	1	-	-	0	0	1	
0	0	1	-	0	-	0	0	1	
0	0	1	-	1	-	0	1	1	
0	1	1	-	-	-	0	1	0	
0	1	0	-	-	0	0	1	1	
0	1	0	-	-	1	1	1	0	
1	1	0	-	-	-	1	0	0	
1	0	0	-	-	-	0	0	0	

$a \setminus bc$	00	01	11	10
0	0	0	0	TC
1	0	-	-	1

$$a_{+1} = ab + TCbc'$$

$a \setminus bc$	00	01	11	10
0	0	$TC8$	1	1
1	0	-	-	0

$$b_{+1} = a'b + TC8c$$

$a \setminus bc$	00	01	11	10
0	L	1	0	TC'
1	0	-	-	0

$$c_{+1} = b'c + La'b' + TC'a'bc'$$

$a \setminus bc$	00	01	11	10
0	0	0	1	0
1	0	-	-	1

$$ENCONT = ab + bc = b(a + c)$$



Ascensore Polo Ferrari

Tema d'esame febbraio 2011

Ascensore con prenotazioni

- Ogni chiamata (interna od esterna) viene registrata fino a quando il piano corrispondente viene raggiunto
- L'ascensore si trova in ogni istante in una di due modalità: **salita** o **discesa**

Modalità salita

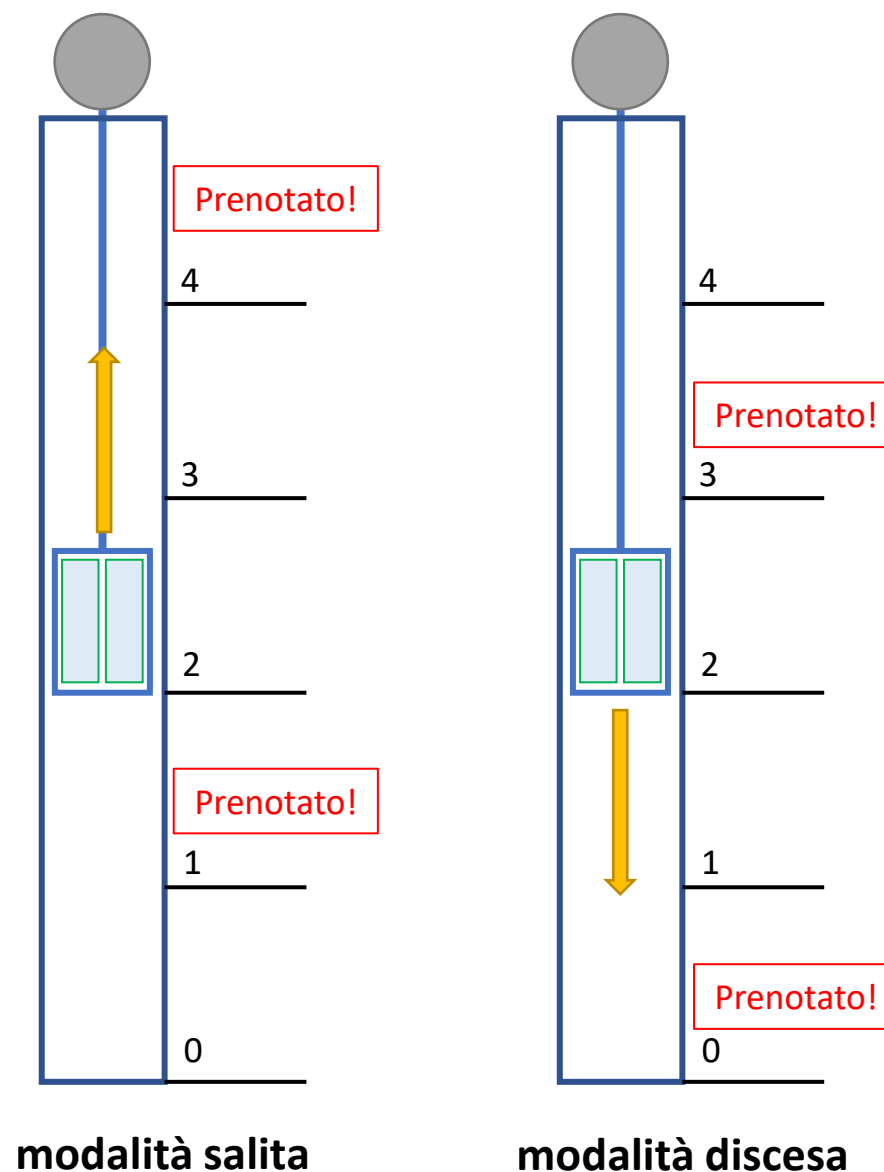
- L'ascensore deve **salire se c'è una prenotazione per un piano più alto** (anche se c'è una prenotazione per uno più basso, magari più vicino)
- **Altrimenti si pone in modalità di discesa**

Modalità discesa

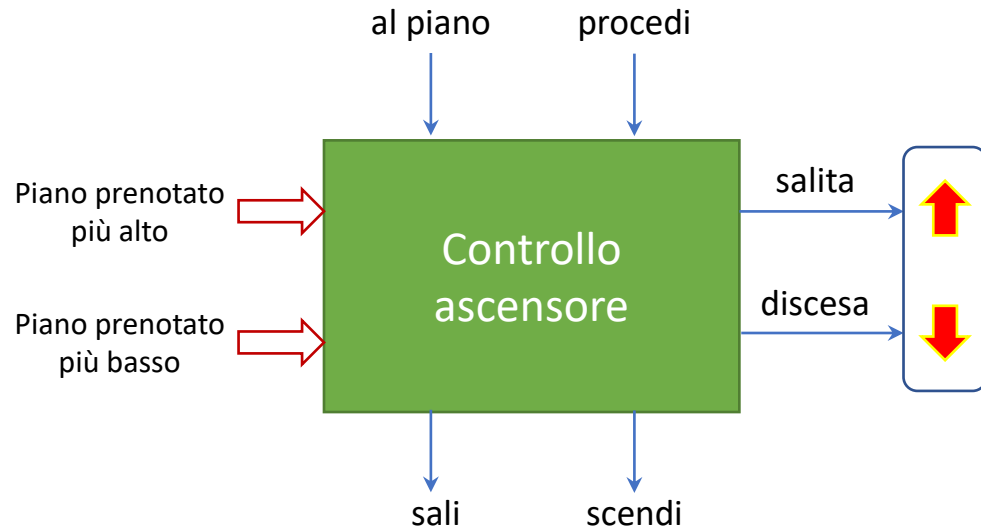
- L'ascensore deve **scendere se c'è una prenotazione per un piano più basso** (anche se c'è una prenotazione per uno più alto)
- **Altrimenti si pone in modalità di salita**

In assenza di prenotazioni

- L'ascensore resta fermo
- Occorre garantire che alla prima prenotazione l'ascensore si muova



Specifica



Progettare il data path

- Dovete in qualche modo tenere traccia del piano a cui è l'ascensore
- Dovete anche confrontare con le prenotazioni

Ingressi e uscite

- Valori dei piani prenotati più in alto e più in basso
- **al_piano**: attivo **per un solo ciclo** di clock quando l'ascensore arriva ad un piano
- **procedi**: attivo quando da un piano è consentito muoversi (disattivo quando persone entrano o escono dall'ascensore)
- **sali**, **scendi**: controllo movimento ascensore. Da non attivare mai contemporaneamente
- **salita**, **discesa**: accendono indicatore della modalità corrente

Progettare il controllo

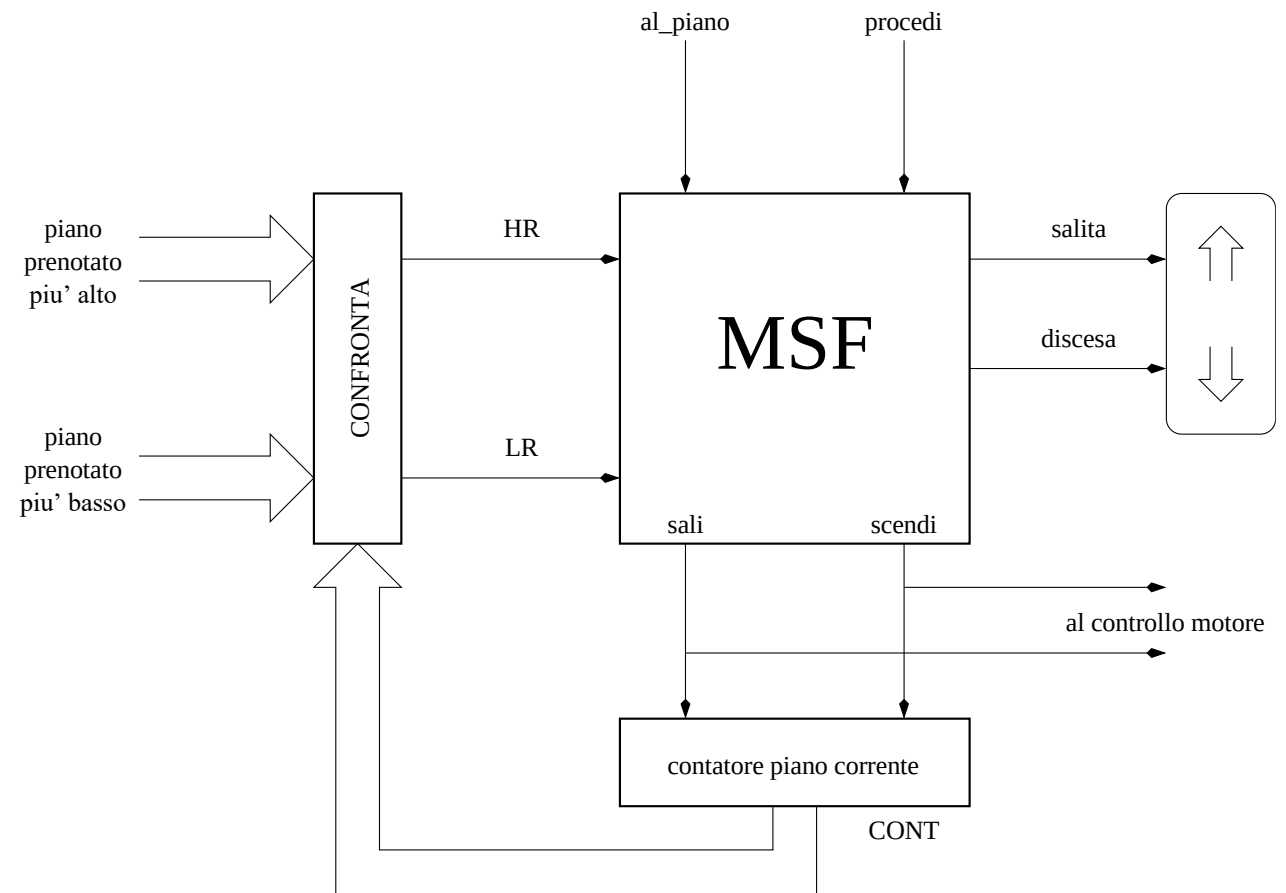
- Non conosciamo esattamente quali piani sono prenotati, quindi per semplicità ci si muove di piano in piano, fermandosi sempre al nuovo piano salvo ripartire subito se **procedi** è attivo
- Deve tenere traccia della modalità corrente

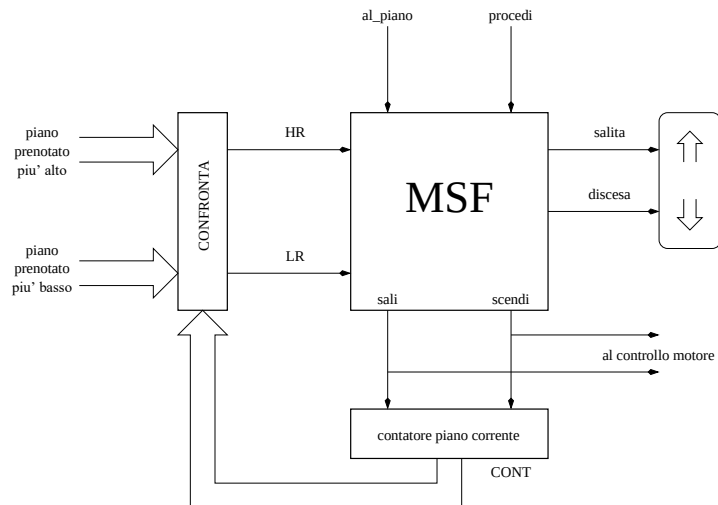
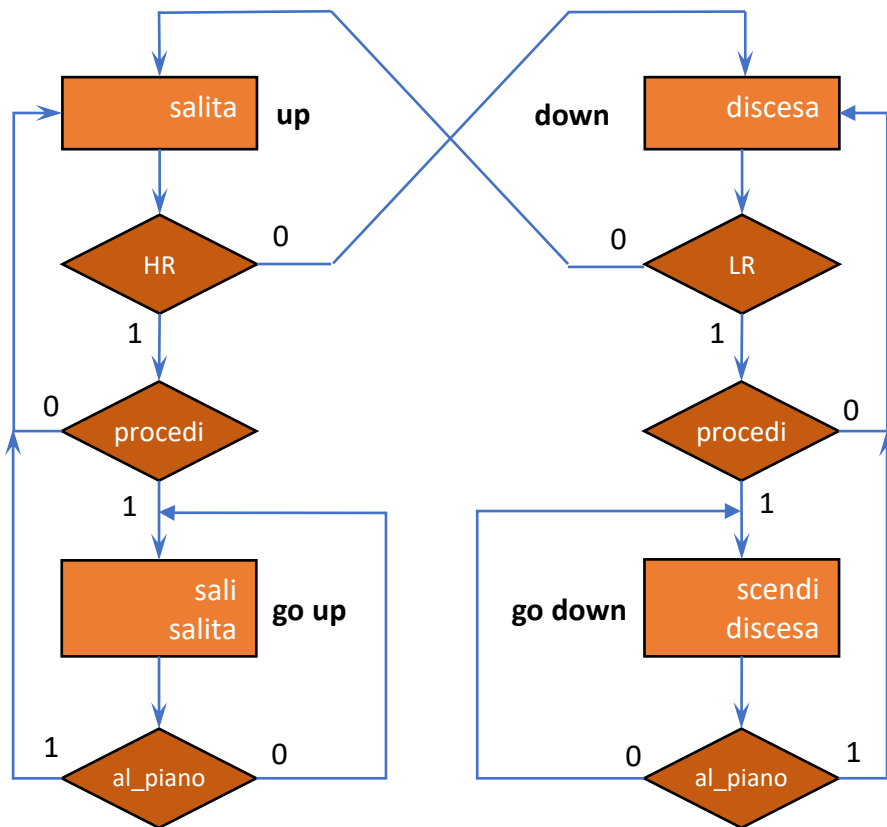
Contatore

- Tiene traccia del piano corrente
- Per cambiare, è un contatore **asincrono**
- Ogni volta che i segnali di ingresso hanno un fronte di discesa il contatore si incrementa (**sali**) oppure si decrementa (**scendi**)
- Essendo asincrono, **l'incremento o decremento è immediato** non appena si verifica la transizione sui segnali di controllo

CONFRONTA

- Confronta il piano corrente con le prenotazioni
- Due semplici comparatori combinatori
- **HR** (higher reservation) è attivo se il piano più alto prenotato è maggiore del contatore
- **LR** (lower reservation) è attivo se il piano più basso prenotato è minore del contatore





Simmetria

- Quando il problema presenta diverse modalità di funzionamento simili, spesso la macchina a stati che ne risulta è simmetrica
- Un lato gestisce la modalità di salita, l'altro la modalità di discesa

Stato up, go up

- Stato di partenza della modalità salita
- Verifica (**HR**) se ci sono prenotazioni verso piani più alti
- Se no (**HR** = 0), si porta in modalità discesa
- Se si (**HR** = 1) verifica se può salire guardando **procedi**
- Se non può muoversi, rimane in modalità salita. **HR** rimane a 1, quindi comincia ad iterare fino a quando si abilita **procedi**
- Quando **procedi** è a 1, si entra in **go up** e si abilita il segnale **sali** che fa salire l'ascensore
- Si resta in **go up** fin tanto che il segnale **al_piano** è 0
- Quando si arriva al piano si ritorna in **up**, rimanendo in modalità salita. Non si va in discesa, perché potremmo non aver ancora raggiunto il piano prenotato più alto
- Quando si torna in up, il segnale **sali** torna a 0, l'ascensore si ferma, ed il contatore si incrementa di 1 **immediatamente**. Si esegue quindi di nuovo il confronto ed **HR** si aggiorna
- Nei due stati teniamo **salita** attivo per accendere la spia luminosa

Stati down, go down

- Funzionano in maniera simmetrica rispetto alla salita

Alternanza

- Quando non ci sono prenotazione (**HR** = **LR** = 0), l'ascensore alterna le due modalità di funzionamento ad ogni ciclo di clock
- Le due spie luminose lampeggiano (se lo fanno abbastanza velocemente le vedremo entrambe accese)

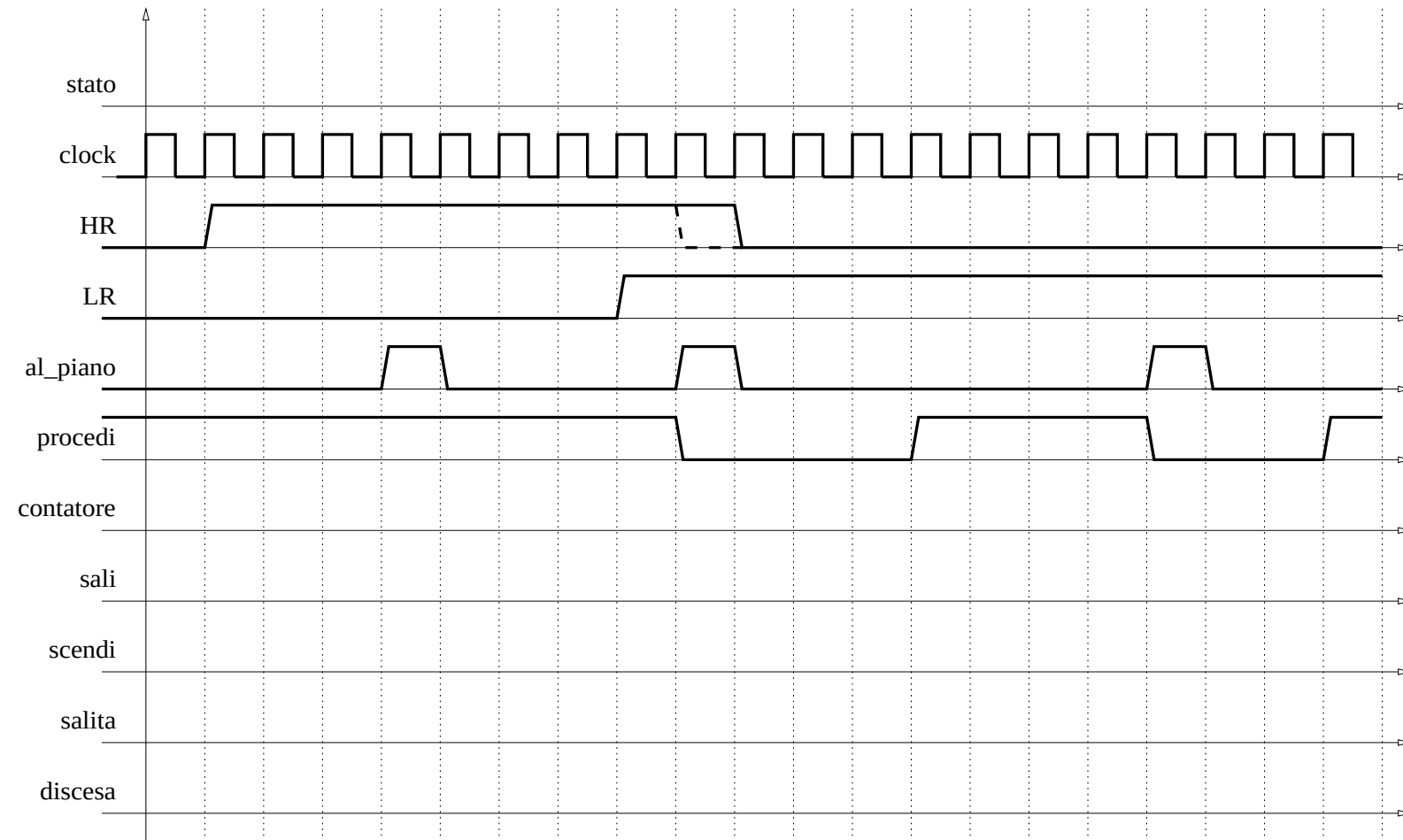
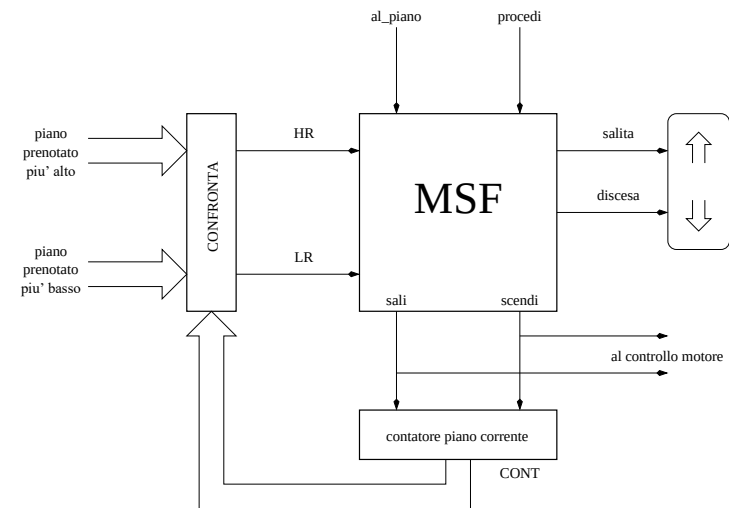
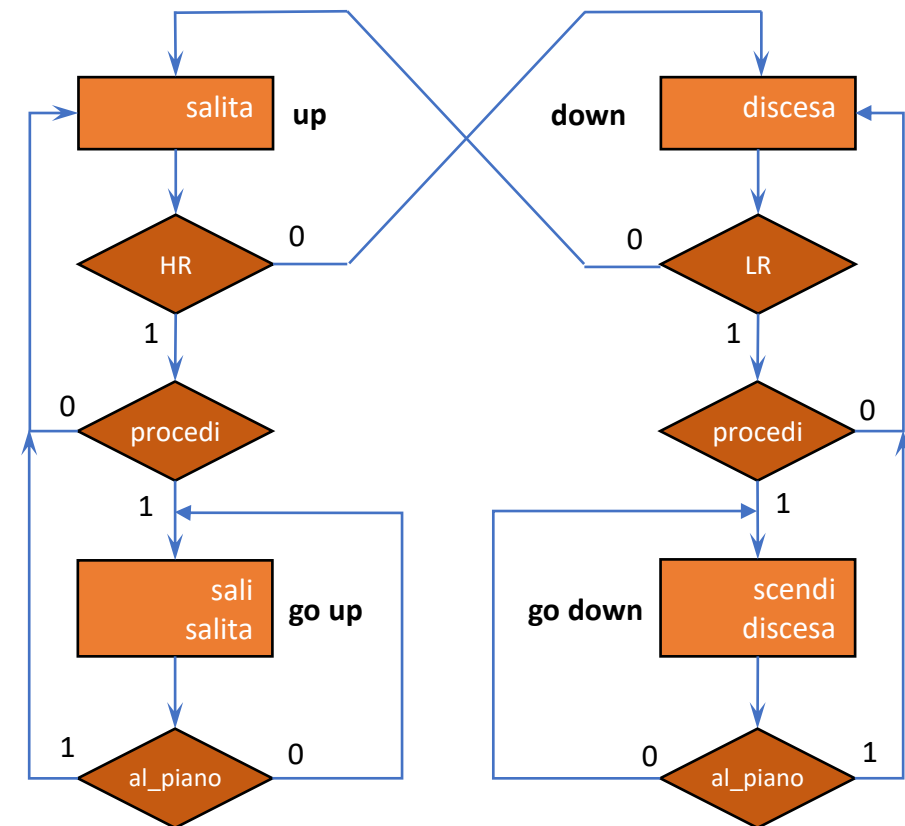
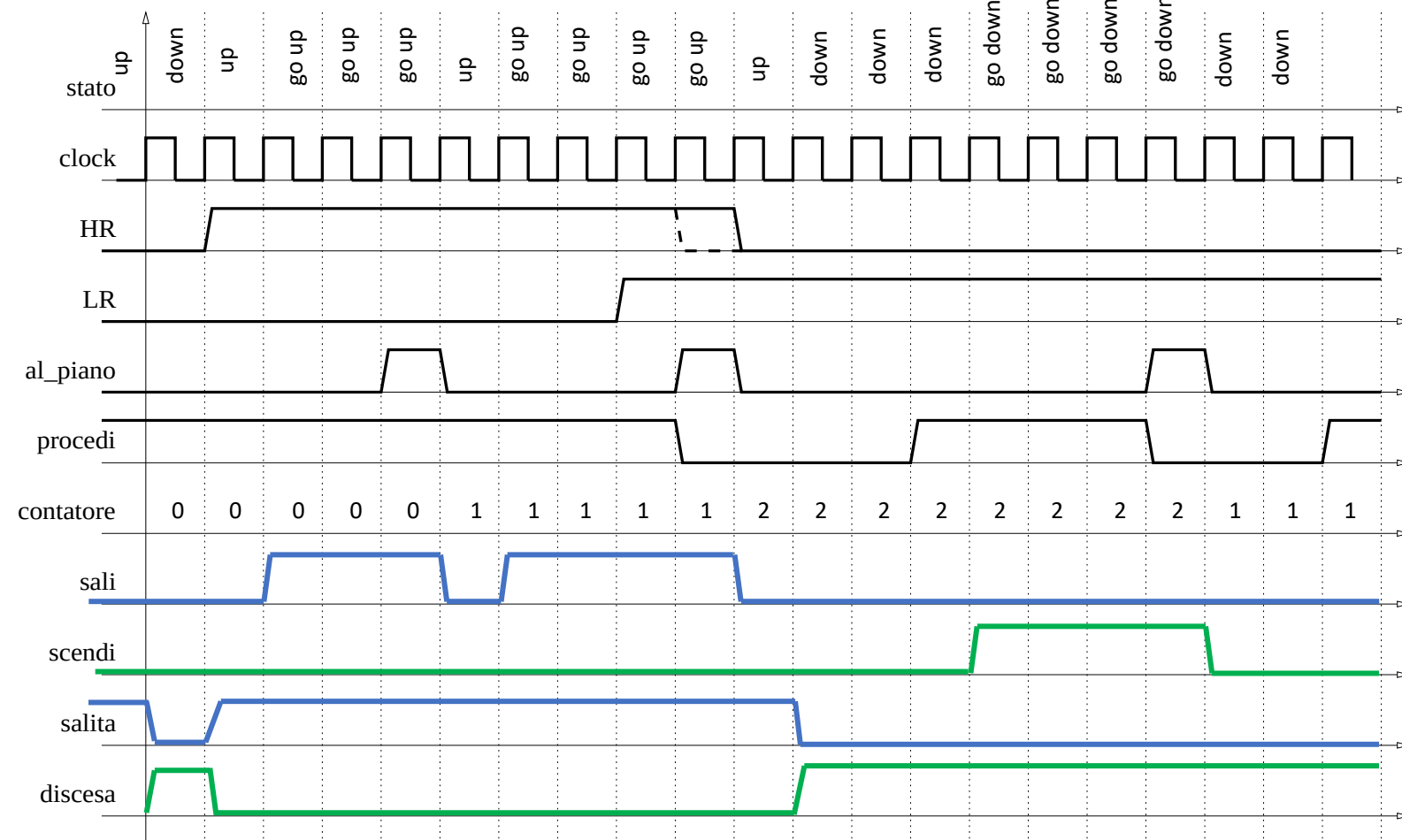


Diagramma temporale:

Inizialmente c'è una prenotazione più in alto, poi anche una più in basso

HR cambia a seconda di quando attivate il contatore

Indicare il valore del contatore nella linea corrispondente. Si assuma di partire dal piano 0



Provetta Aprile 2017

Domande di teoria

Domanda 1

- (4 pts) Cosa si intende per implicante, implicante primo ed implicante essenziale? Se ne fornisca la definizione e si accompagni la risposta identificando tutti gli implicanti primi ed essenziali della funzione $f(a,b,c,d)$ mostrata in Figura 1 in forma di mappa di Karnaugh.

Risposta

Siano f e g due funzioni di n variabili. Si dice che g è un implicante di f , se quando g è 1 allora anche f è 1. Formalmente

$$g(x_1, \dots, x_n) = 1 \rightarrow f(x_1, \dots, x_n) = 1$$

Se invece g è 0, f può assumere qualunque valore. Si dice che g copre certi 1 di f .

Consideriamo solo implicanti in forma di prodotto di letterali. Un implicante è primo se *non* è completamente contenuto in nessun altro implicante.

Un implicante primo è essenziale se copre un 1 non coperto da nessun altro implicante *primo*

Implicanti primi di f

$a'b$ $ab'd$ $a'cd$ $b'cd$ $ac'd$ $bc'd$

Implicanti primi essenziali di f

L'unico implicante primo essenziali è $a'b$, perché è l'unico a coprire gli 1 in 0100 e 0110

The Karnaugh map for function $f(a,b,c,d)$ is shown with prime implicants highlighted. The map is a 4x4 grid with rows labeled $ab \setminus cd$ (00, 01, 11, 10) and columns labeled cd (00, 01, 11, 10). The cells containing 1s are at (00,01), (01,00), (01,01), (01,11), (11,01), and (10,01). The prime implicants are: a vertical group of four 1s in column 01 (highlighted in orange), a horizontal group of four 1s in row 01 (highlighted in blue), a vertical group of two 1s in column 11 (highlighted in green), a horizontal group of two 1s in row 11 (highlighted in red), and a horizontal group of two 1s in row 10 (highlighted in pink). A green arrow points up and another points down from the top of the map.

$ab \setminus cd$	00	01	11	10
00	0	0	1	0
01	1	1	1	1
11	0	1	0	0
10	0	1	1	0

Domanda 2

- (4 pts) Si derivi la forma minima della funzione $f(a,b,c,d)$ della Figura 1, eliminando la ridondanza. Si discuta quindi il ruolo dei don't care nella semplificazione, evidenziando come possano essere utilizzati nell'ottimizzazione a partire dalla mappa di Karnaugh. Si sintetizzi la funzione $h(a,b,c,d)$ mostrata in Figura 1, e si confronti con il risultato del caso senza don't care.

Risposta

La forma minima di f deve includere $a'b$. Per i restanti 1 possiamo scegliere $ac'd$ e $b'cd$

$$f(a, b, c, d) = a'b + ac'd + b'cd$$

I don't care rappresentano combinazioni delle variabili di ingresso per le quali non ci interessa il valore delle uscite. Possono avere luogo per esempio se la combinazione di ingresso corrispondente non si presenta mai, oppure se il valore non sarebbe comunque osservabile.

Si possono usare per ottimizzare le funzioni considerandoli come 1 quando si cercano gli implicant primari, in modo da avere implicant più grossi, e come zero quando si scelgono gli implicant, in modo da sceglierne di meno.

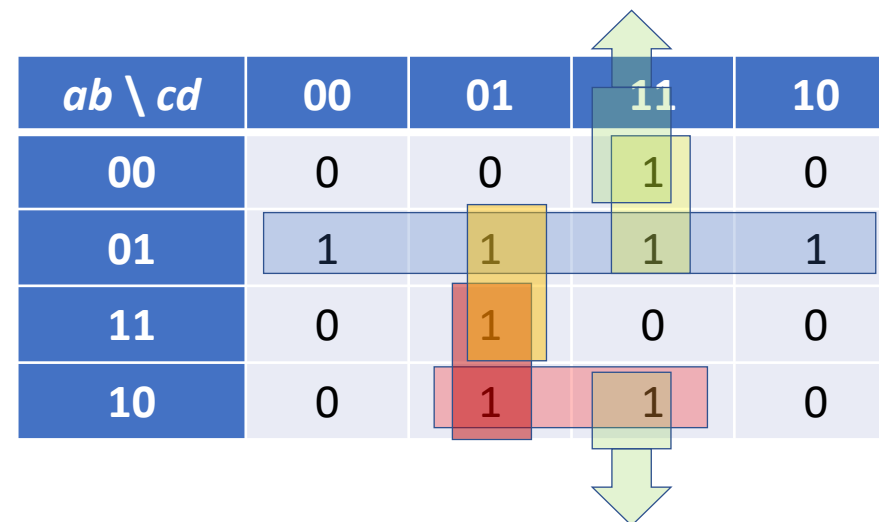
Gli implicant primari di h sono

$a'b$ $a'd$ $b'd$ $c'd$ bcd'

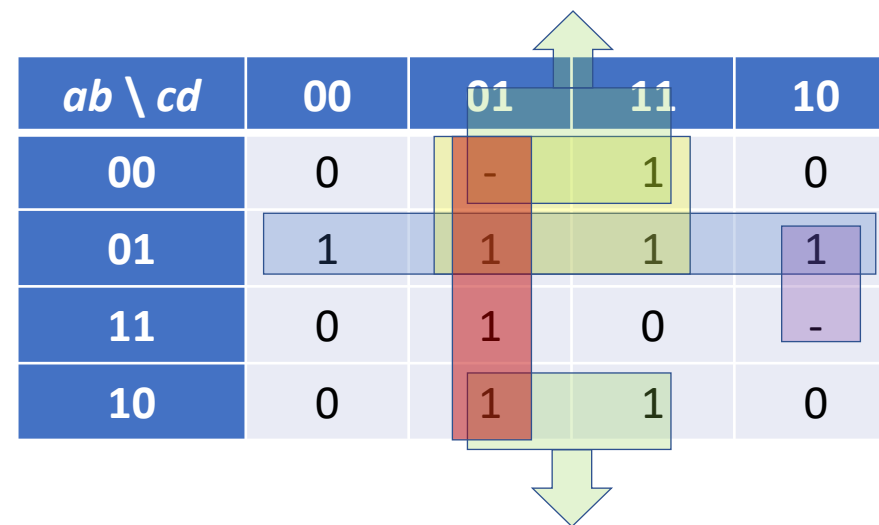
Una copertura minima comprende certamente $a'b$ e $c'd$ che sono essenziali. Possiamo quindi aggiungere $b'd$ per completare la copertura:

$$h(a, b, c, d) = a'b + c'd + b'd$$

Sono 6 contro 8 letterali.



$ab \setminus cd$	00	01	11	10
00	0	0	1	0
01	1	1	1	1
11	0	1	0	0
10	0	1	1	0



$ab \setminus cd$	00	01	11	10
00	0	-	1	0
01	1	1	1	1
11	0	1	0	-
10	0	1	1	0

Domanda 3

- (4 pts) Si consideri il circuito mostrato in Figura 2. Si derivi e si disegni il diagramma a stati corrispondente mostrando inoltre la tabella della verità della funzione di calcolo dello stato futuro.

Risposta

Sulla base della tabella della verità della porta NAND costruiamo la tabella degli stati, indicando lo stato futuro sulla base dello stato presente e dell'ingresso L. Le espressioni per lo stato futuro sono

$$A_{+1} = (LB)' \quad B_{+1} = A$$

E' facile allora costruire il diagramma a stati.

- Lo stato 00 finisce in modo incondizionato nello stato 01
- Lo 01 finisce in 11
- Lo stato 11 va in 11 se L = 0, in 10 se L = 1
- Lo stato 10 va in 01 se L = 0, in 00 se L = 1

x	y	z
0	0	1
0	1	1
1	0	1
1	1	0

nand

Stato presente		Ingr.	Stato futuro	
B	A	L	B ₊₁	A ₊₁
0	0	0	0	1
0	0	1	0	1
0	1	0	1	1
0	1	1	1	1
1	0	0	0	1
1	0	1	0	0
1	1	0	1	1
1	1	1	1	0

tabella degli stati

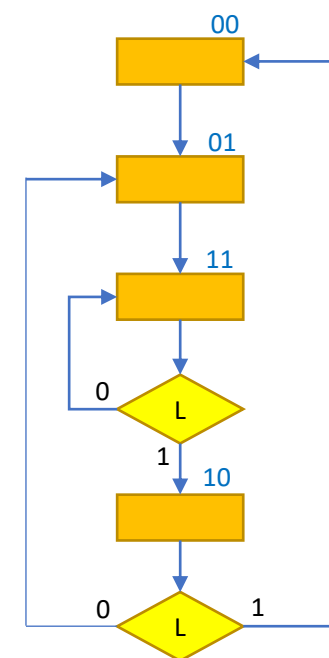
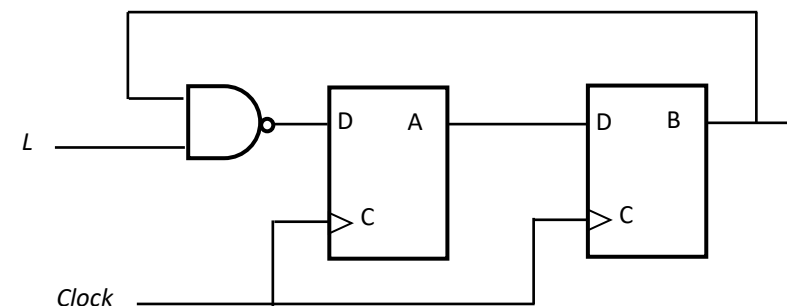


diagramma a stati

Domanda 4

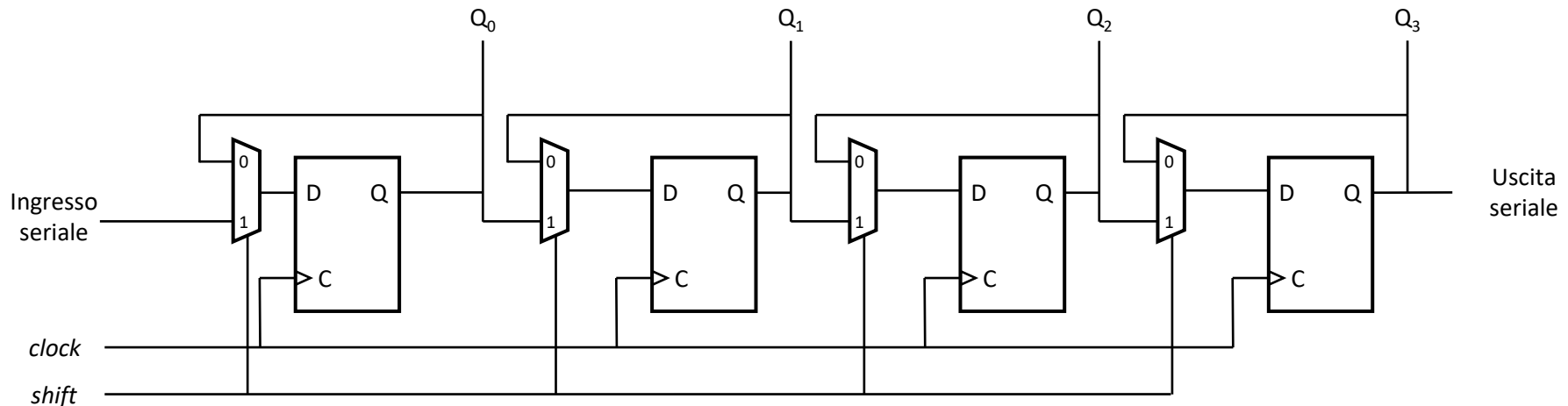
- (4 pts) Si discuta la realizzazione di un registro a scorrimento sincrono a 4 bit con shift enable ed uscita parallela, usando flip flop edge triggered di tipo D e multiplexer. Si spieghi il funzionamento e il ruolo dei vari componenti, e se ne disegni il circuito. Supponendo che il registro contenga i valori 1001, e che l'ingresso seriale valga 0, si indichi il contenuto del registro dopo il fronte attivo del clock, nei due casi in cui sia abilitato e non sia abilitato allo scorrimento.

Risposta

Il registro a scorrimento è costituito da una serie di flip flop collegati in cascata, cioè con l'uscita Q di uno collegata all'ingresso D del seguente. Per consentire un controllo di shift enable sincrono, usiamo un multiplexer davanti a ciascun flip flop per scegliere se caricare il valore proveniente dal flip flop precedente (shift abilitato), oppure se ricaricare il valore contenuto nel flip flop stesso (shift disabilitato). L'uscita parallela si ottiene semplicemente portando fuori l'uscita di ciascun flip flop. Il circuito è mostrato in figura.

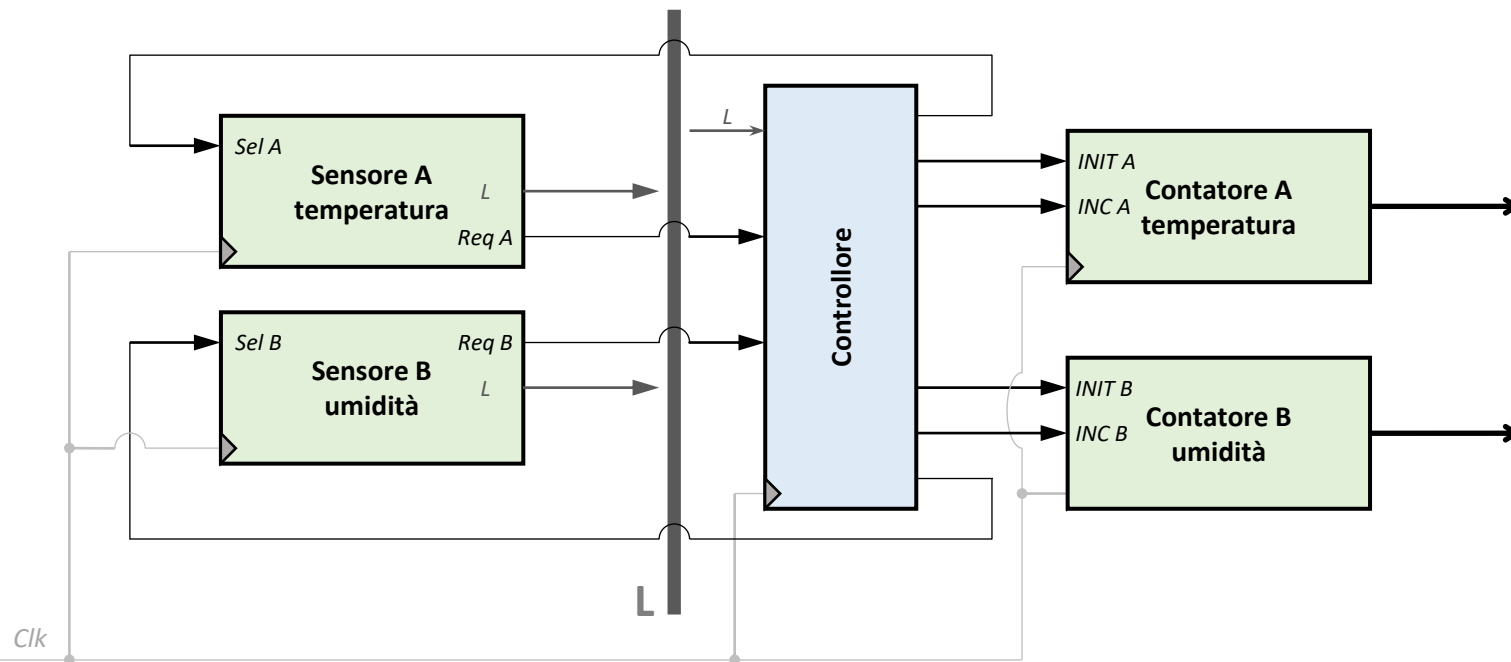
Se il registro contiene il valore 1001 e l'ingresso vale 0, il contenuto del registro sarà

- Con shift abilitato: 0100 (lo 0 in ingresso entra in prima posizione a sinistra, gli altri scorrono, l'1 in prima posizione a destra esce)
- Con shift disabilitato: 1001 (rimane invariato)



Controllore di rete token ring

Prova in itinere Aprile 2017



Scopo

- Il controllore deve arbitrare l'accesso dei due sensori alla linea di comunicazione seriale L
- Il controllore inoltre riceve il dato caricandolo nei due contatori

1. Request e grant

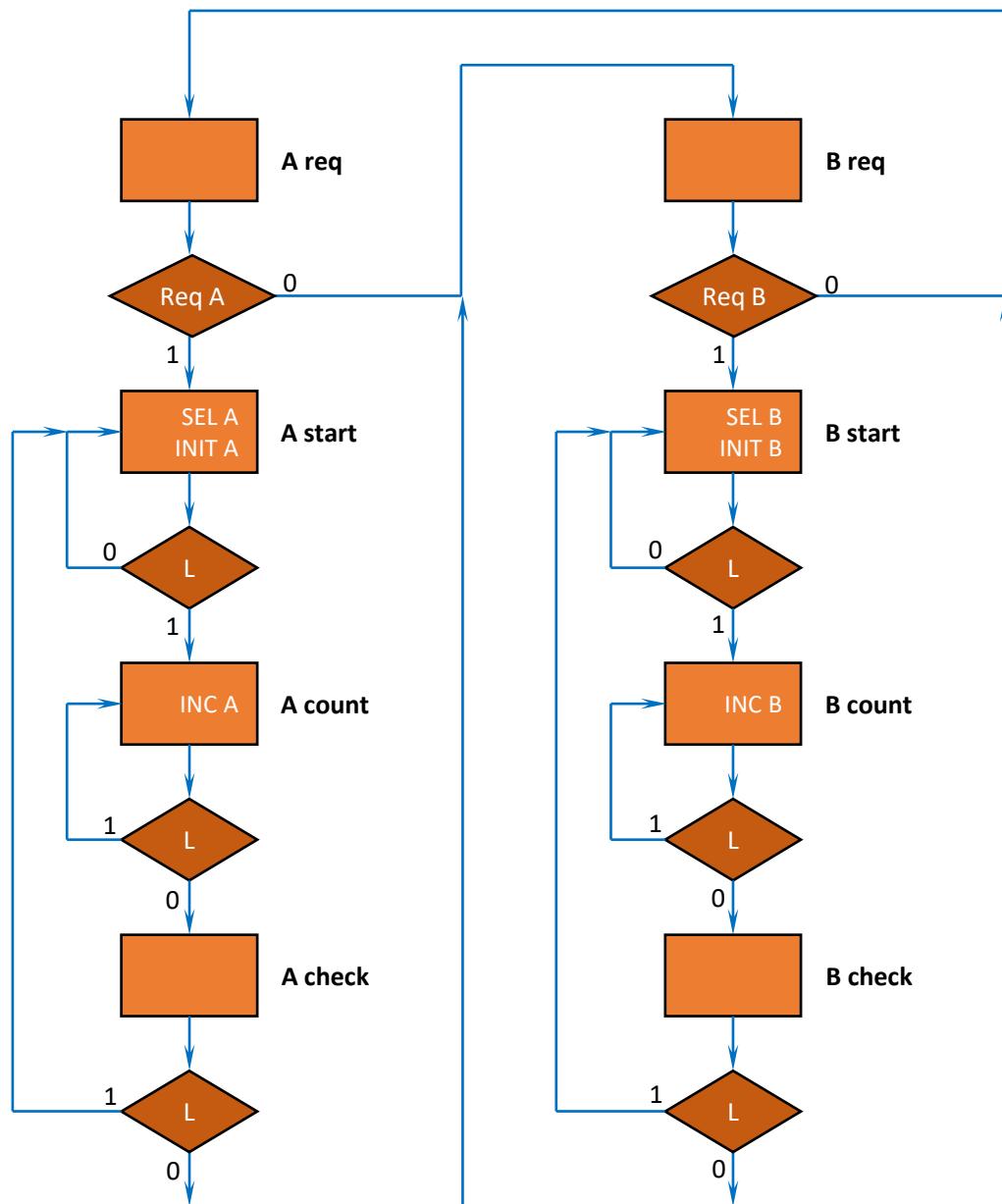
- I sensori richiedono l'accesso alla linea seriale L attivando il rispettivo segnale **Req A** o **Req B**
- Il controllore concede l'accesso ad uno solo dei sensori attivando il rispettivo segnale **Sel A** o **Sel B**. Tale segnale deve rimanere attivo fino a quando il sensore non inizia la trasmissione, indicata da un bit di start a 1 su L. Quindi deve essere disattivato
- Il master opera in priorità alternata: se al ciclo di clock n la priorità è per il sensore A, al ciclo $n + 1$ la priorità è per B, e viceversa

2. Trasmissione

- I sensori trasmettono il dato tramite un bit di start a 1, seguito da un numero di 1 pari al valore che si vuole trasmettere.
- Il pacchetto termina mandando due 0 consecutivi. Esempio: 1-11111-00 indica il valore 5 (5 uni dopo lo start, più due bit di stop)
- Uno 0 seguito da 1 è da considerarsi errore di trasmissione, esempio 1-110100. Il controllore attiva il segnale **Sel** per indicare l'errore, ed il sensore disattiva immediatamente la sua uscita ed attende alcuni cicli prima di ritentare la trasmissione. Il controllore si rimette in attesa del bit di start.
- Terminata la trasmissione con successo il controllore riprende lo schema, assegnando inizialmente la priorità all'altro sensore, e quindi alternando

3. Contatori

- Il controllore li usa per registrare il valore trasmesso su L
- **INIT A/B**: ingresso sincrono di inizializzazione a 0
- **INC A/B**: ingresso sincrono di incremento
- Si possono ignorare eventuali problemi di overflow
- Per semplicità, è consentito contare anche il bit di start



Simmetria

- La macchina a stati è simmetrica, perché il funzionamento è analogo per A e B
- Un lato gestisce la priorità per A, l'altro la priorità per B

Stato A Req

- Verifica se A richiede la linea L. Se sì, si va in **A start**, altrimenti si va in priorità B

Stato A start

- Fornisce il segnale di selezione (**SEL A**) ed inizializza il contatore di A (**INIT A**) per ricevere il dato
- Attende il bit di start

Stato A count

- Dopo il bit di start, conta tutti gli 1 che arrivano su L attivando **INC A**.
- Se invece si vede uno 0 si va in **A check** per verificare se si tratta del primo bit di stop o di un errore
- **Nota:** si conta un bit in più, perché quando L = 0 siamo ancora in **A count**, e **INC A** è attivo. In pratica si conta anche il bit di start. Per evitare, occorrerebbe usare uscite condizionate.

Stato A check

- Se L = 1 si tratta di un errore. Si torna quindi in **A start** in modo da attivare **SEL A**, ed aspettare di nuovo il bit di start. Poiché il sensore disattiva l'uscita non appena vede **SEL A**, si può evitare di usare una uscita condizionata da **A check** su **SEL A**. Se così non fosse, mentre siamo in **A start** potremmo vedere un 1 della trasmissione, invece del nuovo bit di start. Allora **SEL A** dovrebbe essere attivato con una uscita condizionata già in **A check**
- Se L = 0 la trasmissione è terminata, si passa quindi ad aspettare una nuova richiesta con priorità B

Nome e matricola:

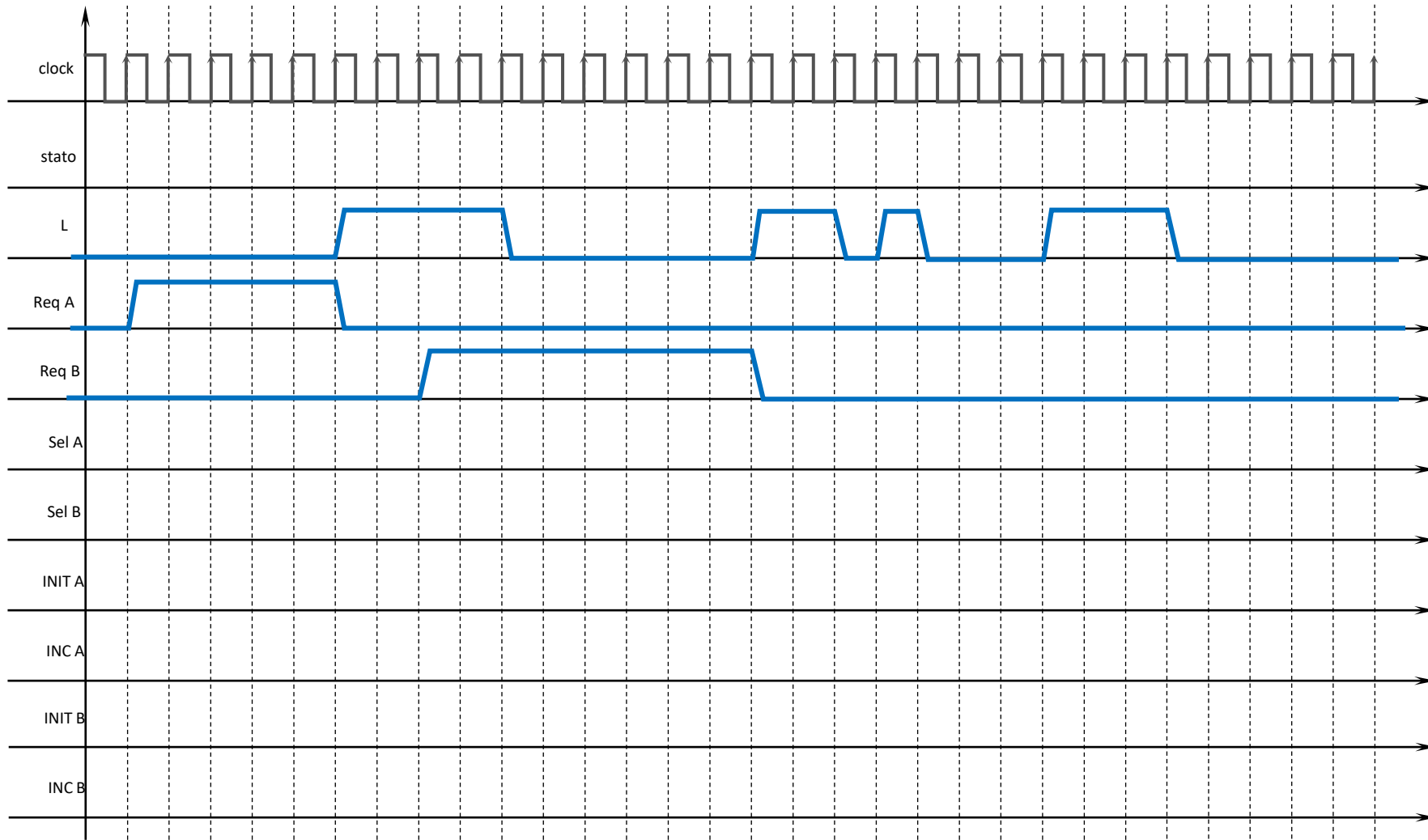
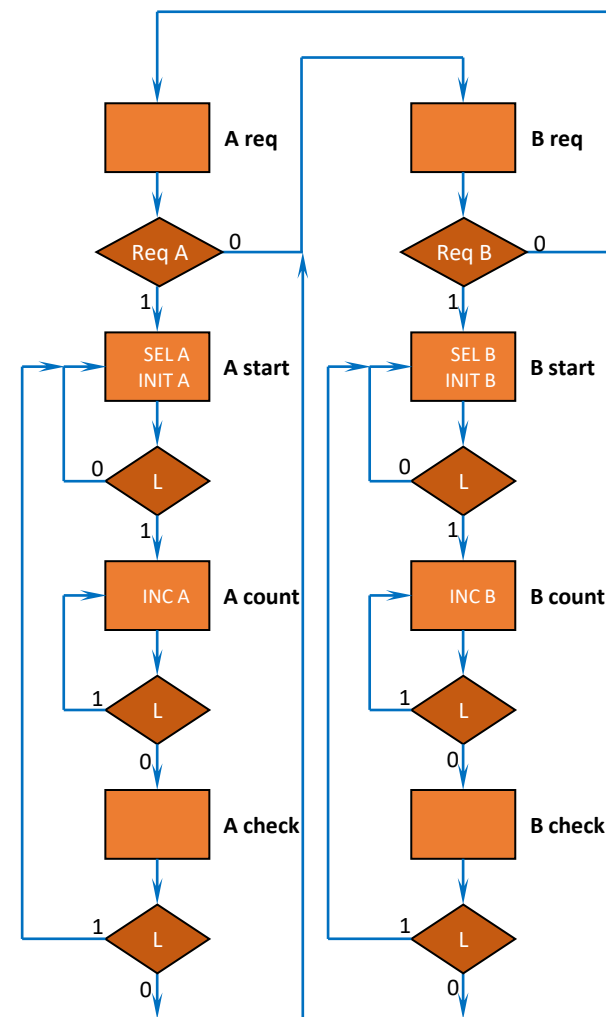


Diagramma temporale:

Prima A richiede la linea di trasmissione, e trasmette quattro bit a 1 senza errori

Poi B richiede la linea di trasmissione. Inizia a trasmettere ma c'è un errore. Quando riprova la trasmissione va a buon fine con tre bit a 1

J. S. Newton



Provetta Aprile 2019

Domande di teoria

Domanda 1

- Semplificazione logica** (6 pts). Si consideri la funzione di quattro variabili $f(a,b,c,d)$ data da:

$$f(a,b,c,d) = a'b'd' + a'b'c + ab'c' + bc'd' + b'cd + bcd'$$

Si derivi l'espressione minima di f (suggerimento: si costruisca la mappa di Karnaugh).

Si supponga quindi che il valore di $f(0,0,0,1)$ sia un don't care. Si derivi nuovamente l'espressione minima di f facendo uso del don't care.

Risposta

Come suggerito costruiamo la mappa di Karnaugh. Ogni termine dell'espressione data corrisponde ad un implicante, ma non necessariamente primo. Eseguiamo quindi la semplificazione sulla mappa.

Senza cercare tutti gli implicanti primi, vediamo che una copertura minima è:

$$f(a, b, c, d) = c'd' + bd' + ab'd + a'b'c$$

un'altra è

$$f(a, b, c, d) = bd' + a'd' + ab'c' + b'cd$$

Se ora supponiamo che $(0,0,0,1)$ sia un don't care, si ottiene la seconda mappa

$$f(a, b, c, d) = b'd + bd' + a'd' + c'd'$$

(ci sono altri due implicanti primi... quali?)

$ab \setminus cd$	00	01	11	10
00	1	0	1	1
01	1	0	0	1
11	1	0	0	1
10	1	1	1	0

$ab \setminus cd$	00	01	11	10
00	1	-	1	1
01	1	0	0	1
11	1	0	0	1
10	1	1	1	0

Domanda 2

- **Contatore offset 3 (6 pts).** Si costruisca, tramite un registro ed un sommatore, un contatore a 4 bit che conti in avanti di 3 alla volta (invece che di 1), spiegandone il funzionamento. Si preveda un ingresso di count enable per abilitare il conteggio. Si derivi quindi la sequenza di conteggio, supponendo di partire da 0000 fino a ritornare allo stesso valore (per semplicità, si può indicare la sequenza in decimale invece che in binario). Si indichi come ottenere una uscita di terminal count che sia attiva ogni volta che il contatore sia in procinto di passare per lo 0 o di superarlo.

Risposta

Come di consueto, costruiamo il contatore tramite un registro a 4 bit ed un sommatore a 4 bit al quale è imposto il valore 3 ad uno dei due ingressi, come mostrato in figura. Per il count enable si può usare un registro con load enable, o usare direttamente il segnale di count enable per mettere 0 o 3 al sommatore. La figura mostra la prima soluzione.

La sequenza di conteggio sarà la seguente (in binario e decimale):

0000 (0), 0011 (3), 0110 (6), 1001 (9), 1100 (12), 1111 (15) -> **overflow** ->

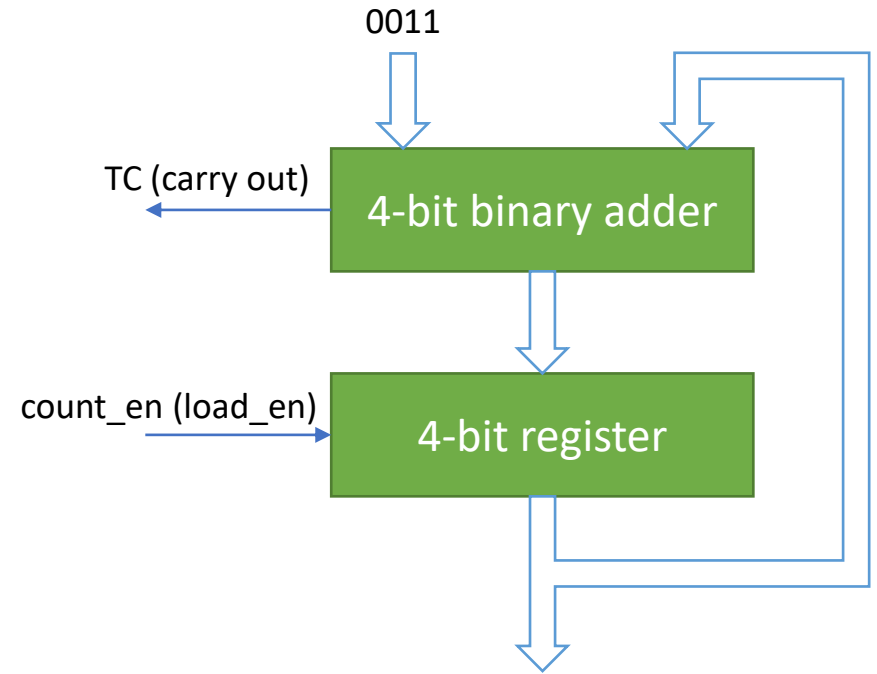
0010 (2), 0101 (5), 1000 (8), 1011 (11), 1110 (14) -> **overflow** ->

0001 (1), 0100 (4), 0111 (7), 1010 (10), 1101 (13) -> **overflow** ->

e ricomincia da 0000.

Il contatore passa per lo 0 (cioè fa overflow) ogni qualvolta l'uscita vale 15, 14 o 13. Si potrebbe quindi usare un circuito che metta a 1 un segnale in corrispondenza di queste combinazioni.

Più semplicemente, si va in overflow quando fa overflow il sommatore. L'uscita carry out del sommatore può quindi essere utilizzata come terminal count secondo quanto richiesto.



Domanda 3

- **Analisi circuito sequenziale (6 pts).** Si consideri il circuito sequenziale mostrato in Figura 1. Si derivi il diagramma a stati supponendo che lo stato iniziale sia $A = 0$, $B = 0$, $C = 1$. Si mostri la tabella della verità del calcolo dello stato futuro limitatamente agli stati raggiungibili dalla macchina.

Risposta

Invece di derivare l'intera tabella degli stati, consideriamo solo quelli raggiungibili partendo da 001. Dal circuito deriviamo che

$$A_{+1} = LB' \quad B_{+1} = L'B' \quad C_{+1} = B$$

Lo stato futuro non dipende da A e neanche da C. E' facile allora costruire la tabella degli stati ed il diagramma a stati.

- Lo stato 001 finisce in 010 se $L = 0$, in 100 se $L = 1$
- Lo stato 010 finisce in 001 in modo incondizionato
- Lo stato 100 va in 010 se $L = 0$, in 100 se $L = 1$ (come per 001, visto che hanno gli stessi valori di B e C!)

Stato presente			Ingr.	Stato futuro		
A	B	C	L	A_{+1}	B_{+1}	C_{+1}
0	0	1	0	0	1	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	0	1
1	0	0	0	0	1	0
1	0	0	1	1	0	0

tabella degli stati

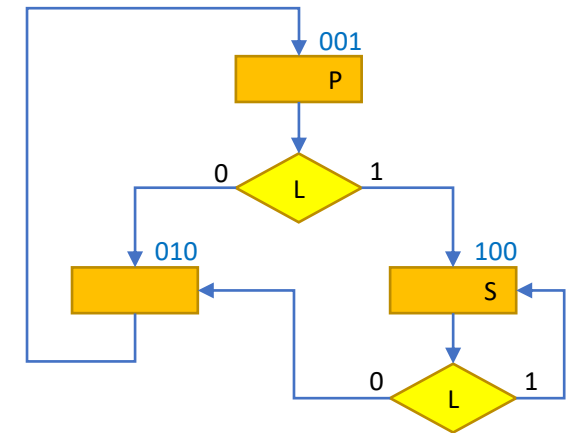
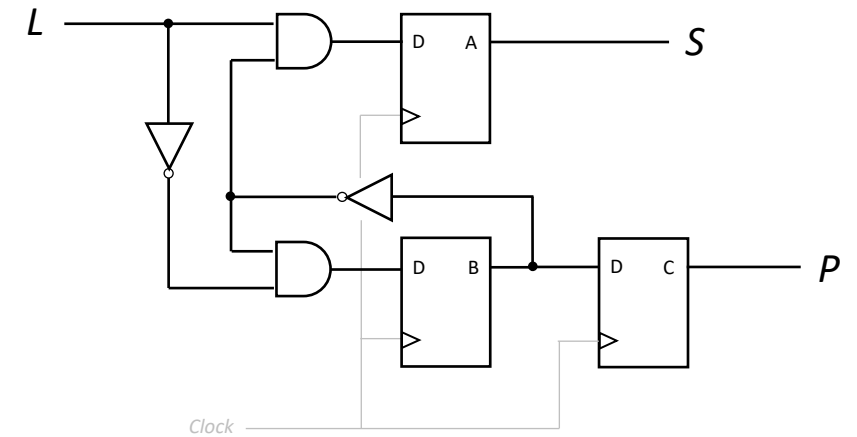


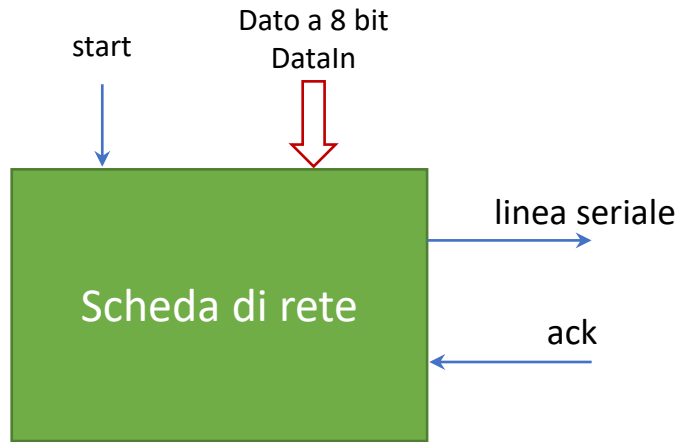
diagramma a stati



Scheda di rete

Prova in itinere Aprile 2019

Specifica



Progettare il data path

- Dovete in qualche modo **acquisire il dato e serializzarlo**
- Dovete **sapere a che punto della trasmissione** siete arrivati (meglio tenerne conto nel data path)
- Dovete **attendere ack per un certo tempo**. Potete attendere un tempo uguale a quello della trasmissione
- Per **attendere un tempo casuale** ci sono molti modi, si può prevedere un componente che ogni tanto attiva casualmente un segnale, e poi progettarlo successivamente

Funzionamento

- La scheda **riceve un dato a 8 bit** sulla linea di ingresso, assieme ad un segnale di start
- Deve allora **serializzare il dato** sulla linea seriale, un bit ogni ciclo di clock, preceduto da un bit di start a 1 e seguito da un bit di stop a 0
- Quando non in uso la linea seriale deve essere a 0
- Terminata la trasmissione, si **attende il segnale ack** per un certo tempo
- **Se non arriva** entro un tempo predefinito, si attende un tempo casuale e si **ritrasmette il dato**

Progettare il controllo

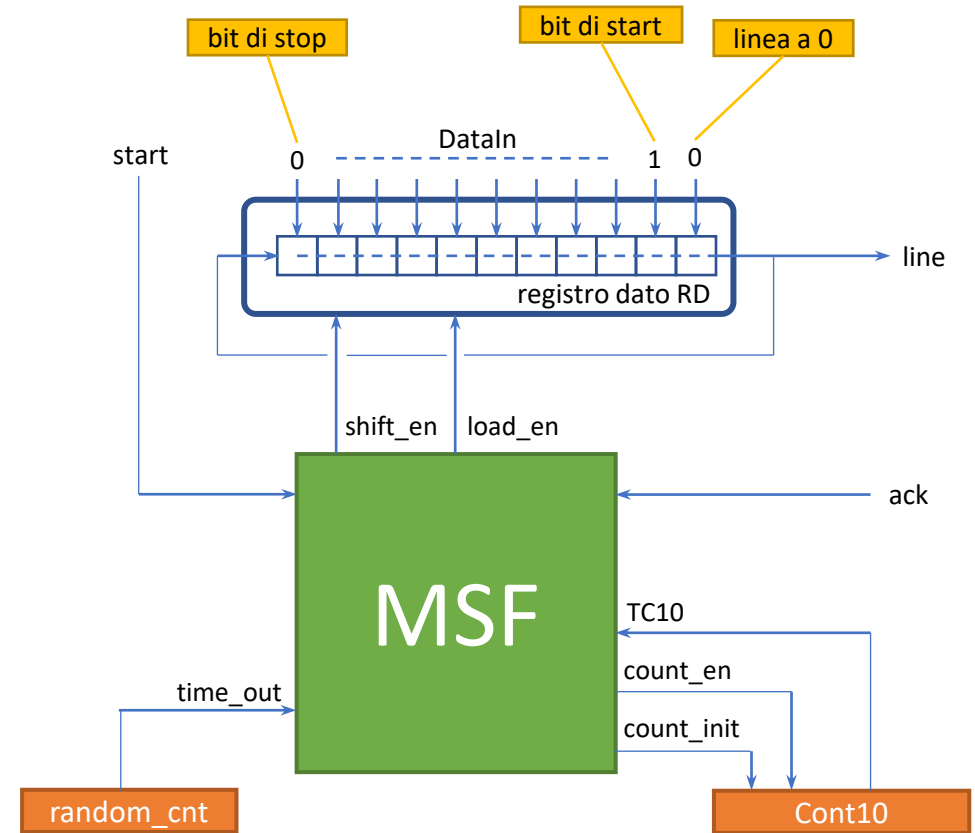
- Non si dice per quanto è attivo start, potrebbe essere uno o più cicli di clock
- **Il dato in ingresso potrebbe essere valido anche per solo un ciclo di clock (in cui start è attivo)**. Questo influenza anche eventualmente il data path

1. Acquisizione

- Un registro a scorrimento a caricamento parallelo si occupa di acquisire il dato
- Dato acquisito quando **load_en** è attivo
- La linea seriale ha un valore uguale all'uscita del flip flop più a destra del registro a scorrimento
- Oltre al dato, mettiamo nel registro anche il bit di stop (lo 0 più a sinistra), il bit di start, e uno 0 che tiene la linea a 0, così da non doverli fare a parte
- Scorrimento abilitato quando **shift_en** è attivo
- L'uscita seriale del registro è collegata al suo ingresso seriale: in questo modo, mentre si trasmette si ricarica il dato, che è quindi pronto nel registro a scorrimento se occorre ritrasmetterlo qualora non si dovesse ricevere **ack**. In questo modo non è necessario avere un registro a parte per memorizzare il dato e nel caso rimetterlo nel registro a scorrimento (il dato in ingresso su **DataIn**, si è detto, potrebbe non essere più disponibile)

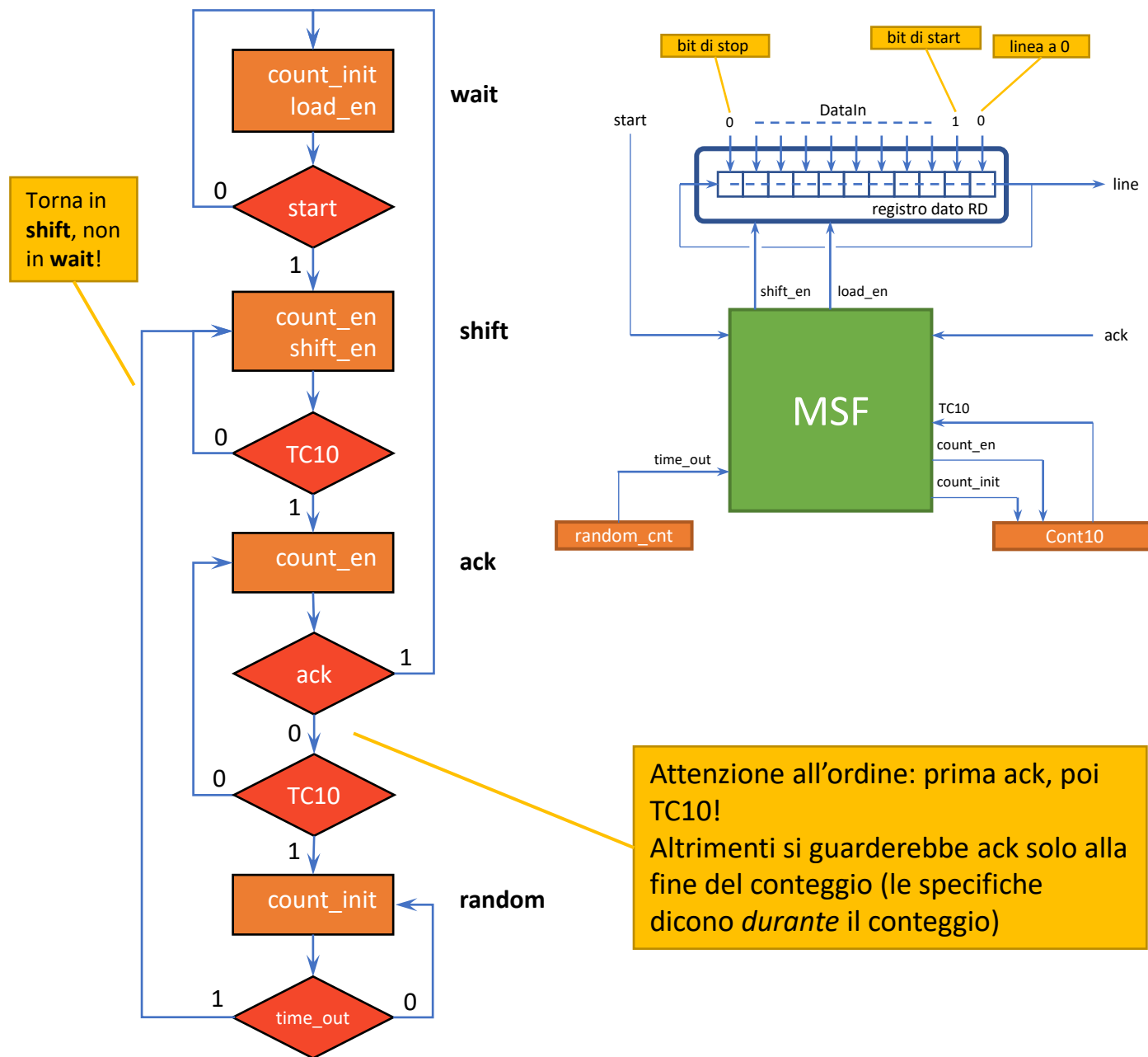
2. Conteggi

- Il contatore **Cont10** (conta modulo 11, da 0 a 10) è usato per contare i bit trasmessi, e sapere quando la trasmissione è terminata
- Dotato di segnali sincroni di inizializzazione (**count_init**) e di abilitazione al conteggio (**count_en**)
- L'uscita di terminal count (**TC10**) è attiva quando il contatore si trova al fondo scala, cioè a 10
- Usiamo lo stesso contatore anche per misurare il tempo entro il quale deve essere ricevuto il segnale **ack** dopo il termine della trasmissione



3. Tempo casuale

- Il contatore **random_cnt** è un contatore *free running*, che mette a 1 la sua uscita di terminal count **time_out** alla fine di ogni conteggio
- Non è proprio un tempo casuale, ma se i dati arrivano in maniera scorrelata rispetto al funzionamento del dispositivo, il tempo è abbastanza casuale
- Una soluzione alternativa può prevedere un vero generatore di tempo casuale innescato dalla MSF quando occorre attendere prima di ritrasmettere



Stato wait

- Attende il segnale di **start**
- Contestualmente inizializza **Cont10** (**count_init**) e carica anche il dato in ingresso nel registro a scorrimento (**load_en**)

Stato shift

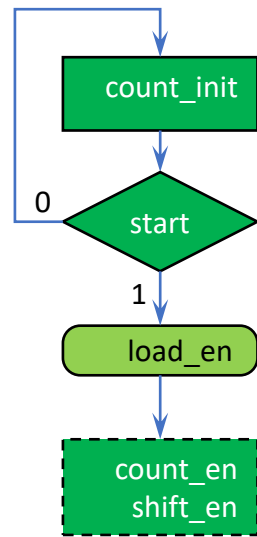
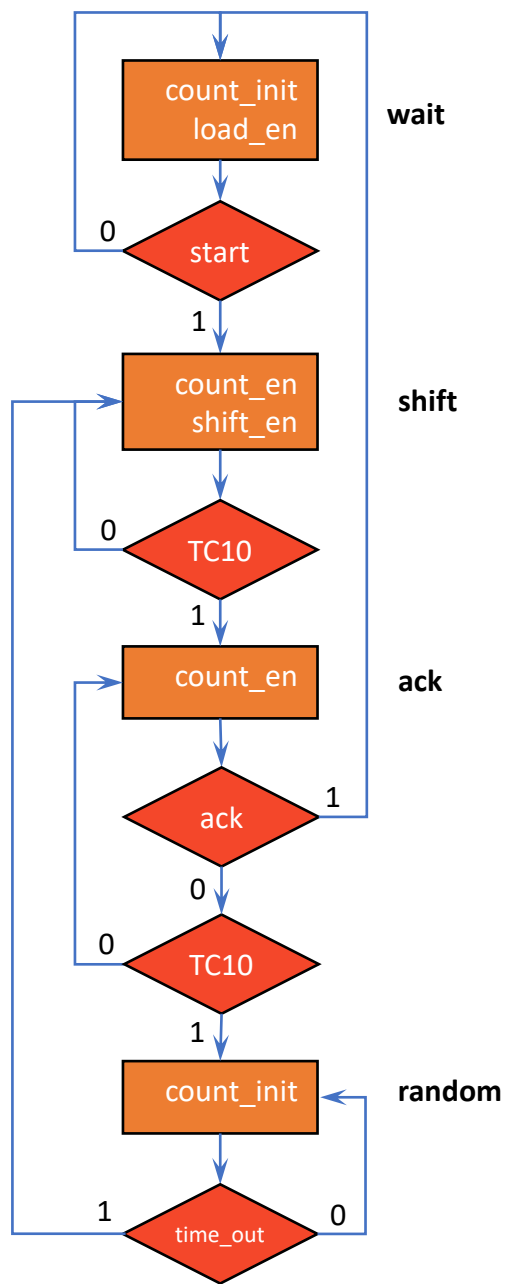
- Abilita lo shift register allo scorrimento (**shift_en**) ed il contatore al conteggio (**count_en**) fino al terminal count (**TC10**)
- In questo stato i bit vengono messi sulla linea seriale uno dopo l'altro

Stato ack

- Attende l'arrivo del segnale **ack**. Se arriva torna in **wait**
- Tiene **count_en** attivo. Il contatore aveva fatto overflow, quindi ricomincia da zero, non è necessario iniziarlo con uno stato apposta (ma non sarebbe errato, si aspetta un ciclo in più)
- Se arriva prima **TC10** va in **random**. Se **TC10** arriva assieme ad **ack** vince **ack**

Stato random

- Attende il segnale random di ritrasmissione
- Quando arriva (**time_out**), non torna in **wait**, ma torna in **shift** per rifare la trasmissione. Il dato è stato ricaricato nel registro a scorrimento durante lo scorrimento
- Attiva anche **count_init** in modo da cominciare con il contatore a zero in shift. Non sarebbe in realtà necessario, visto che siamo entrati in random dopo il **TC10**, quindi il contatore è già a zero



Caricamento del dato

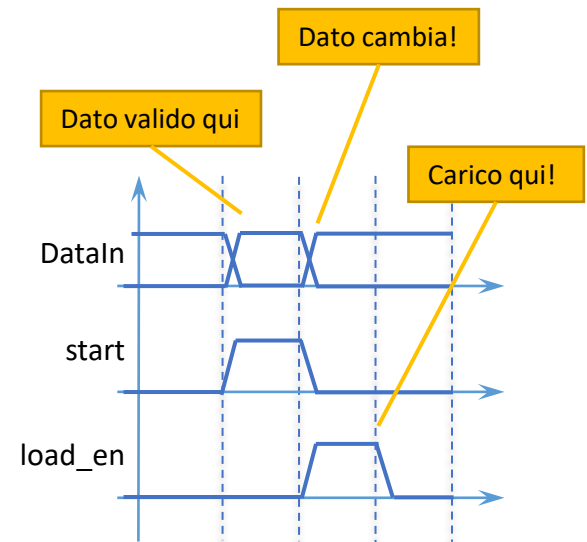
- La MSF carica il dato anche quando `start` non è attivo
- Non è un problema, fosse anche sbagliato ad un certo punto carica comunque quello giusto
- Ha il vantaggio che se il dato è valido contestualmente al segnale `start` abbiamo già il segnale attivo quando `start` va a 1
- Le specifiche non dicono nulla: dobbiamo assumere che il dato valido e il segnale `start` siano presenti contemporaneamente nello stesso ciclo di clock

Alternativa: uscita condizionata

- Il segnale `load_en` è attivato con una uscita condizionata
- Funziona se `start` e il dato sono validi nello stesso ciclo di clock
- **Può andare bene**, soprattutto se `start` è sincronizzato con il clock
- Se non lo è, potrebbero esserci problemi di corse critiche

load_en in uno stato a parte

- In un ciclo vediamo lo `start`
- Il `load_en` è attivo nel ciclo successivo
- **Potrebbe non funzionare**: se il dato è valido solo con lo `start`, il ciclo dopo potrebbe essere già cambiato. Il `load` lo vediamo al ciclo dopo, quindi il dato andrebbe perso



Nome:

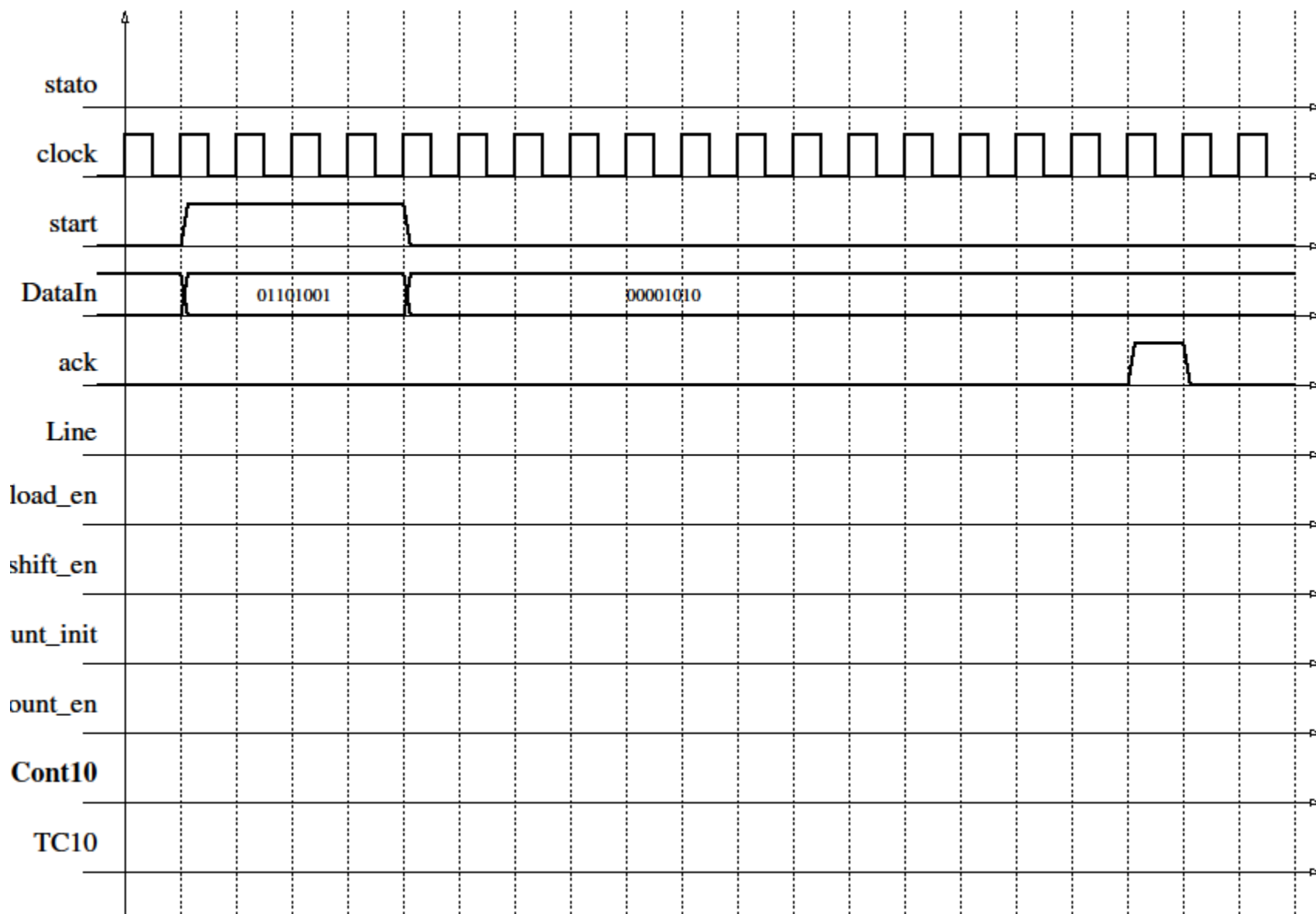


Diagramma temporale:

Arriva un dato con lo start. In questo diagramma non ci sono problemi che il dato sia valido un solo ciclo, ma sarebbe opportuno progettare la MSF in modo che possa gestire anche quella situazione

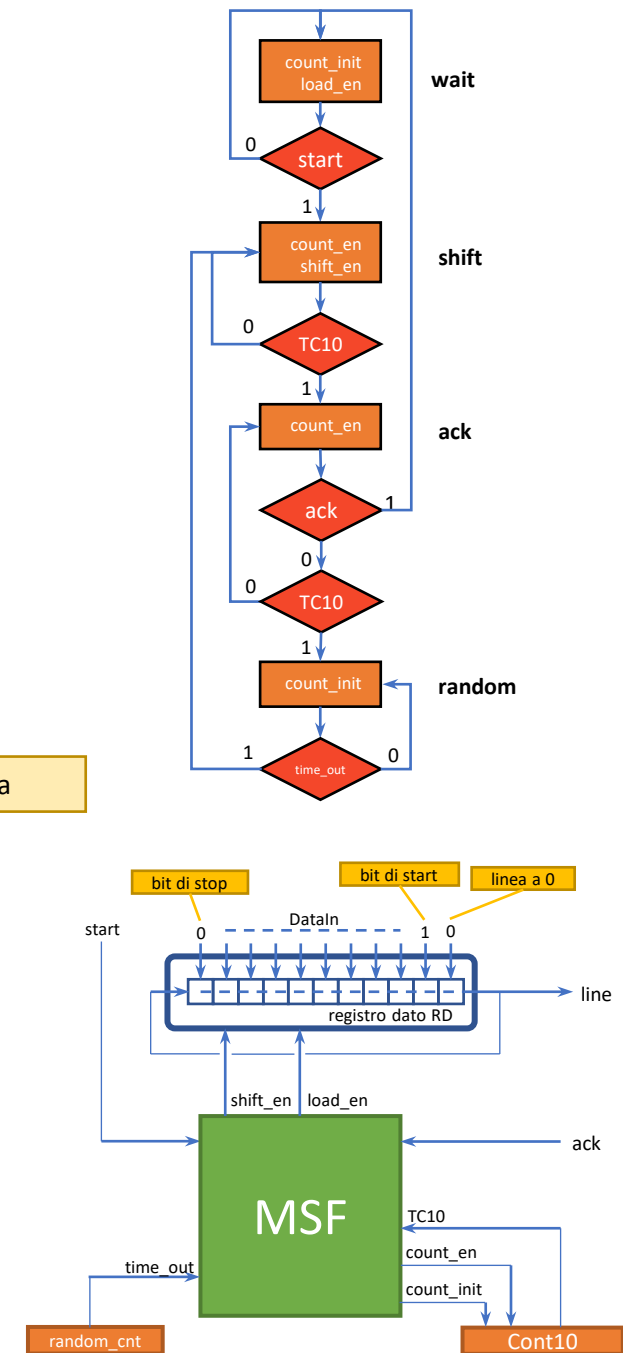
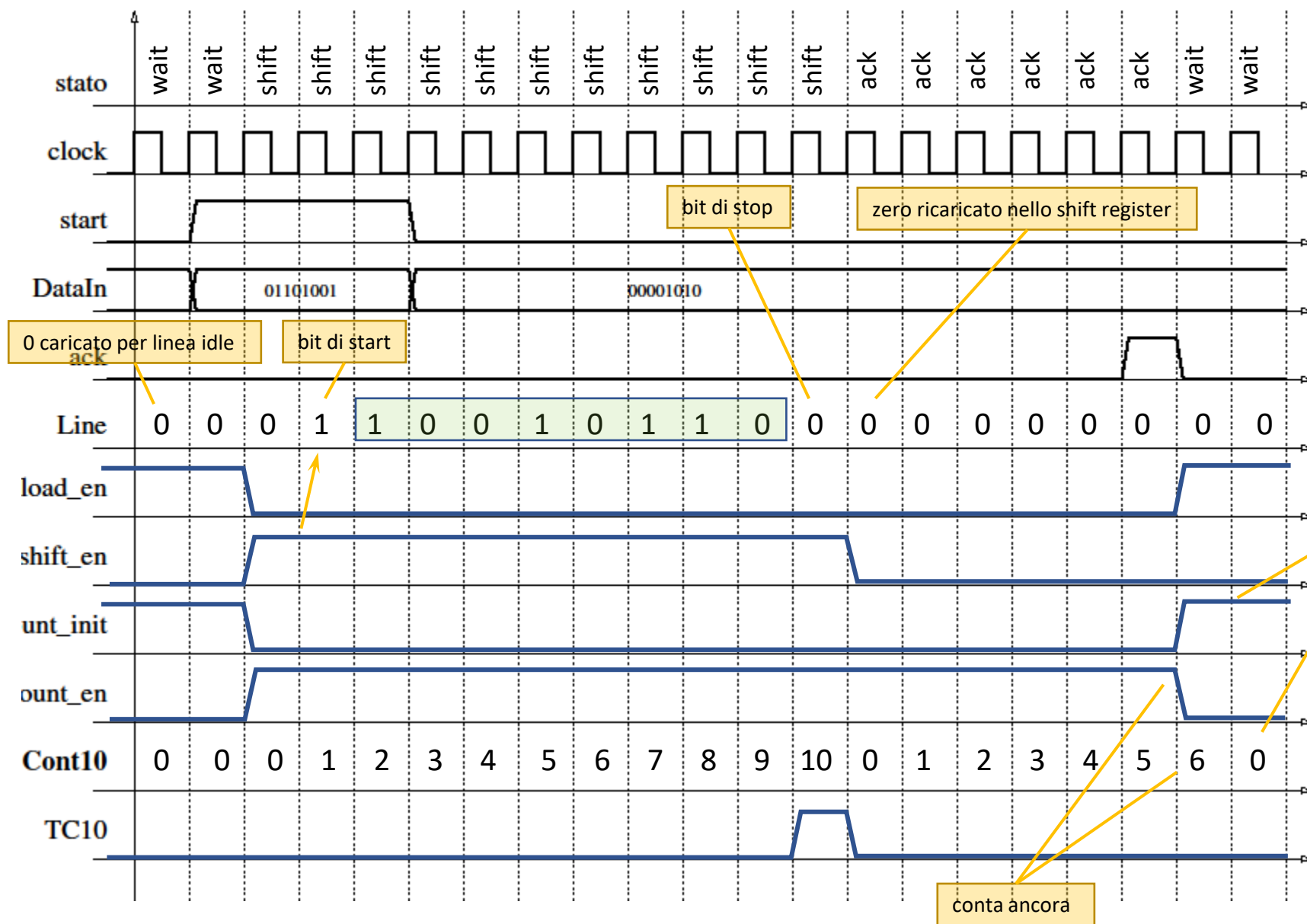
Il dato cambia dopo un po', non si potrebbe ricaricarlo se occorre rifare la trasmissione

In questo caso l'ack arriva entro il tempo stabilito

Usare linea Cont10 per indicare il valore del contatore

Indicare su Line il valore di uscita del registro a scorrimento, sulla base del dato in ingresso

Nome: Hermione Granger



Provetta Aprile 2018

Domande di teoria

Domanda 1

- (6 pts) Considerando implicanti in forma di prodotto, si discuta cosa si intende per implicante, implicante primo ed implicante primo essenziale. Si identifichino tutti gli implicanti primi della funzione $f(a,b,c,d)$ mostrata in Figura 1, scrivendo la loro espressione. Si identifichino quindi gli implicanti primi essenziali, giustificando la risposta (sono essenziali perché...). Infine, si identifichino due coperture per f , identificando sia la copertura minima, sia una copertura alternativa non minima ma comunque non ridondante (e facente uso di soli implicanti primi).

Risposta

Siano f e g due funzioni di n variabili. Si dice che g è un implicante di f , se quando g è 1 allora anche f è 1. Formalmente

$$g(x_1, \dots, x_n) = 1 \rightarrow f(x_1, \dots, x_n) = 1$$

Se invece g è 0, f può assumere qualunque valore. Si dice che g copre certi 1 di f . Un implicante è primo se *non* è completamente contenuto in nessun altro implicante. Un implicante primo è essenziale se copre un 1 non coperto da nessun altro implicante *primo*.

Implicanti primi di f

bc $b'd$ cd $a'b'c'$ $a'bd'$ $a'c'd'$

Implicanti primi essenziali di f

bc perché è l'unico a coprire 1110, $b'd$ perché è l'unico a coprire 1001

Tolti gli essenziali, rimangono scoperti gli uni a 0000 e 0100. Possono essere coperti con $a'c'd'$. La copertura minima di f è quindi

$$f(a, b, c, d) = bc + b'd + a'c'd'$$

Questi uni possono essere coperti anche con due implicanti, ottenendo una copertura non ridondante ma non minima

$$f(a, b, c, d) = bc + b'd + a'b'c' + a'bd'$$

$ab \setminus cd$	00	01	11	10
00	1	1	1	0
01	1	0	1	1
11	0	0	1	1
10	0	1	1	0

Domanda 2

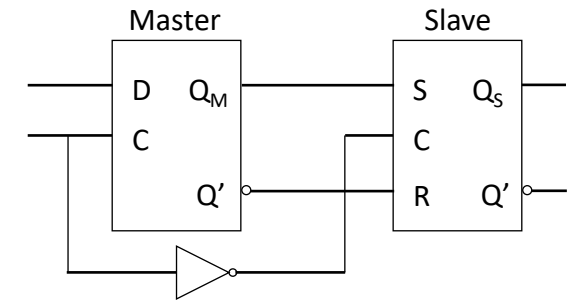
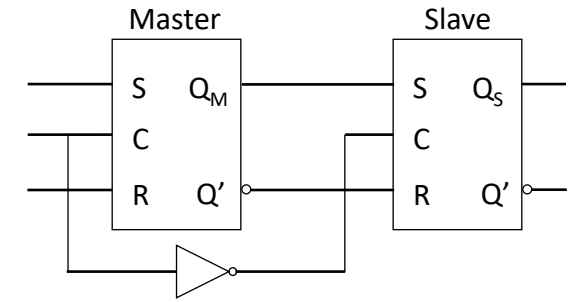
(6 pts) Si disegni lo schema di un flip-flop master-slave di tipo SR e di un flip-flop di tipo D edge-triggered, discutendone il funzionamento. Supponendo che l'uscita sia al valore '0', si costruiscano per entrambi i casi i diagrammi temporali con i segnali di interesse (clock, S, R, D, le uscite ed i segnali interni del circuito) specificando i valori che devono essere applicati ai rispettivi ingressi per avere una transizione in uscita al valore '1' al fronte attivo del clock. Assumendo ora l'uscita al valore '1', si mostri tramite un diagramma temporale come un glitch sugli ingressi, che si esaurisce prima del fronte attivo del clock, possa determinare la transizione dell'uscita al valore '0'. Per quale dei due circuiti si verifica il fenomeno?

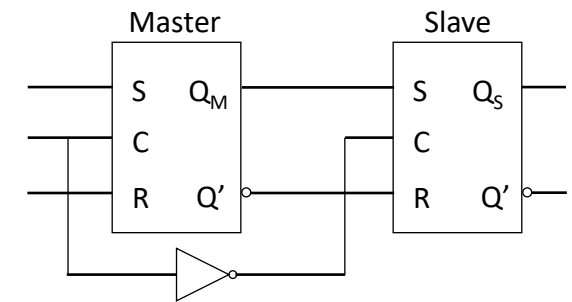
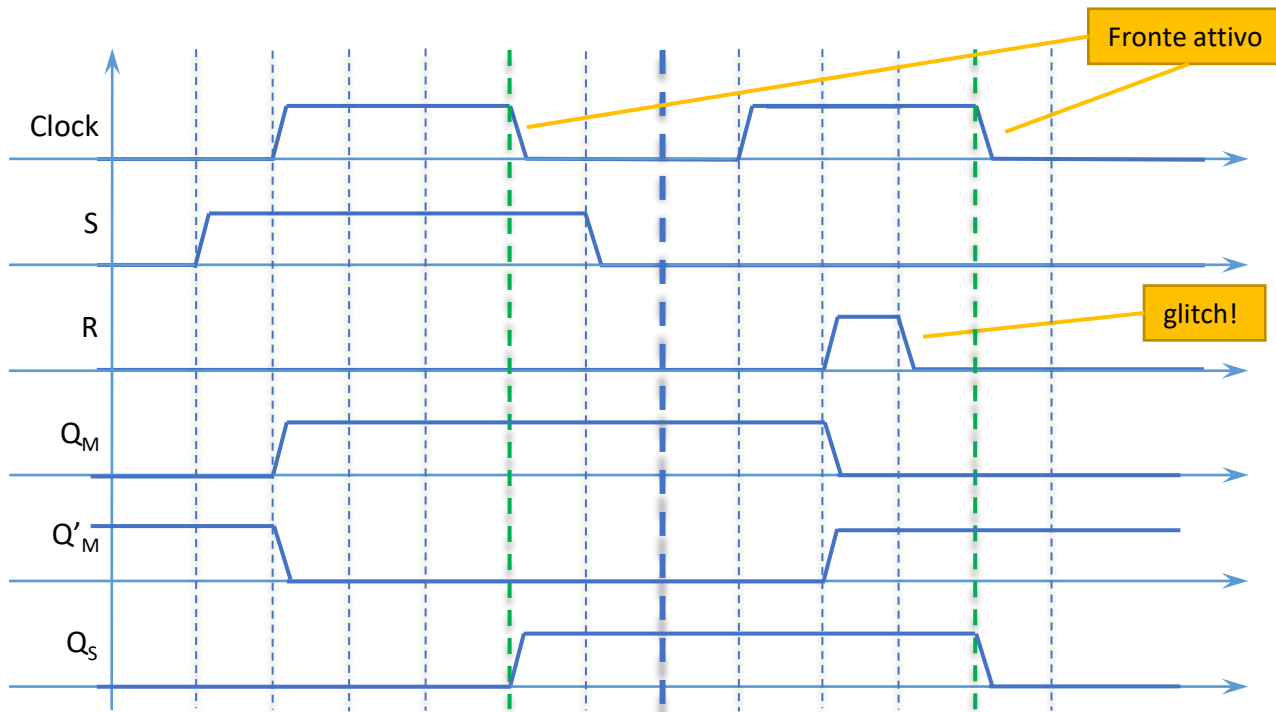
Risposta

Il diagramma dei flip flop master slave ed edge triggered è mostrato a lato in figura, ed è costituito da **due latch in cascata con clock complementare**. In entrambi i casi il funzionamento è simile

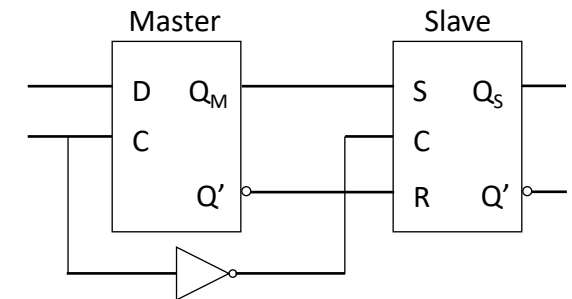
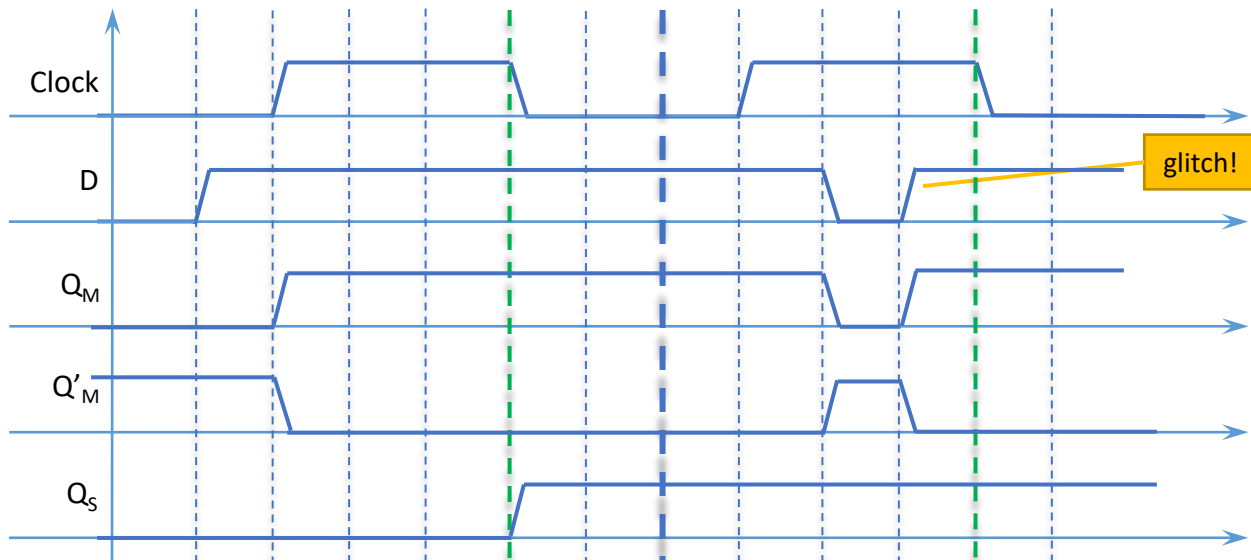
- Quando il clock è a valore logico 1, il master è attivo ed è possibile impostare il valore dell'uscita Q_M tramite gli ingressi. Questo valore non raggiunge l'uscita, perché lo slave è in memoria.
- Alla transizione del clock da 1 a 0, il valore presente in uscita dal master viene riportato sull'uscita Q_S , poiché lo slave diventa trasparente. Il valore dell'uscita del master rimane fisso perché ora il master è in memoria.

Pertanto l'uscita si aggiorna con il valore di Q_M ad ogni transizione di discesa del clock. Nel caso del flip flop edge triggered, il valore di Q_M è uguale all'ingresso D (a parte i ritardi), quindi di fatto il dispositivo campiona il valore di D al fronte di discesa del clock.





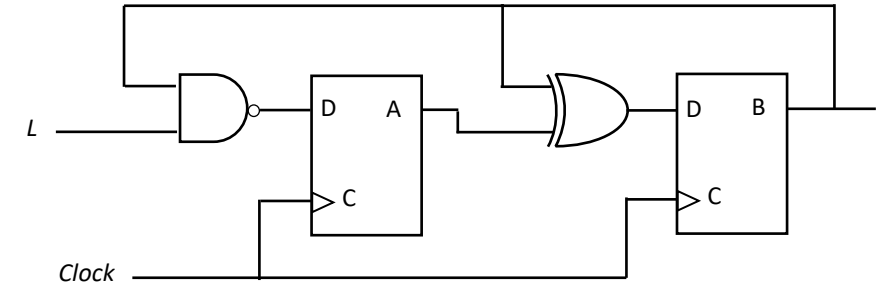
Per il master slave, occorre mettere $S = 1$ ed $R = 0$ per impostare a 1 l'uscita del master, che viene poi registrata dallo slave. S può quindi tornare a 0. Se poi si ha un glitch su R mentre il clock è a 1, l'uscita del master si porta a 0, e vi rimane anche quando il glitch su R è esaurito. Al fronte attivo successivo, quindi, l'uscita dello slave si porta a 0.



Per l'edge triggered, è sufficiente mettere l'ingresso D a 1 per far fare la transizione dell'uscita da 0 a 1. A questo punto un eventuale glitch di D da 1 a 0 e poi di nuovo a 1 viene riportato sull'uscita del master, e quindi si estingue. L'uscita dello slave rimane quindi a 1 al fronte attivo successivo.

Domanda 3

- (6 pts) Si consideri il circuito mostrato in Figura 2. Si derivi e si disegni il diagramma a stati corrispondente mostrando inoltre la tabella della verità della funzione di calcolo dello stato futuro. Inventare delle uscite (inclusendo, se si vuole, una rete combinatoria) in modo da realizzare una macchina di Moore ed una di Mealy. Considerando il diagramma degli stati, e mantenendo la stessa codifica, semplificare il calcolo dello stato futuro di B. (notare che la porta exor può considerarsi equivalente a tre porte logiche)



Risposta

Sulla base della tabella della verità della porta NAND costruiamo la tabella degli stati, indicando lo stato futuro sulla base dello stato presente e dell'ingresso L. Le espressioni per lo stato futuro sono

$$A_{+1} = (LB)' \quad B_{+1} = A \oplus B$$

E' facile allora costruire il diagramma a stati.

- Lo stato 00 finisce in modo incondizionato nello stato 01
- Lo 01 finisce in modo incondizionato nello stato 11
- Lo stato 11 va in 01 se $L = 0$, in 00 se $L = 1$

Per avere una macchina di Moore prendo B come uscita. Per una macchina di Mealy, prendo A_{+1} come uscita. Alternativamente potrei inventare un'uscita tipo $S = B + L$. L'importante è che dipenda dall'ingresso L.

Stato presente		Ingr.	Stato futuro	
B	A	L	B_{+1}	A_{+1}
0	0	0	0	1
0	0	1	0	1
0	1	0	1	1
0	1	1	1	1
1	0	0	1	1
1	0	1	1	0
1	1	0	0	1
1	1	1	0	0

tabella degli stati

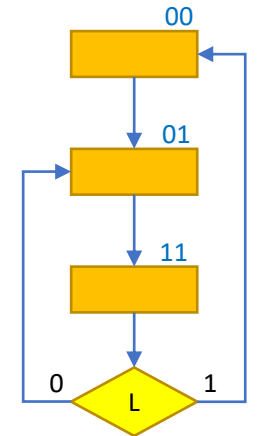


diagramma a stati

- (6 pts) Si consideri il circuito mostrato in Figura 2. Si derivi e si disegni il diagramma a stati corrispondente mostrando inoltre la tabella della verità della funzione di calcolo dello stato futuro. Inventare delle uscite (inclusendo, se si vuole, una rete combinatoria) in modo da realizzare una macchina di Moore ed una di Mealy. Considerando il diagramma degli stati, e mantenendo la stessa codifica, semplificare il calcolo dello stato futuro di B. (notare che la porta exor può considerarsi equivalente a tre porte logiche)

Per la seconda parte della domanda, osserviamo dal diagramma a stati che lo stato 10 non è raggiunto. Posso quindi considerare lo stato futuro da 10 come un don't care, e rideterminare la rete di calcolo per lo stato futuro.

$$B_{+1} = AB' + A'B = A \oplus B$$
$$B_{+1} = AB'$$

Ovviamente, il diagramma a stati rimane immutato.



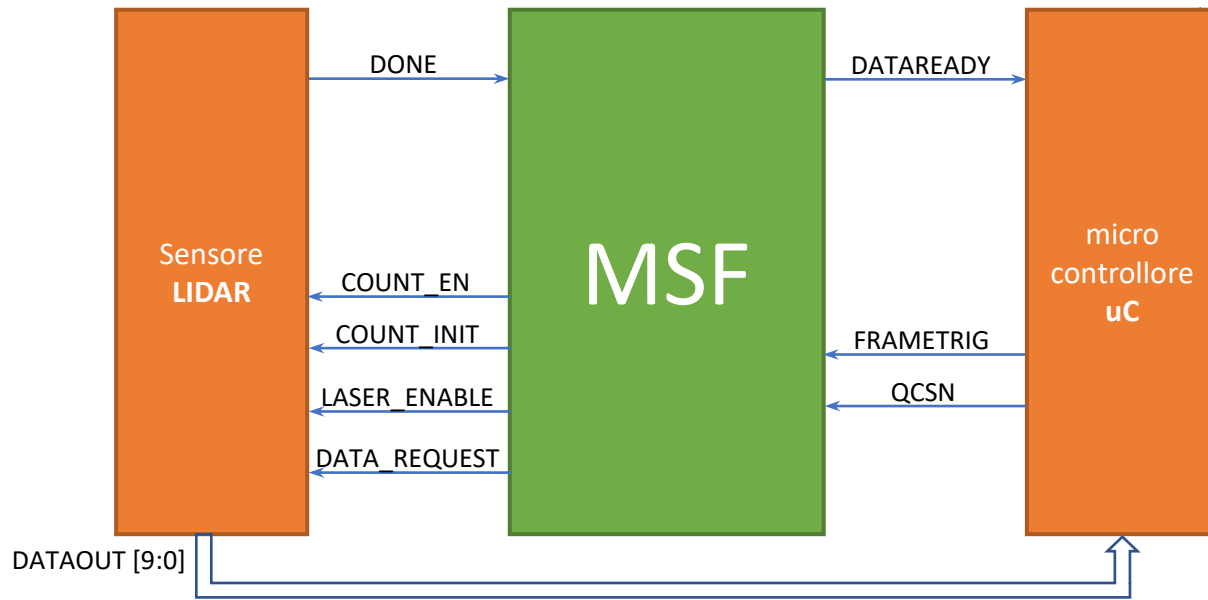
```
graph TD; S00[00] --> S01[01]; S01 --> S11[11]; S11 --> L{L}; L -- 0 --> S00; L -- 1 --> S01;
```

diagramma a stati

$$A_{+1} = L' + B' = (LB)'$$
$$B_{+1} = AB'$$

Controllo sensore Lidar

Prova in itinere Aprile 2018



Funzionamento

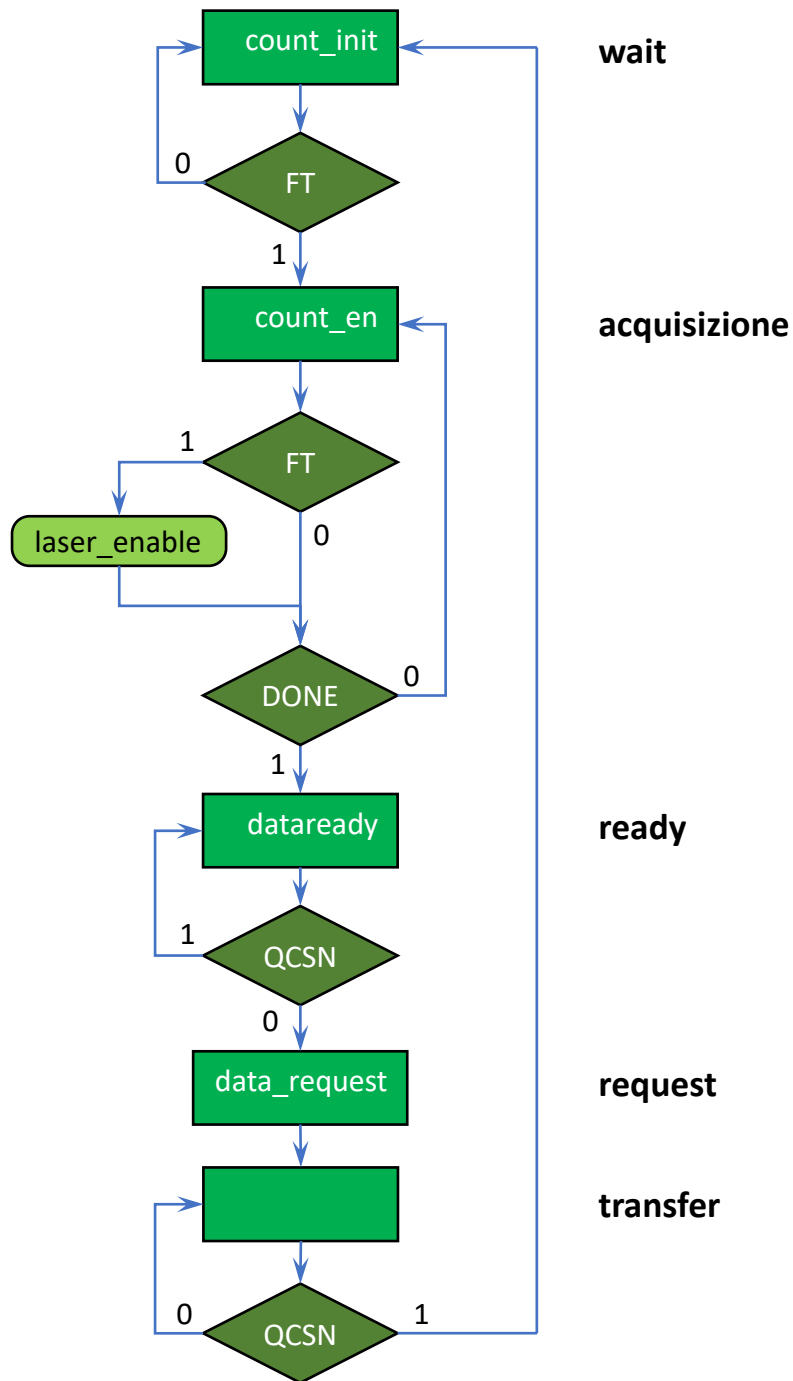
- La **MSF** deve interfacciare un sensore Lidar con un microcontrollore
- Si comporta come una sorta di traduttore di protocollo di comunicazione
- Il **uC** comanda l'inizio di una **acquisizione** tramite **FRAMETRIG**, attende **DATAREADY** e poi comanda il **trasferimento dei dati** tramite **QCSN** (attivo basso)
- La **MSF** abilita il Lidar adattando le richieste ai rispettivi segnali di controllo

Acquisizione

- Il **uC** abilita **FRAMETRIG**, che rimane attivo per tutta l'acquisizione
- Ogni acquisizione è costituita da diversi frame, contati all'interno del sensore da un contatore sincrono
- La **MSF** attiva **COUNT_EN** per contare i frame. Quando l'acquisizione termina, il sensore attiva **DONE**
- Il **uC** rileva eventuali situazioni di pericolo. In tal caso, durante l'acquisizione, disabilita **FRAMETRIG**
- La **MSF** deve spegnere il laser **al più presto**, disattivando **LASER_ENABLE**, che è normalmente attivo
- Lo riattiva quando **FRAMETRIG** torna a 1
- **L'acquisizione deve però continuare**, quindi **COUNT_EN** deve rimanere attivo anche se **FRAMETRIG** è disabilitato

Trasferimento dati

- Finita l'acquisizione, la **MSF** attiva **DATAREADY**, ed attende che il **uC** sia pronto a ricevere i dati
- Il **uC** è pronto quando mette **QCSN** a 0 (il segnale è attivo basso)
- La **MSF** comunica al sensore di cominciare il trasferimento di dati, attivando **DATA_REQUEST** per un solo ciclo di clock
- Il sensore trasmette i dati autonomamente su **DATAOUT**. Il trasferimento termina quando il **uC** riporta il valore di **QCSN** a 1
- La **MSF** si predispone per una nuova acquisizione. Occorre inoltre inizializzare il contatore del Lidar



Stato wait

- Attende il segnale di **FRAMETRIG**
- Contestualmente inizializza il contatore del Lidar (**count_init**)

Stato acquisizione

- Abilita il contatore del Lidar al conteggio (**count_en**)
- Abilita il laser, ma solo se **FRAMETRIG** è attivo, tramite una uscita condizionata. Il conteggio invece è sempre attivo, come richiesto
- Continua l'acquisizione fin quando si attiva **DONE**
- Si potrebbe guardare prima **DONE** e poi **FRAMETRIG**. E' più o meno lo stesso, si disabilita il laser un ciclo prima

Stato ready

- Abilita il segnale **DATAREADY** ed attende che il **uC** sia pronto a ricevere i dati

Stato request

- Abilita il segnale **DATA_REQUEST** per un singolo ciclo di clock, come da specifica

Stato transfer

- Il trasferimento dei dati avviene autonomamente
- La **MSF** attende semplicemente che venga disattivato **QCSN** (che deve tornare al valore 1)

Nome e matricola:

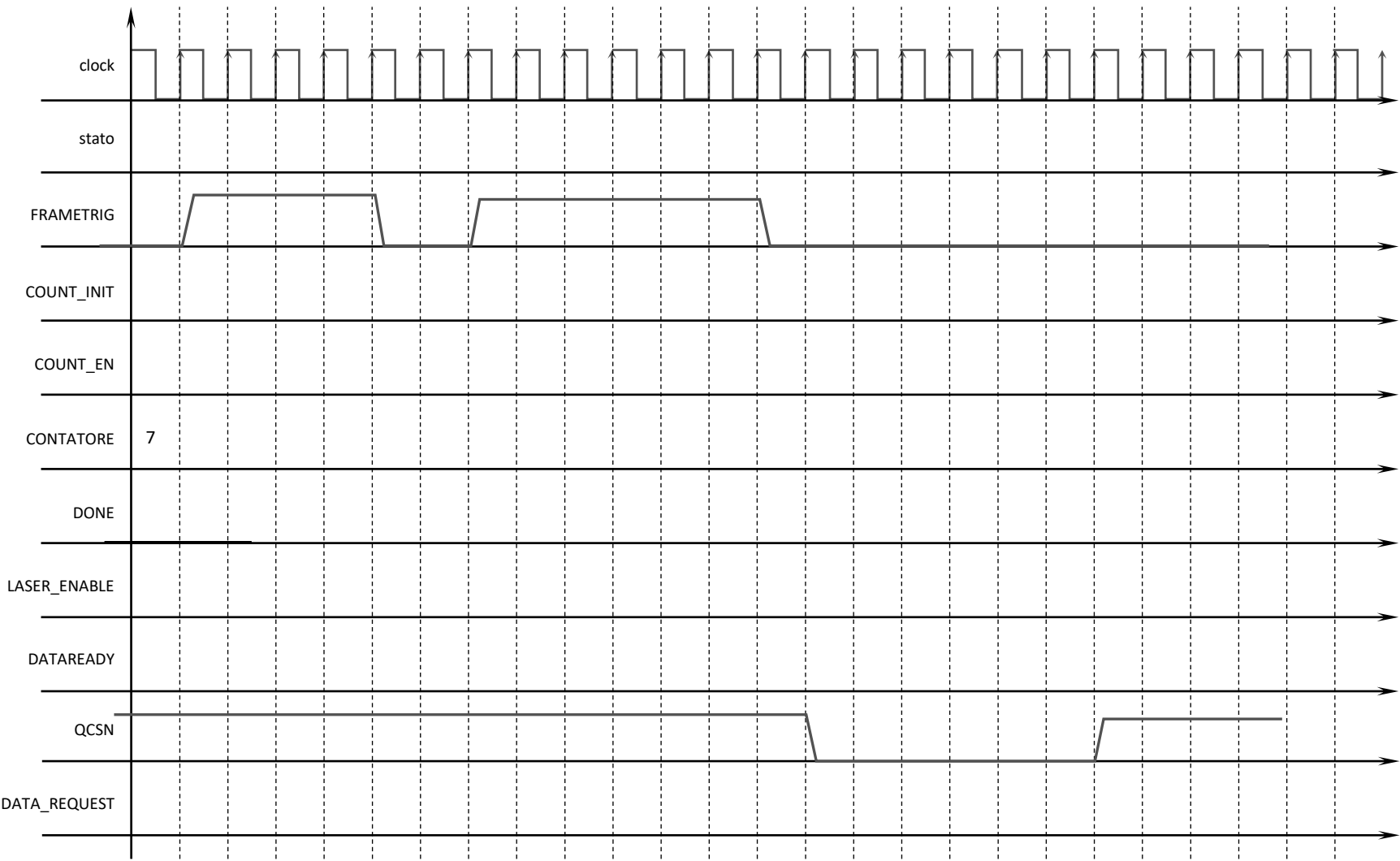


Diagramma temporale:

Il **uC** richiede l'acquisizione tramite FRAMETRIG. Per due cicli di clock viene disattivato per spegnere il laser

Il contatore del Lidar conta all'indietro, partendo dal valore 7. Segnare questo valore nella riga CONTATORE. Il segnale DONE viene attivato quando il contatore ha valore 0

Dopo l'acquisizione, QCSN va a 0 per comandare il trasferimento dei dati

Nome e matricola: Leila Regina Solo

